INTRODUCTION · EXAM STRATEGY - ALL DOMAINS

Databricks Certified Data Engineer Professional

A complete, original preparation course built against the current exam guide.

QUESTIONS 59 scored TIME LIMIT 120 minutes GUIDE VERSION 3 July 2026

admin@sthithapragna.com

The Associate exam asks whether you can build a pipeline. This one asks whether you can **keep it running in production**.

What changes at Professional level

THE ASSOCIATE ASKS

Can you build it?

- Which tool reads this file format
- Write the transformation
- Schedule the job
- Grant the right privilege

THE PROFESSIONAL ASKS

Can you run it?

- Why did this pipeline get slow
- Which of two working designs costs less
- What breaks when this stream restarts
- Prove the deletion actually happened

THE TIEBREAKER

If you can do a thing but cannot say why the alternative is worse, you are not ready for this paper.

This certification is aimed at you if

- You already build and operate pipelines on Databricks, and want the credential to match.
- Databricks recommends **one year** of hands-on data engineering, plus the related training.
- There is **no formal prerequisite**. The Associate is recommended, not required.
- This is not an entry point. If Delta Lake and Unity Catalog are new to you, sit the Associate first.

> ON THE EXAM
>
> Recommended training is the instructor-led Advanced Data Engineering With Databricks, plus four self-paced Academy courses.

What exam day looks like

	DETAIL
Questions	59 scored, plus unmarked unscored items
Time limit	120 minutes, about 2 minutes each
Format	Multiple choice, four options, one answer
Cost	200 US dollars plus local tax
Delivery	Online proctored, or a test centre
Test aids	None at all, including API documentation
Validity	2 years, recertify on the then-live exam
Pass mark	Not published by Databricks

ON THE EXAM

No API documentation means syntax has to be recalled rather than looked up. That should change how you revise.

Ten sections, and what each is worth

SECTION	WEIGHT
1. Developing code with Python and SQL	22%
2. Data ingestion and acquisition	7%
3. Transformation, cleansing and quality	10%
4. Data sharing and federation	5%
5. Monitoring and alerting	10%
6. Cost and performance optimization	13%
7. Data security and compliance	10%
8. Data governance	7%
9. Debugging and deploying	10%
10. Data modeling	6%

ON THE EXAM

Five sections carry 65% between them: developing code, cost and performance, monitoring, security, and debugging and deploying.

What this exam actually dwells on

- **Streaming.** Triggers, checkpoints and state come up more than any other single topic.
- **Declarative pipelines** dominate section one. Streaming tables against materialized views is a recurring decision.
- **Delta internals.** The transaction log, statistics, time travel, and shallow against deep clone.
- **Data quality.** Expectations and quarantining bad records, far more than the weights alone suggest.

> ON THE EXAM
>
> Streaming is not a section of its own. It is spread across sections one, two, three and six, which is exactly why it gets under-prepared.

Original content, and why that matters

- **Every scenario here is original**, written from Databricks documentation and the current exam guide.
- A bank of **339 practice questions** was analysed to find which decisions the exam tests, and how often.
- **No question from that bank appears in this course.** Recycled dumps train recognition, not reasoning.
- Wrong options are explained too. That is what transfers to a question you have never seen.

> ON THE EXAM
>
> Of those 339 questions, only 100 carried an answer and only 42 an explanation. That gap is a large part of why this course exists.

What you may have learned it as

YOU MAY KNOW IT AS	WHAT THE EXAM CALLS IT NOW
Delta Live Tables, or DLT	Lakeflow Spark Declarative Pipelines
Workflows	Lakeflow Jobs
Repos	Git folders
Databricks Asset Bundles	Declarative Automation Bundles, still DABs
APPLY CHANGES	AUTO CDC, with identical syntax

ON THE EXAM

The July 2026 guide uses every name on the right, and prints the old one beside it in brackets. This course does the same.

Where the guide has not caught up

- **Predictive optimization** runs OPTIMIZE, VACUUM and ANALYZE for you on Unity Catalog managed tables.
- On by default for accounts created after **November 2024**. The exam guide never mentions it.
- So an answer that schedules OPTIMIZE weekly is often the **worse** one now. Older banks are full of those.
- Bundles also stopped using Terraform by default. The direct deployment engine is now generally available.

> ON THE EXAM
>
> Where the guide and the live product disagree, this course teaches the examinable answer and then tells you what is actually true.

The conventions used throughout

- An **amber bar** at the foot of a slide is the exam takeaway. If you note one thing down, note that.
- Teal marks something that is true, useful, and worth reaching for.
- Red marks an anti-pattern, or something that looks right and is not.
- The small label above each heading tells you which idea the slide belongs to.

> ON THE EXAM
>
> The marker down the left edge of every slide shows which of the ten sections you are currently in.

Worked scenarios, and the one rule

- Most modules end with **four worked scenarios**: a situation, a question, and four options.
- **Pause and commit to an answer** before the walkthrough. Say it aloud, or write it down.
- The walkthrough then takes every option in turn and says why it wins or why it loses.
- Watching without answering first feels productive and teaches almost nothing.

> ON THE EXAM
>
> Being wrong and understanding why beats being right by accident. The wrong options are where most of the teaching is.

How the course is ordered

01

Build

Sections 1 to 3. Code, pipelines, ingestion, transformation and quality.

02

Share and watch

Sections 4 and 5. Delta Sharing, federation, system tables and alerts.

03

Tune

Section 6. Delta internals, clustering, and what compute costs.

04

Secure

Sections 7 and 8. Masking, purging, and Unity Catalog permissions.

05

Ship

Sections 9 and 10. Debugging, bundles, and data modelling.

ON THE EXAM

Two cross-cutting modules and three practice exams follow. The order is deliberate: you cannot tune what you have not built.

How to actually study this

- Watch in order. Later modules lean on vocabulary the earlier ones set up.
- Then sit **practice exam one**, and mark it section by section rather than only overall.
- Rewatch **only** your weakest section. Then space exams two and three several days apart.
- Revising what you already know keeps the scores high and moves nothing.

> ON THE EXAM
>
> Which section your misses cluster in matters more than the score itself. That is the number to act on.

Ten sections, and **nowhere to hide**. The smallest is still five per cent of your result.

DEVELOPING CODE · DOMAIN 1 - DEVELOPING CODE FOR DATA PROCESSING

Python project structure for bundles

Section 1. Modular code, deployment automation, and where tests actually live.

SECTION 1 of 10 WEIGHT 22% TOPIC Bundles

admin@sthithapragna.com

A bundle turns your whole project into one deployable, version-controlled unit. The exam tests what that buys you, and what it validates.

Why the notebook-first habit breaks

- Logic lives inside notebooks, so it cannot be imported, unit tested, or reused by a second job.
- Promoting to production means **editing paths and cluster IDs by hand**, in the workspace UI.
- Two engineers deploying at once overwrite each other's jobs. Nothing is namespaced.
- A bundle fixes all three by making the project a **source-controlled artefact**, not a set of workspace objects.

> ON THE EXAM
>
> The exam frames this as scale. If an answer involves manually copying notebooks between workspaces, it is almost certainly wrong.

What the default Python template generates

PATH	WHAT IT IS FOR
databricks.yml	The bundle definition. Name, includes, artifacts, variables, targets.
resources/	One file per job or pipeline. Pulled in by a glob.
src/	Your importable Python package. The code that is actually tested.
tests/	Unit tests, run locally or as part of the build.
pyproject.toml	Package metadata and the wheel build definition.
README.md	Generated. How to use this particular bundle.

ON THE EXAM

Created by 'databricks bundle init' with the default-python template. Prompts ask whether to include a notebook job, an ETL pipeline, a stub package and serverless.

What makes this structure scalable

- Logic sits in an **importable package** under the source folder, so a notebook and a job can both call it.
- Each job gets its **own resource file**. Two people editing two jobs never touch the same file.
- Tests import the package directly and run on a laptop, with no cluster and no workspace.
- Environment differences live in **targets**, not in the code. The code never knows which environment it is in.

> ON THE EXAM
>
> Modular development, deployment automation and continuous delivery are the three words the exam guide uses. This slide is all three.

Top-level mappings you should recognise

MAPPING	WHAT IT DOES
bundle	Name, and deployment settings such as the lock.
include	Globs that pull in resource files, typically resources/*.yml.
variables	Named values, referenced elsewhere as ${var.name}.
artifacts	How to build a wheel or JAR before deploying it.
resources	Jobs, pipelines, schemas, volumes and more.
targets	Named environments. Each can override almost anything.
workspace	Host, and the paths artifacts and files are written to.
run_as	The identity a deployed workflow runs under.

ON THE EXAM

Anything set inside a target overrides the same setting at the top level. That override rule is the whole reason one file can serve every environment.

Notebook task or Python wheel task

NOTEBOOK TASK

Fast, but hard to share

- Code and output live together
- Cannot be imported by another job
- Testing means running the notebook
- Cannot take a task environment spec

PYTHON WHEEL TASK

Built, versioned, importable

- Built from your source package
- One entry point, called by name
- Unit tested without a cluster
- Same artefact promoted to every target

WORTH REMEMBERING

A wheel task names a package and an entry point. That entry point is a function declared in the project file, not a file path.

The artifacts mapping, field by field

FIELD	MEANING
`type`	whl or jar. Required for a Python wheel.
`build`	The command to run. Defaults to python setup.py bdist_wheel.
`path`	Where the build runs. Defaults to the bundle root.
`dynamic_version`	Patch the wheel version from its file timestamp.
`files`	Paths to the built artefact files.
`executable`	bash, sh or cmd.

ON THE EXAM

Omit the artifacts mapping entirely and the CLI assumes the wheel is already built, deploying whatever the libraries whl entries point at.

Why yesterday's code keeps running

- You change the code, redeploy, and the job runs the **old behaviour**. Nothing errored.
- Compute caches an installed wheel **by version**. Same version number means no reinstall.
- Set **dynamic_version to true** and the version is patched from the file timestamp on every build.
- New code then deploys without anyone editing a version number in the project file.

> ON THE EXAM
>
> If a question describes redeployed code that behaves as though it never changed, the version number is the thing that did not move.

The command line cycle

01

init

Generate a project from a template, default-python or your own.

02

validate

Check the configuration parses and resolves. Catches errors before any deploy.

03

deploy

Build artefacts, upload files, create or update every resource in the target.

04

run

Trigger a named job or pipeline in that target and return a run URL.

05

summary

Show what is currently deployed. Use destroy to remove it.

ON THE EXAM

Every one of these takes a target. Forgetting which target you are on is the most common cause of a surprise in the wrong workspace.

Development mode against production mode

MODE: DEVELOPMENT

Built for iteration

- Prefixes resources with [dev yourname]
- Pauses all schedules and triggers
- Allows concurrent runs
- Disables the deployment lock
- Lets you override the cluster

MODE: PRODUCTION

Built for safety

- No prefix. Real names, real schedules
- Validates pipelines are not in dev
- Validates the Git branch matches
- Expects a service principal
- Refuses cluster overrides

OFTEN MISSED

Modes are optional. They are a convenience that applies a group of settings at once, not something a bundle requires.

What it validates before it will deploy

- Every declarative pipeline is marked **development: false**.
- The current Git branch matches the branch named in the target. Override with **--force**.
- Databricks recommends **run_as set to a service principal** for production.
- Without one: paths must not be overridden to a specific user, and run_as and permissions must be set.
- Cluster definitions **cannot** be overridden. No compute override is accepted.

> ON THE EXAM
>
> Deployments fail these checks far more often than they fail on syntax. Read the failure message, not the YAML.

Presets, and which setting wins

- Presets adjust mode behaviour: **name_prefix**, **trigger_pause_status**, **jobs_max_concurrent_runs**, **pipelines_development**, **tags**.
- Precedence runs: an **individual resource** beats a **preset**, which beats the **mode default**.
- So a job with max concurrent runs of one stays at one, even if the preset says twenty.
- And a schedule explicitly set to unpaused stays unpaused, even where the preset pauses everything.

> ON THE EXAM
>
> Most specific wins. When a question sets the same thing in two places, find the narrower one.

What the workspace looks like after a deploy

THING	WHERE IT GOES
Your files	Under the bundle root path for the target.
The built wheel	An internal artifacts folder beneath that root.
Jobs and pipelines	Created as real workspace objects, named per the mode.
Deployment state	Tracked by the bundle. This is how updates replace rather than duplicate.

ON THE EXAM

Set workspace.artifact_path to a Unity Catalog volume to keep wheels and JARs governed rather than loose in the workspace.

Where third-party libraries attach

- Classic compute: a **libraries** list on the task, pointing at a wheel path or a PyPI name.
- Serverless: an **environments** block with an environment key, a version, and a dependency list.
- A **notebook task cannot take an environment spec**. Use a notebook-scoped install instead.
- Tasks reference an environment by key, so several tasks can share one definition.

> ON THE EXAM
>
> The next module goes deeper on dependency resolution. What matters here is that the attachment point differs by compute type.

Where tests fit in this structure

- Tests import your package from the source folder and run locally. No cluster, no workspace.
- The **build command can run the tests first**, so a failing test blocks the artefact from being built.
- That makes the deploy step the enforcement point, rather than relying on discipline.
- Integration tests run afterwards, against a deployed development target.

> ON THE EXAM
>
> A structure that cannot be tested locally has failed the objective, whatever else it does well.

Bundles defined in Python

- Resources can be declared in **Python** instead of YAML, using the databricks-bundles package.
- A python section in the bundle file names a virtual environment and a loader function.
- Useful when jobs are generated from data, rather than written out one by one.
- It does not replace YAML. Both can coexist in one bundle, and YAML remains the default.

> ON THE EXAM
>
> Recognise it if it appears in an option. The exam's centre of gravity is still the YAML form.

Variables and substitutions

REFERENCE	RESOLVES TO
`${var.catalog}`	A variable you declared, overridden per target.
`${bundle.target}`	The name of the target being deployed.
`${workspace.current_user.short_name}`	The deploying user. This is what the dev prefix uses.
`${workspace.root_path}`	Where this bundle's files live for this target.
`${bundle.git.commit}`	The commit being deployed. Useful in artefact paths.

ON THE EXAM

Substitutions are why one configuration file serves every environment. Hard-coded catalog names in an answer are a red flag.

The prefix protects jobs, not tables

- The dev prefix separates **jobs**. It does nothing at all about where those jobs write.
- Two engineers can still write to the same table, and silently corrupt each other's results.
- The template offers a **personal schema per user** for exactly this reason.
- Catalog and schema become variables, overridden in the dev target to something user-specific.

> ON THE EXAM
>
> Compute isolation and data isolation are separate problems. A question about two developers interfering is usually about the second.

Git folders or bundles

GIT FOLDERS

Source control in the workspace

- Clone a repo into the workspace
- Edit and commit from the UI
- Good for interactive development
- Does not create jobs or pipelines

BUNDLES

Deployment of the whole project

- Defines jobs, pipelines and permissions
- Deploys from a laptop or a pipeline
- Targets carry environment differences
- Does not give you an editing surface

NOT EITHER-OR

They are complements, not alternatives. Git folders are where a human edits; bundles are how the project reaches an environment.

sync and permissions

- **sync** controls which local files are uploaded. Include and exclude patterns.
- Useful for keeping test fixtures, notebooks or large data out of the workspace.
- **permissions** grants access on the deployed resources, declared alongside them.
- Declaring permissions in the bundle means access is **reviewed in version control**, not clicked in a UI.

> ON THE EXAM
>
> Production mode expects permissions to be stated explicitly when you are not deploying as a service principal.

What to carry into the exam

- Logic in an importable package. Jobs in their own resource files. Environment differences in targets.
- **Development mode** prefixes, pauses schedules, and permits cluster overrides.
- **Production mode** validates pipelines, branch, run identity and permissions, and forbids cluster overrides.
- Most specific setting wins: resource over preset over mode.
- Redeployed code behaving unchanged means the wheel version did not move.

> ON THE EXAM
>
> If an answer requires a person to edit something in the workspace to promote a change, it is the wrong answer.

A team's bundle deploys cleanly to their dev target. The same command against the prod target fails immediately, before anything is created.

The prod target sets mode to production and names a Git branch of main. The engineer is working on a feature branch, and has also passed a flag pointing the deployment at an existing cluster to save start-up time.

What is the most likely cause of the failure?

A. The bundle is missing an artifacts mapping, so no wheel could be built.

B. Production mode validated the Git branch against the target, and refused the cluster override.

C. Production mode requires a service principal, so deployment always fails without one.

D. The prod target inherits the dev prefix, colliding with existing production job names.

◆ PAUSE AND DECIDE

What is the most likely cause of the failure?

A The bundle is missing an artifacts mapping, so no wheel could be built.

Missing artifacts does not fail. The CLI assumes the wheel is already built.

B Production mode validated the Git branch against the target, and refused the cluster override.

Both checks are tripped at once. Either alone would stop the deploy.

C Production mode requires a service principal, so deployment always fails without one.

Recommended, not mandatory. Without one you get extra validations, not a hard stop.

D The prod target inherits the dev prefix, colliding with existing production job names.

The prefix is a development mode behaviour. Production mode never applies it.

ON THE EXAM

Read the failure message before the configuration. Production deployments fail validations far more often than syntax.

A data engineer fixes a rounding bug in a shared transformation function, commits it, and redeploys the bundle to the development target. The deployment succeeds with no warnings.

The wheel task runs, completes successfully, and the output still shows the old rounding. Nothing in the logs indicates an error, and the fix is definitely present in the committed source.

Which change is most likely to resolve this?

A. Set dynamic_version to true so the wheel version changes on every build.

B. Switch the wheel task to a notebook task so code is read fresh on each run.

C. Add the transformation module to the job's libraries list as well as the wheel.

D. Run bundle destroy, then bundle deploy, to force a clean workspace state.

◆ PAUSE AND DECIDE

Which change is most likely to resolve this?

A Set dynamic_version to true so the wheel version changes on every build.

Compute caches an installed package by version. Patching it from the build timestamp forces a reinstall.

B Switch the wheel task to a notebook task so code is read fresh on each run.

It would work, which is what makes it dangerous. It trades away testability and reuse for a one-line setting.

C Add the transformation module to the job's libraries list as well as the wheel.

Listing it twice does not change the version, so the cached wheel still wins.

D Run bundle destroy, then bundle deploy, to force a clean workspace state.

Fixes it once by accident. The same bug returns on the very next change.

ON THE EXAM

Redeployed code that behaves as though it never changed is a version problem, not a deployment problem.

> Two engineers work on the same bundle in the same workspace. Both deploy to a target with mode set to development. The job definition in the repository specifies a nightly schedule.
>
> Neither wants their in-progress work running on a schedule. One engineer additionally needs their own copy to run on a schedule for a latency test.

Which statement describes the resulting behaviour?

A. Both deployments overwrite the same job; the nightly schedule runs for whoever deployed last.

B. Both get separately named jobs with schedules paused; the second sets pause_status to unpaused on their own job.

C. Both get separately named jobs, but schedules stay active because pausing needs an explicit preset.

D. Development mode blocks deployment of any job carrying a schedule.

◆ PAUSE AND DECIDE

Which statement describes the resulting behaviour?

A Both deployments overwrite the same job; the nightly schedule runs for whoever deployed last.

Overwriting is precisely what the dev prefix exists to prevent.

B Both get separately named jobs with schedules paused; the second sets pause_status to unpaused on their own job.

The prefix carries the user's short name, and an individual resource setting beats the mode default.

C Both get separately named jobs, but schedules stay active because pausing needs an explicit preset.

Inverted. Pausing is already what development mode does.

D Development mode blocks deployment of any job carrying a schedule.

No such restriction exists.

ON THE EXAM

Development mode gives you isolation and paused schedules by default, and both can be overridden per resource.

A team is restructuring a project currently made of six notebooks that duplicate the same cleansing logic.

They want the logic unit tested on a laptop, reused by both a scheduled job and an interactive notebook, promoted unchanged from development to production, and they want a failing test to prevent a release.

Which approach best meets all four requirements?

A. Keep the notebooks in Git folders and run them directly as tasks in each environment.

B. Move the logic into a source package, build it as a wheel, and run the tests in the build command.

C. Move the logic into one shared notebook and call it from the others.

D. Define every job in Python with the bundles package, keeping the cleansing logic in each job definition.

◆ PAUSE AND DECIDE

Which approach best meets all four requirements?

A Keep the notebooks in Git folders and run them directly as tasks in each environment.

Versions notebooks, but does not make them importable or locally testable.

B Move the logic into a source package, build it as a wheel, and run the tests in the build command.

Importable means testable locally and callable from both. Tests in the build block the artefact.

C Move the logic into one shared notebook and call it from the others.

Removes duplication and solves nothing else. It still needs a cluster to run at all.

D Define every job in Python with the bundles package, keeping the cleansing logic in each job definition.

Python-defined bundles describe resources. Business logic does not belong in a job definition.

ON THE EXAM

Four requirements, and only one option meets all four. Check each requirement against each option rather than picking on feel.

> A bundle writes to a table named explicitly as main.analytics.orders inside the job definition. It deploys to a development target with mode set to development, and two engineers use it daily.
>
> Both engineers report that their test runs are producing results they cannot explain, with rows appearing that neither of them created. Each has confirmed their own job runs successfully.

What is the underlying problem?

A The development prefix is not being applied, so both engineers share one job.

B The table name is hard-coded, so both isolated jobs write to the same table.

C The deployment lock is disabled in development mode, corrupting concurrent deploys.

D Concurrent runs are enabled, so each engineer's job runs twice.

◆ PAUSE AND DECIDE

What is the underlying problem?

A. The development prefix is not being applied, so both engineers share one job.
The prefix is automatic in development mode, and both report their own job running.

B. The table name is hard-coded, so both isolated jobs write to the same table.
Development mode isolates jobs, not data. Only variables for catalog and schema fix this.

C. The deployment lock is disabled in development mode, corrupting concurrent deploys.
The lock governs deploys, not table writes.

D. Concurrent runs are enabled, so each engineer's job runs twice.
Concurrent runs would duplicate work, not introduce another person's rows.

ON THE EXAM

Isolated compute with shared data is a classic trap. Look at where the job writes, not just what it is called.

> A team wants analysts to explore and edit shared notebooks directly in the workspace with full version history. Separately, they want their nightly production jobs created and updated automatically from the same repository, with no manual steps.
>
> They are debating whether to adopt Git folders or bundles.

Which approach meets both needs?

A Git folders only, running the notebooks in the folder as scheduled tasks.

B Bundles only, with analysts editing the deployed files in the workspace.

C Git folders for interactive editing, and bundles to deploy the jobs.

D Neither. Use the Jobs UI and export definitions to the repository afterwards.

◆ PAUSE AND DECIDE

Which approach meets both needs?

A Git folders only, running the notebooks in the folder as scheduled tasks.

Covers the editing need and leaves job creation manual.

B Bundles only, with analysts editing the deployed files in the workspace.

Deployed files are outputs of a deploy. Editing them there puts changes outside version control.

C Git folders for interactive editing, and bundles to deploy the jobs.

They solve different problems and are designed to be used together.

D Neither. Use the Jobs UI and export definitions to the repository afterwards.

Exporting after the fact makes the workspace the source of truth, which is the thing to avoid.

ON THE EXAM

When two requirements point at two tools, the answer is usually both. Watch for options that force an unnecessary choice.

Libraries and dependencies that actually resolve

Section 1. Scopes, sources, access modes, and why the install worked yesterday.

SECTION 1 of 10 WEIGHT 22% TOPIC Dependencies

admin@sthithapragna.com

Every library question is really a scope question. **Who can see it, and how long does it last?**

Where a dependency can be attached

SCOPE	WHO SEES IT, AND FOR HOW LONG
Notebook-scoped	One notebook session only. Lost on detach or restart.
Compute-scoped	Every notebook and job on that compute. Reinstalled on restart.
Serverless environment	Tasks that reference the environment key. Declared, not installed.
Init script	Runs on every node at start-up. For things pip cannot do.

ON THE EXAM

Notebook-scoped libraries do not persist across sessions. They must be reinstalled at the start of every session, and whenever the notebook is detached.

The pip magic command

- **%pip install** is the recommended way to install for a single notebook session.
- Only that notebook, and jobs associated with it, can see the library. Neighbours are unaffected.
- Works in notebooks scheduled as jobs. In Scala, SQL or R notebooks, reach it via **%python**.
- On clusters of ten nodes or more it drives extra traffic to the driver. Size the driver accordingly.

> ON THE EXAM
>
> On modern runtimes %pip, %sh pip and !pip all install notebook-scoped. On very old runtimes only %pip behaved consistently.

The install succeeded and the import still fails

- **%pip does not restart the Python process.** The old version may stay loaded in the session.
- Call **dbutils.library.restartPython()** after installing or upgrading, then re-import.
- Upgrading or removing core packages such as IPython can break notebook features outright.
- If the session gets into a strange state, restart the cluster or start a new session to reset it.

> ON THE EXAM
>
> An install that reports success followed by an import of the old version is almost always a missing Python restart.

What you are not allowed to uninstall

- You **cannot** uninstall a library that ships with the runtime, or one installed as a compute library.
- You can revert to that version. If you installed a different one, uninstalling drops you back to the underlying one.
- The **-y** flag is required on an uninstall. Without it the command does not proceed.
- Egg files are not supported from volumes or workspace files. Use wheels, or install from the index.

> ON THE EXAM
>
> Uninstall reverts to the layer beneath. It never removes the runtime's own copy.

Where a package can be installed from

SOURCE	NOTES
Public package index	The default. Pin the version.
Unity Catalog volume	Governed storage for private wheels. The preferred private option.
Workspace files	Supported for wheels held beside your code.
Version control	Install directly from a repository URL.
Private repository	Point at an index URL, with credentials from a secret.
DBFS	Discouraged. Any workspace user can modify what is stored there.

ON THE EXAM

Volumes and workspace files are the recommended homes for private packages. DBFS is called out in the docs as a risk because it is writable by everyone.

Requirements files

- Install a whole set at once with **%pip install -r** and a path to the requirements file.
- Capture the current environment with **%pip freeze** redirected to a file, to reproduce it later.
- The target directory must already exist. Freezing into a new folder fails.
- Keep the file in the workspace or a volume so it is versioned alongside the code that needs it.

> ON THE EXAM
>
> A requirements file turns 'it worked on my cluster' into something another person can reproduce exactly.

Why unpinned installs break on serverless

- Databricks explicitly recommends moving away from **unpinned installs**.
- They are **not compatible with the serverless compute architecture**.
- Pin to an exact version. Every install then resolves to the same thing on every run.
- This is also what makes a failure reproducible. An unpinned job fails differently each week.

> ON THE EXAM
>
> If an option installs a package without a version, treat it as suspect. That is now a documented incompatibility, not a style preference.

Credentials without leaking them

- Fetch the token with **dbutils.secrets.get**, then reference it as a variable in the magic command.
- Point at the private repository with an **index URL** option carrying the user and token.
- Add an **extra index URL** for the public index so ordinary dependencies still resolve.
- Never paste a token into a notebook cell. It is then in the source, the history and the repository.

> ON THE EXAM
>
> The same pattern works for private version control repositories. Secret, then variable, never a literal.

Compute library or notebook-scoped

COMPUTE-SCOPED

Everything attached sees it

- Survives the session, reinstalled on restart
- One definition for every job on the cluster
- Changing it affects everyone else too
- Right for a shared production dependency

NOTEBOOK-SCOPED

One session, isolated

- Gone on detach or restart
- Neighbours on the cluster are unaffected
- Safe place to try a different version
- Right for exploration and one-off work

THE TIEBREAKER

A production job that must not depend on someone's session should never rely on a notebook-scoped install.

Environments instead of installs

- On serverless you **declare** dependencies in an environment rather than installing them.
- A job task names an **environment key**; the environment carries a version and a dependency list.
- In a notebook, the **Environment side panel** is the reproducible place to declare dependencies.
- A notebook task still cannot take an environment spec. Use a notebook-scoped install there.

> ON THE EXAM
>
> Declared beats installed. The environment is part of the definition, so it is deployed with the job rather than configured on a cluster.

Base environments and faster installs

- A workspace admin can define a **base environment**: a version plus a standard set of dependencies.
- It can be exported as a file and stored in a workspace folder or a volume, so teams share one baseline.
- On recent serverless environment versions, **%uv pip** is a faster alternative sharing the same virtual environment.
- It does not check conflicts against preinstalled packages, and cannot fully remove them.

> ON THE EXAM
>
> Use the faster tool for interactive iteration. Use the environment panel when you need the result to be reproducible.

What the compute will let you install

ACCESS MODE	WHAT IT MEANS FOR LIBRARIES
Standard	Shared between users. JARs, Maven coordinates and init scripts need an allowlist.
Dedicated	Assigned to one user or group. The assigned principal's identity is used.
Serverless	No cluster to install onto. Dependencies are declared in an environment.

ON THE EXAM

Standard was formerly called shared, and dedicated was formerly single user. Both names appear in older material and in the question bank.

What it does, and what it does not do

- Controls which init scripts, JAR files and Maven coordinates may run on **standard access mode** compute.
- It is **empty by default and cannot be disabled**. Modifying it needs the MANAGE ALLOWLIST privilege.
- Allowlisting a path grants **permission to use it**, not permission to read it. READ VOLUME is still required.
- Prefix matching applies. A trailing slash scopes the entry to everything inside that directory.

> ON THE EXAM
>
> Two separate gates: the allowlist says the artefact may be used, and Unity Catalog permissions say whether you may read it. Both must pass.

The Java-side rules

- On standard access mode with Unity Catalog, JAR classes **must be in a named package**. Default-package classes fail on import.
- Libraries used as **JDBC drivers or custom Spark data sources** require ANY FILE permission.
- Maven coordinates go on the allowlist the same way paths do, and can be allowed by group.
- Some libraries write to one shared temporary directory for all users, which undermines isolation.

> ON THE EXAM
>
> An import that fails only on shared compute, and works on a dedicated cluster, points at the named-package rule or the allowlist.

When pip is not enough

- Init scripts run on **every node** as compute starts. For system packages, agents, certificates, drivers.
- **Cluster-scoped is recommended** over global. Global scripts run on every cluster in the workspace.
- Store them in workspace files, a Unity Catalog volume, or cloud object storage.
- On standard access mode they must be allowlisted, like JARs.

> ON THE EXAM
>
> Reach for an init script only when the need is genuinely below the Python layer. A package install is simpler and easier to reason about.

Diagnosing a dependency failure in order

01

Which scope

Was it installed for the notebook, the compute, or declared in an environment?

02

Which session

Notebook-scoped installs are gone after a detach. Was the session restarted?

03

Was Python restarted

An install without a restart leaves the previously imported version in place.

04

Which access mode

Standard mode blocks JARs, Maven and init scripts that are not allowlisted.

05

Which version

Unpinned installs resolve differently over time. Pin, then reproduce.

ON THE EXAM

Work down this list rather than guessing. Almost every library question in this exam is answered by one of these five.

What to carry into the exam

- Scope decides visibility and lifetime. Notebook-scoped installs **do not survive a detach**.
- An install that succeeded but imports the old version means **Python was not restarted**.
- You cannot uninstall a runtime or compute library. You revert to it.
- Standard access mode needs an **allowlist** for JARs, Maven and init scripts. It is not a permission.
- Unpinned installs are incompatible with serverless. Pin the version.

> ON THE EXAM
>
> Production dependencies belong in something deployed with the job. If an answer depends on someone remembering to run a cell, it is wrong.

A scheduled job runs a notebook that begins with a cell installing a specific parsing library. It has run nightly without incident for months.

An engineer adds the same library as a compute library on the shared cluster to make it available to a second team, then removes the install cell from the notebook to tidy it up. The nightly job now fails with an import error.

What is the most likely explanation?

A The job runs on its own job compute, which never received the compute library.

B Compute libraries are only visible to interactive notebooks, not to scheduled jobs.

C Removing the install cell also removed the library from the cluster.

D The compute library needs a Python restart that only a notebook can trigger.

◆ PAUSE AND DECIDE

What is the most likely explanation?

A The job runs on its own job compute, which never received the compute library.

The library was added to the shared interactive cluster. The job's compute is a different machine entirely.

B Compute libraries are only visible to interactive notebooks, not to scheduled jobs.

They are visible to everything attached, jobs included.

C Removing the install cell also removed the library from the cluster.

Editing a notebook does not uninstall anything.

D The compute library needs a Python restart that only a notebook can trigger.

Compute libraries are present from start-up. No restart is involved.

ON THE EXAM

Where a library was installed and where the job actually runs are two different questions. Check both.

A developer runs an install command to upgrade a serialisation library from version 3.1 to 3.4 in their notebook. The output ends with a success message naming 3.4.

In the very next cell they import the library and print its version. It reports 3.1. Re-running the install reports that 3.4 is already satisfied.

What should they do?

A Install the library as a compute library instead, since notebook installs cannot upgrade.

B Call dbutils.library.restartPython(), then import the library again.

C Uninstall version 3.1 first, then install 3.4.

D Detach and reattach the notebook, then re-run the install cell.

◆ PAUSE AND DECIDE

What should they do?

A Install the library as a compute library instead, since notebook installs cannot upgrade.

They can. And this would change the version for everyone else on the cluster.

B Call dbutils.library.restartPython(), then import the library again.

The install did land. The already-imported module stayed loaded because the Python process was never restarted.

C Uninstall version 3.1 first, then install 3.4.

The install already succeeded, as the second run confirms. Removal is not the problem.

D Detach and reattach the notebook, then re-run the install cell.

Works, but throws away the whole session state to solve a one-line problem.

ON THE EXAM

Success on install plus the old version on import is the signature of a missing Python restart.

A team stores a proprietary wheel in a Unity Catalog volume and installs it successfully on a dedicated cluster. They then try the same wheel, and a JDBC driver JAR from the same volume, on a standard access mode cluster shared across two teams.

An administrator has already granted the group READ VOLUME on the volume. The JAR install still fails.

What is required to make the JAR install succeed?

A Nothing more. READ VOLUME is sufficient; the failure must be a transient error.

B Move the JAR to DBFS, which standard access mode clusters can read without extra configuration.

C Add the path to the Unity Catalog allowlist, and grant ANY FILE for use as a JDBC driver.

D Switch the cluster to dedicated access mode, since JARs are unsupported on standard mode.

◆ PAUSE AND DECIDE

What is required to make the JAR install succeed?

A Nothing more. READ VOLUME is sufficient; the failure must be transient.

Read permission and allowlisting are two separate gates. Both must pass.

B Move the JAR to DBFS, which standard access mode clusters can read without extra configuration.

DBFS is discouraged precisely because any workspace user can modify it.

C Add the path to the Unity Catalog allowlist, and grant ANY FILE for use as a JDBC driver.

Standard mode requires allowlisting for JARs, and driver use carries the additional ANY FILE requirement.

D Switch to dedicated access mode, since JARs are unsupported on standard mode.

They are supported. Abandoning shared compute to avoid a governance step is the wrong trade.

ON THE EXAM

Two gates, always: the allowlist decides whether the artefact may be used, and permissions decide whether you may read it.

A nightly job on serverless compute has run reliably for a year. Its notebook installs three libraries without specifying versions. Last night it failed with an error deep inside one of them, and nothing in the team's code had changed.

The team wants the job to stop failing, and wants any future failure to be reproducible on demand.

Which change best addresses both requirements?

A Add a retry policy to the task so transient dependency failures resolve themselves.

B Pin every dependency to an exact version, declared in the job's environment.

C Move the installs into an init script so they run before the notebook starts.

D Install the libraries as compute libraries on a dedicated cluster instead.

◆ PAUSE AND DECIDE

Which change best addresses both requirements?

A. Add a retry policy so transient dependency failures resolve themselves.

A new version will fail on every retry. It also hides the cause.

B. Pin every dependency to an exact version, declared in the job's environment.

Pinning stops the silent upgrade, and declaring it in the environment makes it part of the deployed job.

C. Move the installs into an init script so they run before the notebook starts.

Serverless has no cluster for an init script, and unpinned installs would still drift.

D. Install as compute libraries on a dedicated cluster instead.

Abandons serverless to work around a versioning problem, and still nothing is pinned.

ON THE EXAM

Nothing changed in the code, so something changed outside it. Unpinned dependencies are the usual answer to that sentence.

> Two analysts share a standard access mode cluster. One needs version 2.0 of a statistics package for an experiment. The cluster has version 1.8 installed as a compute library, which four production jobs depend on.
>
> The analyst must not disturb those jobs, and needs the newer version only for the current piece of work.

What should the analyst do?

A. Install 2.0 notebook-scoped in their own notebook, accepting it is lost when the session ends.

B. Upgrade the compute library to 2.0 and ask the job owners to test against it.

C. Uninstall the compute library, then install 2.0 notebook-scoped.

D. Add 2.0 to the allowlist so both versions can be resolved at once.

◆ PAUSE AND DECIDE

What should the analyst do?

A Install 2.0 notebook-scoped in their own notebook, accepting it is lost when the session ends.

Isolation is exactly what notebook scope provides. Nothing else on the cluster is affected, and impermanence is fine for an experiment.

B Upgrade the compute library to 2.0 and ask the job owners to test against it.

Changes the version for four production jobs to serve one experiment.

C Uninstall the compute library, then install 2.0 notebook-scoped.

Breaks the four jobs, and a compute library cannot be uninstalled from a notebook anyway.

D Add 2.0 to the allowlist so both versions can be resolved at once.

The allowlist governs JARs, Maven and init scripts. It does not resolve Python versions.

ON THE EXAM

Impermanence is a feature here, not a drawback. An experiment that vanishes with the session is exactly what was wanted.

> A platform team must make an internal wheel available to every production job across three workspaces. The wheel is rebuilt weekly, must not be readable by contractors, and installs must not require anyone to edit a notebook.
>
> They currently email the file around and install it by hand.

Which approach best meets all three requirements?

A. Upload the wheel to DBFS in each workspace and reference it from each job definition.

B. Store it in a Unity Catalog volume with permissions, and reference that path from the job definitions.

C. Have each notebook install it from an internal repository at the top of every run.

D. Add a global init script in each workspace that installs the wheel on every cluster.

◆ PAUSE AND DECIDE

Which approach best meets all three requirements?

A Upload the wheel to DBFS in each workspace and reference it from each job definition.

Fails the access requirement outright. Any workspace user can read and modify DBFS.

B Store it in a Unity Catalog volume with permissions, and reference that path from the job definitions.

Governed storage gives real permissions, and referencing it from the job definition removes the manual step.

C Have each notebook install it from an internal repository at the top of every run.

Puts the dependency back inside a notebook cell, which was explicitly ruled out.

D Add a global init script in each workspace that installs the wheel on every cluster.

A global script runs on every cluster in the workspace. Enormous blast radius for one team's package.

ON THE EXAM

Three requirements: governed access, weekly refresh, no manual step. Test each option against all three rather than the first one.

DEVELOPING CODE · DOMAIN 1 - DEVELOPING CODE FOR DATA PROCESSING

User-defined functions and what they cost

Section 1. Python, pandas, and the question of whether you needed one at all.

SECTION 1 of 10 WEIGHT 22% TOPIC UDFs

admin@sthithapragna.com

A user-defined function is a hole in the optimiser. Spark cannot see inside it, so it cannot improve it.

When Databricks says to use one

- **Good fits:** ad hoc queries, manual data cleaning, exploratory analysis, small to medium datasets.
- Common cases named in the docs: **encryption, decryption, hashing, JSON parsing, validation**.
- **Use built-in Spark functions instead** for large datasets, and for anything that runs regularly or continuously.
- That explicitly includes **ETL jobs and streaming**, which is most of what this exam is about.

> ON THE EXAM
>
> Databricks' own guidance is that UDFs suit logic which is difficult to express with built-in functions. Difficult, not merely unfamiliar.

From cheapest to most expensive

01

Built-in

Fully optimised, eligible for vectorised execution. Always the first choice.

›

02

SQL UDF

Expressed in SQL, so the optimiser can still see through it.

›

03

Scala UDF

Runs in the JVM, so no data has to leave it.

›

04

pandas UDF

Leaves the JVM, but in Arrow batches. Up to 100x faster than a Python UDF.

›

05

Python UDF

Row at a time, serialised each way. The most expensive option.

ON THE EXAM

Built-in functions and SQL UDFs are the most efficient. Everything below them pays to move data out of the JVM and back.

Why the plain Python one is so slow

- Every row is **serialised out of the JVM** to a Python interpreter, then the result is serialised back.
- Scalar Python functions rely on **cloudpickle**, which becomes a bottleneck on large inputs and outputs.
- The optimiser cannot reason about it, so filters cannot be pushed through it.
- The vectorised engine cannot accelerate it either, so that part of the plan falls back.

> ON THE EXAM
>
> Four separate costs, all from the same cause: the work is happening outside the engine that was built to optimise it.

The three shapes you should recognise

SHAPE	WHAT IT DOES
Series to Series	Vectorises a scalar operation. Input and output are the same length.
Iterator of Series	Same, but streams batches. Good when set-up per batch is expensive.
Series to Scalar	Collapses many values to one. Used for aggregation with a group by.

ON THE EXAM

The type hint on the function is not decoration. It is how Spark decides which of these three shapes you meant.

The contract of a Series to Series function

- Decorate with **pandas_udf** and declare the return type, for example **double**.
- Annotate the arguments as pandas Series and the return as a pandas Series.
- Returning a **different length** than you received breaks the contract. This shape is row-preserving.
- Inside, you write ordinary vectorised pandas. No loops over rows.

> ON THE EXAM
>
> If the body of a pandas UDF contains a loop over rows, the vectorisation benefit has been thrown away.

Which pandas API fits the shape of your problem

IF YOU NEED	USE
One output row for every input row	A pandas UDF
To process each group as a unit	applyInPandas, on grouped data
A different number of rows out than in	mapInPandas, on the DataFrame
To avoid the pandas conversion entirely	mapInArrow or applyInArrow

ON THE EXAM

Row count is the fastest way to choose. Same number out means a pandas UDF; a different number means one of the map functions.

pandas UDF against grouped apply

PANDAS UDF

Column at a time

- Operates on Series
- One result per input row
- Batches sized by the engine
- Cannot see other rows in a group

APPLYINPANDAS

Group at a time

- Operates on a whole DataFrame per group
- Row count per group may change
- The split, apply, combine pattern
- Loads the entire group into memory

THE TRADE-OFF

The grouped form is powerful because the function sees a whole group. That is also its risk: one huge group can exhaust the executor.

When the map form is the only option

- **mapInPandas** applies a function to an iterator of DataFrames, one batch of a partition at a time.
- Output row count may differ from input. This is the form for exploding, or for heavy filtering.
- It works on the whole DataFrame, so no grouping is involved and no group is held in memory.
- Because it yields batches, it suits work where you cannot hold everything at once.

> ON THE EXAM
>
> If a question describes one input record producing an unknown number of output records, this is the shape being asked for.

The Arrow function APIs

- **mapInArrow** and **applyInArrow** mirror the pandas versions but work on Arrow objects directly.
- They skip the conversion between Arrow and pandas, removing that overhead entirely.
- Arrow-optimised Python UDFs also exist, bypassing the older serialisation path.
- Recognise them. They are not a substitute for using a built-in function in the first place.

> ON THE EXAM
>
> Every improvement in this space narrows the gap with built-in functions. None of them closes it.

Unity Catalog governed against session-scoped

UNITY CATALOG GOVERNED

Shared and permanent

- Persisted in the catalog, reusable across sessions
- Governed by Unity Catalog permissions
- Discoverable in Catalog Explorer
- SQL, Python, Scala and Java

SESSION-SCOPED

Local and temporary

- Exists only in the current Spark session
- Not governed and not shared
- Invisible to anyone else
- SQL, Python and Scala

THE TIEBREAKER

A hashing function used by three teams for compliance should be governed, not redefined in each notebook.

The Unity Catalog types and where they run

TYPE	NOTABLE SUPPORT
Unity Catalog Python UDF	Serverless, standard access mode from DBR 13.3 LTS, SQL warehouses, pipelines.
Batch Unity Catalog Python UDF	Standard access mode from DBR 16.3. Batches values instead of row-by-row.
Unity Catalog Python UDTF	Standard access mode from DBR 17.1. Returns many rows per input row.
Unity Catalog Scala or Java UDF	Also dedicated access mode and classic SQL warehouses.

ON THE EXAM

Nearly all of these require standard access mode on classic compute, with a runtime floor. A dedicated cluster on an old runtime is the usual reason one is unavailable.

Batch Unity Catalog Python UDFs

- Registered in the catalog like any governed function, but it **operates on batches of values**.
- That reduces the row-by-row overhead, which is the whole reason plain Python functions are slow.
- Declared with a pandas parameter style and a named handler, rather than a plain expression body.
- So it gives you governance and vectorisation together, which previously meant choosing one.

> ON THE EXAM
>
> Governed plus vectorised. If a question wants a shared function that also has to run at scale, this is the shape it is describing.

Availability, not syntax

- Non-scalar Python forms need a **higher runtime floor** on classic compute than scalar ones do.
- Scala aggregate functions require **dedicated** access mode, not standard.
- Serverless supports the Python forms broadly, which is one reason it is often the simpler answer.
- A precompiled Java function can be registered from a JAR, subject to the allowlist rules from module 3.

ON THE EXAM

When a UDF 'does not work' in a question, check the access mode and runtime before questioning the code.

How to test a UDF properly

- Test the **plain Python function** directly, before it is wrapped. It is just a function.
- That runs on a laptop in milliseconds, with no Spark session and no cluster.
- Then test the wrapped version once, on a small DataFrame, to confirm types and null handling.
- Testing only through Spark makes a fast unit test into a slow integration test.

> ON THE EXAM
>
> This is why the module 2 structure matters. A UDF defined inside a notebook cell cannot be imported, so it cannot be tested this way.

What to carry into the exam

- A UDF is **opaque to the optimiser**. That is the reason behind every performance answer here.
- Built-in first, then SQL, then Scala, then pandas, then plain Python.
- Row count picks the API: **same** means pandas UDF, **grouped** means applyInPandas, **different** means mapInPandas.
- Grouped apply loads a **whole group into memory**. Watch for skewed groups.
- Governed for sharing and permissions. Session-scoped for quick local work.

> ON THE EXAM
>
> For a regular ETL or streaming job at scale, the best answer is very often not to use a UDF at all.

A nightly ETL job processes 400 million rows. One step uses a Python UDF that takes a postcode string, strips whitespace, uppercases it, and returns the first half.

The step accounts for most of the job's runtime. The team has been asked to reduce cost without changing the output.

What is the best change?

A Convert it to a pandas UDF so the work is vectorised in Arrow batches.

B Replace it with built-in Spark string functions.

C Register it as a Unity Catalog Python UDF so it can be governed and reused.

D Increase the number of shuffle partitions so the UDF runs on more executors.

◆ PAUSE AND DECIDE

What is the best change?

A Convert it to a pandas UDF so the work is vectorised in Arrow batches.

A real improvement, and the trap. It moves one rung up the ladder when the top rung was available all along.

B **Replace it with built-in Spark string functions.**

Trimming, uppercasing and substring are all built in. Full optimisation, vectorised execution, no data leaves the JVM.

C Register it as a Unity Catalog Python UDF so it can be governed and reused.

Governance is not the problem described, and this does not make it faster.

D Increase shuffle partitions so the UDF runs on more executors.

Buys parallelism with more compute. The per-row serialisation cost is unchanged.

ON THE EXAM

Before optimising a UDF, check whether a built-in function does the job. Trim, upper and substring all exist.

A data scientist needs to fit a small regression per store and return the fitted coefficients, one row per store. There are 1,200 stores. The logic exists in a Python library with no Spark equivalent.

One store is a national distribution centre and holds roughly 60% of all rows. The job fails on that partition with an out-of-memory error.

What best explains the failure?

A applyInPandas loads each entire group into executor memory, and one group is enormous.

B A pandas UDF cannot return fewer rows than it receives, so the job overflowed.

C Arrow serialisation has a fixed batch limit that this group exceeded.

D The Python interpreter cannot hold more than one partition at a time.

◆ PAUSE AND DECIDE

What best explains the failure?

A applyInPandas loads each entire group into executor memory, and one group is enormous.

The defining property of grouped apply, and the reason skewed groups are dangerous with it.

B A pandas UDF cannot return fewer rows than it receives, so the job overflowed.

True of the Series to Series shape, but that is not what a per-group fit uses, and it would not cause an OOM.

C Arrow serialisation has a fixed batch limit that this group exceeded.

Invented. Batch sizing is not the constraint being described.

D The Python interpreter cannot hold more than one partition at a time.

Not a real limitation, and it would not single out one group.

ON THE EXAM

Grouped apply plus a skewed grouping key is a recognisable failure pattern. The dataset size is irrelevant; the largest group is what matters.

> A pipeline reads records where each row contains a compressed archive path. For each row, a Python library must open the archive and emit one output row per file inside it. Some archives contain a single file, others contain thousands.
>
> The team has tried a pandas UDF and cannot make it work.

Which approach fits this problem?

A applyInPandas, grouping by the archive path so each archive is one group.

B A Series to Scalar pandas UDF, aggregating the file list per row.

C mapInPandas, yielding a variable number of rows per input batch.

D A scalar Python UDF returning an array, since arrays can hold many values.

◆ PAUSE AND DECIDE

Which approach fits this problem?

A applyInPandas, grouping by the archive path so each archive is one group.

Contrives a group per row, and still constrains you to what fits in memory per group.

B A Series to Scalar pandas UDF, aggregating the file list per row.

That shape collapses many values to one. This problem needs the opposite.

C mapInPandas, yielding a variable number of rows per input batch.

The row count differs from input to output, which is precisely what this form exists for.

D A scalar Python UDF returning an array, since arrays can hold many values.

Works, then needs an explode, and pays row-at-a-time cost to build a potentially huge array per row.

ON THE EXAM

One input row producing an unknown number of output rows is the signature of the map form.

> Three teams each maintain their own copy of a function that tokenises customer identifiers before analysis. The definitions have drifted apart, and an auditor has asked who is able to run the tokenisation.
>
> The function must also run inside a nightly job over several billion rows, and inside SQL warehouse queries.

Which option best satisfies all of this?

A. A session-scoped pandas UDF, defined in a shared notebook each team imports.

B. A batch Unity Catalog Python UDF, registered once in the catalog.

C. A scalar Unity Catalog Python UDF, registered once in the catalog.

D. A Scala UDAF on a dedicated cluster, called by all three teams.

◆ PAUSE AND DECIDE

Which option best satisfies all of this?

A A session-scoped pandas UDF, defined in a shared notebook each team imports.

Fast, but ungoverned. Nobody can answer the auditor's question, and drift can recur.

B A batch Unity Catalog Python UDF, registered once in the catalog.

Governed and permissioned, works on SQL warehouses, and batches values so it scales to billions of rows.

C A scalar Unity Catalog Python UDF, registered once in the catalog.

Governed and shared, but processes row by row. It solves the audit and not the scale.

D A Scala UDAF on a dedicated cluster, called by all three teams.

An aggregate function is the wrong shape, and dedicated access mode does not serve three teams well.

ON THE EXAM

Three requirements: one definition, an auditable permission, and scale. The batch governed form is the only option carrying all three.

> A pandas UDF applies a machine learning model to each row. Loading the model from storage takes about four seconds. The current implementation loads the model inside the function body, and the job is far slower than expected.
>
> The output must remain one prediction per input row.

Which change addresses the cause?

A Switch to an Iterator of Series pandas UDF and load the model once per batch.

B Switch to applyInPandas so the model loads once per group.

C Switch to a scalar Python UDF, which caches imports between rows.

D Broadcast the model file to every executor with a cluster library.

◆ PAUSE AND DECIDE

Which change addresses the cause?

A Switch to an Iterator of Series pandas UDF and load the model once per batch.

This shape exists for exactly this: expensive set-up done once per batch, then reused across it.

B Switch to applyInPandas so the model loads once per group.

Invents a grouping the problem does not have, and loads whole groups into memory.

C Switch to a scalar Python UDF, which caches imports between rows.

Moves down the cost ladder, and does not solve the repeated loading.

D Broadcast the model file to every executor with a cluster library.

Makes the file local, but the code still loads it on every call.

ON THE EXAM

Expensive set-up that should happen once per batch rather than once per call is the reason the Iterator shape exists.

A team writes a Python function that validates an identifier's checksum. They want it unit tested on a laptop in continuous integration, reused by two jobs and one notebook, and running acceptably over 50 million rows nightly.

There is no built-in Spark function for this checksum algorithm.

Which combination best meets the requirements?

A. Define it in a notebook cell and register it with the udf decorator where needed.

B. Put the plain function in the source package, test it directly, and wrap it as a pandas UDF.

C. Write it as a SQL UDF so the optimiser can see through it.

D. Use mapInPandas so the validation can filter out invalid rows as it goes.

◆ PAUSE AND DECIDE

Which combination best meets the requirements?

A Define it in a notebook cell and register it with the udf decorator where needed.

Not importable, so it cannot be unit tested on a laptop, and it invites the drift of three copies.

B Put the plain function in the source package, test it directly, and wrap it as a pandas UDF.

The body is testable as ordinary Python, the package is importable by both jobs and the notebook, and the pandas wrapper handles the volume.

C Write it as a SQL UDF so the optimiser can see through it.

Cheapest if the algorithm can be expressed in SQL. A checksum loop generally cannot.

D Use mapInPandas so the validation can filter out invalid rows as it goes.

Changes the output by dropping rows. Validation was asked for, not filtering.

ON THE EXAM

This is module 2 and module 4 together: an importable package makes the function testable, and the wrapper makes it fast.

DEVELOPING CODE · DOMAIN 1 - DEVELOPING CODE FOR DATA PROCESSING

Declarative pipelines

Section 1. Streaming tables, materialized views, and the refresh that destroys data.

SECTION 1 of 10 WEIGHT 22% TOPIC Pipelines

admin@sthithapragna.com

You declare **what the table should contain**. The engine works out how to keep it that way.

Three dataset types

TYPE	WHAT IT IS
Streaming table	A Unity Catalog managed table that is a streaming target. Each row processed once.
Materialized view	A managed table holding pre-computed results, kept current by incremental refresh.
Temporary view	Pipeline-scoped. Organises logic without materialising anything to storage.

ON THE EXAM

Both streaming tables and materialized views are Unity Catalog managed tables. The difference is how they are kept up to date, not what they are.

Streaming table against materialized view

STREAMING TABLE

Process each row once

- Right for ingestion and low latency
- Append-only and high-volume workloads
- Event-driven, from storage or a message bus
- Cost scales with new data, not total data

MATERIALIZED VIEW

Keep a result correct

- Right for complex transformations and analytics
- Handles joins, aggregations, late updates
- Results pre-computed, so queries are fast
- You cannot modify the data directly

THE TIEBREAKER

Ask whether the source is an unbounded stream of new records, or a query whose answer must stay correct as inputs change.

What 'processed once' buys and costs

- **Buys:** the cost of an update depends on how much arrived, not how much you hold.
- Ideal for bronze ingestion. Databricks recommends streaming tables for **most ingestion cases**.
- **Costs:** a row already processed is not revisited. Late corrections to old rows are not naturally handled.
- Changing stateful logic invalidates existing state, which forces a full refresh.

> ON THE EXAM
>
> For files landing in cloud object storage, Auto Loader gives incremental and idempotent loading into a streaming table.

Correct by construction

- The **query definition controls the output**. You cannot write into a materialized view directly.
- The engine performs **incremental refresh** where it can, processing only what changed.
- Handles updates and deletes in the source naturally, because it recomputes the answer.
- When incremental refresh is not possible, it recomputes, and that cost scales with total data.

> ON THE EXAM
>
> Use them for silver logic with enrichment joins, and for gold aggregations, metrics and summaries.

Which type for which layer

LAYER	USUAL CHOICE
Bronze	Streaming table. Ingest once, append, keep the raw record.
Silver	Streaming table for cleansing. Materialized view for enrichment joins.
Gold	Materialized view. Pre-computed aggregations, metrics and summaries.

ON THE EXAM

Silver is the layer where the answer genuinely depends on the work. Joins against dimensions favour a materialized view; row-level cleansing does not.

The difference is one word

- A **batch read** in the function body produces a materialized view.
- A **streaming read** in the same body produces a streaming table.
- In SQL: **CREATE MATERIALIZED VIEW** against **CREATE STREAMING TABLE**.
- Not every source supports a streaming read. Some sources should always be read as a stream.

> ON THE EXAM
>
> Pipeline dataset functions must return a DataFrame, and must not have side effects. Side effects inside them cause unpredictable behaviour.

The Python API was renamed after the exam guide

YOU WILL SEE	CURRENT FORM
`import dlt`	from pyspark import pipelines as dp
`@dlt.table with a streaming read`	@dp.table
`@dlt.table with a batch read`	@dp.materialized_view
`@dlt.view`	@dp.temporary_view
`dlt.apply_changes(...)`	dp.create_auto_cdc_flow(...)
`@dlt.expect(...)`	dp.expect(...)

ON THE EXAM

Documented on 10 July 2026, one week after the exam guide was published. The older module still works, and SQL did not change at all.

SQL did not change, and neither did the concepts

- **CREATE STREAMING TABLE**, **CREATE MATERIALIZED VIEW** and **CREATE FLOW** are identical in both.
- If you are unsure which Python form an exam question expects, the concepts are what is really being tested.
- Some DLT naming survives on purpose: classic pipeline billing codes and event log schemas still say DLT.
- Apache Spark 4.1 includes declarative pipelines, but **AUTO CDC, expectations and temporary views are Databricks-only**.

> ON THE EXAM
>
> The exam tests which dataset type suits a workload far more than it tests which decorator spells it.

How data actually gets into a dataset

FLOW	WHAT IT DOES
Append flow	The default for both dataset types. Adds new records to the target.
AUTO CDC flow	Streaming flow handling out-of-order changes. SCD Type 1 and Type 2.
Materialized view flow	A batch flow that processes only new data and changes where it can.
REPLACE WHERE flow	Incremental batch processing of joins and aggregations.

ON THE EXAM

A flow is usually created implicitly by your query. You define one explicitly when several sources must write into one target.

Append flows avoid a full refresh

- An append flow adds a **new streaming source** to an existing streaming table without a full refresh.
- It is also how you **backfill** historical data into a streaming table that is already live.
- And how you combine several sources into one target without a union in a single query.
- Without it, adding a source to a streaming table generally means reprocessing everything.

> ON THE EXAM
>
> If a question needs a new source or a backfill added to a live streaming table without disruption, this is the mechanism.

Triggered against continuous

TRIGGERED

Refresh, then stop

- Processes data available when the update began
- Compute runs only long enough to finish
- Cheaper, and the usual default
- New data waits for the next trigger

CONTINUOUS

Keep it fresh

- Processes new data as it arrives
- Requires an always-running cluster
- More expensive, lower latency
- For millisecond latency, use real-time mode

COMMONLY CONFUSED

Pipeline mode is independent of dataset type. Both materialized views and streaming tables update in either mode.

Use a continuous job, not continuous mode

- Databricks recommends running a continuous pipeline via a **continuous job**, rather than setting pipeline mode to continuous.
- The job then manages the execution lifecycle, and **unlocks serverless performance modes** that built-in continuous mode does not support.
- Trigger interval is best set **per table**, because streaming and batch queries have different defaults.
- Standalone materialized views and streaming tables **always** run triggered, whatever orchestrates them.

> ON THE EXAM
>
> This is a recent recommendation and older material contradicts it. Job orchestration is now the preferred route to continuous execution.

What a full refresh does to a streaming table

- It **truncates the table, clears the checkpoints**, and reprocesses all source data from the beginning.
- If the source no longer holds that data, it **cannot be recovered**. That is data loss.
- Downstream dependent tables must be fully refreshed too, or the pipeline fails.
- Block it with the table property **pipelines.reset.allowed** set to false.

> ON THE EXAM
>
> A Kafka source with 24-hour retention plus a full refresh run after that window equals messages that no longer exist anywhere.

Materialized views are not affected this way

- A materialized view returns the **same result** whether refreshed normally or fully.
- It is recomputing an answer from sources, so there is no checkpoint state to destroy.
- A streaming table resets all state and checkpoint information, and can drop records.
- Full refreshes must be triggered **explicitly**. Nothing runs one by accident.

> ON THE EXAM
>
> Same words, entirely different consequences. Which dataset type is being refreshed changes whether the operation is safe.

Legitimate reasons for a full refresh

- You changed **stateful logic**, such as a watermark threshold or the aggregation columns.
- The streaming checkpoint has been corrupted or deleted.
- Schema tracking or schema location files have been lost.
- Previously processed data would now be inconsistent with updated logic or schema.

> ON THE EXAM
>
> Streaming state is built incrementally. Change the logic and the existing state no longer matches it, so it has to be rebuilt.

Standalone against a full pipeline

STANDALONE

One dataset, managed for you

- SQL only, from a warehouse or serverless notebook
- Databricks manages the pipeline behind it
- Always triggered. Schedule or trigger on update
- Shows as MV/ST on the jobs page

LAKEFLOW PIPELINE

Many datasets, authored as a unit

- SQL and Python, with dependency orchestration
- Sinks, private datasets, snapshot CDC
- Triggered or continuous
- Shows as ETL on the jobs page

ONE-WAY DOOR

A standalone dataset cannot be moved into a pipeline later. It has to be recreated there.

What to carry into the exam

- Streaming table processes each row **once**. Materialized view keeps a query's **answer correct**.
- Batch read gives a materialized view, streaming read gives a streaming table.
- Full refresh on a streaming table **clears checkpoints and can lose data**. On a materialized view it is harmless.
- Append flows add sources or backfill **without** a full refresh.
- Mode is independent of dataset type. Triggered is cheaper, continuous is fresher.

> ON THE EXAM
>
> When a question describes late-arriving corrections to already-processed rows, a plain streaming table is usually the wrong answer.

A pipeline ingests clickstream events from a message bus into a streaming table. The bus retains messages for 24 hours. The table holds 18 months of history and feeds three downstream materialized views.

An engineer changes the watermark threshold on a stateful aggregation and is told the pipeline requires a full refresh to apply it.

What is the most important consequence to raise?

A The refresh will be slow, but all 18 months will be rebuilt correctly from the message bus.

B Only data still within the 24-hour retention window can be reprocessed; the rest is lost.

C The materialized views will need rebuilding, but the streaming table itself is unaffected.

D Full refresh applies only to materialized views, so the watermark change will not take effect.

◆ PAUSE AND DECIDE

What is the most important consequence to raise?

A. The refresh will be slow, but all 18 months will be rebuilt correctly from the message bus.

The bus holds 24 hours. It cannot supply 18 months, however long you wait.

B. Only data still within the 24-hour retention window can be reprocessed; the rest is lost.

Full refresh truncates the table and clears checkpoints, then replays from source. The source no longer has it.

C. The materialized views will need rebuilding, but the streaming table itself is unaffected.

Backwards. The streaming table is truncated, and downstream tables must be refreshed too or the pipeline fails.

D. Full refresh applies only to materialized views, so the watermark change will not take effect.

It applies to both, and changing stateful logic is a documented reason to need one.

ON THE EXAM

Retention on the source versus history in the table is the pairing to spot. When the table holds more than the source can replay, a full refresh destroys the difference.

A silver dataset joins order records against a customer dimension table. Customer records are updated regularly, including corrections to addresses on customers who ordered months ago.

The team implemented this as a streaming table and reports that corrected customer details never appear for historical orders.

What is the correct fix?

A Add an append flow so the corrections are appended to the streaming table.

B Switch the dataset to a materialized view, so the join result is recomputed as sources change.

C Run the pipeline in continuous mode so corrections are picked up as they arrive.

D Shorten the trigger interval so the streaming table updates more frequently.

◆ PAUSE AND DECIDE

What is the correct fix?

A. Add an append flow so the corrections are appended to the streaming table.

Appends new rows. It does not revise rows already written.

B. Switch the dataset to a materialized view, so the join result is recomputed as sources change.

An enrichment join where old rows must change is exactly what materialized views are for.

C. Run the pipeline in continuous mode so corrections are picked up as they arrive.

Changes latency, not semantics. Rows already processed are still not revisited.

D. Shorten the trigger interval so the streaming table updates more frequently.

Same mistake as C. More frequent updates to a process-once table still never revisit old rows.

ON THE EXAM

Process-once is a semantic property, not a timing one. No scheduling change makes a streaming table revisit a row it has already handled.

A live streaming table ingests transactions from one regional system. A second region is being onboarded, and its historical transactions must also be loaded into the same table.

The table has 400 million rows and downstream gold materialized views that must stay available throughout.

Which approach adds the second region with least disruption?

A Rewrite the query as a union of both sources and run a full refresh.

B Define an append flow for the second region's source, and another to backfill its history.

C Create a second streaming table and a materialized view that unions the two.

D Switch the table to continuous mode so it picks up the new source automatically.

◆ PAUSE AND DECIDE

Which approach adds the second region with least disruption?

A Rewrite the query as a union of both sources and run a full refresh.

Works, and reprocesses 400 million rows while clearing checkpoints. The expensive route.

B Define an append flow for the second region's source, and another to backfill its history.

Append flows exist to add sources and backfill history to a live streaming table without a full refresh.

C Create a second streaming table and a materialized view that unions the two.

Workable but adds a dataset and a layer, and changes what downstream views read from.

D Switch the table to continuous mode so it picks up the new source automatically.

Mode controls when work happens, not which sources feed a table.

ON THE EXAM

Adding a source, or backfilling, without a full refresh is the exact purpose of append flows. Recognise the requirement wording.

> An analyst needs a single pre-aggregated table to accelerate a dashboard. They work entirely in the SQL editor against a serverless warehouse, want it refreshed on a schedule, and need no orchestration or intermediate datasets.
>
> A colleague suggests building a full pipeline with Python so it can be extended later.

What is the better recommendation?

A. Build the full pipeline now, since a standalone dataset can be moved into one later if needed.

B. Create a standalone materialized view with a schedule, accepting it must be recreated if a pipeline is needed later.

C. Create a standalone streaming table, since scheduled refresh requires streaming semantics.

D. Build the full pipeline, because standalone datasets cannot be refreshed on a schedule.

◆ PAUSE AND DECIDE

What is the better recommendation?

A Build the full pipeline now, since a standalone dataset can be moved into one later if needed.

The reasoning is wrong. Standalone datasets cannot be moved into a pipeline; they must be recreated.

B Create a standalone materialized view with a schedule, accepting it must be recreated if a pipeline is needed later.

One dataset, SQL, warehouse, scheduled refresh, no pipeline-only features. Exactly the standalone case.

C Create a standalone streaming table, since scheduled refresh requires streaming semantics.

Invented. A pre-aggregated dashboard table is a materialized view.

D Build the full pipeline, because standalone datasets cannot be refreshed on a schedule.

They can, with SCHEDULE, TRIGGER ON UPDATE, or a SQL task in a job.

ON THE EXAM

Note that A reaches the wrong conclusion via a true-sounding claim. The one-way door is real, but it argues for honesty about the trade-off, not for over-building.

A team runs a pipeline in continuous mode to keep a gold layer fresh for an operations dashboard. Costs are higher than expected and they are told serverless performance modes could reduce them.

They confirm the freshness requirement is genuine and cannot be relaxed to a triggered schedule.

What should they change?

A Nothing. Continuous mode is the only way to achieve continuous freshness.

B Orchestrate the pipeline with a continuous job instead of setting pipeline mode to continuous.

C Convert the gold materialized views to streaming tables, which are cheaper in continuous mode.

D Set a short trigger interval on the pipeline and switch to triggered mode.

◆ PAUSE AND DECIDE

What should they change?

A Nothing. Continuous mode is the only way to achieve continuous freshness.

A continuous job achieves the same freshness and manages the lifecycle.

B Orchestrate the pipeline with a continuous job instead of setting pipeline mode to continuous.

Databricks' current recommendation. Job orchestration unlocks serverless performance modes built-in continuous mode does not support.

C Convert the gold materialized views to streaming tables, which are cheaper in continuous mode.

Mode is independent of dataset type, and gold aggregations are materialized view work.

D Set a short trigger interval on the pipeline and switch to triggered mode.

Relaxes the freshness requirement the question said was fixed.

ON THE EXAM

A recent recommendation that older material contradicts. Job orchestration is now the preferred route to continuous execution.

A bronze streaming table ingests files landing in cloud storage. A junior engineer has twice run a full refresh on it while debugging, each time forcing an expensive reprocess of the entire history and breaking three downstream tables.

The team wants to prevent this while keeping the ability to perform a full refresh deliberately during a planned migration.

What is the most appropriate control?

A. Set the table property pipelines.reset.allowed to false, and change it during the planned migration.

B. Convert the bronze table to a materialized view so full refresh becomes harmless.

C. Remove the junior engineer's permission on the pipeline entirely.

D. Enable skipChangeCommits on the downstream tables so they survive a full refresh.

◆ PAUSE AND DECIDE

What is the most appropriate control?

A Set the table property pipelines.reset.allowed to false, and change it during the planned migration.

Purpose-built for this. It blocks accidental refreshes and can be lifted deliberately when one is genuinely wanted.

B Convert the bronze table to a materialized view so full refresh becomes harmless.

Bronze ingestion is streaming table work. Changing the dataset type to dodge an operation is the wrong trade.

C Remove the junior engineer's permission on the pipeline entirely.

Addresses one person, not the risk. Anyone else can still do it tomorrow.

D Enable skipChangeCommits on the downstream tables so they survive a full refresh.

Helps downstream tables tolerate it, and does nothing about the bronze table being truncated.

ON THE EXAM

Both requirements matter: prevent the accident, and keep the deliberate operation available. Only a reversible control does both.

DEVELOPING CODE · DOMAIN 1 - DEVELOPING CODE FOR DATA PROCESSING

Structured Streaming

Section 1. Triggers, checkpoints, watermarks, and what you may change on restart.

SECTION 1 of 10 WEIGHT 22% TOPIC Streaming

admin@sthithapragna.com

The checkpoint **is** the query. Change the query enough and the checkpoint no longer describes it.

What is actually stored

CONTENTS	WHY IT MATTERS
Source offsets	How far into each source the query has read. This is what makes restart incremental.
Intermediate state	Accumulated data for aggregations, joins and deduplication.
Metadata	Configuration and a unique identity for the query.
Commit log	Which micro-batches completed. This is what prevents reprocessing.

ON THE EXAM

Every query needs its own checkpoint directory in durable cloud storage. Two queries sharing one location is a corruption waiting to happen.

The checkpoint you did not specify

- Some sinks **generate a temporary checkpoint automatically** if you omit the location. Display output and the memory sink both do.
- Those temporary locations offer **no fault tolerance and no consistency guarantee**, and may not be cleaned up.
- Databricks recommends **always specifying a checkpoint location**, even for these.
- It is the difference between something that appears to work in a notebook and something that survives a restart.

> ON THE EXAM
>
> A stream that works interactively and behaves oddly in production is often a stream that was never given a real checkpoint.

How often the query looks for work

TRIGGER	BEHAVIOUR
Default	Starts the next micro-batch as soon as the previous one finishes.
processingTime interval	Fixed interval micro-batches. Checks for new data every N seconds.
processingTime of 0	Next batch begins immediately when data is available. Lowest micro-batch latency.
availableNow	Consumes all available records as an incremental batch, then stops.
Once	Deprecated from DBR 11.3 LTS. Use availableNow instead.
Continuous	Not supported on Databricks. Experimental in open-source Spark only.

ON THE EXAM

On serverless compute only availableNow and Once are supported, and Databricks recommends availableNow.

Incremental batch against operational streaming

AVAILABLENOW

Process what is there, then stop

- Compute runs only while there is work
- Right for periodic incremental loads
- Batch size tunable with rate limits
- The recommended trigger on serverless

PROCESSINGTIME OF 0

Keep going, as fast as possible

- Lowest micro-batch latency
- Right for fraud, alerting, monitoring
- Costs more in cloud storage API calls
- Explicitly not availableNow or Once

STATED IN THE DOCS

For operational workloads the docs say plainly: do not use availableNow, Once or Continuous. Those are for incremental batch, not for low latency.

Real-time mode

- Achieves end-to-end latency **under one second at the tail**, and around **300 milliseconds** typically.
- This is what replaced the idea of continuous processing, which was never supported here.
- For operational latency targets in the tens to hundreds of milliseconds, this is the answer.
- Micro-batch tuning cannot reach these numbers. There is a floor to how short a batch can usefully be.

> ON THE EXAM
>
> If a question sets a sub-second latency requirement, micro-batch tuning is not the answer being looked for.

What gets written each trigger

MODE	WHAT IT EMITS, AND THE CATCH
Append	Only new rows, never revised. Required for stream-stream joins.
Update	Rows that changed, re-emitted until the watermark expires. The sink must be idempotent.
Complete	The entire result table every trigger. Not for low latency.

ON THE EXAM

Update mode gives the lowest latency where it is supported, but it re-emits the same key repeatedly. A sink that cannot absorb that will duplicate rows.

Multiple stateful operators

- From **DBR 13.3 LTS** you can chain stateful operators, feeding a windowed aggregation into a join.
- **Only append output mode is supported** when you do.
- FlatMapGroupsWithState is not supported in a chain.
- So chaining and update mode are mutually exclusive. Choose which property you need.

> ON THE EXAM
>
> A question that chains stateful operations and then asks for update mode is describing something that does not work.

The trade-off you are actually setting

- A watermark says how long the query keeps **accepting out-of-order event-time data** before dropping it and evicting state.
- Too short and it **silently discards valid late records**. Nothing errors.
- Too long and you carry more state, and pay in latency and memory.
- A small multiple of your latency target, such as **twice** it, is a reasonable starting point.

> ON THE EXAM
>
> The failure mode is silence. A watermark that is too short does not fail loudly; it just quietly loses data you needed.

What you may change and still resume

CHANGE	CAN YOU KEEP THE CHECKPOINT?
Adding or removing a filter	Yes. Generally safe.
Rate limits, such as max files per trigger	Yes.
Trigger interval	Yes. Can be changed between runs.
Number or type of input sources	No. Needs a new checkpoint.
Kafka topics or Auto Loader paths	No.
Stateful operator type, or state schema	No.
Output sink type	No.

ON THE EXAM

The pattern: anything that changes what state means, or which sources exist, breaks the checkpoint. Anything that changes only pace or filtering does not.

Sources are identified by position

- By default Structured Streaming identifies sources by their **position in the query plan**, not by name.
- So **reordering** sources breaks checkpoint compatibility, even though nothing about them changed.
- **Source evolution**, from DBR 18.2, lets you name each source and then reorder, add or remove them.
- Deduplication operators can also fail to restart when the compute access mode changes.

> ON THE EXAM
>
> Adding a source to an existing union is the classic version of this. It looks additive and it invalidates the checkpoint.

The state store and how to make it cheaper

- The **RocksDB state store** is required for both changelog checkpointing and asynchronous state checkpointing.
- **Changelog checkpointing** persists only the incremental changes to state, rather than the whole thing.
- **Asynchronous state checkpointing** overlaps the checkpoint write with the next micro-batch.
- The state management scheme **cannot be changed** between restarts.

> ON THE EXAM
>
> Reach for these when checkpoint writing is the bottleneck in your batch duration, not before. Async checkpointing has recovery and resizing caveats.

Asynchronous progress tracking

- Updates the offset and commit logs **without blocking data processing**.
- Useful when offset checkpointing is adding latency to a continuously running stream.
- **Not compatible with the availableNow or Once triggers.**
- So it belongs with operational streaming, alongside a processing time trigger.

> ON THE EXAM
>
> Two separate optimisations with similar names: state checkpointing is about accumulated state, progress tracking is about offsets.

The rules Databricks states directly

- Run streams as **Lakeflow Jobs on jobs compute**. Never on all-purpose compute.
- Schedule with the job's **Continuous mode**. That is job scheduling, not the streaming trigger.
- **Do not enable autoscaling** for Structured Streaming jobs. Scaling down has known limitations.
- Remove display and count calls from production notebooks.

> ON THE EXAM
>
> Continuous job mode prevents concurrent runs, restarts on failure, and uses exponential backoff. That is the restart-on-failure requirement, solved.

Where exactly-once stops

- **foreachBatch provides at-least-once**, not exactly-once. This is stated explicitly.
- So anything written through it must be **idempotent**, or duplicates will appear after a retry.
- Checkpoints plus an idempotent sink are what deliver exactly-once for the sources that support it.
- Update output mode carries the same requirement, for the same reason.

> ON THE EXAM
>
> Exactly-once is a property of the whole path, not of Structured Streaming alone. The sink has to hold up its end.

Multiple tasks or multiple queries

	RETRY BEHAVIOUR
Multiple tasks in one job	All tasks must fail before the job retries.
Multiple queries in one task	The task retries if any single query fails.

ON THE EXAM

With multiple tasks, compute is sized per task. With multiple queries, they share compute and can be assigned to scheduler pools.

Do not await termination inside a job

- In Lakeflow Jobs, **do not call awaitTermination**. The Jobs service already prevents a run finishing while a query is active.
- It blocks the cell, stops the service tracking the query, and **breaks backlog metrics and notifications**.
- Use it in interactive notebooks, so query state is visible and failures surface.
- And locally, where the process would otherwise exit as soon as the main thread finished.

> ON THE EXAM
>
> The same line of code is correct in a notebook and wrong in a job. Context decides.

What to carry into the exam

- The checkpoint records offsets, state, metadata and commits. Change what state **means** and it cannot be reused.
- Once is **deprecated**, Continuous is **unsupported**. availableNow for incremental batch, processingTime of 0 for operational.
- A watermark that is too short **silently drops** valid data.
- Update mode and foreachBatch both require an **idempotent sink**.
- Jobs compute, continuous job scheduling, no autoscaling.

> ON THE EXAM
>
> When a restart fails after a code change, ask what the change did to the state or the sources. That is where the answer nearly always is.

A streaming query aggregates orders by region and writes to Delta. To improve reporting granularity, a developer changes the grouping from region alone to region and product category, and redeploys.

The query fails on restart with an error about the checkpoint. The developer points out that they only changed a group by clause, and no sources or sinks were touched.

Why does the restart fail?

A The output sink schema changed, and Delta requires schema evolution to be enabled first.

B Changing the grouping keys changes the state schema, which the checkpoint cannot carry forward.

C Streaming aggregations cannot be modified at all once a query has been started.

D The trigger interval must be reset whenever the query plan changes.

◆ PAUSE AND DECIDE

Why does the restart fail?

A The output sink schema changed, and Delta requires schema evolution to be enabled first.

Plausible-sounding, but the failure is about state, not the sink's table schema.

B Changing the grouping keys changes the state schema, which the checkpoint cannot carry forward.

State schema is on the not-allowed list. Any change to the number or type of grouping keys invalidates it.

C Streaming aggregations cannot be modified at all once a query has been started.

Too absolute. Filters and rate limits can change freely; it is the state schema that cannot.

D The trigger interval must be reset whenever the query plan changes.

Trigger interval is explicitly safe to change on the same checkpoint.

ON THE EXAM

The developer's reasoning is the trap. A small code change can still be a large state change, and the checkpoint tracks state rather than lines of code.

A fraud detection stream must act on card transactions within a few hundred milliseconds. The current implementation uses a 10-second processingTime trigger and writes with complete output mode.

The team has tried reducing the trigger to 1 second and then 500 milliseconds. Latency improved but plateaued well above the target, and cost rose sharply.

What is the appropriate change?

A Reduce the trigger interval further, to 100 milliseconds.

B Switch to the availableNow trigger, which removes the fixed interval entirely.

C Move to real-time mode, and change the output mode away from complete.

D Enable asynchronous state checkpointing to remove the checkpoint write from the batch.

◆ PAUSE AND DECIDE

What is the appropriate change?

A Reduce the trigger interval further, to 100 milliseconds.

The plateau is the point. Micro-batch overhead is a floor no interval gets under.

B Switch to the availableNow trigger, which removes the fixed interval entirely.

availableNow stops after processing. The docs say explicitly not to use it for operational workloads.

C Move to real-time mode, and change the output mode away from complete.

Real-time mode reaches around 300 ms, and complete mode is explicitly not for low latency.

D Enable asynchronous state checkpointing to remove the checkpoint write from the batch.

Helps only when checkpoint writing is the bottleneck. Nothing here says it is.

ON THE EXAM

The word plateaued is the clue. Diminishing returns from tuning means you have hit a structural limit, not a configuration one.

A stream unions events from two Kafka topics and writes to a bronze streaming table. A third topic must now be added to the same query.

The team adds it to the union, redeploys, and the query fails on restart. They confirm the two original topics are unchanged and the sink is untouched.

What explains this, and what is the modern remedy?

A Adding a source changes the number of input sources, which breaks the checkpoint. Source evolution allows named sources to be added.

B Kafka topics must be subscribed with a pattern rather than a list, so no new source is created.

C The union operation is stateful, so any change to it requires clearing the state store.

D Each Kafka topic requires its own checkpoint, so the query needs three checkpoint locations.

◆ PAUSE AND DECIDE

What explains this, and what is the modern remedy?

A Adding a source changes the number of input sources, which breaks the checkpoint. Source evolution allows named sources to be added.

Sources are identified by position by default. Source evolution assigns stable names so they can be added and reordered.

B Kafka topics must be subscribed with a pattern rather than a list, so no new source is created.

Changing the subscribed topics is itself on the not-allowed list.

C The union operation is stateful, so any change to it requires clearing the state store.

A union is not a stateful operator. The problem is source identity, not state.

D Each Kafka topic requires its own checkpoint, so the query needs three checkpoint locations.

One checkpoint per query, not per source.

ON THE EXAM

Adding a source feels purely additive and is not. Position-based identity is why, and named sources are the fix.

> A stream uses foreachBatch to write aggregated results into an external operational database via a JDBC connection. The job is configured to restart automatically on failure.
>
> After an executor failure and automatic restart, the operations team reports duplicate rows in the target database for the period around the failure.

What is the correct explanation?

A The checkpoint was corrupted by the executor failure and replayed committed offsets.

B foreachBatch provides at-least-once, so the write must be made idempotent.

C Automatic restart should be disabled, since it causes reprocessing.

D The output mode should be complete so the target is fully rewritten each batch.

◆ PAUSE AND DECIDE

What is the correct explanation?

A The checkpoint was corrupted by the executor failure and replayed committed offsets.

Nothing suggests corruption. This behaviour is the documented guarantee, not a fault.

B foreachBatch provides at-least-once, so the write must be made idempotent.

Stated explicitly in the docs. A batch can be re-executed after failure, so the write must tolerate it.

C Automatic restart should be disabled, since it causes reprocessing.

Databricks recommends automatic restart. Removing it trades duplicates for an outage.

D The output mode should be complete so the target is fully rewritten each batch.

Rewriting an operational database every batch is not viable, and does not address the guarantee.

ON THE EXAM

Exactly-once is a property of the whole path. Writing through an arbitrary sink hands the guarantee to you.

> A sessionisation stream uses a 30-second watermark on event time. Analysts report that a small but material number of mobile events are missing from the results entirely. Mobile clients buffer events when offline and can upload them up to 20 minutes later.
>
> No errors appear in the logs, and the stream reports healthy throughput.

What is happening, and what is the trade-off in fixing it?

A The watermark is dropping late events silently. Lengthening it retains them, at the cost of more state and latency.

B The trigger interval is too long, so events arrive after the batch closes. Shorten the interval.

C Append output mode is discarding revised rows. Switch to update mode.

D The state store has evicted entries under memory pressure. Enable RocksDB.

◆ PAUSE AND DECIDE

What is happening, and what is the trade-off in fixing it?

A The watermark is dropping late events silently. Lengthening it retains them, at the cost of more state and latency.

A 30-second watermark against 20-minute lateness. The watermark drops what arrives beyond it, with no error.

B The trigger interval is too long, so events arrive after the batch closes. Shorten the interval.

Trigger interval governs when batches run, not how much lateness is tolerated.

C Append output mode is discarding revised rows. Switch to update mode.

These events never enter the result at all. Output mode governs what is emitted, not what is accepted.

D The state store has evicted entries under memory pressure. Enable RocksDB.

Eviction here is by watermark policy, not memory pressure.

ON THE EXAM

Silence is the signature. A watermark that is too short produces no error, no warning, and no entry in the logs.

A team runs eight streaming queries. They are currently defined as eight queries inside a single task, on one autoscaling all-purpose cluster, and the notebook calls awaitTermination at the end.

One query fails roughly weekly, taking all eight down. They want a more resilient arrangement.

Which set of changes best addresses this?

A. Keep one task, add scheduler pools, and increase the autoscaling maximum.

B. Split into separate tasks on jobs compute, disable autoscaling, use continuous job scheduling, and remove awaitTermination.

C. Keep the single task but wrap each query in retry logic inside the notebook.

D. Move all eight queries onto a single availableNow trigger so failures stop the batch cleanly.

◆ PAUSE AND DECIDE

Which set of changes best addresses this?

A Keep one task, add scheduler pools, and increase the autoscaling maximum.

Scheduler pools help contention, not isolation. Autoscaling is advised against for streaming.

B Split into separate tasks on jobs compute, disable autoscaling, use continuous job scheduling, and remove awaitTermination.

Separate tasks isolate the failure, and every other element is a stated Databricks recommendation.

C Keep the single task but wrap each query in retry logic inside the notebook.

Rebuilds by hand what continuous job scheduling already provides, and keeps every other problem.

D Move all eight queries onto a single availableNow trigger so failures stop the batch cleanly.

Converts continuous streams into periodic batches, changing the workload rather than fixing it.

ON THE EXAM

Four separate recommendations are being broken at once here. The question rewards recognising the whole set rather than the loudest one.

DEVELOPING CODE · DOMAIN 1 - DEVELOPING CODE FOR DATA PROCESSING

Structured Streaming or declarative pipelines?

Section 1. A selection module. Eight scenarios, and the rules that decide them.

SECTION 1 of 10 WEIGHT 22% TOPIC Choosing

admin@sthithapragna.com

The default has flipped. Pipelines are now the recommended starting point, and **Structured Streaming is what you reach for when a pipeline cannot do the job.**

admin@sthithapragna.com

What a pipeline stops you having to build

- **Orchestration and lineage** across datasets, worked out from your queries rather than declared.
- **Checkpoint and state management**, handled rather than configured per query.
- **Expectations** for data quality, and **AUTO CDC** for slowly changing dimensions.
- **Enhanced autoscaling**. Ordinary autoscaling has known scale-down limitations for streaming.

> ON THE EXAM
>
> That last point is more than convenience. The production guidance for hand-written streaming is to disable autoscaling entirely.

admin@sthithapragna.com

Control against convenience

STRUCTURED STREAMING

You own every decision

- Any trigger, including real-time mode
- Any output mode, including complete
- Arbitrary sinks and arbitrary state
- You also own checkpoints and restarts

DECLARATIVE PIPELINES

The engine owns the plumbing

- Dependencies and lineage inferred
- Expectations and AUTO CDC built in
- Enhanced autoscaling and retries
- You accept the engine's constraints

HOW IT IS TESTED

Neither column is the safe default answer. The exam decides these on a specific capability the question quietly requires.

admin@sthithapragna.com

What pipelines cannot do

CONSTRAINT	CONSEQUENCE
Real-time mode is unsupported	Sub-second latency requirements rule pipelines out.
pivot() is unsupported	It needs eager loading to compute the output schema.
Time travel on materialized views	Not supported. Streaming tables only.
Iceberg reads	Cannot be enabled on materialized views or streaming tables.
External access	Published datasets are reachable only by Databricks clients.
A dataset is defined once	Only one operation may target it, unless append flows are used.

ON THE EXAM

These are the sentences a question hides inside a scenario. Learn them as triggers, not as trivia.

admin@sthithapragna.com

Real-time mode is Structured Streaming only

- **Not supported on pipelines**, classic or serverless. Not supported on plain serverless either.
- Runs on **dedicated** compute, and on **standard** compute for Python only.
- **Update output mode only.** Append and complete are not supported.
- Kafka, Event Hubs and Kinesis as sources. **Delta is not supported** as a source or a sink.

> ON THE EXAM
>
> So real-time mode is not simply a faster pipeline. It is a different set of constraints, and it excludes the lakehouse's own table format as a source.

admin@sthithapragna.com

What it will not do

- No **stream-to-stream joins**, no session windows, no deduplication within a watermark.
- Legacy arbitrary state operators are unsupported. **transformWithState works, with differences.**
- **Exactly-once sinks are not supported.** The sink mode must be set to at-least-once.
- Plain deduplication works, but its state is **unbounded**, so it grows without a watermark to evict it.

> ON THE EXAM
>
> A sub-second requirement combined with a stream-stream join is a contradiction. Notice when a question is asking for both.

admin@sthithapragna.com

Sinks brought arbitrary writes into pipelines

- Pipelines now have **sinks**, including a batch-level sink that runs your own Python for each micro-batch.
- That covers custom merge logic, writing to several destinations, and systems with no streaming write.
- **Python only.** SQL is not supported for sinks.
- **Only append flows can write to a sink**, and **expectations are not supported** for sinks.

> ON THE EXAM
>
> Older material says arbitrary sinks force you out of pipelines. That is no longer true, but the constraints matter.

admin@sthithapragna.com

Custom stateful logic in either place

- **transformWithState** is now recommended over the older map and flatMap state operators.
- It works in Structured Streaming **and inside a pipeline flow**, so custom state no longer forces a choice.
- Available from DBR 16.2. RocksDB became the default state store from DBR 17.3.
- Below that runtime, the RocksDB state store must be configured explicitly.

> ON THE EXAM
>
> Another gap that has closed. Needing custom state used to mean hand-written streaming; now it does not by itself.

admin@sthithapragna.com

Rules that settle most questions

IF THE SCENARIO REQUIRES	CHOOSE
Sub-second latency, or real-time mode	Structured Streaming
Multi-stage dependencies and lineage	Pipelines
Data quality expectations	Pipelines
SCD Type 1 or Type 2 handling	Pipelines, via AUTO CDC
Time travel on a derived aggregate	Structured Streaming, or a streaming table
Consumption by a non-Databricks client	Structured Streaming, or a pipeline sink
A new pipeline with no special constraint	Pipelines. That is the stated default.

ON THE EXAM

Work from the requirement to the tool, never the reverse. The final row is the tie-breaker when nothing else decides it.

admin@sthithapragna.com

What actually differs day to day

- Pipelines autoscale properly for streaming. Hand-written streams are told **not to autoscale at all**.
- Pipeline failures surface in an event log with dataset-level detail, rather than in cluster logs.
- Hand-written streams need continuous job scheduling, jobs compute and restart policies configured by you.
- Both are billed for compute. The difference is engineering time and the cost of mistakes.

> ON THE EXAM
>
> When a question mentions operational burden, retries, or a team spending time on plumbing, it is pointing at pipelines.

admin@sthithapragna.com

What migration actually involves

- There is **no in-place conversion**. A hand-written stream becomes a pipeline by being rewritten.
- Checkpoints do not transfer. The pipeline manages its own, so the new streaming table starts fresh.
- That makes source retention the first question, exactly as it was for a full refresh.
- A standalone dataset also cannot be moved into a pipeline. It has to be recreated there.

> ON THE EXAM
>
> Migration questions are really retention questions. If the source cannot replay the history, the migration needs a plan for it.

admin@sthithapragna.com

What to carry into the exam

- **Pipelines are the stated default** for new streaming work. Use that as the tie-breaker.
- **Real-time mode does not run on pipelines.** Sub-second latency is the clearest discriminator.
- Arbitrary sinks and custom state **no longer force** hand-written streaming. Both gaps have closed.
- Expectations are unsupported for sinks. Time travel is unsupported on materialized views.
- Migration means rewriting, and starting from a fresh checkpoint.

> ON THE EXAM
>
> Find the one capability the scenario requires, then check which tool lacks it. That is faster than weighing general merits.

admin@sthithapragna.com

> A payments team must flag suspicious card transactions and publish alerts to a Kafka topic within 400 milliseconds end to end. Events arrive from Kafka.
>
> The platform team's standard is to build all new streaming work as declarative pipelines.

What should the payments team do?

A Build it as a pipeline in continuous mode, with a short trigger interval.

B Build it as Structured Streaming in real-time mode, on dedicated compute.

C Build it as a pipeline, using a foreach batch sink to write to Kafka.

D Build it as a pipeline on serverless, which supports real-time mode.

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What should the payments team do?

A. Build it as a pipeline in continuous mode, with a short trigger interval.

Continuous mode still runs micro-batches. It cannot reach 400 milliseconds.

B. Build it as Structured Streaming in real-time mode, on dedicated compute.

Real-time mode reaches around 300 milliseconds and is not available on pipelines. Kafka to Kafka is supported.

C. Build it as a pipeline, using a foreach batch sink to write to Kafka.

Solves the sink and not the latency. A sink does not make a pipeline sub-second.

D. Build it as a pipeline on serverless, which supports real-time mode.

Factually wrong. Real-time mode is unsupported on pipelines and on serverless.

ON THE EXAM

A team standard is not a requirement. When a stated latency target is unreachable by the standard approach, the target wins.

admin@sthithapragna.com

> A team is building a new bronze to gold pipeline over supplier data. It has four stages, needs record-level quality rules with rejected rows quarantined, and must apply Type 2 history to a supplier dimension.
>
> Latency requirements are relaxed: hourly is acceptable.

Which approach fits best?

A Structured Streaming with foreachBatch, implementing merge logic and validation in Python.

B A declarative pipeline using expectations and AUTO CDC.

C Structured Streaming for bronze, then a pipeline for silver and gold.

D Four standalone streaming tables, refreshed hourly on a schedule.

◆ PAUSE AND DECIDE

admin@sthithapragna.com

Which approach fits best?

A Structured Streaming with foreachBatch, implementing merge logic and validation in Python.

Hand-builds expectations and SCD handling that already exist, and inherits at-least-once semantics.

B A declarative pipeline using expectations and AUTO CDC.

Multi-stage dependencies, quality rules and Type 2 history are three separate pipeline strengths, with no constraint against them.

C Structured Streaming for bronze, then a pipeline for silver and gold.

Splits one pipeline in two and loses end-to-end lineage for no stated reason.

D Four standalone streaming tables, refreshed hourly on a schedule.

Standalone datasets have no orchestration between them, no expectations and no AUTO CDC.

ON THE EXAM

Three requirements each point the same way, and nothing points the other. That is the shape of an easy pipelines answer.

admin@sthithapragna.com

> An existing pipeline computes a daily revenue aggregate as a materialized view. Finance now needs to reproduce the exact figures published on any past date, for audit.
>
> The team proposes adding time travel queries against the materialized view.

What is the problem, and what should they do?

A. Nothing is wrong. Time travel works on any Delta table, including materialized views.

B. Time travel is unsupported on materialized views. Write the aggregate to a streaming table instead.

C. Time travel requires change data feed to be enabled first on the materialized view.

D. Materialized views only retain the current version, so they must be fully refreshed daily.

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What is the problem, and what should they do?

A Nothing is wrong. Time travel works on any Delta table, including materialized views.

Stated limitation: time travel is supported on streaming tables, not materialized views.

B Time travel is unsupported on materialized views. Write the aggregate to a streaming table instead.

The limitation is specific to materialized views, and streaming tables do support history.

C Time travel requires change data feed to be enabled first on the materialized view.

Different feature. Change data feed does not enable time travel.

D Materialized views only retain the current version, so they must be fully refreshed daily.

Daily full refresh would not create queryable history either.

ON THE EXAM

Both dataset types are Unity Catalog managed tables, which makes A feel right. The limitation is specific and stated.

admin@sthithapragna.com

A pipeline enriches events and must also push each micro-batch into a third-party operational system that has no streaming connector, using a vendor Python client.

A senior engineer says this rules out declarative pipelines entirely and the team must rewrite in Structured Streaming.

Is that correct?

A. Yes. Pipelines can only write to their own streaming tables and materialized views.

B. No. A foreach batch sink runs custom Python per micro-batch, though expectations do not apply to sinks.

C. No. Pipelines can write anywhere, and expectations still apply to the sink output.

D. Yes, unless the third-party system supports Delta Sharing.

◆ PAUSE AND DECIDE

Is that correct?

A Yes. Pipelines can only write to their own streaming tables and materialized views.

This was true once. Sinks changed it, and older material still says this.

B No. A foreach batch sink runs custom Python per micro-batch, though expectations do not apply to sinks.

Correct on both halves: the capability exists, and the quality mechanism does not extend to it.

C No. Pipelines can write anywhere, and expectations still apply to the sink output.

Right conclusion, wrong reason. Expectations are explicitly unsupported for sinks.

D Yes, unless the third-party system supports Delta Sharing.

Delta Sharing is for sharing tables, not for writing into an operational system.

ON THE EXAM

C is the dangerous one. It reaches the correct conclusion through a false claim, which is exactly how a distractor is built.

admin@sthithapragna.com

A team runs a hand-written stream on an autoscaling cluster. Costs are high because the cluster rarely scales back down after nightly peaks, and one engineer spends most of a day each week on failed restarts and checkpoint issues.

The workload is a straightforward three-stage transformation. Latency of a few minutes is acceptable.

What is the strongest recommendation?

A Keep Structured Streaming, disable autoscaling, and size the cluster for peak.

B Rewrite as a declarative pipeline, gaining enhanced autoscaling and managed checkpoints.

C Keep Structured Streaming and move to real-time mode, which manages state automatically.

D Keep Structured Streaming and switch the trigger to availableNow to reduce cost.

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What is the strongest recommendation?

A Keep Structured Streaming, disable autoscaling, and size the cluster for peak.

Follows the guidance and makes the cost worse, since peak sizing runs continuously.

B Rewrite as a declarative pipeline, gaining enhanced autoscaling and managed checkpoints.

Addresses both complaints at once, and nothing in the workload needs hand-written streaming.

C Keep Structured Streaming and move to real-time mode, which manages state automatically.

Real-time mode is for latency, adds constraints, and does not manage checkpoints for you.

D Keep Structured Streaming and switch the trigger to availableNow to reduce cost.

Helps cost, does nothing about the operational burden, and changes the workload to periodic batch.

ON THE EXAM

Two complaints, cost and operational burden, and one option addresses both. A addresses one by worsening the other.

admin@sthithapragna.com

A pipeline builds a wide reporting table. Part of the logic requires pivoting a long key-value table into columns, where the set of keys is not known in advance.

The pipeline fails when this step is added.

What is the cause and the best remedy?

A The pivot function is unsupported in pipelines. Compute the pivot outside and read the result in.

B The pivot must be applied to a materialized view rather than a streaming table.

C Pivots require the target schema to be declared explicitly on the dataset.

D The pipeline needs a full refresh to pick up the new columns.

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What is the cause and the best remedy?

A The pivot function is unsupported in pipelines. Compute the pivot outside and read the result in.

A stated limitation. Pivot needs eager loading to compute the output schema, which pipelines do not do.

B The pivot must be applied to a materialized view rather than a streaming table.

The limitation is at pipeline level, not per dataset type.

C Pivots require the target schema to be declared explicitly on the dataset.

Declaring a schema does not restore the eager loading that pivot needs.

D The pipeline needs a full refresh to pick up the new columns.

Full refresh does not enable an unsupported operation.

ON THE EXAM

The reason matters: pivot needs to read the data to know its own output columns, and a declarative engine plans before it reads.

admin@sthithapragna.com

> An analytics team outside the Databricks estate needs to query a curated table produced by a pipeline, using their own Spark cluster and open source tooling. They report they cannot read the streaming table directly.
>
> The pipeline owners want to keep the pipeline as it is.

What is the right approach?

A Grant the external team SELECT on the streaming table in Unity Catalog.

B Use a pipeline sink to write a copy to an external Delta table the other team can read.

C Enable Iceberg reads on the streaming table so open source clients can query it.

D Rewrite the whole pipeline as Structured Streaming so the output is an ordinary Delta table.

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What is the right approach?

A Grant the external team SELECT on the streaming table in Unity Catalog.

Permissions are not the barrier. Pipeline-published datasets are reachable only by Databricks clients.

B Use a pipeline sink to write a copy to an external Delta table the other team can read.

The documented route for external access, and it leaves the pipeline otherwise intact.

C Enable Iceberg reads on the streaming table so open source clients can query it.

Iceberg reads cannot be enabled on streaming tables or materialized views.

D Rewrite the whole pipeline as Structured Streaming so the output is an ordinary Delta table.

Would work, and discards a working pipeline when a sink solves it.

ON THE EXAM

Two stated limitations in one question: external clients cannot read published datasets, and Iceberg reads cannot be enabled on them.

admin@sthithapragna.com

A team plans to migrate a hand-written stream to a declarative pipeline. The stream reads from a Kafka topic with seven days of retention and has been running for two years, maintaining a streaming table of 3 billion rows.

They expect to switch over with no gap in the data.

No constraint in the workload requires hand-written streaming.

What must they plan for?

A Nothing special. The pipeline can adopt the existing checkpoint and continue where the stream left off.

B The pipeline starts from a fresh checkpoint, so history beyond seven days must be backfilled separately.

C The migration is impossible; a stream with existing state cannot become a pipeline.

D They must run a full refresh on the existing streaming table before migrating.

◆ PAUSE AND DECIDE

What must they plan for?

A Nothing special. The pipeline can adopt the existing checkpoint and continue where the stream left off.

Checkpoints do not transfer. The pipeline manages its own.

B The pipeline starts from a fresh checkpoint, so history beyond seven days must be backfilled separately.

Kafka retains seven days against two years of history. The gap has to be filled from the existing table.

C The migration is impossible; a stream with existing state cannot become a pipeline.

It is a rewrite, not an impossibility.

D They must run a full refresh on the existing streaming table before migrating.

Would destroy the very history that needs preserving, since Kafka holds only seven days.

ON THE EXAM

The same retention arithmetic as module 5. Two years of history behind a seven-day source means the difference must come from somewhere else.

admin@sthithapragna.com

DEVELOPING CODE · DOMAIN 1 - DEVELOPING CODE FOR DATA PROCESSING

AUTO CDC and slowly changing dimensions

Section 1. Out-of-order changes, SCD Type 1 and Type 2, and why MERGE is not enough.

SECTION 1 of 10 WEIGHT 22% TOPIC AUTO CDC

admin@sthithapragna.com

A merge applies changes in the order they arrive. AUTO CDC applies them in the order they happened.

AUTO CDC and APPLY CHANGES are the same thing

- **AUTO CDC replaces APPLY CHANGES, with identical syntax.** Databricks recommends the newer name.
- APPLY CHANGES still works. Both names appear across documentation, courses and question banks.
- In Python: **create_auto_cdc_flow** replaced **apply_changes**.
- **Not supported by Apache Spark Declarative Pipelines.** This is a Databricks extension, not open source.

> ON THE EXAM
>
> The July 2026 exam guide writes it as 'AUTO CDC APIs (formerly APPLY CHANGES)', so expect either name in a question.

Which one depends on the source

API	USE IT WHEN
AUTO CDC	The source has a change feed. You receive inserts, updates and deletes as events.
AUTO CDC FROM SNAPSHOT	The source has no change feed. You receive whole-table snapshots.

ON THE EXAM

AUTO CDC works in both SQL and Python. AUTO CDC FROM SNAPSHOT is Python only, and the SQL interface does not support it.

What has to be true before you can use it

- The pipeline must be on **serverless**, or on the **Pro or Advanced** pipeline editions.
- The Core edition will not run it. This is an edition-level gate, not a runtime version.
- You must **declare the target streaming table first**, then define the flow that writes into it.
- The source must be a **streaming source**. In SQL, read it with the stream keyword.

> ON THE EXAM
>
> Two-step construction: create the streaming table, then create the flow. A question showing only one step is describing something that does not run.

What each part of the statement does

CLAUSE	MEANING
KEYS	The columns that identify a record uniquely.
SEQUENCE BY	Which change happened first. This is the whole point.
APPLY AS DELETE WHEN	The condition that marks an event as a delete.
APPLY AS TRUNCATE WHEN	Clears the table at that point in the sequence.
COLUMNS * EXCEPT	Which source columns land in the target.
STORED AS	SCD TYPE 1, SCD TYPE 2, or BITEMPORAL.
TRACK HISTORY ON	Which columns cause a new version. Defaults to all of them.

ON THE EXAM

SEQUENCE BY is what makes this different from a merge.

SCD Type 1 against Type 2

SCD TYPE 1

Only the current truth

- Updates records in place
- No history is retained
- The target holds one row per key
- No start or end columns are added

SCD TYPE 2

Every version, dated

- A new row for each version
- Adds a start-at and an end-at column
- An end-at of null means currently active
- History survives deletes as a closed row

THE TIEBREAKER

Type 1 answers 'what is true now'. Type 2 answers 'what did we believe on this date'. Audit and point-in-time reporting need Type 2.

One user, three cities, two outcomes

	RESULT FOR USER 125
Input events	Tijuana at sequence 2, Mexicali at sequence 5, Guadalajara at sequence 6.
Arrival order	Guadalajara arrives before Mexicali. The feed is out of order.
SCD TYPE 1 target	One row: Guadalajara. The Mexicali update is dropped as superseded.
SCD TYPE 2 target	Three rows: Tijuana 2 to 5, Mexicali 5 to 6, Guadalajara 6 to null.

ON THE EXAM

Note what Type 1 did with the late arrival: it dropped it, because a higher sequence number had already been applied. A merge would have overwritten with the stale value.

Start at, end at, and what null means

- Type 2 adds a **start-at** and an **end-at** column, holding values from the sequencing column.
- A row with **end-at of null is the current version**. That is how you filter for present state.
- If you declare the target schema yourself, you **must include both columns**, with the same type as the sequence column.
- A deleted record does not disappear. Its final version is closed with an end value.

> ON THE EXAM
>
> Deletion in Type 2 means ending the current version, not removing rows. History of a deleted entity is preserved deliberately.

Not every change deserves a new version

- By default, **any** column change creates a new version. That is TRACK HISTORY ON everything.
- Name a subset, or exclude columns, and changes to the untracked ones **overwrite the current row in place**.
- Useful when a dimension has volatile attributes you do not want cluttering the history.
- The excluded column still exists and still updates. It just does not trigger a new row.

> ON THE EXAM
>
> Excluding a column from history tracking does not exclude it from the table. It changes when a version boundary is created.

Two conditions, both optional

- **APPLY AS DELETE WHEN** marks matching events as deletions rather than upserts.
- **APPLY AS TRUNCATE WHEN** clears the table at that point in the sequence.
- A truncate is **positional**. Records inserted before it are removed; later ones survive.
- For Type 2, deleted rows are held briefly as tombstones, with a view filtering them out.

> ON THE EXAM
>
> Truncate applies at its sequence position, not on arrival. That is why it can remove records that were inserted earlier in the sequence.

Initial hydration with a once flow

- A **once** flow performs a one-time insert, or backfill, into the target.
- It is **not re-run** when the pipeline refreshes, except during a full refresh.
- The documented pattern for replicating a database: hydrate once from a snapshot, then process the ongoing feed.
- This solves the problem that a change feed usually starts partway through a table's life.

> ON THE EXAM
>
> A change feed tells you what changed since it was enabled. It does not tell you what was already there. That gap is what a once flow fills.

Choosing the sequence column carefully

- The sequencing column must be a **sortable type**, and **null values are not supported**.
- To sequence by more than one column, combine them in a **struct**. It orders by the first field, then breaks ties on the second.
- That is how you handle a timestamp that is not unique, adding an identifier to break ties.
- Bitemporal processing adds a second sequence, for system time alongside business time.

> ON THE EXAM
>
> Nulls in the sequence column are a stated limitation, not a warning. If the source can emit them, that has to be handled upstream.

When there is no change feed

- The snapshot API compares successive snapshots to work out what changed, then applies those changes.
- Snapshots can come from a Delta table, cloud storage files, or a database connection.
- Two patterns: rely on the **pipeline run** for ordering, or supply a **version function** for explicit control.
- With a version function, a snapshot that arrives **out of order is skipped**, not reordered.

> ON THE EXAM
>
> Under Type 1 the target looks exactly like the latest snapshot. The value is that downstream consumers can read the change feed instead of the whole table.

What to carry into the exam

- AUTO CDC exists because **merge applies arrival order**, and change feeds arrive out of order.
- **SEQUENCE BY** is the clause that fixes it. Type 1 keeps current state, Type 2 keeps versions.
- Type 2 adds start and end columns. **End of null means current.**
- Requires **serverless or Pro or Advanced**. Snapshot processing is **Python only**.
- A **once** flow handles initial hydration before the change feed takes over.

> ON THE EXAM
>
> If an option proposes hand-written merge logic for an out-of-order CDC feed, it is describing the problem this API exists to solve.

A team replicates a customer table using MERGE INTO in a scheduled job, matching on customer id and updating when matched. The source emits changes with a monotonically increasing version number.

Auditors find several customers whose current address is an older value than one recorded earlier. The merge logic has been reviewed and matches on the correct key.

What is the most likely cause?

A The merge is matching on a non-unique key, so multiple rows are being updated.

B Change events are arriving out of order, and MERGE applies them in arrival order.

C The target table needs change data feed enabled to preserve update ordering.

D The scheduled job interval is too long, so updates are being batched together.

◆ PAUSE AND DECIDE

What is the most likely cause?

A The merge is matching on a non-unique key, so multiple rows are being updated.

The question states the key is correct, and this would produce different symptoms.

B Change events are arriving out of order, and MERGE applies them in arrival order.

The documented failure mode. A version number exists in the source and the merge is ignoring it.

C The target table needs change data feed enabled to preserve update ordering.

Change data feed records changes to a Delta table. It does not order incoming events.

D The scheduled job interval is too long, so updates are being batched together.

Batching is not the problem. Within a batch the order is still arrival order.

ON THE EXAM

The giveaway is that a version number exists and the merge does not use it. That column is exactly what SEQUENCE BY consumes.

> A product dimension must support point-in-time reporting: analysts need to reconstruct what each product's category and price were on any past date. Products are occasionally deleted from the source.
>
> The team implements AUTO CDC with STORED AS SCD TYPE 1 and reports that deleted products vanish and old prices are unavailable.

What should change?

A Keep Type 1 and enable change data feed on the target so history can be reconstructed.

B Use SCD TYPE 2, which retains every version and closes deleted records rather than removing them.

C Keep Type 1 and remove the APPLY AS DELETE WHEN clause so deletions are ignored.

D Use AUTO CDC FROM SNAPSHOT, which retains history automatically.

◆ PAUSE AND DECIDE

What should change?

A Keep Type 1 and enable change data feed on the target so history can be reconstructed.

Reconstructing a dimension from a change feed is exactly the manual work Type 2 removes.

B Use SCD TYPE 2, which retains every version and closes deleted records rather than removing them.

Point-in-time reporting is the definition of Type 2, and deletions close a version rather than dropping the row.

C Keep Type 1 and remove the APPLY AS DELETE WHEN clause so deletions are ignored.

Products would linger as though still active, which is a different kind of wrong.

D Use AUTO CDC FROM SNAPSHOT, which retains history automatically.

The snapshot API also offers Type 1 or Type 2. The source API is not what governs history.

ON THE EXAM

D is worth noticing: it swaps the wrong axis. Snapshot versus feed is about the source, not about whether history is kept.

A supplier dimension uses SCD Type 2. It has a last_contacted timestamp that updates several times a day, and a credit_rating that changes twice a year.

Analysts complain that the dimension has grown enormous and that finding credit rating changes means wading through thousands of versions per supplier.

What is the appropriate fix?

A Switch the dimension to SCD Type 1 and store credit rating history in a separate table.

B Use TRACK HISTORY ON with credit_rating, so only that column creates a new version.

C Exclude last_contacted with the COLUMNS clause so it is not written to the target.

D Add last_contacted to the KEYS clause so versions are grouped by supplier and time.

◆ PAUSE AND DECIDE

What is the appropriate fix?

A. Switch to SCD Type 1 and store credit rating history in a separate table.

Rebuilds by hand the history mechanism already available, and abandons Type 2 for the whole dimension.

B. Use TRACK HISTORY ON with credit_rating, so only that column creates a new version.

Untracked columns update the current row in place. The timestamp stays available and stops creating versions.

C. Exclude last_contacted with the COLUMNS clause so it is not written to the target.

Removes the column from the table entirely. It was wanted, just not as a version trigger.

D. Add last_contacted to the KEYS clause so versions are grouped by supplier and time.

Keys identify a record. Adding a changing column would make every event a new supplier.

ON THE EXAM

C and B differ in one word: whether the column is in the table, or whether it creates a version. Read which one the scenario actually wants.

A team must replicate an operational database table into the lakehouse. Change data capture was enabled on the source last week, but the table has eight years of existing rows.

They configure AUTO CDC against the change feed. The target contains only the last week of activity, and rows never touched since CDC was enabled are missing entirely.

What is the correct approach?

A Run a full refresh on the target streaming table to pull in the historical rows.

B Use a once flow to hydrate from a full extract, then continue with the change feed.

C Switch to AUTO CDC FROM SNAPSHOT, which reads the whole table each run.

D Extend the change feed retention on the source database to eight years.

◆ PAUSE AND DECIDE

What is the correct approach?

A Run a full refresh on the target streaming table to pull in the historical rows.

Replays the same change feed from the beginning. The feed still only holds one week.

B Use a once flow to hydrate from a full extract, then continue with the change feed.

The documented pattern for replicating an existing database. The once flow fills the gap the feed cannot.

C Switch to AUTO CDC FROM SNAPSHOT, which reads the whole table each run.

Would work, and discards a real change feed to repeatedly copy an eight-year table.

D Extend the change feed retention on the source database to eight years.

Retention cannot create events for changes that happened before CDC was enabled.

ON THE EXAM

A change feed knows what changed since it started. Rows that never changed since then were never events, so no amount of replay produces them.

An engineer writes an AUTO CDC flow sequencing by an event timestamp. The source system generates timestamps at second resolution and can emit several changes to the same record within one second.

Downstream, some records show the wrong final value, and which one is wrong varies between pipeline runs.

What is the cause and remedy?

A Ties in the sequence column are resolved arbitrarily. Sequence by a struct of timestamp and a tie-breaking id.

B Second resolution is too coarse for AUTO CDC. Convert the timestamp to milliseconds.

C The sequence column must be numeric. Cast the timestamp to a long.

D Add the timestamp to the KEYS clause so each change is treated as distinct.

◆ PAUSE AND DECIDE

What is the cause and remedy?

A Ties in the sequence column are resolved arbitrarily. Sequence by a struct of timestamp and a tie-breaking id.

A struct orders by the first field and breaks ties on the second. This is the documented pattern for non-unique timestamps.

B Second resolution is too coarse for AUTO CDC. Convert the timestamp to milliseconds.

Does not add information that was never captured. The source only knows the second.

C The sequence column must be numeric. Cast the timestamp to a long.

It must be sortable, and timestamps are. Casting changes nothing about the ties.

D Add the timestamp to the KEYS clause so each change is treated as distinct.

Every change would become a separate record rather than a version of one.

ON THE EXAM

Varying between runs is the signal. Non-determinism means something is being decided arbitrarily, and ties are the usual cause.

> A team on the Core pipeline edition wants to build an SCD Type 2 dimension. Their source is a legacy system with no change data capture, delivering a nightly full extract as CSV files to cloud storage.
>
> They plan to write the logic in SQL, as nobody on the team writes Python.

What must they change before this will work?

A. Nothing. AUTO CDC FROM SNAPSHOT works in SQL on any pipeline edition.

B. Move to serverless or the Pro or Advanced edition, and accept that snapshot processing requires Python.

C. Enable change data capture on the legacy source, which AUTO CDC requires.

D. Upgrade to a newer Databricks Runtime, which unlocks the CDC APIs.

◆ PAUSE AND DECIDE

What must they change before this will work?

A Nothing. AUTO CDC FROM SNAPSHOT works in SQL on any pipeline edition.

Wrong twice: the edition gate is real, and snapshot processing has no SQL interface.

B Move to serverless or the Pro or Advanced edition, and accept that snapshot processing requires Python.

Two independent blockers, and both must be cleared. Snapshots suit the source; Python is the only interface for them.

C Enable change data capture on the legacy source, which AUTO CDC requires.

Would let them use the SQL interface, and the snapshot API exists precisely so this is not required.

D Upgrade to a newer Databricks Runtime, which unlocks the CDC APIs.

The gate is the pipeline edition, not the runtime version.

ON THE EXAM

Two blockers, not one. Scan a scenario for every stated constraint before choosing, because these questions often hide a second.

DEVELOPING CODE · DOMAIN 1 - DEVELOPING CODE FOR DATA PROCESSING

Jobs, tasks and control flow

Section 1. Dependencies, branching, loops, retries, and the at-most-once problem.

SECTION 1 of 10 WEIGHT 22% TOPIC Jobs

admin@sthithapragna.com

That last objective is **almost word for word** the section headings of one documentation page. When a guide gets that specific, it is telling you where to look.

Three concepts, and they nest

CONCEPT	WHAT IT IS
Job	The unit you schedule and run. Holds tasks, triggers and parameters.
Task	One piece of work. A notebook, a pipeline, a wheel, a SQL query, another job.
Trigger	What starts the job. A schedule, a file arrival, a continuous run, or manual.

ON THE EXAM

Tasks form a directed acyclic graph. Databricks runs upstream tasks first, and runs as many in parallel as the graph allows.

The six run-if conditions

CONDITION	WHEN THE TASK RUNS
All succeeded	Every dependency ran and succeeded. This is the default.
At least one succeeded	One or more dependencies succeeded.
None failed	No dependency failed. Skipped ones are acceptable.
All done	Every dependency finished, whatever the outcome.
At least one failed	Used for error handling branches.
All failed	Used when everything upstream has gone wrong.

ON THE EXAM

When All succeeded is unmet, the downstream task is marked Upstream failed rather than simply skipped. That distinction shows up in monitoring.

The If/else condition task

- Runs part of the graph based on a **boolean expression**, rather than on whether something failed.
- The condition can read **task values**, **job parameters**, or dynamic values.
- Downstream tasks default to the **true** branch. The false branch must be selected explicitly.
- **It fails if the upstream task supplying its condition value is disabled.**

> ON THE EXAM
>
> Run if branches on outcome. If/else branches on value. A question about 'only if new data arrived' is asking for If/else.

Task values carry state between tasks

- A task can **set a value** that later tasks read. That is how a computation's result leaves a task.
- They are what makes a meaningful If/else condition possible. Count the rows, then branch on the count.
- They can also be passed as parameters into downstream tasks.
- They are not a general message bus. They carry small results, not data.

> ON THE EXAM
>
> Without task values, an If/else can only branch on things known before the job started. With them, it can branch on what actually happened.

The For each task needs two parts

- It runs **another task** in a loop, passing different parameters to each iteration.
- You must define **both** the For each task **and a nested task**. One alone does nothing.
- The nested task is an ordinary task type. There is nothing special about it.
- The input array can be written by hand, or generated dynamically from an earlier task.

> ON THE EXAM
>
> Dynamically generated arrays are the powerful case: one task discovers which regions have new files, and the loop processes exactly those.

The default depends on the configuration

CONFIGURATION	RETRY BEHAVIOUR
Most job configurations	No retries on failure. This is the default.
Serverless jobs	Auto-optimisation retries failed tasks by default.
Continuous trigger mode	Automatic retries with exponential backoff.

ON THE EXAM

You cannot set a retry policy on a continuous job. Its retry behaviour is system-defined and not configurable.

Some features assume you retry

- **Schema evolution with Structured Streaming assumes retries are configured.**
- When the schema changes, the stream stops. The **retry is what restarts it** with the new schema.
- Without retries, that stream simply stays failed until somebody notices.
- So a retry policy is not only about transient errors. It is a required part of some designs.

> ON THE EXAM
>
> This is the reason a question about a stream that stops on every schema change often has 'configure retries' as its answer.

Serverless auto-optimisation picks your compute **and retries your failed tasks**. Both halves are on by default.

At least once against at most once

LEAVE IT ENABLED

At least once

- Databricks' recommendation
- Transient failures resolve themselves
- Critical workloads do complete
- A task may run more than once

DISABLE IT

At most once

- For workloads that are not idempotent
- A payment, an email, an external API call
- Uncheck it in the Retry Policy dialog
- In configuration: disable auto optimization

STATED IN THE DOCS

The docs recommend leaving it on so critical workloads run at least once. The exception they name is jobs that are not idempotent.

High memory is notebook tasks only

- You can configure a **higher memory size** for serverless notebook workloads.
- **Only for notebook task types.** Not for wheel tasks, not for pipelines.
- Set through the **Memory setting in the notebook's Environment side panel**, not in the job.
- It carries a **higher DBU emission rate** than standard memory, so it is not free.

> ON THE EXAM
>
> Note where the setting lives. It is configured on the notebook environment, which is also where dependencies are declared.

Environments and dependencies on a task

- A serverless task references an **environment by key**. The environment carries a version and dependencies.
- Several tasks can share one environment definition, so a change lands in one place.
- A **notebook task cannot take a task environment spec**. Its dependencies come from the notebook environment panel.
- Which is why the memory setting and the dependency list live together.

> ON THE EXAM
>
> This is the module 3 material, seen from the job side. The attachment point differs by task type, and notebook tasks are the exception.

What you no longer control

- **Autoscaling and Photon are enabled automatically.** There is no switch.
- Spark configuration can be set at **session level only**, from a notebook, and only from an allowed list.
- **Egress traffic cannot be controlled**, so serverless jobs have full internet access.
- A **Performance optimized** setting on the job affects only its serverless tasks.

> ON THE EXAM
>
> The egress point is a security consideration, not a performance one. It can rule serverless out on its own.

Creating and automating jobs

INTERFACE	USE IT FOR
UI	Authoring and debugging. The visual graph is genuinely useful for branching.
REST API	Programmatic creation, triggering from outside, and monitoring.
CLI	Scripting and automation from a terminal or a delivery pipeline.
Bundles	Defining the job as version-controlled configuration. The production answer.

ON THE EXAM

The objective names UI, API and CLI. In practice bundles wrap the CLI, which is how a job definition gets into version control.

What to carry into the exam

- **Run if** branches on outcome. **If/else** branches on a value, usually a task value.
- **For each needs a nested task.** The array can be generated dynamically.
- Default is **no retries**, except on serverless and in continuous mode. Continuous cannot be configured.
- Disable serverless auto-optimisation for **at most once** work. Leave it on otherwise.
- **High memory is notebook tasks only**, set in the notebook environment panel.

> ON THE EXAM
>
> That completes Section 1, the largest section on the exam at 22%. Section 2 is next, and it is much smaller.

A job has an ingestion task followed by three expensive transformation tasks. On most days no new files arrive, and the transformations run anyway, producing identical output at significant cost.

The team wants the transformations to run only when ingestion actually loaded new records.

What is the right construction?

A Set the transformations' run-if condition to At least one succeeded.

B Have ingestion set a task value with the row count, then use an If/else task to branch on it.

C Configure the ingestion task to fail when no new files arrive, so downstream tasks are skipped.

D Add a For each task over the ingested files, so transformations run once per file.

◆ PAUSE AND DECIDE

What is the right construction?

A Set the transformations' run-if condition to At least one succeeded.

Run if branches on outcome. Ingestion succeeds whether or not it found anything.

B Have ingestion set a task value with the row count, then use an If/else task to branch on it.

Branching on a value computed by an earlier task is exactly what task values and If/else exist for.

C Configure the ingestion task to fail when no new files arrive, so downstream tasks are skipped.

Makes a normal day look like an incident, and pollutes alerting and run history.

D Add a For each task over the ingested files, so transformations run once per file.

Solves a different problem, and changes the transformation from one run to many.

ON THE EXAM

Outcome or value is the first question. Nothing failed here, so run-if conditions cannot express what is wanted.

A serverless job task calls a third-party payment provider's API to issue refunds. During a network incident the task failed, and the team later found that a number of customers had been refunded twice.

The task itself has no retry policy configured, and the team is confident their code runs once per invocation.

What most likely happened, and what should they change?

A Serverless auto-optimisation retried the failed task. Disable auto-optimisation for this task.

B The job was in continuous mode, which retries with exponential backoff. Switch to a scheduled trigger.

C Nothing retried it. The provider's API must have duplicated the requests.

D The default retry policy is three attempts. Set max retries to zero.

◆ PAUSE AND DECIDE

What most likely happened, and what should they change?

A Serverless auto-optimisation retried the failed task. Disable auto-optimisation for this task.

On serverless, auto-optimisation retries failed tasks by default. No retry policy is needed for a retry to happen.

B The job was in continuous mode, which retries with exponential backoff. Switch to a scheduled trigger.

Nothing describes a continuous job, and refunds are not a continuous workload.

C Nothing retried it. The provider's API must have duplicated the requests.

Blames the third party for documented default behaviour on this platform.

D The default retry policy is three attempts. Set max retries to zero.

There is no default of three, and setting max retries does not disable auto-optimisation.

ON THE EXAM

This is the exact case the docs name: a workload that must run at most once because it is not idempotent.

A nightly job must process files for whichever of 40 country codes received new data that day. The list changes daily. Currently there are 40 hard-coded tasks, most of which exit immediately.

The team wants one construct that processes only the countries with new data.

What should they build?

A An If/else task per country, each branching on whether that country has data.

B A task that produces the list of active countries, then a For each task with a nested processing task.

C A single task that loops over countries inside the notebook code.

D A For each task alone, with the country list as its input array.

◆ PAUSE AND DECIDE

What should they build?

A An If/else task per country, each branching on whether that country has data.

Replaces 40 tasks with 80. The count is the problem, not the exit behaviour.

B A task that produces the list of active countries, then a For each task with a nested processing task.

Dynamically generated input array plus a nested task. Exactly what For each is for.

C A single task that loops over countries inside the notebook code.

Works, and hides the loop from the job graph, so per-country failures and retries disappear.

D A For each task alone, with the country list as its input array.

A For each task with no nested task has nothing to run.

ON THE EXAM

D is the structural trap. A For each task is half a construction; without a nested task it does nothing.

> A serverless notebook task fails intermittently with out-of-memory errors on the largest partitions. The logic cannot easily be restructured before a deadline. A colleague suggests configuring high memory.
>
> A second task in the same job is a Python wheel task that has the same problem.

What is true about this suggestion?

A. High memory can be applied to both tasks in the job settings.

B. High memory applies to the notebook task only, set in its environment panel, and bills at a higher rate.

C. High memory is a cluster policy setting and does not apply to serverless.

D. High memory is enabled automatically by auto-optimisation, so no change is needed.

◆ PAUSE AND DECIDE

What is true about this suggestion?

A. High memory can be applied to both tasks in the job settings.

Wrong twice: it is notebook-only, and it is configured on the notebook environment, not the job.

B. High memory applies to the notebook task only, set in its environment panel, and bills at a higher rate.

All three constraints matter here, and the wheel task will need a different remedy.

C. High memory is a cluster policy setting and does not apply to serverless.

It is specifically a serverless feature.

D. High memory is enabled automatically by auto-optimisation, so no change is needed.

Auto-optimisation selects resources, but high memory is an explicit opt-in.

ON THE EXAM

The wheel task is in the question for a reason. Half the problem has a quick fix and half does not.

> A Structured Streaming job stops every few weeks with a schema-related error whenever the upstream producer adds a column. An engineer restarts it manually and it picks up the new schema and continues without further intervention.
>
> They want this to stop requiring a human.

What is the appropriate change?

A Configure a retry policy, since schema evolution assumes automatic retries.

B Pin the schema explicitly so new columns are ignored.

C Disable schema evolution and handle new columns in a separate job.

D Move the stream to a continuous job, which never fails on schema changes.

◆ PAUSE AND DECIDE

What is the appropriate change?

A Configure a retry policy, since schema evolution assumes automatic retries.

The documented design: the stream stops on a schema change and the retry restarts it with the new schema.

B Pin the schema explicitly so new columns are ignored.

Stops the failures by discarding the new data, which is a silent loss.

C Disable schema evolution and handle new columns in a separate job.

Adds a second job to solve what a retry policy already handles.

D Move the stream to a continuous job, which never fails on schema changes.

A continuous job does retry with backoff, but the claim that it never fails is false.

ON THE EXAM

The giveaway is that a manual restart already fixes it completely. That is a retry, done by a person.

> A regulated financial team must move a batch workload to serverless jobs. Their security policy requires that all outbound network traffic from compute be restricted to an approved list of endpoints.
>
> They also want cleanup to run whether or not the main task succeeds.

Which statement correctly assesses their plan?

A. Both requirements are met: configure egress rules on the serverless environment, and use All done for cleanup.

B. Egress cannot be controlled on serverless for workflows, so that requirement blocks it. All done handles the cleanup.

C. Egress is controlled by the workspace network policy, so serverless is fine. Use At least one failed for cleanup.

D. Both requirements are unmet: serverless has no egress control and no way to run a task unconditionally.

◆ PAUSE AND DECIDE

Which statement correctly assesses their plan?

A. Both requirements are met: configure egress rules on the serverless environment, and use All done for cleanup.

The cleanup half is right. Serverless for workflows does not support controlling egress traffic.

B. Egress cannot be controlled on serverless for workflows, so that requirement blocks it. All done handles the cleanup.

The docs state that jobs have full internet access because egress cannot be controlled. All done runs regardless of outcome.

C. Egress is controlled by the workspace network policy, so serverless is fine. Use At least one failed for cleanup.

Wrong on both: the limitation is stated, and At least one failed skips cleanup on success.

D. Both requirements are unmet: serverless has no egress control and no way to run a task unconditionally.

Half right. All done exists precisely for unconditional cleanup.

ON THE EXAM

Two requirements, one met and one not. Options that make both true or both false are the easiest to write and the easiest to spot.

INGESTION · DOMAIN 2 - DATA INGESTION AND ACQUISITION

Formats and sources

Section 2. Eight file formats, four message buses, and which mechanism reads them.

SECTION 2 of 10 WEIGHT 7% TOPIC Ingestion

admin@sthithapragna.com

Some formats **carry** their schema. Others make you guess. Almost every ingestion decision follows from which kind you have.

admin@sthithapragna.com

Eight file formats, in two groups

FORMAT	WHAT YOU GET
Parquet	Columnar, compressed, schema embedded. The default for analytics at rest.
ORC	Columnar, schema embedded. Common when arriving from a Hadoop estate.
Avro	Row-oriented, schema embedded. Common on message buses.
JSON	Text. Schema must be inferred or supplied. Nested by nature.
CSV	Text. Schema must be inferred or supplied. Flat and fragile.
XML	Text. Schema must be inferred or supplied. Deeply nested.
Text	One string column. You do the parsing.
Binary	The file's bytes, plus path and metadata. Nothing is parsed.

ON THE EXAM

The same eight are supported by Auto Loader and by COPY INTO. If a question offers a format outside this list, that is the discriminator.

What Parquet, ORC and Avro give you

- The schema is **read from the file**, not guessed. No sampling, no inference cost.
- Columnar formats let a query **read only the columns it needs**, which is why they suit analytics.
- Avro is row-oriented instead, which suits writing one record at a time rather than scanning columns.
- They are not human-readable, so debugging a bad file means tooling rather than a text editor.

> ON THE EXAM
>
> Row-oriented against columnar is the Avro versus Parquet question. Write one record at a time, or scan a few columns across many rows.

Where the cost and the risk live

- Schema must be **inferred from a sample** or supplied. Inference reads data before any work happens.
- A column that looks numeric in the sample and is not later will fail, or land in rescued data.
- Supplying a schema removes both the cost and the guesswork, at the price of maintaining it.
- JSON and XML are nested; CSV is flat. That shapes the transformation work far more than the parsing.

> ON THE EXAM
>
> Schema evolution is only interesting for text formats. A Parquet file that gains a column tells you so; a CSV file just has an extra comma.

Text and binary as formats

- The **text** format returns one string column per line. Log files and anything with a custom layout.
- The **binary file** format returns the file's **content, path, length and modification time**.
- That is how you ingest images, PDFs and audio into a table without parsing them.
- Do not use them as a shortcut for a real format. That bypasses inference, evolution and rescued data.

> ON THE EXAM
>
> Databricks' stated guidance is to set the real format rather than reading everything as text or binary first. Doing so pushes parsing into your own code.

Four ways to get files in

MECHANISM	CHARACTER
Auto Loader	Streaming source. Incremental, exactly-once, scales to billions of files.
COPY INTO	SQL command. Idempotent, re-triable, skips files already loaded.
read_files	SQL table-valued function. Reads files directly in a query.
Lakeflow Connect	Governed connectors. Extends Auto Loader to sources such as SFTP.

ON THE EXAM

Auto Loader and COPY INTO support the same eight formats. What differs is how they track what has already been read.

Auto Loader against COPY INTO

AUTO LOADER

A stream over a directory

- Tracks files in a RocksDB store in the checkpoint
- Handles millions of files per hour
- Schema inference, evolution and rescued data
- Recommended inside Lakeflow pipelines

COPY INTO

A command you run

- Tracks loaded files in the target table
- Idempotent: reruns skip what was loaded
- Schema inference, mapping, merging, evolution
- Best for a bounded set of files

THE CURRENT ADVICE

Databricks now recommends that SQL users reach for streaming tables rather than COPY INTO, for a more scalable and robust ingestion experience.

Four streaming sources

SOURCE	NOTES
Apache Kafka	The reference implementation. Event Hubs also connects through it.
Amazon Kinesis	AWS-native. Enhanced fan-out mode matters for low latency.
Google Pub/Sub	Read through a streaming function, inside pipelines.
Apache Pulsar	Supported through a streaming function, in Public Preview.

ON THE EXAM

A message bus is unbounded and append-only, so a streaming table is nearly always the right target for one.

Kafka gives you bytes, not data

- The key and value are **always returned as byte arrays**. Always, regardless of what was written.
- You must **cast to string**, or use a function such as from_avro, to get anything usable.
- The reader also returns topic, partition, offset, timestamp and timestamp type.
- Offset and partition are what make the read replayable, and they belong in your bronze table.

> ON THE EXAM
>
> A question showing a Kafka read whose value column is treated as a string without a cast is describing code that does not work.

The table-valued functions and their limits

- **read_kafka** works in **both batch and streaming** queries.
- **read_pulsar** is **streaming only**, and in Public Preview.
- **read_pubsub** is streaming only, and works **only inside Lakeflow pipelines**.
- Streaming in SQL at all requires a pipeline, or a streaming table in Databricks SQL.

> ON THE EXAM
>
> These asymmetries are exactly the kind of detail a professional exam tests. Kafka is the flexible one; the others are narrower.

The append-only assumption

- A streaming query over a Delta table picks up new records **when a version is committed**.
- **By default, streaming queries expect the source to contain only appended records.**
- An update or delete in the source will fail the stream, not silently pass through.
- For sources with updates and deletes, Databricks recommends pipelines with **AUTO CDC**.

> ON THE EXAM
>
> This is the same idea as module 5's process-once semantics, seen from the source side rather than the target side.

Lakeflow Connect

- Governed connectors that **extend Auto Loader** to sources beyond object storage.
- The **SFTP** connector supports the same eight formats, with incremental ingestion and exactly-once.
- Credentials and access are governed by **Unity Catalog** rather than living in a notebook.
- Schema inference, evolution and data rescue come along too. It behaves like Auto Loader.

> ON THE EXAM
>
> When a scenario names a legacy delivery mechanism such as an SFTP drop, this is the modern answer rather than a scripted download.

admin@sthithapragna.com

What to carry into the exam

- Parquet, ORC and Avro **carry a schema**. JSON, CSV and XML do not, and that is where inference cost and evolution risk live.
- Kafka returns **bytes**. Cast the key and value explicitly.
- Auto Loader tracks files in its **checkpoint**; COPY INTO tracks them in the **target table**.
- Databricks now points SQL users at **streaming tables** rather than COPY INTO.
- Delta streaming sources are **append-only by default**. Updates need AUTO CDC.

> ON THE EXAM
>
> Set the real format rather than reading as text first. That one habit preserves inference, evolution and rescued data.

> A team reads a Kafka topic carrying JSON payloads and writes to a bronze streaming table. The code selects the value column directly and applies a JSON parsing function to it.
>
> The job fails with a type error, and the developer insists the topic definitely contains JSON text.

What is wrong?

A. The topic must be configured with a schema registry before Databricks can read it.

B. The value column is returned as binary and must be cast to string before parsing.

C. JSON payloads require the Avro deserialiser, not the JSON parser.

D. Streaming reads from Kafka must specify a schema location, as with Auto Loader.

◆ PAUSE AND DECIDE

What is wrong?

A The topic must be configured with a schema registry before Databricks can read it.

A registry helps with Avro. It is not required to read a topic at all.

B The value column is returned as binary and must be cast to string before parsing.

The Kafka reader always returns key and value as byte arrays, whatever was written.

C JSON payloads require the Avro deserialiser, not the JSON parser.

Different format entirely. from_avro is for Avro payloads.

D Streaming reads from Kafka must specify a schema location, as with Auto Loader.

Schema location is an Auto Loader option for files, not a Kafka one.

ON THE EXAM

The developer is right about the topic and wrong about the reader. What was written and what is returned are different questions.

A pipeline ingests daily CSV extracts. Schema inference samples the files and infers a product code column as an integer. Three months in, one supplier begins sending codes that contain letters.

Records from that supplier stop appearing in the silver table. Nothing errors, and the pipeline reports success.

What is the most likely explanation?

A The malformed values are being captured in the rescued data column rather than the typed column.

B CSV files cannot contain mixed types, so the whole file is being skipped.

C Schema evolution added a new column, and downstream queries are reading the old one.

D The supplier's files are being deduplicated because the codes now collide.

◆ PAUSE AND DECIDE

What is the most likely explanation?

A The malformed values are being captured in the rescued data column rather than the typed column.

Values that do not fit the inferred type are rescued rather than failing, which is why nothing errored.

B CSV files cannot contain mixed types, so the whole file is being skipped.

Files are not skipped for type mismatches, and nothing reported a skip.

C Schema evolution added a new column, and downstream queries are reading the old one.

Closer to plausible, but the column already existed. Its type is the problem.

D The supplier's files are being deduplicated because the codes now collide.

Nothing here deduplicates on a product code.

ON THE EXAM

Silence again. Rescued data is designed to avoid losing records, and it succeeds so quietly that nobody notices until a report is short.

A team must ingest a one-off historical load of 900,000 Parquet files from a decommissioned system, then never read that location again. They are comfortable in SQL and want the simplest thing that works.

A colleague argues they should build an Auto Loader stream because it is the recommended tool.

What is the better assessment?

A The colleague is right; Auto Loader is always preferred for file ingestion.

B COPY INTO suits a bounded one-off load, is idempotent on rerun, and needs no checkpoint to maintain.

C Neither works at this scale; the files must be loaded with a Spark batch read and an insert.

D Auto Loader is required because COPY INTO does not support Parquet.

◆ PAUSE AND DECIDE

What is the better assessment?

A The colleague is right; Auto Loader is always preferred for file ingestion.

The recommendation is for incremental ingestion of arriving files. This load is bounded and finished.

B COPY INTO suits a bounded one-off load, is idempotent on rerun, and needs no checkpoint to maintain.

Idempotence covers the retry risk, and there is no ongoing stream to operate afterwards.

C Neither works at this scale; the files must be loaded with a Spark batch read and an insert.

Both handle far more than this. Auto Loader alone scales to billions of files.

D Auto Loader is required because COPY INTO does not support Parquet.

Both support the same eight formats, Parquet included.

ON THE EXAM

Recommendations have scope. Auto Loader is recommended for incremental ingestion; this load has a defined end.

A silver streaming job reads from a bronze Delta table. An operations team began running corrective UPDATE statements against bronze to fix a data quality issue.

The silver stream now fails on every run with an error about the source table.

What is happening, and what is the recommended remedy?

A Streaming reads expect append-only sources. Use a pipeline with AUTO CDC to handle updates.

B The corrective updates broke the checkpoint. Delete it and restart the stream.

C Streaming reads require change data feed to be enabled on every Delta source.

D The stream should be switched to a batch read, since Delta cannot stream after an update.

◆ PAUSE AND DECIDE

What is happening, and what is the recommended remedy?

A Streaming reads expect append-only sources. Use a pipeline with AUTO CDC to handle updates.

The default assumption is append-only, and AUTO CDC is the documented answer for sources with updates and deletes.

B The corrective updates broke the checkpoint. Delete it and restart the stream.

Deleting the checkpoint reprocesses everything, and the next update breaks it again.

C Streaming reads require change data feed to be enabled on every Delta source.

Not required for ordinary append-only streaming reads.

D The stream should be switched to a batch read, since Delta cannot stream after an update.

Delta streams fine. The constraint is about updates in the source, not about Delta.

ON THE EXAM

The failure is protective. Passing an update through as if it were an append would silently corrupt the target.

A regulated insurer receives daily claim files by SFTP from twelve brokers, in a mix of CSV and XML. A scheduled virtual machine currently downloads them with a shell script and copies them to object storage, using a shared password stored in the script.

Auditors have raised the credential handling, and the team also wants schema evolution and rescued data.

What is the strongest recommendation?

A. Keep the script but move the password into a Databricks secret scope.

B. Use the Lakeflow Connect SFTP connector, which extends Auto Loader with Unity Catalog governance.

C. Have the brokers switch to writing directly into cloud storage, then use Auto Loader.

D. Use COPY INTO against the SFTP path directly, since it supports both formats.

◆ PAUSE AND DECIDE

What is the strongest recommendation?

A. Keep the script but move the password into a Databricks secret scope.
Fixes the credential and leaves the script, with no schema evolution or rescued data.

B. Use the Lakeflow Connect SFTP connector, which extends Auto Loader with Unity Catalog governance.
Governed credentials, both formats supported, and Auto Loader's evolution and rescue come along.

C. Have the brokers switch to writing directly into cloud storage, then use Auto Loader.
Technically ideal and depends on twelve external parties changing their systems.

D. Use COPY INTO against the SFTP path directly, since it supports both formats.
COPY INTO reads cloud storage and volumes, not SFTP servers.

ON THE EXAM

Three requirements: governed credentials, both formats, and Auto Loader's schema handling. Only one option delivers all three.

> A team must ingest scanned claim documents as PDFs so a downstream process can extract text. They also need to record which file each extraction came from and when it landed.
>
> A junior engineer proposes reading them with the text format and splitting on newlines.

What should they use instead?

A. The text format is correct; PDFs are text files with binary headers.

B. The binary file format, which returns content along with path, length and modification time.

C. The XML format, since PDF is a structured document format.

D. COPY INTO with a custom parser registered as a UDF.

◆ PAUSE AND DECIDE

What should they use instead?

A The text format is correct; PDFs are text files with binary headers.

PDFs are binary. Reading them as text corrupts the content.

B The binary file format, which returns content along with path, length and modification time.

Ingests the bytes without parsing, and supplies exactly the provenance columns the team asked for.

C The XML format, since PDF is a structured document format.

Structured is not the same as XML.

D COPY INTO with a custom parser registered as a UDF.

COPY INTO does not run a parser during load, and a UDF here would be the expensive option from module 4.

ON THE EXAM

The second requirement decides it. Binary file is the only option that returns the path and modification time alongside the content.

INGESTION · DOMAIN 2 - DATA INGESTION AND ACQUISITION

Auto Loader in depth

Section 2. Schema evolution, rescued data, file detection, and one pipeline for batch and streaming.

SECTION 2 of 10 WEIGHT 7% TOPIC Auto Loader

admin@sthithapragna.com

Auto Loader is a stream that remembers which files it has read. **Everything else follows from that memory.**

admin@sthithapragna.com

How exactly-once actually works

- File metadata is persisted in a **key-value store inside the checkpoint**, not by listing what already landed.
- So a file is processed once, and a restart resumes rather than reprocessing.
- It scales to **billions of files** for a backfill, and **millions of files per hour** ongoing.
- Delete the checkpoint and that memory is gone. Every file looks new again.

> ON THE EXAM
>
> This is why Auto Loader beats a hand-written 'list the directory and compare' loop, which gets slower as the directory grows.

What you actually configure

OPTION	PURPOSE
`cloudFiles.format`	The real source format. Always set it.
`cloudFiles.schemaLocation`	Where the inferred schema is stored and updated.
`cloudFiles.schemaEvolutionMode`	What happens on a new column.
`cloudFiles.schemaHints`	Override the inferred type for named columns.
`cloudFiles.rescuedDataColumn`	Name of the column holding what did not parse.
`cloudFiles.useManagedFileEvents`	Use the managed file events service.

ON THE EXAM

Schema location and checkpoint location are separate settings storing different things. A production stream needs both.

Five modes, and what each does to the stream

MODE	BEHAVIOUR ON A NEW COLUMN
`addNewColumns`	Stream fails. Column added to the schema. Existing types unchanged.
`rescue`	Schema never evolves, stream never fails. New columns go to rescued data.
`failOnNewColumns`	Stream fails. No restart until you update the schema or remove the file.
`none`	New columns ignored, nothing rescued unless asked. Stream does not fail.
`addNewColumnsWithTypeWidening`	As addNewColumns, plus safe type widening. Unsafe changes rescued.

ON THE EXAM

Notice that addNewColumns fails the stream. The failure is part of the evolution, not an alternative to it.

Why addNewColumns failing is not a bug

- The stream stops with an unknown field error when a new column appears.
- **Before it throws**, it infers from the latest micro-batch and **updates the schema location**, merging new columns to the end.
- On **restart**, the evolved schema is in place and the stream carries on.
- So Databricks recommends running Auto Loader as a job configured to **restart automatically**.
- Without automatic restart, the stream sits failed and somebody has to notice.

> ON THE EXAM
>
> This is the module 9 point exactly: schema evolution assumes retries are configured. The retry is the second half of the mechanism.

The default depends on whether you supplied a schema

- **No schema provided:** the default is **addNewColumns**.
- **Schema provided:** the default is **none**.
- addNewColumns is **not allowed** alongside a provided schema at all.
- It does work if you supply your schema as a **schema hint** instead.

> ON THE EXAM
>
> Widely-repeated blog posts claim the default is to fail, like failOnNewColumns. The documentation says otherwise. Trust the docs.

What lands in the rescued column

- Three causes: a **column missing from the schema**, a **type mismatch**, or a **case mismatch**.
- It holds a **JSON blob** containing the rescued columns and **the source file path** of the record.
- That file path is what makes it possible to trace a bad record back to its origin.
- Case differences are rescued by default. Set the reader to case-insensitive to change that.

> ON THE EXAM
>
> Nothing is lost and nothing errors, which is the point. It is also why a column full of rescued data can go unnoticed for months.

Telling it what you already know

- Override the inferred type for named columns, without supplying a whole schema.
- The right tool when you know a column should be a timestamp and inference keeps calling it a string.
- For Parquet they do **not cast**. They tell the reader to read the column as that type, and a mismatch is rescued.
- They also let you keep addNewColumns while still pinning the columns you care about.

> ON THE EXAM
>
> Hints are the middle ground between full inference and a hard-coded schema. They fix what you know without freezing what you do not.

Directory listing against file events

DIRECTORY LISTING

The default

- Lists the input directory to find new files
- No extra permissions beyond reading the data
- Optimised, and detects lexical ordering
- Cost grows with the size of the directory

FILE EVENTS

The recommendation

- One managed service per external location
- One shared queue serving many consumers
- Backfill and tuning handled for you
- Enable on the location, then set the option

REASSURING

You can switch detection modes across stream restarts and keep exactly-once guarantees. It is not a one-way decision.

Options it makes unnecessary

- **Backfill interval** is handled automatically for locations with file events enabled.
- **Fetch parallelism** is tuned for you. There is no manual knob.
- The choice between incremental listing and notifications disappears. There is one mode.
- Notification cleanup is handled, including when a stream is deleted or fully refreshed.

> ON THE EXAM
>
> If an answer proposes tuning fetch parallelism or setting a backfill interval on a file-events stream, it is describing an older configuration.

Run it at least once a week

- The file events service **caches recently created files**. That cache expires.
- Databricks recommends running Auto Loader streams **at least once every seven days**.
- Otherwise the stored read position becomes invalid and a **full directory listing** is required to resync.
- A full listing also happens when starting a new stream, or migrating from another detection mode.

> ON THE EXAM
>
> Correctness is not at risk here. The cost is a slow first run after a long gap, which matters when the directory is enormous.

One definition, batch or streaming

- The **same Auto Loader definition** runs continuously, or as a batch with the available now trigger.
- The **checkpoint carries over**, so a batch run picks up exactly where the last one stopped.
- That is how one pipeline serves an hourly schedule today and continuous ingestion later, with no rewrite.
- Writing to Delta makes the target **append-only and transactional**, which is what downstream streams expect.

> ON THE EXAM
>
> This is the section 2 objective: an append-only pipeline handling both batch and streaming data. It is one definition and a trigger choice.

What to carry into the exam

- **addNewColumns fails the stream**, updates the schema first, and resumes on restart. Configure automatic retries.
- Default is **addNewColumns** without a schema, **none** with one. Hints avoid that trap.
- Rescued data catches **missing columns, type mismatches and case mismatches**, with the source file path.
- **File events** is the recommendation. Run streams at least weekly or pay for a full listing.
- Same definition, batch or streaming. The trigger decides, and the checkpoint carries over.

> ON THE EXAM
>
> That completes Section 2 at 7%. Section 3 is transformation and data quality, at 10%.

> An Auto Loader stream ingests JSON events with no schema supplied. A producer adds a new field. The stream fails overnight with an unknown field error and an engineer restarts it manually the next morning, after which it runs normally and the new column is present.
>
> This has happened four times this quarter. The team believes Auto Loader is unreliable.

What should they conclude and do?

A. The failure is part of how addNewColumns evolves the schema. Configure the job to restart automatically.

B. Switch to failOnNewColumns so the schema is locked and cannot change unexpectedly.

C. The schema location is corrupt. Delete it and let inference start again.

D. Supply an explicit schema so no inference happens and the stream never fails.

◆ PAUSE AND DECIDE

What should they conclude and do?

A The failure is part of how addNewColumns evolves the schema. Configure the job to restart automatically.

The schema is updated before the error is thrown, so the restart is the second half of the mechanism.

B Switch to failOnNewColumns so the schema is locked and cannot change unexpectedly.

Makes it strictly worse. That mode will not restart at all without manual intervention.

C The schema location is corrupt. Delete it and let inference start again.

It is working correctly. The restart proves the updated schema was written.

D Supply an explicit schema so no inference happens and the stream never fails.

Switches the default to none, so new fields are silently ignored rather than ingested.

ON THE EXAM

The clue is that a plain restart fixes it every time and needs nothing else. That is a handover, not a fault.

A finance team notices a quarterly report is missing a revenue adjustment field that the source system has been sending for two months. The Auto Loader stream has never failed and reports healthy throughput.

The stream was configured with an explicit schema when it was built, to guarantee stable column types.

What is happening?

A The field is in the rescued data column, because a type mismatch prevented parsing.

B Providing a schema set the evolution mode default to none, so new columns are ignored.

C The schema location has not been updated because the checkpoint was cleared.

D The stream is using failOnNewColumns and has been silently skipping the offending files.

◆ PAUSE AND DECIDE

What is happening?

A The field is in the rescued data column, because a type mismatch prevented parsing.

Plausible for a type problem. Here the column is entirely new, and none mode ignores it without rescuing.

B Providing a schema set the evolution mode default to none, so new columns are ignored.

The two-defaults rule. With a schema supplied, new columns are ignored and nothing is rescued unless asked.

C The schema location has not been updated because the checkpoint was cleared.

A cleared checkpoint reprocesses files. It does not silently drop a column.

D The stream is using failOnNewColumns and has been silently skipping the offending files.

That mode fails loudly. Nothing about it is silent.

ON THE EXAM

The team made a reasonable decision for a good reason and inherited a default they never chose. That is the shape of this trap.

> A regulated pipeline must never ingest a record it has not validated, and any unexpected field must stop the pipeline for review before data reaches the target. Uptime is secondary to correctness.
>
> The team currently uses rescue mode because it was described as the safest option.

Which mode fits the requirement?

A. Keep rescue mode; unexpected fields are captured in rescued data and can be reviewed later.

B. Use failOnNewColumns, which stops the stream until the schema is deliberately updated.

C. Use addNewColumns, which fails the stream on each new column.

D. Use none, so unexpected fields are ignored and never reach the target.

◆ PAUSE AND DECIDE

Which mode fits the requirement?

A Keep rescue mode; unexpected fields are captured in rescued data and can be reviewed later.

Data keeps flowing to the target, which is exactly what the requirement forbids.

B Use failOnNewColumns, which stops the stream until the schema is deliberately updated.

It will not restart until a human updates the schema or removes the file. That is the review gate the requirement asks for.

C Use addNewColumns, which fails the stream on each new column.

It fails, then evolves and resumes on restart. The new field is ingested without review.

D Use none, so unexpected fields are ignored and never reach the target.

Nothing stops, and nobody reviews anything.

ON THE EXAM

Safest depends on what you are protecting. Rescue protects uptime; failOnNewColumns protects the target.

A stream ingests CSV files with an explicit rescued data column enabled. A downstream analyst reports that an entire customer identifier column is now null, starting from a specific date. The rescued data column has values for those rows.

The upstream team confirms they made a cosmetic change: the header now reads CustomerID rather than customerid.

What happened, and what is the least disruptive fix?

A. A case mismatch routed the field to rescued data. Set the reader to case-insensitive.

B. The column was renamed, so a full refresh is needed to rebuild the schema.

C. A type mismatch occurred because the new header changed the inferred type.

D. The schema location must be deleted so inference picks up the new header.

◆ PAUSE AND DECIDE

What happened, and what is the least disruptive fix?

A A case mismatch routed the field to rescued data. Set the reader to case-insensitive.

Case mismatch is one of the three documented rescue causes, and the case-insensitive reader option addresses it directly.

B The column was renamed, so a full refresh is needed to rebuild the schema.

Nothing was renamed in substance, and a full refresh is far more disruptive than a reader option.

C A type mismatch occurred because the new header changed the inferred type.

A header's capitalisation does not change a column's type.

D The schema location must be deleted so inference picks up the new header.

Would work, discards the accumulated schema, and leaves the same trap for next time.

ON THE EXAM

Case is the third rescue cause and the one people forget. A purely cosmetic upstream change can empty a column.

> A quarterly compliance extract runs an Auto Loader stream against a bucket holding several million files. It is triggered four times a year. Each run now takes hours before it begins processing, though it processes very little data.
>
> The stream uses file notification mode with file events enabled on the external location.

What explains the slow start?

A. The file events cache has expired, so Auto Loader performs a full directory listing to resynchronise.

B. File events are not supported for quarterly schedules and the stream has fallen back permanently.

C. The checkpoint has grown too large to load efficiently after three months.

D. Backfill interval must be set manually when a stream runs less than daily.

◆ PAUSE AND DECIDE

What explains the slow start?

A The file events cache has expired, so Auto Loader performs a full directory listing to resynchronise.

Streams should run at least once every seven days. A quarterly run is far outside that, so the read position is invalid.

B File events are not supported for quarterly schedules and the stream has fallen back permanently.

There is no such restriction, and the fallback is per-run rather than permanent.

C The checkpoint has grown too large to load efficiently after three months.

Checkpoint size tracks files processed, and this stream processes very little.

D Backfill interval must be set manually when a stream runs less than daily.

Backfill is handled automatically for locations with file events enabled.

ON THE EXAM

Correctness is unaffected here. The symptom is cost and time, which is what makes it hard to diagnose from the output alone.

A team ingests partner files into a bronze Delta table. Today the business wants an hourly schedule to keep costs down. Within a year they expect to move to continuous ingestion for a near-real-time dashboard.

They want to avoid rewriting the ingestion when that happens, and no data may be reprocessed at the switchover.

What should they build now?

A. A batch job reading the directory and comparing against the target to find new files.

B. An Auto Loader stream with the available now trigger, scheduled hourly, switching the trigger later.

C. COPY INTO on an hourly schedule, replaced by an Auto Loader stream when continuous is needed.

D. A continuous Auto Loader stream now, throttled with a long trigger interval to control cost.

◆ PAUSE AND DECIDE

What should they build now?

A. A batch job reading the directory and comparing against the target to find new files.
Hand-built bookkeeping that gets slower as the directory grows, and a full rewrite later.

B. An Auto Loader stream with the available now trigger, scheduled hourly, switching the trigger later.
One definition, one checkpoint. Changing the trigger is not a rewrite, and the checkpoint prevents reprocessing.

C. COPY INTO on an hourly schedule, replaced by an Auto Loader stream when continuous is needed.
Works today and forces the rewrite they asked to avoid, with no checkpoint to carry over.

D. A continuous Auto Loader stream now, throttled with a long trigger interval to control cost.
Keeps compute running continuously, which is the cost they were trying to avoid.

ON THE EXAM

This is the Section 2 objective in a scenario: one append-only pipeline serving batch today and streaming later.

TRANSFORMATION · DOMAIN 3 - DATA TRANSFORMATION, CLEANSING, QUALITY

Window functions, joins and aggregations

Section 3. The transformation surface, and the Databricks parts of it.

SECTION 3 of 10 WEIGHT 10% TOPIC Transformation

admin@sthithapragna.com

A group by **collapses** rows. A window function **keeps** them, and adds an answer computed across their neighbours.

A window specification has three parts

- **PARTITION BY** divides rows into independent groups. Calculations reset at each boundary.
- **ORDER BY** sets the order within a partition. Ranking and offsets are meaningless without it.
- **The frame clause** narrows to a sliding subset of the partition.
- Aliases exist: SORT BY for ORDER BY, DISTRIBUTE BY for PARTITION BY, and CLUSTER BY when there is no ORDER BY.

> ON THE EXAM
>
> Partition by does not collapse rows the way group by does. It only decides where the calculation restarts.

Three functions that look similar

FUNCTION	BEHAVIOUR ON TIES
`row_number`	Always distinct. Ties are broken arbitrarily. 1, 2, 3, 4.
`rank`	Ties share a rank, then a gap follows. 1, 2, 2, 4.
`dense_rank`	Ties share a rank, no gap follows. 1, 2, 2, 3.

ON THE EXAM

Row number is the one to reach for when deduplicating, precisely because it never ties. The other two are for genuine ranking.

Deduplicating with row number

- Partition by the **key that should be unique**, order by whatever decides which copy wins.
- Assign a row number, then keep only the rows numbered one.
- Ordering by a timestamp descending keeps the latest record per key.
- Ordering by nothing meaningful makes the choice arbitrary, and it will change between runs.

> ON THE EXAM
>
> This is deduplication that chooses which copy survives. Distinct cannot do that, because it has no way to prefer one row over another.

QUALIFY filters window results directly

- Window functions are evaluated **after** WHERE and HAVING, so you cannot filter on a rank there.
- **QUALIFY** filters window function results in the same query block. No subquery, no CTE.
- At least one window function must appear in the SELECT list or in the QUALIFY clause itself.
- It **cannot contain aggregate functions**, and it is not available in open-source Spark.

> ON THE EXAM
>
> Available from DBR 10.4 LTS. A genuinely Databricks-specific clause, which makes it a clean discriminator in a question.

ROWS against RANGE

	BEHAVIOUR
ROWS	Physical offsets. Counts a fixed number of rows before or after the current one.
RANGE	Logical offsets on the ORDER BY value. Rows sharing a value form one peer group.
Requirement	RANGE needs an ORDER BY with exactly one expression, or it errors.
The default	With ORDER BY and no explicit frame: RANGE, unbounded preceding to current row.

ON THE EXAM

The default being RANGE rather than ROWS is the detail that catches people. With duplicate ordering values, a running total jumps rather than climbing one row at a time.

Seven join types

TYPE	WHAT COMES BACK
INNER	Rows matching in both. This is the default when you write JOIN alone.
LEFT OUTER	All left rows, with nulls where the right has no match.
RIGHT OUTER	All right rows, with nulls where the left has no match.
FULL OUTER	All rows from both, nulls on whichever side is missing.
LEFT SEMI	Left rows that have a match. No columns from the right.
LEFT ANTI	Left rows that have no match.
CROSS	The Cartesian product. Every left row against every right row.

ON THE EXAM

Semi and anti are filters expressed as joins. Neither adds columns, and neither can multiply rows.

The filter that does not multiply rows

- An inner join used as a filter **duplicates left rows** when the right side has more than one match.
- A **semi join** returns each qualifying left row exactly once, whatever the right side contains.
- An **anti join** is the clean way to find what is missing, and it handles nulls better than NOT IN.
- Both express intent. The reader can see you wanted a filter and not an enrichment.

> ON THE EXAM
>
> A row count that grew after a join that was only supposed to filter is the signature of this mistake.

ON, USING, and the one to avoid

- **ON** takes any boolean expression, including inequalities and multiple conditions.
- **USING** names shared columns and returns them once rather than twice.
- **NATURAL** matches on **every** column with a shared name, implicitly.
- So adding a column upstream can silently change what a natural join matches on.

> ON THE EXAM
>
> Natural join is in the language and is a liability in a pipeline, because the join condition is decided by column names you do not control.

Beyond a plain group by

- **GROUPING SETS** computes several groupings in one pass, rather than unioning separate queries.
- **ROLLUP** adds subtotals up a hierarchy. **CUBE** adds every combination.
- A **FILTER** clause on an aggregate passes only matching rows to that function.
- That gives conditional aggregates without a case expression inside every function.

> ON THE EXAM
>
> One pass over the data instead of several. That is the efficiency the objective is asking about.

What changes when the input is unbounded

- Ranking across an entire partition needs to see every row, so it does not translate to an unbounded stream.
- Time-based windows plus a **watermark** are the streaming equivalent of a bounded aggregation.
- Chained stateful operations support **append output mode only**, from module 6.
- Deduplication has a streaming form, and it needs an event time column so state can be evicted.

> ON THE EXAM
>
> A pattern that works in batch is not automatically available in a stream. Anything that needs the whole partition is the first thing to check.

What to carry into the exam

- Group by collapses rows. A window function keeps them and adds context.
- **row_number** never ties, which is why it deduplicates. Rank gaps, dense rank does not.
- **QUALIFY** filters window results without a subquery. Databricks only, from DBR 10.4 LTS.
- The default frame is **RANGE**, not ROWS. Duplicate ordering values behave as one peer group.
- **Semi and anti joins** filter without adding columns or multiplying rows.

> ON THE EXAM
>
> When a question mentions efficiency, look for the option that reads the data once rather than the one that reads it three times.

> A bronze table receives the same order record multiple times when an upstream system retries. Each copy has an identical order id but a different ingestion timestamp, and later copies may correct earlier values.
>
> A silver job must keep exactly one row per order id, and it must be the most recent copy.

Which approach is correct?

A SELECT DISTINCT across all columns, which removes the duplicate order records.

B row_number over a partition by order id, ordered by ingestion timestamp descending, keeping row 1.

C GROUP BY order id with MAX on every other column.

D dense_rank over a partition by order id, keeping rank 1.

◆ PAUSE AND DECIDE

Which approach is correct?

A SELECT DISTINCT across all columns, which removes the duplicate order records.

The copies differ in timestamp and corrected values, so they are not identical rows. Distinct keeps them all.

B row_number over a partition by order id, ordered by ingestion timestamp descending, keeping row 1.

Partitions on the key that should be unique, orders so the latest wins, and row number guarantees exactly one.

C GROUP BY order id with MAX on every other column.

Takes the maximum of each column independently, producing a row that never existed.

D dense_rank over a partition by order id, keeping rank 1.

Dense rank ties. Two copies sharing a timestamp would both be kept.

ON THE EXAM

C is the dangerous one. It produces a plausible-looking row assembled from different source records, and nothing errors.

> A query must return, for each store, the three highest-value transactions. An engineer writes a ranking function in the SELECT list and then adds a WHERE clause filtering the rank to three or less.
>
> The query fails, saying the rank column cannot be resolved in the WHERE clause.

What is the cleanest fix on Databricks?

A Move the filter into a HAVING clause, which is evaluated later than WHERE.

B Use QUALIFY to filter the window function result in the same query block.

C Wrap the query in a CTE and filter in an outer SELECT.

D Add the rank expression to a GROUP BY so it becomes available to WHERE.

◆ PAUSE AND DECIDE

What is the cleanest fix on Databricks?

A Move the filter into a HAVING clause, which is evaluated later than WHERE.

HAVING filters groups after aggregation. Window functions are still evaluated after it.

B Use QUALIFY to filter the window function result in the same query block.

Exactly what QUALIFY exists for, and it avoids the extra query block entirely.

C Wrap the query in a CTE and filter in an outer SELECT.

Correct, and it is the workaround QUALIFY replaced. On Databricks it is no longer the cleanest option.

D Add the rank expression to a GROUP BY so it becomes available to WHERE.

Grouping by a rank collapses the rows you were trying to keep.

ON THE EXAM

C works and is what you would write in open-source Spark. The question asks for the cleanest fix on Databricks, and that word matters.

A daily job filters a 200 million row transactions table down to those whose customer appears in a small compliance watchlist. The engineer uses an inner join on customer id.

The output has more rows than the input, and finance reports inflated totals. The watchlist contains a few customers listed twice under different case references.

What is the correct fix?

A Deduplicate the watchlist, then keep the inner join.

B Replace the inner join with a LEFT SEMI join.

C Replace the inner join with a LEFT OUTER join and filter out nulls.

D Add SELECT DISTINCT to the output of the join.

◆ PAUSE AND DECIDE

What is the correct fix?

A. Deduplicate the watchlist, then keep the inner join.
Fixes today's data and leaves the design fragile. The next duplicate reintroduces the bug.

B. Replace the inner join with a LEFT SEMI join.
A semi join returns each qualifying left row once regardless of how many matches exist on the right.

C. Replace the inner join with a LEFT OUTER join and filter out nulls.
Still multiplies rows on duplicate matches, and does more work first.

D. Add SELECT DISTINCT to the output of the join.
Masks the duplication at the cost of a shuffle over 200 million rows, and would collapse legitimate identical transactions.

ON THE EXAM

The intent was to filter and the tool multiplied. Choosing a join that cannot multiply removes the whole class of bug.

> A report computes a running total of daily revenue, ordered by date. Several dates share the same revenue value in the test data.
>
> The engineer wrote an ORDER BY with no explicit frame clause, and finds that the running total jumps by several days' worth at once rather than increasing one day at a time.

What explains this?

A The default frame is RANGE, so rows sharing an ordering value are treated as one peer group.

B The ORDER BY is missing a PARTITION BY, so all rows are in one partition.

C Running totals require dense_rank rather than a sum over a window.

D The frame defaults to UNBOUNDED FOLLOWING, so future rows are included.

◆ PAUSE AND DECIDE

What explains this?

A The default frame is RANGE, so rows sharing an ordering value are treated as one peer group.

With ORDER BY and no frame, the default is RANGE unbounded preceding to current row. Peers are included together.

B The ORDER BY is missing a PARTITION BY, so all rows are in one partition.

A running total across all dates is what was wanted. That is not the symptom.

C Running totals require dense_rank rather than a sum over a window.

Ranking functions do not compute totals.

D The frame defaults to UNBOUNDED FOLLOWING, so future rows are included.

The default stops at the current row, not at the end of the partition.

ON THE EXAM

Switching the frame to ROWS gives the row-at-a-time behaviour that was expected. The default is the surprise, not the data.

> An analytics team needs revenue totals by region, by region and product, by product alone, and a grand total. They currently run four separate queries against the same 4 billion row table and union the results.
>
> The job takes hours and the cost has been questioned.

What is the most efficient rewrite?

A Use GROUPING SETS to compute all four groupings in a single pass.

B Cache the table, then run the four queries against the cached copy.

C Use four window functions with different PARTITION BY clauses.

D Create four materialized views, one per grouping.

◆ PAUSE AND DECIDE

What is the most efficient rewrite?

A Use GROUPING SETS to compute all four groupings in a single pass.

One read of the table instead of four, which is precisely what the extension exists for.

B Cache the table, then run the four queries against the cached copy.

Four billion rows will not sit comfortably in cache, and it still scans four times.

C Use four window functions with different PARTITION BY clauses.

Window functions keep every row. This needs collapsed aggregates.

D Create four materialized views, one per grouping.

Helps repeated reads and still computes four separate aggregations.

ON THE EXAM

ROLLUP would also work if the groupings formed a strict hierarchy. They do not here, because product alone is not under region.

> A batch job identifies each customer's single most recent order using a ranking function partitioned by customer. The team now wants the same logic in a Structured Streaming pipeline over an unbounded order feed.
>
> They report that the streaming version will not run.

What is the correct assessment?

A Ranking across a whole partition needs every row, so it does not translate to an unbounded stream.

B Streaming supports ranking only if the output mode is complete.

C The ranking function must be replaced with QUALIFY, which is streaming-compatible.

D Ranking works in streaming provided a checkpoint location is configured.

◆ PAUSE AND DECIDE

What is the correct assessment?

A Ranking across a whole partition needs every row, so it does not translate to an unbounded stream.

The stream never ends, so the engine can never know which order is last. AUTO CDC or a stateful operator is the route.

B Streaming supports ranking only if the output mode is complete.

Complete mode does not make an unbounded partition finite.

C The ranking function must be replaced with QUALIFY, which is streaming-compatible.

QUALIFY filters window results. It does not change what a window can compute.

D Ranking works in streaming provided a checkpoint location is configured.

A checkpoint records progress. It does not bound a partition.

ON THE EXAM

The real answer for 'latest record per key' on a stream is AUTO CDC from module 8, which is built for exactly this.

TRANSFORMATION · DOMAIN 3 - DATA TRANSFORMATION, CLEANSING, QUALITY

Shuffle, skew and broadcast

Section 3. What 'efficient' means once the query is running.

SECTION 3 of 10 WEIGHT 10% TOPIC Execution

admin@sthithapragna.com

Every expensive query has a shuffle in it.
The question is never whether it shuffles, but how much and how evenly.

admin@sthithapragna.com

Narrow and wide

- **Narrow:** each output partition depends on one input partition. Filters, projections, casts.
- **Wide:** output partitions depend on many input partitions. Joins, group by, distinct, sorts, window functions.
- A wide transformation forces a **shuffle**, and a shuffle forces a **stage boundary**.
- Counting stage boundaries in a plan is a quick estimate of how much reorganisation a query needs.

> ON THE EXAM
>
> Filtering before a join is not a style preference. It reduces what has to cross the network, which is the expensive part.

Four strategies, and when each applies

STRATEGY	CHARACTER
Broadcast hash	Copies the small side to every executor. The large side never moves.
Sort merge	Shuffles both sides, sorts, then merges. The default for large-to-large.
Shuffle hash	Shuffles both sides and builds a hash table instead of sorting.
Broadcast nested loop	For non-equi joins and cross joins, where there is no key to hash on.

ON THE EXAM

Seeing a broadcast nested loop join over large data in a plan is a warning rather than a win. It usually means an equality condition went missing.

The fast join that can take out the driver

- No shuffle of the large side at all. When one side truly fits, nothing else comes close.
- The small side is **collected to the driver**, then sent to every executor.
- A **BROADCAST hint is honoured even when the table exceeds the threshold**. That is how a hint causes an out-of-memory failure.
- So a hint is a statement about what you believe, and belief about table size ages badly.

> ON THE EXAM
>
> A broadcast that worked for a year and then failed is usually a dimension table that grew past what the driver can hold.

Four things AQE does at runtime

FEATURE	WHAT IT FIXES
Switch sort merge to broadcast	A join planned as large-to-large where one side turns out small.
Coalesce partitions	Hundreds of tiny partitions after a shuffle, each with per-task overhead.
Handle skew join	One partition far larger than the others, holding up the whole stage.
Propagate empty relations	A branch of the plan that turns out to have no rows at all.

ON THE EXAM

AQE is enabled by default. It works from real statistics gathered at the end of a shuffle, which is why it beats planning from stale table statistics.

Numbers worth knowing

SETTING	DEFAULT
Shuffle partitions	200. Set it to auto for auto-optimized shuffle.
Runtime broadcast threshold	30 MB, for AQE switching a join at runtime.
Advisory partition size	64 MB, the target size after coalescing.
Partition coalescing	Enabled.

ON THE EXAM

Two hundred shuffle partitions is a fixed default that suits neither a small dataset nor a huge one. Coalescing exists largely because of it.

Shuffle partitions and streaming checkpoints

- In Structured Streaming, **shuffle partitions cannot be changed** between restarts from the same checkpoint.
- So a tuning change that is routine in batch is a **checkpoint-invalidating change** in a stream.
- Decide the value before the stream goes live, because changing it later means a new checkpoint.
- This is the same principle as module 6: anything that changes the shape of the state cannot be carried forward.

> ON THE EXAM
>
> A perfectly reasonable batch optimisation becomes a breaking change the moment the workload is streaming.

How the engine decides a partition is skewed

- A partition is skewed when **both** tests pass: it is larger than a **multiple of the median**, and larger than an **absolute size threshold**.
- Requiring both stops the engine treating a slightly uneven but small partition as a problem.
- When detected, the skewed partition is split so the work spreads across more tasks.
- The symptom in the interface is one task running far longer than every other task in the stage.

ON THE EXAM

Skew is why a stage can be 99% complete for twenty minutes. The median task finished long ago; one is still going.

Where AQE skew handling does not reach

- It applies **only to shuffle-based joins**: sort merge and shuffle hash.
- **Broadcast joins are never skew-optimized.**
- In a **LEFT OUTER JOIN, only left-side skew** can be optimized.
- A **FULL OUTER JOIN is not skew-optimized on either side.**

> ON THE EXAM
>
> These limits are exactly the sort of specific fact a professional paper tests. AQE is not a universal answer to skew.

Salting, and its cost

- Add a random component to the join key so one hot value spreads across many partitions.
- The other side must be **expanded to match** every salt value, which multiplies its rows.
- It changes the query, which means it changes what a reader has to understand.
- Reach for it when AQE cannot help: a skewed full outer join, or skew outside a join entirely.

> ON THE EXAM
>
> Salting is the manual version of what AQE does automatically. Try the automatic route first, and know why it failed before adding complexity.

Four join hints, in priority order

HINT	MEANING
BROADCAST	Broadcast this side. Highest priority. Aliases: BROADCASTJOIN, MAPJOIN.
MERGE	Use sort merge. Aliases: SHUFFLE_MERGE, MERGEJOIN.
SHUFFLE_HASH	Use shuffle hash. Smaller side becomes the build side.
SHUFFLE_REPLICATE_NL	Shuffle and replicate nested loop. Lowest priority.

ON THE EXAM

When both sides carry different hints, that order decides. And a hint is a suggestion: a strategy that cannot support the join type will not be used.

The range join hint

- A join on a **range condition** has no equality key, so the engine falls back to nested loops.
- The **range join hint** bins the range values so a hash-based lookup becomes possible.
- That turns something close to a Cartesian product into a manageable join.
- Typical shape: an event timestamp falling between a start and an end column.

> ON THE EXAM
>
> If a plan shows a broadcast nested loop join over large inputs and the condition is a range, this hint is what the question is pointing at.

How to tell AQE actually did something

- Compare the **initial plan** against the **final plan**. Different physical join nodes mean a strategy switched.
- Coalescing shows as a custom shuffle reader marked **Coalesced**.
- Skew handling shows as a sort merge join with an **is skew** field set to true.
- An empty relation shows as part of the plan replaced by a local table scan with nothing in it.

> ON THE EXAM
>
> When a question asks how you would confirm an optimisation happened, these markers are the evidence rather than the runtime.

What to carry into the exam

- Wide transformations shuffle. Filtering before a join reduces what crosses the network.
- **AQE is on by default**: switches joins, coalesces partitions, handles skew, prunes empty relations.
- Skew handling covers **shuffle joins only**. Never broadcast, and never a full outer join.
- A **BROADCAST hint overrides the threshold**, which is how a hint takes out the driver.
- **Shuffle partitions cannot change** between streaming restarts on the same checkpoint.

> ON THE EXAM
>
> Before tuning anything, check whether AQE already handled it. Most manual shuffle tuning advice predates it being on by default.

A nightly job joins a 4 TB transactions table to a country reference table. It has run in eight minutes for a year. Last week it began failing with a driver out-of-memory error.

The query contains a BROADCAST hint on the reference table, added when the job was written. The reference table has grown as new markets were added.

What is the cause?

A The transactions table has grown past what a broadcast join can handle.

B The hint is honoured even above the broadcast threshold, so the grown reference table is collected to the driver.

C AQE has stopped switching the join to broadcast because statistics are stale.

D Shuffle partitions must be increased to accommodate the larger reference table.

◆ PAUSE AND DECIDE

What is the cause?

A The transactions table has grown past what a broadcast join can handle.

The broadcast side is the small one. The large side is never collected.

B The hint is honoured even above the broadcast threshold, so the grown reference table is collected to the driver.

A hint overrides the automatic size check, which is exactly what makes it dangerous as data grows.

C AQE has stopped switching the join to broadcast because statistics are stale.

The hint forces the broadcast. AQE is not making this decision.

D Shuffle partitions must be increased to accommodate the larger reference table.

A broadcast join does not shuffle the large side. Partition count is not involved.

ON THE EXAM

Removing the hint lets AQE decide from real statistics, and it will simply stop broadcasting once the table is too big.

> A full outer join between two large tables runs for hours. The Spark UI shows one task in the join stage running far longer than the rest, with the stage stuck near completion. One join key value accounts for roughly 40% of rows on both sides.
>
> An engineer says AQE will handle the skew automatically.

What is the correct assessment?

A They are right. AQE splits skewed partitions in any shuffle-based join.

B A FULL OUTER JOIN is not skew-optimized on either side, so a manual approach such as salting is needed.

C AQE will handle it once statistics are refreshed with ANALYZE.

D The join should be broadcast, which avoids skew entirely.

◆ PAUSE AND DECIDE

What is the correct assessment?

A They are right. AQE splits skewed partitions in any shuffle-based join.

Nearly right, and the join type limits apply. Full outer is the exception.

B A FULL OUTER JOIN is not skew-optimized on either side, so a manual approach such as salting is needed.

A documented limit. Adaptive execution will not rescue this particular shape.

C AQE will handle it once statistics are refreshed with ANALYZE.

AQE works from runtime statistics precisely so it does not depend on collected ones.

D The join should be broadcast, which avoids skew entirely.

Both sides are large, and broadcast joins are never skew-optimized anyway.

ON THE EXAM

A is the answer most people would give, and it is right about the mechanism and wrong about this join type. Read the join type first.

> A batch job was tuned by reducing shuffle partitions from 200 to 32, which cut its runtime substantially. An engineer applies the same change to a Structured Streaming job that shares the same logic.
>
> The streaming job fails on restart.

Why?

A 32 partitions is below the minimum permitted for streaming queries.

B Shuffle partitions cannot be changed between restarts from the same checkpoint in Structured Streaming.

C Streaming requires shuffle partitions to match the number of source partitions.

D AQE must be disabled before shuffle partitions can be set manually.

◆ PAUSE AND DECIDE

Why?

A 32 partitions is below the minimum permitted for streaming queries.

No such minimum exists.

B Shuffle partitions cannot be changed between restarts from the same checkpoint in Structured Streaming.

Documented explicitly. A routine batch tuning change invalidates a streaming checkpoint.

C Streaming requires shuffle partitions to match the number of source partitions.

Invented requirement.

D AQE must be disabled before shuffle partitions can be set manually.

Both can be set together. AQE coalesces from whatever initial number is configured.

ON THE EXAM

The same edit is routine tuning in batch and a breaking change in a stream. That asymmetry is what the question is testing.

A query joins an events table to a promotions table on the condition that the event timestamp falls between the promotion's start and end times. There is no equality condition.

The plan shows a broadcast nested loop join and the query has never completed on full data.

What should the engineer do?

A Add a BROADCAST hint on the promotions table to force a hash join.

B Use a range join hint so the range values are binned and hashed.

C Increase shuffle partitions so the nested loop is spread over more tasks.

D Rewrite it as a cross join with a WHERE clause, which the optimiser handles better.

◆ PAUSE AND DECIDE

What should the engineer do?

A. Add a BROADCAST hint on the promotions table to force a hash join.

A hash join needs an equality key. Broadcasting does not create one.

B. Use a range join hint so the range values are binned and hashed.

Bins the range so a hash lookup replaces the nested loop. Built for exactly this shape.

C. Increase shuffle partitions so the nested loop is spread over more tasks.

More tasks doing quadratic work is still quadratic work.

D. Rewrite it as a cross join with a WHERE clause, which the optimiser handles better.

That is what the query already is. Writing it explicitly changes nothing.

ON THE EXAM

The absence of an equality condition is the signal. Without a key to hash on, the engine has nothing to organise the join around.

> A team inherited a job containing a long list of Spark configuration settings, added over several years. It sets shuffle partitions to 1000, disables adaptive query execution, and manually specifies broadcast thresholds.
>
> The job is slow and the team wants a starting point for improving it.

What should they try first?

A. Increase shuffle partitions further, since 1000 is clearly insufficient for a slow job.

B. Re-enable AQE and remove the manual settings, then measure before tuning anything by hand.

C. Add BROADCAST hints to every join so no join has to shuffle.

D. Salt all join keys pre-emptively to eliminate any possible skew.

◆ PAUSE AND DECIDE

What should they try first?

A Increase shuffle partitions further, since 1000 is clearly insufficient for a slow job.

More partitions means more tasks and more overhead. Nothing suggests too few.

B Re-enable AQE and remove the manual settings, then measure before tuning anything by hand.

Most of that tuning predates AQE being on by default, and disabling it removes four automatic optimisations.

C Add BROADCAST hints to every join so no join has to shuffle.

Broadcasting a large side is how you take out the driver.

D Salt all join keys pre-emptively to eliminate any possible skew.

Adds complexity everywhere to solve a problem nobody has confirmed exists.

ON THE EXAM

Accumulated configuration is often the cause rather than the cure. Removing it and measuring is a legitimate first step.

> An engineer is asked to prove that a query optimisation actually took effect after enabling adaptive query execution. The query now runs faster, but the change coincided with a cluster upgrade.
>
> They need evidence that AQE specifically did something.

What should they look for?

A. A shorter total runtime in the job history, compared against the previous week.

B. Differences between the initial and final plans, plus markers such as a coalesced shuffle reader or a join with is-skew set.

C. A lower DBU count in the billable usage system table.

D. Fewer files written to the target table.

◆ PAUSE AND DECIDE

What should they look for?

A. A shorter total runtime in the job history, compared against the previous week.
Confounded by the cluster upgrade, which is exactly why the question mentions it.

B. Differences between the initial and final plans, plus markers such as a coalesced shuffle reader or a join with is-skew set.
AQE rewrites between the initial and final plan, and each optimisation leaves a specific marker.

C. A lower DBU count in the billable usage system table.
Also confounded, and it measures cost rather than which optimisation fired.

D. Fewer files written to the target table.
AQE re-plans the query. It is not a file-layout feature.

ON THE EXAM

Runtime tells you something improved. The plan tells you what. When a question mentions a confounding change, it is asking for the plan.

admin@sthithapragna.com

TRANSFORMATION · DOMAIN 3 - DATA TRANSFORMATION, CLEANSING, QUALITY

Expectations and quarantining bad data

Section 3. Warn, drop, fail, and the fourth thing that is not an operator.

SECTION 3 of 10 WEIGHT 10% TOPIC Data quality

admin@sthithapragna.com

An expectation asks one question of every row. **What you do with a no is the whole design decision.**

Three components, every time

- A **name**, used to track and report the expectation. It must be **unique within a dataset**.
- A **constraint**: a SQL Boolean expression evaluated per record.
- An **action** deciding what happens to a failing record.
- The same expectation can be **reused across datasets** in a pipeline, which is how you keep rules consistent.

> ON THE EXAM
>
> Expectations can be defined on streaming tables, materialized views and temporary views. Nothing else supports them.

What the expression cannot do

- No **custom Python functions**. The constraint is SQL, evaluated by the engine.
- No **external service calls**. You cannot check a record against an outside system.
- No **subqueries referencing other tables**. Validation is per-record and self-contained.
- Everything else is fair game: SQL functions, CASE expressions, complex Boolean logic.

> ON THE EXAM
>
> Validation against another table is a join, not an expectation. If a question needs a lookup, expectations are not the mechanism.

Three responses to a failing record

ACTION	WHAT HAPPENS
warn, the default	The invalid record is written to the target, and counted in metrics.
drop	The record is discarded before writing. The count is logged.
fail	The update fails. The transaction is atomically rolled back.

ON THE EXAM

Warn being the default surprises people. An expectation with no violation clause does not stop bad data; it lets it through and counts it.

The decision in one question

- Should the invalid row **stay in the target**? Use warn. You want visibility, not enforcement.
- Should the row **disappear** while processing continues? Use drop.
- Should invalid data **block the update**? Use fail. Any bad record means a serious upstream problem.
- Should the row be **kept somewhere else for inspection**? That is quarantine, and none of the three does it.

> ON THE EXAM
>
> The fourth row is the one the objective names. It is built from the other pieces rather than selected from a list.

Quarantine is a pattern, not an operator

- There is no expect-or-quarantine action. The API has exactly **three**: warn, drop, fail.
- The docs describe quarantining as **advanced logic you implement**, routing invalid records to a separate table.
- A Databricks **Labs** metadata framework does define such a rule type. That is not the core product.
- So expect the name to appear as a plausible distractor, and know it is not a built-in operator.

> ON THE EXAM
>
> The objective says 'develop a quarantining process'. That wording is accurate: you develop it, you do not switch it on.

How a quarantine is actually assembled

- Define the rule **once** as a condition, then use it in both directions.
- The clean table keeps records **where the condition holds**. The quarantine table keeps those where it does not.
- Add the reason for rejection so somebody can triage without re-deriving it.
- Nothing is lost, nothing blocks, and bad records are visible in one place.

> ON THE EXAM
>
> Two datasets reading the same source with opposite filters. That is the whole trick, and it is why it counts as a pattern rather than a feature.

Where the numbers come from, and where they do not

- Metrics are recorded for **warn and drop**, and shown on the Data quality tab of the pipeline.
- **Not for fail.** Because the update fails, nothing is aggregated to report.
- Metrics are also queryable from the **pipeline event log**, which is how you track trends over time.
- For standalone datasets created in Databricks SQL, the Data quality tab is **not available**. Use the event log.

> ON THE EXAM
>
> Fail gives you a detailed error naming the offending record, and no aggregate metrics. Those are different kinds of information.

Fail behaves differently by pipeline mode

- In a **triggered** pipeline, one flow failing does **not** fail other parallel flows.
- In a **continuous** pipeline, the failure stops that flow **and all dependent flows**.
- For real orchestration control, split validation and downstream work into **separate pipelines**, coordinated by task control flow.
- After a failure, the **pipeline code must be fixed** before re-running. It will not simply retry past it.

> ON THE EXAM
>
> Same action, different blast radius depending on the pipeline mode. That asymmetry is worth remembering.

Grouping is Python only

- Both languages support several expectations on one dataset.
- Only **Python** lets you group them and apply a **collective action**.
- The grouped forms take a **dictionary** of name to constraint, so one dictionary serves many datasets.
- That is how the same rule set gets applied with warn at bronze and fail at gold.

> ON THE EXAM
>
> One dictionary, three severities. Warn where you are observing, drop where you are cleaning, fail where you are publishing.

The other half of the objective

- Outside pipelines, the equivalent is Auto Loader's **rescued data column**, from module 11.
- It captures a **missing column, a type mismatch or a case mismatch**, with the source file path.
- Filtering on that column not being null gives you a quarantine table for structural problems.
- It catches **malformed** records, not **invalid** ones. A negative price parses perfectly well.

> ON THE EXAM
>
> Two different failure classes. Rescued data catches records that did not parse; expectations catch records that parsed and are wrong.

Where expectations do not reach

- Only streaming tables, materialized views and temporary views. Nothing else.
- **Not supported for sinks**, which is the module 7 point: writing outside the pipeline loses the quality mechanism.
- **Not supported with AUTO CDC FROM SNAPSHOT.**
- Views are computed only when queried, so metrics may be missing or may appear several times over.

> ON THE EXAM
>
> The sink limitation matters most in design questions. Quality checks stop at the boundary where you leave the pipeline's own datasets.

What to carry into the exam

- **Warn is the default.** An expectation with no violation clause lets bad data through and counts it.
- Drop discards, fail rolls back the transaction atomically.
- **No metrics for fail**, because the update did not complete.
- **Quarantine is a pattern you build**, not an action you select.
- Grouped actions are **Python only**. Rescued data is the classic-jobs equivalent for malformed records.

> ON THE EXAM
>
> That completes Section 3 at 10%. Section 4, Data Sharing and Federation, is next and is the smallest section on the exam.

A team adds an expectation to a bronze streaming table checking that an order amount is not negative. They write CONSTRAINT non_negative_amount EXPECT (amount >= 0) with no further clause.

A week later, an analyst finds negative amounts in the bronze table. The team insists the expectation is working, and the metrics do show violations being counted.

Who is right?

A The team is right; the expectation works, and warn is the default action, so invalid records are still written.

B The analyst is right; the expectation is malformed and is being ignored entirely.

C The team is right; the negative amounts must have been written before the expectation was added.

D The analyst is right; expectations do not apply to streaming tables.

◆ PAUSE AND DECIDE

Who is right?

A The team is right; the expectation works, and warn is the default action, so invalid records are still written.

Both observations are correct. Warn counts violations and writes the record anyway.

B The analyst is right; the expectation is malformed and is being ignored entirely.

The metrics prove it is being evaluated.

C The team is right; the negative amounts must have been written before the expectation was added.

Plausible timing argument, and the metrics show ongoing violations.

D The analyst is right; expectations do not apply to streaming tables.

Streaming tables are one of the three types that do support them.

ON THE EXAM

Both parties observed something true. The expectation is working exactly as written, and as written it does not block anything.

> A regulated pipeline must never publish a record with a missing customer identifier. The compliance team must also review every rejected record and understand why it failed.
>
> An engineer proposes ON VIOLATION DROP ROW, since it prevents bad records reaching the target.

What is the problem with that proposal?

A. Nothing. Drop prevents publication and the metrics record how many were dropped.

B. Drop discards the records entirely, so there is nothing for compliance to review. A quarantine pattern is needed.

C. Drop is not supported on streaming tables; only warn and fail are.

D. Drop should be replaced with expect_or_quarantine, which routes rejects to a quarantine table.

◆ PAUSE AND DECIDE

What is the problem with that proposal?

A Nothing. Drop prevents publication and the metrics record how many were dropped.

Half the requirement. A count is not a reviewable record.

B Drop discards the records entirely, so there is nothing for compliance to review. A quarantine pattern is needed.

Two requirements: block publication, and preserve rejects for review. Drop satisfies only the first.

C Drop is not supported on streaming tables; only warn and fail are.

All three actions are supported.

D Drop should be replaced with expect_or_quarantine, which routes rejects to a quarantine table.

No such operator exists in the core API. It comes from a Databricks Labs framework.

ON THE EXAM

D is the distractor this module exists to inoculate against. The name sounds exactly right and the operator is not real.

A gold table uses ON VIOLATION FAIL UPDATE on a critical constraint. Overnight the pipeline failed. The on-call engineer opens the Data quality tab expecting to see how many records violated the constraint, and finds no metrics for that run at all.

They conclude the monitoring is broken.

What should they be told?

A Metrics are not recorded for fail, because the update did not complete. The error message names the offending record.

B Metrics take up to 24 hours to appear in the Data quality tab.

C The pipeline must be configured with a metrics reporter before any expectation metrics appear.

D Fail records metrics only in the event log, not the UI. Query the event log for the counts.

◆ PAUSE AND DECIDE

What should they be told?

A Metrics are not recorded for fail, because the update did not complete. The error message names the offending record.

Documented behaviour. Fail gives a detailed violation message instead of aggregate counts.

B Metrics take up to 24 hours to appear in the Data quality tab.

Invented delay.

C The pipeline must be configured with a metrics reporter before any expectation metrics appear.

A real setting, and it would have suppressed warn and drop metrics too, which the team has been seeing.

D Fail records metrics only in the event log, not the UI. Query the event log for the counts.

There are no aggregate counts to find, in either place.

ON THE EXAM

Fail trades a count for a specific record. That is usually the better trade when diagnosing, and it is not what the tab is for.

> A team wants an expectation that rejects any order whose customer identifier does not appear in the customers dimension table. They write a constraint containing a subquery against that table.
>
> The pipeline will not start.

Why, and what should they do instead?

A. Subqueries referencing other tables are not allowed in a constraint. Use a join, or an anti join to isolate orphans.

B. The subquery is allowed but must be wrapped in a Python UDF for the pipeline to evaluate it.

C. The customers table must be declared as a temporary view before a constraint can reference it.

D. Constraints can only reference columns of the same name in both tables.

◆ PAUSE AND DECIDE

Why, and what should they do instead?

A Subqueries referencing other tables are not allowed in a constraint. Use a join, or an anti join to isolate orphans.

A stated limitation. Cross-table validation is a join, and module 12's anti join is the natural way to find orphans.

B The subquery is allowed but must be wrapped in a Python UDF for the pipeline to evaluate it.

Custom Python functions are also explicitly disallowed in a constraint.

C The customers table must be declared as a temporary view before a constraint can reference it.

The restriction is on referencing other tables at all, not on how they are declared.

D Constraints can only reference columns of the same name in both tables.

Invented rule.

ON THE EXAM

Expectations validate a record against itself. Anything requiring another dataset is a join, and belongs in the query rather than the constraint.

A pipeline defines the same eight validation rules on bronze, silver and gold datasets. On bronze the team wants visibility only, on silver they want bad rows removed, and on gold nothing invalid may pass.

The pipeline is currently written in SQL, and the rules are copied out three times.

What is the better approach?

A Keep SQL and use a different ON VIOLATION clause on each copy of the rules.

B Write the datasets in Python and apply one dictionary of rules with expect_all, expect_all_or_drop and expect_all_or_fail.

C Define the rules once in SQL and reference them from the other datasets by name.

D Use a single gold-level fail expectation, since bronze and silver feed it anyway.

◆ PAUSE AND DECIDE

What is the better approach?

A. Keep SQL and use a different ON VIOLATION clause on each copy of the rules.
Works, and leaves three copies of eight rules to drift apart.

B. Write the datasets in Python and apply one dictionary of rules with expect_all, expect_all_or_drop and expect_all_or_fail.
One definition, three severities. Collective actions are Python only, which is exactly why Python is the answer here.

C. Define the rules once in SQL and reference them from the other datasets by name.
SQL supports multiple expectations but has no mechanism for a shared, named rule set with a collective action.

D. Use a single gold-level fail expectation, since bronze and silver feed it anyway.
Loses the graduated response, and a gold failure would then be the first sign of a bronze problem.

ON THE EXAM

This is the case where language choice is driven by a capability rather than preference. Grouped actions do not exist in SQL.

> A classic job uses Auto Loader to ingest JSON into a bronze table, with a rescued data column enabled. The team wants to catch two things: records whose fields did not parse, and orders with a negative total.
>
> They plan to rely on the rescued data column for both.

What is wrong with that plan?

A Nothing. The rescued data column captures any record that fails validation.

B Rescued data catches records that did not parse. A negative total parses fine, so it needs a separate check.

C Rescued data works only for CSV, not JSON.

D The rescued data column must be paired with an expectation to function at all.

◆ PAUSE AND DECIDE

What is wrong with that plan?

A. Nothing. The rescued data column captures any record that fails validation.
It captures parsing failures, not business-rule failures.

B. Rescued data catches records that did not parse. A negative total parses fine, so it needs a separate check.
Two different failure classes. Structural problems go to rescued data; a valid number that is wrong does not.

C. Rescued data works only for CSV, not JSON.
It works across the inference-based formats, JSON included.

D. The rescued data column must be paired with an expectation to function at all.
It is an Auto Loader feature and works independently.

ON THE EXAM

Malformed and invalid are different problems. A quarantine that catches only one of them will look like it is working.

SHARING · DOMAIN 4 - DATA SHARING AND FEDERATION

Delta Sharing

Section 4. Two protocols, one read-only promise, and what WITH HISTORY really exposes.

SECTION 4 of 10 WEIGHT 5% TOPIC Sharing

admin@sthithapragna.com

Delta Sharing is now OpenSharing

- Announced **10 June 2026**. Databricks' own release notes say plainly: Delta Sharing is now OpenSharing.
- The exam guide is dated **3 July 2026** and still says delta sharing throughout. Expect the old name.
- **Nothing breaks.** Existing shares keep working. It adds capability rather than replacing anything.
- What it adds: AI models, agent skills, unstructured data, and **Iceberg client support**.

> ON THE EXAM
>
> The protocol is now a Linux Foundation project. For the exam, the concepts are unchanged and the old name is the one you will see.

Nothing is copied. The recipient reads your files, with credentials you control and can revoke.

Three things you create

OBJECT	WHAT IT IS
Provider	You, when sharing. The metastore that holds the data.
Share	A named collection of objects from your metastore, made available together.
Recipient	A named object representing the person or organisation you are sharing with.

ON THE EXAM

Creating a recipient requires metastore admin, or the CREATE RECIPIENT privilege on the metastore. It is not an ordinary user action.

Databricks-to-Databricks against open sharing

D2D

Both sides on Databricks

- Recipient has a Unity Catalog workspace
- No token, no credential file, no activation link
- Shares views, volumes, models and notebooks too
- Identities are handled between metastores

D2O, OPEN SHARING

The recipient is anywhere

- Bearer token, or OIDC federation
- Credential file plus an activation link
- Delta-format tables
- Any tool that speaks the protocol

THE TIEBREAKER

The question that decides it is simply whether the recipient has a Unity Catalog-enabled Databricks workspace. If yes, use D2D; there is no reason not to.

How a non-Databricks recipient authenticates

- Either a **long-lived bearer token**, or **OIDC federation** through the recipient's own identity provider.
- With a token, Databricks generates a **credential file** and an **activation link**.
- The activation link must be sent over a **secure channel**. It is the only thing standing between the data and anyone.
- Token lifetime is configurable at recipient creation, and OIDC avoids a long-lived secret entirely.

> ON THE EXAM
>
> OIDC federation is the stronger option when available, because authentication stays with the recipient's identity provider rather than a token you issued.

Not everything can be shared both ways

ASSET	WHERE IT WORKS
Delta tables	Both protocols.
Views	Databricks-to-Databricks only.
Unity Catalog volumes	Databricks-to-Databricks only.
Unity Catalog models	Databricks-to-Databricks only.
Notebook files	Databricks-to-Databricks only.

ON THE EXAM

A requirement to share a view or a volume with an external partner is a requirement that open sharing does not meet.

WITH HISTORY is not only about history

- Sharing with history enables **cloud tokens**: temporary, path-scoped credentials to the table's storage.
- The recipient then reads the files **directly**, with performance the docs call comparable to direct access.
- **Default from DBR 16.2 and above.** Sharing a whole schema shares all its tables with history.
- Existing shares are not upgraded automatically. You must alter the share.

> ON THE EXAM
>
> The name undersells it. This is the setting that decides whether a recipient reads through the sharing server or straight from your storage.

What a cloud token actually grants

- Credentials are scoped to the table's **root directory**, giving read access to the data files **and the Delta log**.
- The log holds the **commit history**, **who committed each change**, and **deleted data not yet vacuumed**.
- So sharing with history can expose more than the table's current contents.
- If that matters, vacuum before sharing, or share without history and accept the performance cost.

> ON THE EXAM
>
> This is stated directly in the documentation and it is easy to miss. A performance setting has a data privacy consequence.

Partition filters, and what they cost

- A **partition specification** shares only part of a table, set when you add it to a share.
- That is how one table serves several recipients who should each see only their own slice.
- But on Databricks-to-Databricks, a partition filter makes the table **ineligible for cloud tokens**.
- So filtering and direct-read performance are in tension. You get one or the other.

> ON THE EXAM
>
> The tension makes sense: a credential scoped to a directory cannot enforce a row filter. Restricting what is visible means reading through the server.

Read-only, live, and on any platform

- **Shared data is read only.** A recipient cannot write back through a share.
- It is live. There is no refresh to schedule, because the recipient reads your current table.
- Recipients use their own tools: pandas, open-source Spark, Power BI, Tableau, or a Databricks workspace.
- That is the second objective's phrase: sharing live data to any computing platform.

> ON THE EXAM
>
> Live and read-only together are why sharing beats an export. No copy exists to drift, and no copy exists to leak.

Where sharing can and cannot reach

- Databricks-to-Databricks supports sharing **across regulatory domains**.
- Commercial workspaces can share with government clouds and with Azure China.
- Sharing between **Azure China and the government clouds is not supported**.
- These matrices change. Check the current support matrix rather than trusting a course.

> ON THE EXAM
>
> Cross-cloud and cross-region sharing works, which is often the actual reason an organisation chooses Delta Sharing over an export pipeline.

What to carry into the exam

- **D2D when the recipient has Unity Catalog**, open sharing otherwise. That single fact decides it.
- Views, volumes, models and notebooks are **D2D only**. Open sharing carries Delta tables.
- **WITH HISTORY enables cloud tokens** and direct reads. It also exposes the Delta log.
- A partition filter and cloud tokens are **mutually exclusive** on D2D.
- Shared data is **read only** and **live**. Nothing is copied.

> ON THE EXAM
>
> If an answer involves exporting, copying or scheduling a sync between two organisations, sharing is almost certainly the intended alternative.

A retailer must give a logistics partner access to a curated shipments table. The partner runs their own Databricks workspace with Unity Catalog enabled.

An engineer proposes creating an open sharing recipient, generating a bearer token, and emailing the activation link to the partner's data team.

What is the better approach?

A The proposal is correct; open sharing is the standard route for any external organisation.

B Use Databricks-to-Databricks sharing, which needs no token, credential file or activation link.

C Export the table nightly to a shared storage bucket the partner can read.

D Use open sharing but with OIDC federation rather than a bearer token.

◆ PAUSE AND DECIDE

What is the better approach?

A The proposal is correct; open sharing is the standard route for any external organisation.

External does not mean non-Databricks. The partner has Unity Catalog, so D2D applies.

B Use Databricks-to-Databricks sharing, which needs no token, credential file or activation link.

The recipient has a Unity Catalog workspace, which is the only fact that decides the protocol.

C Export the table nightly to a shared storage bucket the partner can read.

Creates a copy that goes stale and cannot be revoked. Sharing exists to avoid this.

D Use open sharing but with OIDC federation rather than a bearer token.

Better than a bearer token, and still the wrong protocol for this recipient.

ON THE EXAM

The distinction is not internal against external. It is whether the recipient has Unity Catalog.

> A healthcare provider shares a patient events table with an analytics partner using WITH HISTORY for performance. Records are routinely deleted from the source table when patients exercise their right to erasure.
>
> A privacy review asks whether deleted records could still be visible to the recipient.

What is the correct answer?

A. No. Deletes are applied immediately and the recipient sees only the current table.

B. Yes. Cloud tokens grant access to the Delta log, which retains deleted data until it is vacuumed.

C. No. The Delta log contains metadata only, never data.

D. Yes, but only if the recipient also has the CREATE RECIPIENT privilege.

◆ PAUSE AND DECIDE

What is the correct answer?

A. No. Deletes are applied immediately and the recipient sees only the current table.

True of a query against the table. Not true of direct file access to its directory.

B. Yes. Cloud tokens grant access to the Delta log, which retains deleted data until it is vacuumed.

Stated in the docs: the log contains commit history, committer information, and deleted data not yet vacuumed.

C. No. The Delta log contains metadata only, never data.

The log references data files that still exist until vacuum removes them.

D. Yes, but only if the recipient also has the CREATE RECIPIENT privilege.

That privilege is about creating recipients on the provider side. It is unrelated.

ON THE EXAM

A performance setting with a data privacy consequence. In a regulated context, vacuum before sharing, or share without history.

A data team maintains a curated view that joins three tables and masks two columns. They need to make this view available to an external partner who uses Snowflake, not Databricks.

They add the view to a share and create an open sharing recipient. The view does not appear for the recipient.

What is happening, and what should they do?

A Views are Databricks-to-Databricks only. Materialise the view as a table and share that.

B The view needs to be shared WITH HISTORY before it becomes visible.

C The recipient must install a Delta Sharing connector before views appear.

D Views require the recipient to be granted SELECT on the underlying tables as well.

◆ PAUSE AND DECIDE

What is happening, and what should they do?

A Views are Databricks-to-Databricks only. Materialise the view as a table and share that.

Open sharing carries Delta tables. A view needs a Unity Catalog on the other side to resolve against.

B The view needs to be shared WITH HISTORY before it becomes visible.

WITH HISTORY concerns cloud token eligibility for tables, and views are not eligible at all.

C The recipient must install a Delta Sharing connector before views appear.

A connector is needed to read anything, and it does not make an unsupported asset type appear.

D Views require the recipient to be granted SELECT on the underlying tables as well.

That would defeat the masking the view exists to provide.

ON THE EXAM

Materialising the view is not a workaround, it is the intended pattern. The masking is applied when the table is written.

> A provider shares one large transactions table with four regional partners. Each partner must see only their own region, so the table is added to each share with a partition specification.
>
> Partners report slower query performance than a similar unfiltered share they receive from another provider.

What explains the difference?

A Partition filters are evaluated by the recipient, so they always cost more.

B A partition filter makes the table ineligible for cloud tokens, so reads go through the sharing server rather than directly to storage.

C The table needs OPTIMIZE run before each share refresh.

D Four shares of the same table cause contention on the provider's storage.

◆ PAUSE AND DECIDE

What explains the difference?

A Partition filters are evaluated by the recipient, so they always cost more.

Backwards. The filter exists so the recipient never sees the other partitions.

B A partition filter makes the table ineligible for cloud tokens, so reads go through the sharing server rather than directly to storage.

A credential scoped to a directory cannot enforce a row filter, so filtered shares lose direct file access.

C The table needs OPTIMIZE run before each share refresh.

File layout is worth attention generally, and it does not explain the difference against an unfiltered share.

D Four shares of the same table cause contention on the provider's storage.

Invented. Shares do not contend with one another.

ON THE EXAM

This is a genuine trade-off rather than a misconfiguration. Restricting visibility and granting direct file access are incompatible by design.

> A team creates an open sharing recipient with a bearer token and emails the activation link to a distribution list at the partner organisation. Some weeks later, a security review finds the link was forwarded outside the intended team.
>
> The partner's organisation runs its own identity provider.

What is the strongest remediation?

A Regenerate the token and email the new activation link to a narrower list.

B Move the recipient to OIDC federation, so authentication is handled by the partner's identity provider.

C Shorten the token lifetime and keep the current process.

D Switch to Databricks-to-Databricks sharing to remove the token.

◆ PAUSE AND DECIDE

What is the strongest remediation?

A. Regenerate the token and email the new activation link to a narrower list.

Repeats the same process and hopes for a better outcome. The mechanism is unchanged.

B. Move the recipient to OIDC federation, so authentication is handled by the partner's identity provider.

Removes the long-lived secret entirely. Access follows the partner's own identity management, including leavers.

C. Shorten the token lifetime and keep the current process.

Reduces the window and keeps a forwardable secret at the centre of it.

D. Switch to Databricks-to-Databricks sharing to remove the token.

Only available if the partner has a Unity Catalog workspace, which nothing here states.

ON THE EXAM

The problem is not the token's length of life. It is that a bearer token is a secret that can be forwarded by anyone who holds it.

A share was created two years ago and carries a large fact table. A recipient on Databricks reports that queries against it are markedly slower than against a table shared by a newer provider.

Both are Databricks-to-Databricks shares of unfiltered managed Delta tables on current runtimes.

What is the most likely cause?

A The older share predates WITH HISTORY being the default, and was never altered to enable it.

B The older share needs to be recreated, since shares expire after two years.

C The recipient is reading a stale copy that needs refreshing.

D The older table needs its statistics recomputed on the provider side.

◆ PAUSE AND DECIDE

What is the most likely cause?

A The older share predates WITH HISTORY being the default, and was never altered to enable it.

WITH HISTORY became the default from DBR 16.2, and existing shares are not upgraded automatically.

B The older share needs to be recreated, since shares expire after two years.

Shares do not expire. Recipient tokens can, which is a different thing.

C The recipient is reading a stale copy that needs refreshing.

Nothing is copied. The recipient always reads the current table.

D The older table needs its statistics recomputed on the provider side.

Would affect both shares equally, and does not explain a difference between them.

ON THE EXAM

Defaults change, and existing objects keep the behaviour they were created with. That gap is a recurring source of unexplained differences.

SHARING · DOMAIN 4 - DATA SHARING AND FEDERATION

Lakehouse Federation and its limits

Section 4. Query the data where it lives, and know what that costs.

SECTION 4 of 10 WEIGHT 5% TOPIC Federation

admin@sthithapragna.com

Federation puts an external database **inside your catalog**. Same three-level names, same grants, same lineage. The data never moves.

Two securables, and one more for catalog federation

OBJECT	WHAT IT HOLDS
Connection	The path and credentials for reaching an external system. A securable in its own right.
Foreign catalog	A mirror of an external database inside Unity Catalog, created from a connection.
External location	Only for catalog federation. Where the underlying files actually live.

ON THE EXAM

Creating a foreign catalog needs CREATE CATALOG on the metastore, plus either ownership of the connection or CREATE FOREIGN CATALOG on it.

Query federation against catalog federation

QUERY FEDERATION

Pushed to their database

- Translated to SQL and sent over JDBC
- Runs on Databricks and their compute
- For ad hoc reporting, BI, proof of concept
- Relational databases

CATALOG FEDERATION

Read from their storage

- Reads the files directly in object storage
- Runs on Databricks compute only
- More cost-effective and faster
- Catalog platforms: Hive, Glue, Snowflake

WORTH KNOWING

Catalog federation avoids the other system's compute entirely, which is why the docs call it more cost-effective and performance-optimized.

What each type connects to

TYPE	SOURCES
Query federation	MySQL, PostgreSQL, Oracle, Teradata, Redshift, Snowflake, SQL Server, Synapse, BigQuery, Salesforce Data 360, and Databricks.
Catalog federation	Legacy and external Hive metastores, AWS Glue, Snowflake, Salesforce Data 360, Palantir Foundry.

ON THE EXAM

Some sources appear in both lists. Snowflake and Salesforce can be reached either way, and the choice then follows the cost and performance argument.

The same shape, in four steps

- Create a **connection** holding the credentials and the address of the system.
- Create a **foreign catalog** from that connection. For catalog federation, also give it an external location.
- **Grant privileges** on the tables in the foreign catalog. This is where governance actually happens.
- Run queries. They are pushed down, or read from storage, depending on the type.

> ON THE EXAM
>
> Notice the third step. Registering the source is plumbing; granting access with Unity Catalog is the objective's 'proper governance'.

What Unity Catalog actually brings

- **Table-level access controls** on foreign tables, using ordinary grants.
- **Lineage**, so a dashboard's dependency on an external system is visible rather than folklore.
- **Search and discovery** alongside your own tables.
- Credentials live in the connection, so users query the source **without ever holding them**.

> ON THE EXAM
>
> That last point is the strongest argument. Federation replaces a connection string circulating in notebooks with a governed, revocable grant.

Federated queries are read-only

- You cannot write back to a federated source. Federation is a read path.
- The docs name one exception: catalog federation of a workspace's **legacy Hive metastore**, where foreign tables are writeable.
- If you need write access, the answer is the **Spark data source interface**, not federation.
- The documentation is not entirely consistent on the exception. Treat read-only as the rule.

> ON THE EXAM
>
> A requirement to update the external system rules federation out. That is usually the whole question.

Your query becomes their query

- Databricks translates your SQL into statements **pushed down** into the foreign source.
- Inspect the generated SQL from the **foreign data source scan node** in the Query Profile, or with EXPLAIN FORMATTED.
- Coverage varies by source. What cannot be pushed down is executed on Databricks after the data arrives.
- So a filter that does not push down means the whole table crosses the network first.

> ON THE EXAM
>
> When a federated query is slow, look at what actually got pushed down before blaming the source. The scan node tells you.

No caching, and their concurrency limits are yours

- **Databricks query caching is not supported** for federated queries. Neither result cache nor disk cache.
- So a repeated dashboard query hits the source every time, with no free second run.
- Throttling follows the Databricks SQL concurrent query limit, and you are competing with the source's own workload.
- A federated dashboard refreshed by fifty analysts is fifty queries against a production database.

> ON THE EXAM
>
> Federation makes an operational database look like a table. It does not make it behave like one under analytical load.

When Databricks says not to federate

- When a source supports **both federation and Lakeflow Connect**, Databricks recommends **Lakeflow Connect**.
- The stated condition: when performance at **higher data volumes** or **lower latency** is a priority.
- Federation is positioned for **ad hoc reporting, BI and proof of concept**, not for the production read path.
- That is a narrower recommendation than most material implies.

> ON THE EXAM
>
> Federation is a way to reach data, not a substitute for ingesting it. The docs are explicit about which is preferred at volume.

When to use the Spark data source interface

- When the source is **not supported** by Lakehouse Federation at all.
- When you need **write access** to the external system.
- When you need control over **execution and parallelisation**, such as partitioned reads.
- The runtime bundles connectors for PostgreSQL, SQL Server, MySQL, Snowflake and Redshift.

> ON THE EXAM
>
> Three reasons, each decisive on its own. Any of them makes federation the wrong tool rather than a slower one.

What to carry into the exam

- Connection plus foreign catalog. Governance is the **grants**, not the registration.
- **Query federation** pushes down over JDBC. **Catalog federation** reads storage on Databricks compute only.
- **Read-only**, **no caching**, and you inherit the source's concurrency limits.
- Databricks prefers **Lakeflow Connect** when volume or latency matter.
- Need to write, or an unsupported source? **Spark data source interface.**

> ON THE EXAM
>
> That completes Section 4, the smallest section at 5%. Section 5 is monitoring and alerting, at 10%.

> An analytics team federates a production PostgreSQL order system so analysts can query it without extracts. The proof of concept was fast. Six weeks after rollout, the database team reports sustained load and slow application response during business hours.
>
> A dashboard built on the foreign catalog is refreshed by around sixty analysts each morning.

What is the most likely explanation?

A Federated queries are not cached, so every refresh runs against the production database.

B The foreign catalog is holding a stale copy that needs refreshing more often.

C Query pushdown is disabled by default and must be enabled on the connection.

D The connection needs a higher concurrency limit configured in Unity Catalog.

◆ PAUSE AND DECIDE

What is the most likely explanation?

A Federated queries are not cached, so every refresh runs against the production database.

Neither result cache nor disk cache applies. Sixty refreshes are sixty queries on a production system.

B The foreign catalog is holding a stale copy that needs refreshing more often.

A foreign catalog holds no data. Nothing is copied.

C Query pushdown is disabled by default and must be enabled on the connection.

Pushdown is automatic. Coverage varies, but it is not a switch.

D The connection needs a higher concurrency limit configured in Unity Catalog.

Raising concurrency would increase load on the struggling database, not reduce it.

ON THE EXAM

A proof of concept that went well is the setup here. Federation scales differently from how it demonstrates.

> A team needs to read reference data from an on-premises system and also write status updates back to it after processing. The source is a supported Lakehouse Federation database.
>
> An architect proposes federating it and using MERGE to update the foreign tables.

What is wrong with the proposal?

A. Nothing. Foreign tables support MERGE like any Unity Catalog table.

B. Federated queries are read-only. Writing needs the Spark data source interface.

C. MERGE is supported but only through catalog federation, not query federation.

D. Foreign tables must be cloned into Unity Catalog before they can be written to.

◆ PAUSE AND DECIDE

What is wrong with the proposal?

A. Nothing. Foreign tables support MERGE like any Unity Catalog table.
Federation is a read path. Foreign tables are not writeable.

B. Federated queries are read-only. Writing needs the Spark data source interface.
Write access is one of the three documented reasons to choose the Spark data source interface instead.

C. MERGE is supported but only through catalog federation, not query federation.
Catalog federation is read-only too, apart from the legacy Hive metastore case.

D. Foreign tables must be cloned into Unity Catalog before they can be written to.
Cloning would write into Unity Catalog, not into the external system.

ON THE EXAM

A write requirement rules federation out on its own. It does not matter how well it would have handled the read.

> Data engineers currently paste a Snowflake connection string and service account password into each notebook that reads from it. A security review requires that no user holds those credentials, while analysts keep read access to two specific tables.
>
> The team is comfortable with the data staying in Snowflake.

What best meets the requirement?

A. Move the password into a Databricks secret scope and keep the current notebooks.

B. Create a connection and foreign catalog, then grant SELECT on the two tables.

C. Ingest both tables nightly with Lakeflow Connect and grant on the copies.

D. Use Delta Sharing to share the two Snowflake tables into Unity Catalog.

◆ PAUSE AND DECIDE

What best meets the requirement?

A. Move the password into a Databricks secret scope and keep the current notebooks.
Better than a literal, and anyone who can read the scope still holds the credential.

B. Create a connection and foreign catalog, then grant SELECT on the two tables.
Credentials live in the connection, so users query without ever holding them, and grants are table-level.

C. Ingest both tables nightly with Lakeflow Connect and grant on the copies.
Solves the credential problem by moving the data, which the team explicitly did not want.

D. Use Delta Sharing to share the two Snowflake tables into Unity Catalog.
Sharing works outward from a Databricks provider. It does not pull data in from Snowflake.

ON THE EXAM

This is the governance argument in its purest form: the credential stops being something a person holds and becomes something a grant confers.

> A federated query against a large Oracle table takes far longer than expected. The query filters on a date column and aggregates by region. The Oracle team reports seeing a full table scan.
>
> An engineer wants to know what Databricks actually sent.

How should they investigate?

A Enable query logging on the connection object in Unity Catalog.

B Open the Query Profile and read the foreign data source scan node, or run EXPLAIN FORMATTED.

C Check the Delta transaction log of the foreign table.

D Compare the initial and final plans to see what AQE changed.

◆ PAUSE AND DECIDE

How should they investigate?

A Enable query logging on the connection object in Unity Catalog.

Not a feature of connections.

B Open the Query Profile and read the foreign data source scan node, or run EXPLAIN FORMATTED.

Both show the statement actually pushed into the source, which tells you whether the filter went down.

C Check the Delta transaction log of the foreign table.

A foreign table is not a Delta table and has no transaction log.

D Compare the initial and final plans to see what AQE changed.

Right technique from module 13, wrong problem. AQE re-plans Databricks execution, not what is pushed down.

ON THE EXAM

A full table scan on their side alongside a filter on yours means the filter did not push down. The scan node is where you confirm it.

> A company is migrating from an external Hive metastore to Unity Catalog. The migration will take about a year, and during that time some tables must remain managed by the Hive metastore while new work happens in Unity Catalog.
>
> They want governed access to both from one place, at the lowest cost.

Which approach fits?

A Query federation against the Hive metastore, which pushes queries down over JDBC.

B Catalog federation, which reads the files directly in object storage using only Databricks compute.

C Migrate everything at once to avoid running two systems.

D Delta Sharing between the Hive metastore and Unity Catalog.

◆ PAUSE AND DECIDE

Which approach fits?

A Query federation against the Hive metastore, which pushes queries down over JDBC.

A Hive metastore holds metadata, not a query engine to push SQL into.

B Catalog federation, which reads the files directly in object storage using only Databricks compute.

The documented use case: incremental migration or a long-term hybrid, and cheaper because no external compute is involved.

C Migrate everything at once to avoid running two systems.

Ignores the stated constraint that some tables must remain for a year.

D Delta Sharing between the Hive metastore and Unity Catalog.

Sharing operates between Unity Catalog metastores, not from a Hive metastore.

ON THE EXAM

Catalog federation exists for exactly this shape of problem. Recognising incremental migration in a scenario is the cue.

A team federates a Salesforce source for a dashboard. It works well for a monthly report over a small slice of data. The business now wants the same data feeding an hourly pipeline over several hundred million rows, with tight latency targets.

The source is supported by both Lakehouse Federation and Lakeflow Connect.

What does Databricks recommend?

A Keep federation and increase the warehouse size to handle the volume.

B Use Lakeflow Connect, which Databricks recommends when volume or latency are priorities.

C Keep federation but enable result caching to reduce repeated load.

D Use catalog federation instead, since it is more performance-optimized.

◆ PAUSE AND DECIDE

What does Databricks recommend?

A Keep federation and increase the warehouse size to handle the volume.

The constraint is the source and the network, not Databricks compute.

B Use Lakeflow Connect, which Databricks recommends when volume or latency are priorities.

Stated directly: where a source supports both, prefer Lakeflow Connect when performance at volume or lower latency matters.

C Keep federation but enable result caching to reduce repeated load.

Caching is not supported for federated queries at all.

D Use catalog federation instead, since it is more performance-optimized.

True in general, and it is a different query path for catalog platforms rather than a faster federation setting.

ON THE EXAM

Federation is scoped to ad hoc reporting, BI and proof of concept. Growing past that scope is the signal to ingest instead.

MONITORING · DOMAIN 5 - MONITORING AND ALERTING

System tables

Section 5. The platform observing itself, and the traps in reading it.

SECTION 5 of 10 WEIGHT 10% TOPIC System tables

admin@sthithapragna.com

Everything the platform does is written to a table you can query. **Monitoring becomes SQL.**

Five things that are true of all of them

- They live in the **system catalog**, present in every Unity Catalog metastore.
- They are **free**. You pay only for the compute that queries them.
- They are **read-only** and cannot be modified.
- They cover **every workspace in the account in the same region**, not just the one you are in.
- They include data from **non-Unity Catalog workspaces**, even though you need a UC workspace to read them.

> ON THE EXAM
>
> That last pair is the useful one. One query answers a question across every workspace in the region, including ones never migrated to Unity Catalog.

Schemas worth knowing by name

SCHEMA	WHAT IT HOLDS
system.billing	Billable usage, and a historical log of SKU prices.
system.access	Audit events, plus table and column lineage.
system.compute	Cluster and warehouse configuration history, and node utilisation.
system.lakeflow	Jobs, tasks, pipelines and their run timelines.
system.query	Query history for warehouses and serverless compute.
system.storage	Predictive optimization history and related operations.

ON THE EXAM

Cost questions go to billing. Who-did-what goes to access. Why-was-it-slow goes to query and compute. Job outcomes go to lakeflow.

Who can read them

- Account admins and metastore admins have access **by default**.
- Everyone else needs three grants: **USE CATALOG** on system, **USE SCHEMA**, and **SELECT**.
- So access is granted with ordinary Unity Catalog statements, schema by schema.
- That granularity matters. Audit data and billing data rarely suit the same audience.

> ON THE EXAM
>
> Grant per schema rather than per catalog. A finance team needs billing; they almost certainly should not have audit.

Mostly a year, with real exceptions

RETENTION	EXAMPLES
365 days	Most tables, including billing usage, audit, jobs and query history.
180 days	MLflow tables, predictive optimization history.
90 days	Node utilisation timeline, model serving token usage.
30 days	Ingress access denied events.
Indefinite	Price history, node types, workspace metadata.

ON THE EXAM

Node utilisation at 90 days is the one that catches people. A year-on-year capacity comparison is not available from that table.

How a correction appears in billable usage

- When a usage record is wrong, Databricks does not edit it. It adds **two rows**.
- A **retraction** with a **negative** usage quantity that cancels the original, then a **restatement** with the correct values.
- The record type column marks each as original, retraction or restatement.
- So **filtering to original records only** gives you the uncorrected, wrong total.

> ON THE EXAM
>
> A plain sum is correct, because the retraction cancels the original. Being clever and filtering by record type is what breaks it.

When one SKU covers several products

- Some products **bill under the same SKU**. Data quality monitoring, predictive optimization and serverless workflows share one.
- Two columns separate them: **billing origin product** and **product features**.
- So grouping by SKU alone will attribute several different things to one line.
- For job-level cost, join usage metadata to the jobs tables rather than reading the SKU.

> ON THE EXAM
>
> Cost attribution questions usually turn on joining billing to lakeflow or compute. The billing table alone rarely answers 'which job'.

Slowly changing dimensions, and deleted rows

- The jobs, tasks and pipelines tables are **slowly changing dimension type 2**.
- A configuration change emits a **new row**, so the table holds the full history of every entity.
- **A deletion is a new row with a delete time populated**, not the removal of a row.
- So a naive query returns several rows per job. Filter to the current version, or you will double count.

> ON THE EXAM
>
> This is module 8's SCD Type 2 appearing inside the platform's own tables. The same reading rules apply.

Reading system tables incrementally

- Many system tables support streaming reads, which is how you build a live monitoring pipeline.
- **Set skip change commits to true**, or deletes in the system table will disrupt your stream.
- System tables use the default **seven day** vacuum retention.
- So a stream that falls more than seven days behind **breaks**. It is not merely slow.

> ON THE EXAM
>
> A weekly-scheduled stream over a system table is living exactly on the edge of that seven-day window.

Schema drift, and queries that are too broad

- **New columns may be added to system tables at any time.** Existing columns will not change or be removed.
- So a job writing system table data elsewhere should have **schema evolution enabled**.
- A query that is **not selective enough** fails outright, asking for more selective predicates.
- There is also **no real-time monitoring**. Data arrives through the day, so a very recent event may be missing.

> ON THE EXAM
>
> Select star from a system table with no filter is not a slow query. It is an error message.

Do not move this data off the platform

- Databricks **highly discourages** moving system table data outside the platform.
- It can expose sensitive data and put the deployment at risk.
- Audit tables record who accessed what. Lineage records what depends on what.
- If you must export, you are responsible for its security and for preventing misuse.

> ON THE EXAM
>
> An answer that exports audit logs to an external tool for convenience is proposing something the documentation explicitly warns against.

What to carry into the exam

- System catalog, free, read-only, and covering **every workspace in the region**.
- Billing for cost, access for audit and lineage, lakeflow for jobs, query and compute for performance.
- Corrections are a **retraction plus a restatement**. Sum everything; do not filter to original.
- Jobs and pipelines are **SCD Type 2**. Deleted means a delete time, not a missing row.
- Retention is 365 days for most, but **90 for node utilisation** and 30 for ingress denials.

> ON THE EXAM
>
> When a question asks how you would find something out across the whole account, system tables are almost always the answer.

> A finance analyst builds a monthly cost report from the billable usage system table. To avoid double counting, they filter to records where the record type is ORIGINAL.
>
> The reported total is consistently higher than the invoice, and the gap varies month to month.

What is wrong?

A. Filtering to ORIGINAL excludes the retractions that cancel incorrect records, so uncorrected usage is counted.

B. The usage table reports list prices rather than negotiated prices, so it always exceeds the invoice.

C. The analyst is missing a filter on the account identifier, so other accounts are included.

D. Usage records outside the 365-day retention window are being double counted.

◆ PAUSE AND DECIDE

What is wrong?

A Filtering to ORIGINAL excludes the retractions that cancel incorrect records, so uncorrected usage is counted.

A correction is a negative retraction plus a restatement. Excluding retractions leaves the wrong figures in.

B The usage table reports list prices rather than negotiated prices, so it always exceeds the invoice.

The usage table records quantities. A consistent multiplier would not vary month to month.

C The analyst is missing a filter on the account identifier, so other accounts are included.

System tables cover your own account only.

D Usage records outside the 365-day retention window are being double counted.

Expired records are removed, not duplicated.

ON THE EXAM

The filter was added to prevent double counting and caused the opposite. Sum everything and let the retractions cancel.

A platform team wants a report of cost per job. They join the billable usage table to the jobs table on job identifier and sum the usage quantity.

The totals are far higher than expected, and jobs whose definitions have been edited frequently look the most expensive.

What is causing this?

A Frequently edited jobs genuinely cost more because each edit triggers a full recompute.

B The jobs table is SCD Type 2, so a job with many versions joins to usage many times.

C The usage table stores cost in a different currency for edited jobs.

D Job identifiers are reused after deletion, merging unrelated jobs.

◆ PAUSE AND DECIDE

What is causing this?

A Frequently edited jobs genuinely cost more because each edit triggers a full recompute.

Editing a job definition does not run it.

B The jobs table is SCD Type 2, so a job with many versions joins to usage many times.

Every configuration change emits a new row. Joining without filtering to the current version multiplies the usage.

C The usage table stores cost in a different currency for edited jobs.

Invented.

D Job identifiers are reused after deletion, merging unrelated jobs.

Identifiers are not reused, and deletion is recorded rather than removing the row.

ON THE EXAM

The correlation with edit frequency is the tell. More edits means more rows, and more rows means a larger multiplier.

> A capacity planning exercise needs to compare cluster node utilisation this quarter against the same quarter last year, using the node timeline system table.
>
> The query returns no rows for the earlier period, though the clusters certainly existed then.

What is the explanation?

A The node timeline table has a 90-day retention period, so last year's data no longer exists.

B Node utilisation is only recorded for clusters that are currently running.

C The system schema must be enabled retroactively to backfill historical data.

D Utilisation data is regional, and the clusters ran in another region.

◆ PAUSE AND DECIDE

What is the explanation?

A The node timeline table has a 90-day retention period, so last year's data no longer exists.

One of the documented exceptions to the usual 365 days. A year-on-year comparison is simply not available from it.

B Node utilisation is only recorded for clusters that are currently running.

It records history for terminated clusters too, within retention.

C The system schema must be enabled retroactively to backfill historical data.

There is no retroactive backfill.

D Utilisation data is regional, and the clusters ran in another region.

Possible in general, and it would not produce a clean split at the retention boundary.

ON THE EXAM

Retention decides which questions are answerable at all. Ninety days for node utilisation rules out any year-on-year analysis.

> A security team asks for a monthly extract of the audit system table to be delivered into their own external SIEM tool, so that Databricks activity sits alongside other systems.
>
> A data engineer is asked whether this is a good idea.

What should they say?

A It is straightforward and recommended, since system tables are designed to be exported.

B Databricks highly discourages moving system table data off the platform, and responsibility for its security transfers to whoever exports it.

C It is impossible; system tables cannot be read by anything other than Databricks SQL.

D It is fine provided the extract is limited to the last 30 days.

◆ PAUSE AND DECIDE

What should they say?

A It is straightforward and recommended, since system tables are designed to be exported.

The documentation says the opposite, in its opening warning.

B Databricks highly discourages moving system table data off the platform, and responsibility for its security transfers to whoever exports it.

Stated directly. Audit and lineage data is sensitive, and exporting it moves the risk without removing it.

C It is impossible; system tables cannot be read by anything other than Databricks SQL.

They are ordinary readable tables. The constraint is advisory, not technical.

D It is fine provided the extract is limited to the last 30 days.

Volume is not the concern. Sensitivity is.

ON THE EXAM

The request is reasonable and the warning is explicit. The useful answer names the trade-off rather than simply refusing.

> A team builds a streaming job that reads the billable usage system table into their own cost dashboard table. It runs on a weekly schedule.
>
> After a public holiday shifts one run by several days, the stream fails and will not restart.

What is the most likely cause?

A. System tables use the default seven-day VACUUM retention, so a stream lagging beyond that breaks.

B. Streaming reads of system tables are only supported for the audit schema.

C. The weekly schedule exceeded the system table's 365-day retention window.

D. The stream must be recreated whenever new columns are added to the source.

◆ PAUSE AND DECIDE

What is the most likely cause?

A System tables use the default seven-day VACUUM retention, so a stream lagging beyond that breaks.

A weekly schedule already sits on the seven-day edge. A few days' delay pushes it past, and the files are gone.

B Streaming reads of system tables are only supported for the audit schema.

Many system tables support streaming, billing usage included.

C The weekly schedule exceeded the system table's 365-day retention window.

Off by two orders of magnitude.

D The stream must be recreated whenever new columns are added to the source.

New columns are a real hazard, and they call for schema evolution rather than causing a hard failure like this.

ON THE EXAM

The design was fragile before the holiday. A weekly stream over a seven-day vacuum window has no margin at all.

> A cost review shows a large and growing line under the serverless jobs SKU. The team cannot account for it from their job schedule, which has not changed.
>
> They are asked to identify what is actually driving the spend.

What should they do?

A. Group by SKU alone, since the SKU uniquely identifies the product.

B. Break the usage down by billing origin product and product features, since several products share that SKU.

C. Check the price history table, since a SKU price increase would explain it.

D. Compare against the node utilisation table to find over-provisioned clusters.

◆ PAUSE AND DECIDE

What should they do?

A Group by SKU alone, since the SKU uniquely identifies the product.

It does not. Several products share the serverless jobs SKU.

B Break the usage down by billing origin product and product features, since several products share that SKU.

Data quality monitoring, predictive optimization and serverless workflows all bill under it. These two columns separate them.

C Check the price history table, since a SKU price increase would explain it.

Worth a glance, and it would not explain growth when the schedule is unchanged.

D Compare against the node utilisation table to find over-provisioned clusters.

Serverless has no clusters of yours to over-provision.

ON THE EXAM

Predictive optimization is a likely culprit here, and it bills under the serverless jobs SKU while running automatically.

MONITORING · DOMAIN 5 - MONITORING AND ALERTING

Reading the Spark UI and the Query Profiler

Section 5. Where the evidence lives, and how to read it in order.

SECTION 5 of 10 WEIGHT 10% TOPIC Profiling

admin@sthithapragna.com

The profile tells you **what the query did**. The Spark UI tells you **how the work was distributed**. Slow queries usually need both.

What the query summary gives you before you dig

METRIC	WHAT IT TELLS YOU
Wall-clock duration	Total elapsed time, broken into scheduling, optimisation and pruning, and execution.
Aggregated task time	Combined time across all cores of all nodes.
Query metrics	Includes filter icons showing the percent of data pruned during scanning.
Top operators	The most expensive operators in the query, ranked.
Input and output	How much data was actually read and written.

ON THE EXAM

The breakdown of wall-clock time matters. A query that spent most of its time in scheduling has a queueing problem, not a query problem.

Aggregated task time against wall-clock

- **Aggregated task time much longer than wall-clock** means work ran in parallel. That is healthy.
- **Aggregated task time shorter than wall-clock** means tasks were **waiting for available nodes**.
- So the second case points at compute contention rather than at the query.
- Both directions are documented behaviour. Neither is an error.

> ON THE EXAM
>
> Comparing these two numbers is the fastest way to tell a slow query from a query that was waiting its turn.

When there is no profile at all

- **A query served from the query cache has no profile.** You get a message saying it is not available.
- The documented workaround: make a **trivial change** to the query, such as altering or removing the LIMIT.
- That bypasses the cache and forces a real execution you can profile.
- It is also a reminder that a fast second run may be telling you nothing about the query.

> ON THE EXAM
>
> Profiling a cached query is measuring the cache. Change something trivial, then profile the run that actually did work.

Six operations, and which one to worry about

OPERATOR	WHAT IT DID
Scan	Read data from a source and produced rows.
Filter	Applied criteria and returned a subset.
Join	Combined rows from several relations.
Union	Concatenated rows from relations sharing a schema.
Hash / Sort	Grouped rows by a key and applied an aggregate.
Shuffle	Redistributed data between executors. Explicitly called expensive.

ON THE EXAM

The documentation singles out shuffle as expensive because it moves data between executors. That is module 13's point, stated here as a reading instruction.

Three metrics, one at a time

- The graph can display **time spent**, **memory peak**, or **rows**. Switch between them.
- Time spent finds the slow operator. **Rows** finds the operator producing more than it should.
- An operator emitting far more rows than it consumed is the signature of an **exploding join**.
- The Top operators panel gives you the same ranking without reading the graph.

> ON THE EXAM
>
> Switching the graph to rows is the underused move. Time tells you where the cost landed; rows often tells you why.

Verbose mode, and a Photon caveat

- Metrics for some operations are **hidden by default**, on the basis that they are unlikely to be bottlenecks.
- **Enable verbose mode** from the kebab menu to see every operation and additional metrics.
- **Non-Photon operations executed as a group share metrics**, so each shows the parent operator's value.
- So identical numbers across several operators may be an artefact, not a coincidence.

> ON THE EXAM
>
> If several adjacent operators report exactly the same figure, check whether they are grouped before drawing a conclusion from it.

Performance insights

- Databricks surfaces insights automatically, **ranked by projected effect on total task duration**.
- They appear as a summary in the query details panel, and in full in a **Performance insights** tab.
- Where actionable, an **Optimize** button opens Genie Code, which rewrites the query for your approval.
- For table or compute changes it summarises the recommendation in plain language instead.

> ON THE EXAM
>
> Ranked by projected effect is the useful part. It tells you which of five suggestions is worth doing first.

Two routes, and one that does not persist

- If the other person has **CAN MANAGE** on the query, share the URL directly.
- Otherwise **download the profile as JSON**, and they import it from the query history.
- An imported profile is loaded into the **browser session only**. It does not persist in the workspace.
- That makes JSON the route for anyone outside the workspace, including support.

> ON THE EXAM
>
> The JSON route works for someone who is not a workspace member at all, which is how a profile reaches Databricks support.

The profile is not only in query history

- From the **SQL editor**, via the link showing elapsed time and rows returned.
- From a **notebook** attached to a warehouse or serverless compute, via See performance.
- From the **Lakeflow pipelines UI**, in its Query History tab.
- From the **jobs UI**, for jobs run on warehouses and serverless compute.

> ON THE EXAM
>
> That pipelines entry point matters. It means a declarative pipeline's queries can be profiled like any other, which is not obvious.

Open in Spark UI

- For Databricks SQL queries, the kebab menu offers **Open in Spark UI**.
- That is the move once the profile has told you **which operator** is expensive.
- The profile works in operators. The Spark UI works in **stages and tasks**.
- Skew and spill are task-level properties, so the profile alone will not show them clearly.

> ON THE EXAM
>
> Use the profile to find the operator, then the Spark UI to find out what the tasks inside it were doing.

Start at the longest stage, and get three numbers

- Open **Stages**, sort by duration descending, and take the longest one. Everything else is noise until you understand it.
- Inside it, read the **median task duration**, the **maximum**, and the **shuffle read** figures.
- The **ratio of maximum to median** is your first discriminator.
- Everything else in that interface is confirmation, not discovery.

> ON THE EXAM
>
> Discipline matters more than knowledge here. The Spark UI shows hundreds of numbers, and three of them do most of the work.

What the pattern of task times means

PATTERN	LIKELY CAUSE
A few tasks far slower than the median, with skewed shuffle read	Data skew
Spill appearing on most tasks, not just outliers	Memory pressure
All tasks uniformly slow	Insufficient resources, or too few partitions
High garbage collection time across executors	Memory pressure again

ON THE EXAM

A few slow tasks means skew. Uniformly slow tasks means resources. That single distinction resolves most of these questions.

Wall-clock is not evidence

- A single faster run is not validation. Cluster warmth, caching and cluster load all move wall-clock time.
- Confirm that the **underlying metric moved**: less spill, lower garbage collection, a balanced max-to-median ratio.
- Before manual tuning, check whether **adaptive execution** already handled it. Much tuning advice predates it being default.
- And remember that a cached query has no profile at all, so a fast rerun may not have run.

> ON THE EXAM
>
> This is the same instinct as module 13. Prove the mechanism changed, not just that the number got smaller.

What to carry into the exam

- You need **CAN MONITOR** on the warehouse, or ownership of the query, to see a profile.
- **Cached queries have no profile.** Change the LIMIT to force a real run.
- **Aggregated task time shorter than wall-clock** means tasks waited for nodes.
- Switch the graph to **rows** to find an exploding join. Insights are ranked by projected effect.
- Profile finds the operator. **Spark UI** finds the task pattern: few slow tasks is skew, all slow is resources.

> ON THE EXAM
>
> Two tools, two altitudes. Reach for the profile first, and cross to the Spark UI once you know which operator to interrogate.

> An analyst reports that a dashboard query takes 40 seconds. An engineer opens the query profile and sees the wall-clock duration is 38 seconds, while the aggregated task time is 6 seconds.
>
> The query itself is a simple aggregate over a well-clustered table, and the operators show no obvious problem.

What should they investigate?

A The query logic, since a simple aggregate should not take 38 seconds.

B Compute availability, since aggregated task time far below wall-clock means tasks waited for nodes.

C Data skew, since one task must be dominating the total.

D File layout, since poor data skipping would explain the elapsed time.

◆ PAUSE AND DECIDE

What should they investigate?

A The query logic, since a simple aggregate should not take 38 seconds.

The work took six seconds. The query is not the problem.

B Compute availability, since aggregated task time far below wall-clock means tasks waited for nodes.

Documented behaviour: task time shorter than wall-clock indicates tasks waiting for available nodes.

C Data skew, since one task must be dominating the total.

Skew would show as a long aggregated task time with an uneven distribution, not a short one.

D File layout, since poor data skipping would explain the elapsed time.

Bad pruning means more data scanned, which raises task time rather than lowering it.

ON THE EXAM

Six seconds of actual work inside thirty-eight seconds of elapsed time. Thirty-two seconds were spent somewhere other than computing.

> An engineer changes a query, reruns it, and sees it complete in two seconds instead of ninety. Pleased, they open the query profile to document the improvement.
>
> The profile page says a query profile is not available.

What has happened?

A The optimisation was so effective that no profile was generated.

B The result came from the query cache, which produces no profile. The two-second run may not prove anything.

C Profiles are only generated for queries exceeding a minimum duration.

D The engineer lacks CAN MONITOR permission on the warehouse.

◆ PAUSE AND DECIDE

What has happened?

A The optimisation was so effective that no profile was generated.

Profiles are produced regardless of duration.

B The result came from the query cache, which produces no profile. The two-second run may not prove anything.

Documented: no profile exists for a cached result. Change the LIMIT trivially to force a real execution.

C Profiles are only generated for queries exceeding a minimum duration.

No such threshold exists.

D The engineer lacks CAN MONITOR permission on the warehouse.

That gives a permission error, not a 'not available' message.

ON THE EXAM

The missing profile is itself the diagnosis. It tells you the query did not actually run.

> A join between two large tables is the slowest operator in a profile. The engineer opens the Spark UI, sorts stages by duration, and examines the longest stage.
>
> Median task duration is 4 seconds. Maximum task duration is 26 minutes. Shuffle read is similarly lopsided. Spill appears on only two tasks.

What is the diagnosis?

A Memory pressure, because spill is present.

B Data skew, because a few tasks far exceed the median and shuffle read is also skewed.

C Insufficient cluster resources, because the maximum duration is very high.

D Too few shuffle partitions, because the median is only 4 seconds.

◆ PAUSE AND DECIDE

What is the diagnosis?

A. Memory pressure, because spill is present.

Spill on two tasks out of many is a consequence of skew. Memory pressure shows spill on most tasks.

B. Data skew, because a few tasks far exceed the median and shuffle read is also skewed.

Both conditions hold: an enormous max-to-median ratio, and shuffle read lopsided the same way.

C. Insufficient cluster resources, because the maximum duration is very high.

Resource limits make all tasks slow. The median here is four seconds.

D. Too few shuffle partitions, because the median is only 4 seconds.

Too few partitions would make tasks uniformly large, raising the median.

ON THE EXAM

Spill on two tasks is the trap. Skew causes localised spill; memory pressure causes widespread spill.

> A query takes eleven minutes. The profile shows a join operator consuming most of the time. The engineer switches the graph metric from time spent to rows and sees the join consuming 4 million rows and emitting 900 million.
>
> The join condition matches on a customer identifier.

What does this indicate?

A. An exploding join, likely caused by duplicate keys on one side.

B. A broadcast join that failed and fell back to sort merge.

C. Poor data skipping, since too many rows were scanned.

D. Data skew, since the row count is unbalanced.

◆ PAUSE AND DECIDE

What does this indicate?

A An exploding join, likely caused by duplicate keys on one side.

Output vastly exceeding input is the definition. Duplicate keys multiply matching rows, and the docs name exploding joins explicitly.

B A broadcast join that failed and fell back to sort merge.

A strategy change alters cost, not row counts.

C Poor data skipping, since too many rows were scanned.

That would show at the Scan operator, not as growth across a join.

D Data skew, since the row count is unbalanced.

Skew is about distribution across tasks, not about total output exceeding input.

ON THE EXAM

Switching the graph to rows is what surfaced this. Time alone would have told you the join was slow and not why.

> A team is debugging a failing query with Databricks support. Support asks for the query profile. The support engineer is not a member of the workspace.
>
> A team member tries to send the profile URL and support cannot open it.

What is the correct approach?

A Grant the support engineer CAN MONITOR on the warehouse so the URL works.

B Download the query profile as JSON and send that; support imports it into their own session.

C Take screenshots of each operator in the DAG.

D Export the query history system table and send the relevant rows.

◆ PAUSE AND DECIDE

What is the correct approach?

A Grant the support engineer CAN MONITOR on the warehouse so the URL works.

Adds an external identity to the workspace to share one diagnostic, when a file does the job.

B Download the query profile as JSON and send that; support imports it into their own session.

The documented route for anyone without CAN MANAGE or outside the workspace entirely.

C Take screenshots of each operator in the DAG.

Loses every metric that is not on screen, and cannot be explored.

D Export the query history system table and send the relevant rows.

Query history is not the profile, and module 17 warns against exporting system table data.

ON THE EXAM

The JSON route exists precisely for people outside the workspace. Note it loads into their browser session and does not persist.

> An engineer is asked why a pipeline's gold materialized view refresh is slow. They say query profiles are only available for Databricks SQL queries, so the pipeline cannot be profiled and they will have to read cluster logs instead.
>
> The pipeline runs on serverless.

Is that correct?

A Yes. Query profiles apply only to SQL warehouse queries.

B No. The Lakeflow pipelines UI has a Query History tab from which profiles can be opened.

C No, but only if the pipeline is switched to a SQL warehouse first.

D Yes, unless verbose mode is enabled on the pipeline.

◆ PAUSE AND DECIDE

Is that correct?

A Yes. Query profiles apply only to SQL warehouse queries.

Profiles are reachable from the pipelines UI, the jobs UI and notebooks as well.

B No. The Lakeflow pipelines UI has a Query History tab from which profiles can be opened.

A documented entry point. Pipeline queries are profiled like any others.

C No, but only if the pipeline is switched to a SQL warehouse first.

No such change is needed.

D Yes, unless verbose mode is enabled on the pipeline.

Verbose mode reveals hidden metrics within a profile. It does not create one.

ON THE EXAM

A common and costly assumption. Reaching for cluster logs when a profile is one tab away wastes a great deal of time.

MONITORING · DOMAIN 5 - MONITORING AND ALERTING

Event logs, alerts and notifications

Section 5. Four objectives, and one question: how do you find out something went wrong?

SECTION 5 of 10 WEIGHT 10% TOPIC Alerting

admin@sthithapragna.com

Monitoring is what you can go and look at.
Alerting is what comes and finds you.
Production needs both.

admin@sthithapragna.com

Everything a pipeline did, in one table

- Contains **audit logs, data quality checks, pipeline progress and data lineage**. All four.
- Readable from the pipelines UI, the pipelines API, or by querying it directly as a table.
- The **details** field holds JSON, parsed with the colon operator.
- In Unity Catalog, views support streaming, so the event log can be **read as a stream**.

> ON THE EXAM
>
> Expectation metrics from module 14 live here. This is where you go to see whether quality is degrading over weeks rather than in one run.

Hidden by default, and owned by one person

- By default it is a **hidden Delta table** in the pipeline's default catalog and schema.
- Named **event log, then the pipeline ID**, with the dashes replaced by underscores.
- By default **only the pipeline owner can query it**.
- You can **publish it** from the pipeline's Advanced settings, which makes it an ordinary named table.

> ON THE EXAM
>
> Hidden is not the same as inaccessible. Sufficiently privileged users can query it; the default is simply that only the owner is.

Things that will break a pipeline

- **Do not delete the event log**, or the catalog or schema it was published to.
- Doing so **may cause the pipeline to fail on future updates**. It is not merely a loss of history.
- The event log location also serves as the **schema location for any Auto Loader queries** in that pipeline.
- So a tidy-up that removes it takes the Auto Loader schema tracking with it.

> ON THE EXAM
>
> That second point explains the first. The event log location is doing more work than its name suggests.

Create a view before granting access

- Databricks **recommends creating a view** over the event log table before modifying privileges.
- Some compute settings may let users reach **schema metadata** if the table itself is shared directly.
- The view is also what the documented example queries are written against.
- That is the same instinct as a masking view in module 15: expose the result, not the object.

> ON THE EXAM
>
> A recommendation with a security reason behind it, which makes it more likely to appear as an answer than a style preference would.

The event log table-valued function

- There is an **event log function** that returns the log for a pipeline or a streaming table.
- It **only applies to Unity Catalog pipelines using legacy publishing mode**. Otherwise, query the table.
- It can be called **only by the owner**, and a view built on it **cannot be shared**.
- So for a modern pipeline, the answer is the published table rather than the function.

> ON THE EXAM
>
> This is a function you will meet in older material and mostly should not use. Recognise it, and know it does not apply to current pipelines.

Event hooks and the account-wide view

- **Event hooks** define custom actions to run when events are logged, such as sending an alert.
- There is also a **system table** holding pipeline event log entries for the whole region.
- That is the route when you want one query across every pipeline rather than one at a time.
- Remember its retention and delivery caveats from module 17. It is not real-time.

> ON THE EXAM
>
> Per-pipeline event log for depth on one pipeline. System table for breadth across all of them.

A query, a condition, and a notification

- Now **generally available**. Authored in the SQL editor, evaluated on a schedule or inside a job.
- Databricks names the uses directly: a table **not refreshed within an hour**, row counts below baseline, null rates above a threshold.
- Also duplicate keys, distribution shift, and business-metric drift.
- So this is the mechanism for **data** quality, as opposed to **job** outcomes.

> ON THE EXAM
>
> Expectations catch bad records as they pass. Alerts catch the absence of records, and conditions that only appear in aggregate.

The SQL alert task, and its status

- A **SQL alert task** evaluates an alert as a step in a job, on a warehouse you specify.
- The task reports **Succeeded whenever the alert evaluated successfully**, whether or not the condition triggered.
- **Failed** means an evaluation error, such as a warehouse connectivity problem or a bad query.
- It evaluates the alert **independently of any schedule** set on the alert itself.

> ON THE EXAM
>
> Task status tells you whether the check ran, not what it found. That distinction is exactly the kind of thing a professional paper tests.

How the alert outcome reaches the rest of the job

- The task exposes an **alert state** value, compared against **TRIGGERED**.
- An **If/else task** reads that and branches, which is module 9's control flow applied to data quality.
- So you can halt publication, or run a remediation path, when a check fails.
- Relying on task status instead would branch on whether the check ran, not on its verdict.

> ON THE EXAM
>
> This is the pattern the objective is really pointing at: a quality gate that stops bad data reaching consumers.

Five triggers

TRIGGER	FIRES WHEN
On start	The run begins.
On success	The run completes successfully.
On failure	Internal error, or a failed or timed-out result.
Duration warning	The run exceeds the configured run duration threshold.
Streaming backlog	A streaming backlog threshold is exceeded.

ON THE EXAM

A maximum of three destinations per trigger. Both email and webhook destinations are available for each.

The health metrics behind the triggers

- Duration warnings use a **run duration in seconds** metric set in the job's health rules.
- Streaming backlog thresholds come in four flavours: **bytes, records, seconds and files**.
- Seconds is usually the one that matches a business requirement, since it expresses lateness.
- A notification without a configured threshold has nothing to fire on.

> ON THE EXAM
>
> Choosing the metric is a modelling decision. Backlog in records tells you volume; backlog in seconds tells you how stale your data is.

How backlog alerting actually behaves

- Alerting is based on the **10-minute average** of the metric, not on an instantaneous spike.
- If the issue persists, notifications are **resent every 30 minutes**.
- That averaging is deliberate. A brief burst will not wake anybody up.
- **Streaming backlog notifications are not supported for pipeline tasks.**

> ON THE EXAM
>
> So a declarative pipeline needs a different route for backlog alerting. The event log carries backlog metrics in its flow progress events.

The API and command line

- The **Jobs API** lists runs and returns run detail, which is how an external system learns a job's outcome.
- The **pipelines API** exposes updates and event log entries in the same way.
- The **command line** wraps both, which is what makes monitoring scriptable from a delivery pipeline.
- Use these when something **outside** Databricks needs to know. Use notifications when a person does.

> ON THE EXAM
>
> The distinction is who is asking. A human wants a notification; an orchestrator wants an API it can poll or a webhook it can receive.

What to carry into the exam

- The event log holds **audit, quality, progress and lineage**. Hidden and owner-only by default.
- **Do not delete it.** It is also the Auto Loader schema location for that pipeline.
- **SQL alerts** monitor the data. **Job notifications** monitor the job. Different objects, different questions.
- A SQL alert task reports **Succeeded when it evaluated**, not when it triggered. Branch on alert state.
- Backlog alerting is a **10-minute average**, resent every 30, and **not supported for pipeline tasks**.

> ON THE EXAM
>
> That completes Section 5 at 10%. Section 6, cost and performance, is next and is the second-largest section at 13%.

> A platform team wants three engineers to monitor data quality trends across a pipeline by querying its event log. The pipeline was created with default settings and none of the three is the owner.
>
> An engineer suggests they each call the event log table-valued function with the pipeline ID.

Why will that not work, and what should they do?

A The function can only be called by the owner. Publish the event log, then create a view and grant on it.

B The function requires DBR 13.3 LTS, so they need to upgrade their compute.

C The event log must be enabled in Advanced settings before any query works.

D They should each be made owner of the pipeline in turn.

◆ PAUSE AND DECIDE

Why will that not work, and what should they do?

A The function can only be called by the owner. Publish the event log, then create a view and grant on it.

Two documented facts: the function is owner-only and its views cannot be shared, and publishing plus a view is the recommended route.

B The function requires DBR 13.3 LTS, so they need to upgrade their compute.

A real requirement, and not the blocker here.

C The event log must be enabled in Advanced settings before any query works.

It exists by default as a hidden table. Publishing changes where it lives, not whether it exists.

D They should each be made owner of the pipeline in turn.

Changes who the pipeline runs as, to solve a read-access problem.

ON THE EXAM

Also note the function applies only to legacy publishing mode. For a current pipeline it is the wrong tool twice over.

> During a workspace clean-up, an old catalog is deleted. It contained the published event log for a production pipeline. Nobody was reading that log and the team considered it disposable.
>
> The next scheduled pipeline update fails.

What is the most likely explanation?

A Deleting the event log breaks the pipeline, and the location also served as the Auto Loader schema location.

B The pipeline cannot start without a place to write its first event, but it will recover on the next run.

C The catalog deletion revoked the pipeline owner's permissions.

D The pipeline is fine; the failure must be unrelated.

◆ PAUSE AND DECIDE

What is the most likely explanation?

A Deleting the event log breaks the pipeline, and the location also served as the Auto Loader schema location.

The docs warn against deleting it for exactly this reason, and the location does double duty as the schema location.

B The pipeline cannot start without a place to write its first event, but it will recover on the next run.

Half right on the cause, and wrong that it self-heals.

C The catalog deletion revoked the pipeline owner's permissions.

Permissions are not the mechanism described in the warning.

D The pipeline is fine; the failure must be unrelated.

The documentation explicitly warns that deletion may cause future updates to fail.

ON THE EXAM

An event log looks like history and is partly infrastructure. That is why the warning exists.

A team adds a SQL alert task to a job, checking that a loaded table's row count exceeds a baseline. Downstream tasks depend on the alert task with the default All succeeded condition.

A load arrives with almost no rows. The alert triggers and notifies the team, and the downstream publication tasks run anyway.

Why did publication not stop?

A The alert task reports Succeeded whenever it evaluates, whether or not the condition triggered.

B Run-if conditions do not apply to SQL alert tasks.

C The alert's own schedule overrode the task evaluation.

D The alert condition was evaluated after the downstream tasks began.

◆ PAUSE AND DECIDE

Why did publication not stop?

A The alert task reports Succeeded whenever it evaluates, whether or not the condition triggered.

Task status reflects whether the check ran, not its verdict. Branch on the alert state with an If/else task instead.

B Run-if conditions do not apply to SQL alert tasks.

They apply normally. The problem is what the status means.

C The alert's own schedule overrode the task evaluation.

The task evaluates independently of any schedule on the alert.

D The alert condition was evaluated after the downstream tasks began.

Dependencies are respected; the alert task completed first.

ON THE EXAM

This is module 9's distinction again: run-if branches on outcome, If/else branches on a value. The alert's verdict is a value.

> A nightly job normally completes in 20 minutes. Occasionally it takes four hours but still succeeds, and nobody notices until a morning dashboard is stale. The team has on-failure notifications configured.
>
> They want to be told when the job runs long, without failing it.

What should they configure?

A. A timeout, so the job fails after a set duration and the on-failure notification fires.

B. A run duration health threshold with a duration warning notification.

C. A SQL alert on the job's runtime in the system tables.

D. On-start notifications, so the team can time it themselves.

◆ PAUSE AND DECIDE

What should they configure?

A. A timeout, so the job fails after a set duration and the on-failure notification fires.

Meets the notification need by killing a job that would have finished. The question said without failing it.

B. A run duration health threshold with a duration warning notification.

Exactly what the trigger exists for: notify on a long run while letting it complete.

C. A SQL alert on the job's runtime in the system tables.

Would work, with system table delivery latency on top and a mechanism built by hand.

D. On-start notifications, so the team can time it themselves.

Notifies at the wrong moment and asks a human to be the monitor.

ON THE EXAM

Read the constraint: without failing it. That single phrase rules out the timeout answer that otherwise looks reasonable.

A declarative pipeline ingests from Kafka. The team wants to be notified when the pipeline falls more than ten minutes behind. They add a streaming backlog notification with a backlog seconds threshold to the job that runs the pipeline task.

The notification never fires, even during a confirmed two-hour backlog.

What is the explanation?

A Backlog alerting uses a 10-minute average, so a two-hour backlog is averaged away.

B Streaming backlog notifications are not supported for pipeline tasks. Use the event log's flow progress backlog metrics instead.

C Backlog seconds is not a supported metric; only bytes and records are.

D The threshold must be set on the pipeline rather than in the job's health rules.

◆ PAUSE AND DECIDE

What is the explanation?

A Backlog alerting uses a 10-minute average, so a two-hour backlog is averaged away.

The averaging smooths brief spikes. A sustained two-hour backlog would clear any average.

B Streaming backlog notifications are not supported for pipeline tasks. Use the event log's flow progress backlog metrics instead.

A documented limitation, and the event log carries backlog metrics in flow progress events.

C Backlog seconds is not a supported metric; only bytes and records are.

All four exist: bytes, records, seconds and files.

D The threshold must be set on the pipeline rather than in the job's health rules.

Health rules are the right place. The task type is the problem.

ON THE EXAM

The configuration is correct and the task type is unsupported. Nothing errors, which is why this is hard to spot.

> An external orchestration system outside Databricks must know whether a nightly job succeeded before it starts downstream work in another platform. Email notifications already go to the data team.
>
> An engineer proposes parsing the notification emails.

What is the better approach?

A Parse the emails, since they already contain the outcome.

B Use the Jobs API or a webhook notification, so the external system receives or retrieves the outcome directly.

C Grant the external system access to the workspace UI.

D Have the job write a status file to cloud storage that the other system polls.

◆ PAUSE AND DECIDE

What is the better approach?

A. Parse the emails, since they already contain the outcome.

Builds an integration on a human-readable format that can change without notice.

B. Use the Jobs API or a webhook notification, so the external system receives or retrieves the outcome directly.

The documented route for programmatic monitoring. A webhook pushes; the API is polled.

C. Grant the external system access to the workspace UI.

A UI is not an integration point.

D. Have the job write a status file to cloud storage that the other system polls.

Workable, and reinvents an interface that already exists, with a new failure mode if the write is missed.

ON THE EXAM

Ask who needs to know. A person gets a notification; a system gets an API or a webhook.

COST AND PERFORMANCE · DOMAIN 6 - COST & PERFORMANCE OPTIMIZATION

Managed tables and predictive optimization

Section 6. Why the maintenance job you were taught to write is now the wrong answer.

SECTION 6 of 10 WEIGHT 13% TOPIC Managed tables

admin@sthithapragna.com

The question is not where the files live. It is **who is responsible for keeping them tidy**.

Managed, external and foreign

	WHO MANAGES WHAT
Managed	Unity Catalog manages lifecycle, location and optimisations. Automatic optimisations: yes.
External	You specify the location and manage the lifecycle. Automatic optimisations: limited.
Foreign	An external system manages data and metadata. Automatic optimisations: none.

ON THE EXAM

Managed is described in the documentation as the default and recommended table type. That phrasing matters when a question asks what you should use.

Five things, and the last one is underrated

- **Automatic optimisation**, reducing both storage and compute costs.
- **Faster queries across all client types**, not only from Databricks.
- **Automatic table maintenance**, which is the maintenance job you no longer write.
- Secure access for non-Databricks clients through open APIs.
- **Automatic upgrades to the latest platform features.**

ON THE EXAM

That last one compounds. An external table stays on the feature set it was created with unless someone actively migrates it.

What happens when you drop it

- Dropping a **managed** table deletes the metadata **and the underlying data files**.
- Dropping an **external** table removes only the catalog metadata. The files remain.
- That is the safety argument people make for external tables, and it is a real one.
- It is also how organisations accumulate orphaned data nobody is paying attention to but everybody is paying for.

> ON THE EXAM
>
> Both halves are true. External protects you from an accidental drop, and leaves you paying for data that stopped being useful months ago.

Governed, but not optimised

- Foreign tables are registered through **query federation or catalog federation**, from module 16.
- Foreign tables backed by Delta Lake **lack many optimisations** available to managed tables.
- Databricks recommends migrating to managed tables for **production workloads or frequently queried datasets**.
- So federation is for reaching data. It is not a performance strategy.

> ON THE EXAM
>
> This is module 16's recommendation from the other direction. Federation gives you governance and access, not speed.

Predictive optimization runs **OPTIMIZE, VACUUM and ANALYZE** for you, deciding which tables need them and when.

The rollout, in dates

- **Enabled by default** for accounts created on or after **11 November 2024**.
- Existing accounts are being enabled by gradual rollout, **expected to complete by August 2026**.
- It runs **only on Unity Catalog managed tables**. An external table gets nothing from it.
- It is **not available in all regions**.

> ON THE EXAM
>
> The dates matter for a professional exam because they explain why two teams can describe completely different behaviour on the same platform.

Predictive optimization does not run Z-order

- **OPTIMIZE does not run ZORDER when executed by predictive optimization.**
- On a table that uses Z-order, predictive optimization **ignores the Z-ordered files**.
- So a Z-ordered table receives **less** automatic benefit than an unclustered one, not more.
- That is one of the clearest practical arguments for moving from Z-order to liquid clustering.

> ON THE EXAM
>
> Counterintuitive and precisely stated. A table you carefully Z-ordered years ago is now partly excluded from automatic maintenance.

Set retention before you enable it

- Vacuum retention comes from the **deleted file retention duration** table property, defaulting to **seven days**.
- Vacuum removes data files no longer referenced within that window, which **limits how far back time travel reaches**.
- To keep longer history, **set the property before enabling** predictive optimization.
- That is a deliberate decision, not a default to accept quietly, if audit or reproducibility matters.

> ON THE EXAM
>
> Automatic maintenance and long time travel are in tension. The property is where you resolve it, and the docs say to set it first.

How predictive optimization is billed

- It runs on **serverless compute for jobs**, and your account is billed for that compute.
- It appears under the **serverless jobs SKU**, shared with other products.
- So it shows up as a cost line that no job in your schedule explains.
- Break it out with the **billing origin product** column, exactly as module 17 described.

> ON THE EXAM
>
> Module 17's shared-SKU trap and this module's automatic maintenance are the same story from two directions.

Automatic liquid clustering

- Set **CLUSTER BY AUTO** on a managed unpartitioned or liquid table.
- Predictive optimization then **analyses how the table is queried** and selects clustering keys itself.
- It **evaluates whether the performance gain outweighs the clustering overhead** before changing anything.
- Keys can be updated over time as query patterns change, which manual clustering never does.

> ON THE EXAM
>
> This is the argument that layout decisions age badly. A key chosen correctly two years ago may be wrong now, and nobody revisits it.

Finding out what it decided

- **DESCRIBE EXTENDED** shows the reasons predictive optimization **skipped** a table.
- Typical reasons: the table is **already well-clustered**, or **too small to benefit** from compaction.
- Operation history is also tracked in a **system table**, with 180-day retention from module 17.
- So 'it did nothing' is usually a decision rather than a failure, and you can read the reason.

> ON THE EXAM
>
> When a question describes automatic maintenance apparently not running, the first move is to check why it chose not to.

Converting an external table to managed

- Use **ALTER TABLE SET MANAGED**. Requires DBR 17.0 or above, or serverless.
- Preferred over a create-table-as-select because it **retains history**, keeps permissions and views, and handles concurrent writes.
- It also supports **rolling back** to external, and minimises reader and writer downtime.
- **Cancel any running OPTIMIZE jobs first**, and schedule none during the conversion.

> ON THE EXAM
>
> Retaining history is the deciding advantage. A create-table-as-select gives you a new table with no past, and no time travel.

Two behaviours worth knowing

- From **DBR 18.1**, path-based reads and writes to the old external location **automatically redirect** to the managed location.
- That lets legacy code using storage paths keep working, at a slight performance cost.
- **Foreign table conversions do not redirect**, and predictive optimization is set to **inherit** rather than enabled.
- After a foreign conversion, **restart any streaming jobs** reading or writing the table.

> ON THE EXAM
>
> Databricks still recommends moving to name-based access. Redirect is a migration aid, not a destination.

What to carry into the exam

- **Managed is the default and recommended type.** External gives you the location and the maintenance.
- Dropping managed deletes the files. Dropping external leaves them, and the bill.
- Predictive optimization runs **OPTIMIZE, VACUUM and ANALYZE** on managed tables only.
- It **does not run ZORDER**, and **ignores Z-ordered files**. Z-order gets less automatic benefit, not more.
- Convert with **SET MANAGED** to retain history. Set vacuum retention **before** enabling.

> ON THE EXAM
>
> When an option proposes a scheduled weekly OPTIMIZE and VACUUM job on a managed table, ask what predictive optimization was already doing.

> A team maintains a weekly job that runs OPTIMIZE and VACUUM across forty Unity Catalog managed tables. Costs for that job have been questioned, and a review notes that query performance has not changed since predictive optimization was enabled on the account.
>
> The team argues the job is essential and should be kept.

What is the best assessment?

A. Keep the job. Predictive optimization supplements manual maintenance but does not replace it.

B. The job duplicates work predictive optimization already performs on managed tables. Retire it and verify with DESCRIBE EXTENDED and the system table.

C. Keep the job but reduce it to monthly, to halve the cost.

D. Convert the tables to external so the manual job remains meaningful.

◆ PAUSE AND DECIDE

What is the best assessment?

A Keep the job. Predictive optimization supplements manual maintenance but does not replace it.

It removes the need to manually manage maintenance operations on managed tables. That is its stated purpose.

B The job duplicates work predictive optimization already performs on managed tables. Retire it and verify with DESCRIBE EXTENDED and the system table.

All forty tables are managed, so the operations are already being run when they are worth running.

C Keep the job but reduce it to monthly, to halve the cost.

Halves the cost of work that is entirely redundant.

D Convert the tables to external so the manual job remains meaningful.

Gives up automatic maintenance to justify a job. The reasoning runs backwards.

ON THE EXAM

The evidence is in the scenario: performance did not change when predictive optimization arrived, because the work was already being done.

> Two teams run identical workloads on identical data. Team A reports that maintenance happens automatically and they have written no maintenance jobs. Team B reports that nothing happens automatically and they schedule OPTIMIZE weekly.
>
> Both use Unity Catalog managed tables. Team B's account was created in 2021; Team A's in 2025.

What is the most likely explanation?

A. Team B's tables are external despite what they believe.

B. Predictive optimization is default-on for accounts created from November 2024, and older accounts are still being enabled by a gradual rollout.

C. Team B has fewer tables, so predictive optimization judges them not worth optimising.

D. Predictive optimization requires an explicit table property that Team B has not set.

◆ PAUSE AND DECIDE

What is the most likely explanation?

A Team B's tables are external despite what they believe.

Possible in general, and the account dates are the detail the scenario actually gives you.

B Predictive optimization is default-on for accounts created from November 2024, and older accounts are still being enabled by a gradual rollout.

The account creation dates straddle the cutover exactly. This is why two teams describe different platforms.

C Team B has fewer tables, so predictive optimization judges them not worth optimising.

The workloads and data are stated to be identical.

D Predictive optimization requires an explicit table property that Team B has not set.

It is an account-level default, not a per-table property to opt into.

ON THE EXAM

The account creation dates are the only asymmetry in the scenario. When a question supplies dates, they are usually the answer.

> An engineer notices that one large managed table receives far less automatic maintenance than similar tables, despite being heavily queried. The table was carefully Z-ordered on two columns three years ago and the layout has not been revisited.
>
> Predictive optimization is enabled and working on other tables in the same schema.

What explains this?

A Predictive optimization does not run ZORDER, and ignores Z-ordered files on tables that use it.

B Z-ordered tables are already optimal, so no maintenance is required.

C The table exceeds a size threshold above which predictive optimization does not operate.

D Z-order must be re-run manually before predictive optimization will consider the table.

◆ PAUSE AND DECIDE

What explains this?

A Predictive optimization does not run ZORDER, and ignores Z-ordered files on tables that use it.

Stated directly. The Z-ordering is what excludes those files from automatic maintenance.

B Z-ordered tables are already optimal, so no maintenance is required.

A layout chosen three years ago against today's query patterns is unlikely to be optimal.

C The table exceeds a size threshold above which predictive optimization does not operate.

Skipping happens for tables too small to benefit, not too large.

D Z-order must be re-run manually before predictive optimization will consider the table.

Re-running Z-order produces more Z-ordered files to ignore.

ON THE EXAM

The engineering effort is causing the problem. Moving to liquid clustering, or automatic liquid clustering, restores full automatic maintenance.

> A regulated team must be able to reproduce the exact contents of a table as at any date in the past 90 days. They are about to migrate the table to Unity Catalog managed and enable predictive optimization.
>
> Nobody has changed any table properties.

What must they do?

A Nothing. Managed tables retain full history indefinitely.

B Set the deleted file retention duration to cover 90 days before enabling predictive optimization.

C Disable VACUUM entirely on the table after migration.

D Keep the table external, since managed tables cannot support extended time travel.

◆ PAUSE AND DECIDE

What must they do?

A Nothing. Managed tables retain full history indefinitely.

Retention defaults to seven days. Files outside it are removed by VACUUM.

B Set the deleted file retention duration to cover 90 days before enabling predictive optimization.

The documented sequence: set the property first, so automatic VACUUM respects the longer window.

C Disable VACUUM entirely on the table after migration.

Trades one problem for unbounded storage growth, and the property already expresses the requirement.

D Keep the table external, since managed tables cannot support extended time travel.

They support it. It is a property setting, not a table type limitation.

ON THE EXAM

Note the ordering in the answer. The documentation says to set the property before enabling, not afterwards.

> A team must convert 200 external Delta tables to managed. Requirements: retain time travel history, keep existing permissions and views, and avoid breaking legacy notebooks that read tables by storage path.
>
> An engineer proposes a create-table-as-select into new managed tables, then dropping the old ones.

What is the better approach, and why?

A The CTAS approach is fine; it produces clean tables with no legacy baggage.

B Use ALTER TABLE SET MANAGED, which retains history and permissions and redirects path-based access on recent runtimes.

C Use DEEP CLONE into a managed table, which copies history.

D Leave them external and enable predictive optimization on them individually.

◆ PAUSE AND DECIDE

What is the better approach, and why?

A The CTAS approach is fine; it produces clean tables with no legacy baggage.

It also produces tables with no history, and permissions and views must be rebuilt by hand.

B Use ALTER TABLE SET MANAGED, which retains history and permissions and redirects path-based access on recent runtimes.

Meets all three requirements, and is the documented recommendation over CTAS.

C Use DEEP CLONE into a managed table, which copies history.

Deep clone copies data and creates a new table. Permissions, views and path-based access still break.

D Leave them external and enable predictive optimization on them individually.

Predictive optimization runs only on managed tables. There is nothing to enable.

ON THE EXAM

Three requirements, and only one option meets all three. The path-based redirect is the one people do not know exists.

> A cost review flags a growing spend line that no scheduled job accounts for. Investigation shows it under the serverless jobs SKU. The team runs no serverless jobs at all.
>
> They recently migrated 300 tables to Unity Catalog managed.

What is the most likely explanation?

A An unauthorised serverless job is running in the account.

B Predictive optimization runs on serverless compute for jobs and bills under that SKU. Break it out by billing origin product.

C Managed tables are billed for storage under the serverless jobs SKU.

D The migration itself is still running and will stop billing when complete.

◆ PAUSE AND DECIDE

What is the most likely explanation?

A An unauthorised serverless job is running in the account.

A reasonable worry, and the documented explanation fits the facts far better.

B Predictive optimization runs on serverless compute for jobs and bills under that SKU. Break it out by billing origin product.

The migration made 300 tables eligible, and maintenance now runs automatically on all of them.

C Managed tables are billed for storage under the serverless jobs SKU.

Storage and compute are billed separately.

D The migration itself is still running and will stop billing when complete.

Growing spend suits ongoing maintenance rather than a finite migration.

ON THE EXAM

This is module 17's shared-SKU scenario with its cause supplied. The spend is real, expected, and usually cheaper than what it replaced.

COST AND PERFORMANCE · DOMAIN 6 - COST & PERFORMANCE OPTIMIZATION

Delta internals

Section 6. Statistics, data skipping, deletion vectors, and the change feed.

SECTION 6 of 10 WEIGHT 13% TOPIC Delta internals

admin@sthithapragna.com

The log is not a record of the table. **The log is the table.** Everything in this module is a consequence of that.

What gets recorded when you write

- Per file, Delta records the **minimum and maximum values**, the **null count**, and the **total records**.
- Collected **automatically on write**, to Delta Lake and to managed Iceberg tables.
- At query time those are used to **skip irrelevant files** without reading them.
- A column with no statistics collected cannot be used for skipping. The file gets read.

> ON THE EXAM
>
> Data skipping is not an index you build. It is a by-product of writing, used at planning time to avoid opening files.

The 32-column rule is now conditional

- The familiar rule: statistics are collected on the **first 32 columns**. That still applies to **Unity Catalog external tables**.
- For **Unity Catalog managed tables**, statistics columns are chosen intelligently by predictive optimization, with **no 32-column limit**.
- So a managed wide table can have useful statistics on a column in position 90.
- Any material asserting a universal 32-column limit is now out of date.

> ON THE EXAM
>
> This is one of the clearest cases where an older question bank teaches something that has since become conditional rather than wrong.

Two properties, and one important caveat

- **Number of indexed columns**: raises or lowers the count. It depends on **column order**.
- **Statistics columns**: names columns explicitly, and **supersedes** the numeric property. DBR 13.3 LTS and above.
- **Changing either does not recompute statistics for existing data.** It affects future writes only.
- And statistics are **not used at all** for columns outside the current list.

> ON THE EXAM
>
> Naming columns beats counting them, because the numeric property depends on where a column happens to sit in the schema.

Recomputing statistics

- From **DBR 14.3 LTS**, you can recompute statistics for a Delta table on demand.
- That is the fix when you widen the statistics list and existing files have none.
- Predictive optimization runs **ANALYZE** automatically on managed tables, so this is mostly a manual-management concern.
- Statistics are also **required** for any column used in a Z-order statement.

> ON THE EXAM
>
> Widening the list without recomputing gives you statistics on new files only, and queries still read every old one.

Deleting without rewriting

- Without them, **changing a single row rewrites the entire Parquet file** containing it.
- With them, delete, update and merge **mark rows as modified in metadata** instead.
- Reads then **apply the deletion vectors at query time** to resolve the current state.
- That trades a large write now for a small read cost later, which is usually a very good trade.

> ON THE EXAM
>
> This is why they belong to the cost objective. The saving is in write amplification, on tables where small updates are frequent.

Enabling them, and the version floors

- On **Delta Lake tables you must explicitly enable them**. They are not on by default.
- **All Apache Iceberg v3 tables include them by default.** UniForm Iceberg v2 does not support them.
- **Write** with all optimisations from **DBR 14.3 LTS**. **Read** from **DBR 12.2 LTS**.
- From DBR 14.2, tables with deletion vectors support **row-level concurrency**.

> ON THE EXAM
>
> Photon uses deletion vectors to power predictive I/O for updates, so enabling them also unlocks that acceleration.

When you actually need the rows gone

- A row marked deleted is **still in the Parquet file**. It is filtered out on read.
- To force a physical rewrite, use **REORG TABLE with the purge option**, then vacuum.
- By default purge scans **all Parquet footers**, checking for dropped columns and soft-deleted rows. On a large table that is slow.
- Set the purge mode to **rows** to limit it to files that actually contain soft-deleted rows.

ON THE EXAM

This is the module 26 connection. A compliance deletion is not complete when the deletion vector is written; it is complete after the purge and vacuum.

A streaming read tells you **what is new**. The change feed tells you **what changed**, including updates and deletes.

Three metadata columns

- **Change type**: insert, delete, and update as a **pre-image and a post-image pair**.
- **Commit version** and **commit timestamp**, saying exactly when the change happened.
- The pre-image and post-image pair is what lets a downstream consumer apply a real update rather than an insert.
- **You cannot enable it if your schema already has columns with those names.** Rename first.

> ON THE EXAM
>
> An update appears as two rows, not one. Downstream logic that counts rows will double-count updates unless it filters on change type.

Legacy and automatic change data feed

	HOW IT WORKS
Legacy	Enable the change data feed table property, per table. This is the form all older material describes.
Automatic	No property. Requires a managed Delta table with row tracking, or Iceberg v3.
Both	Use the same read options and the same table changes function.
Together	Not possible. Unset the property to move from legacy to automatic.

ON THE EXAM

The documentation now calls the table-property form legacy. Recognise both, since a question written from older material will use the property.

It is not a permanent record

- **Only changes made after enabling are recorded.** Past history is not reconstructed.
- Records are **transient**, available only within a retention window. **Vacuum deletes change data files.**
- Turning legacy change data feed **off and on again leaves the gap permanently unqueryable**.
- Reads always use the **schema of the latest table version**, not the schema as it was.

> ON THE EXAM
>
> An audit requirement that must survive years is not a change data feed requirement. It is a table you write to deliberately.

Streaming from a change feed

- Set the **read change feed** option on a batch or streaming read. Same option either way.
- Rate limits apply, and they apply **atomically to entire commits**. A commit is never split across batches.
- If you name a **starting version that has aged out of table history**, the stream fails to start from a new checkpoint.
- It also works with batch queries and with Databricks-to-Databricks sharing.

> ON THE EXAM
>
> Retention arithmetic again. If the starting version has been cleaned up, the stream cannot begin, which is the same shape as module 5's full refresh problem.

Some operations produce no change files

- **Insert-only writes** and **full-partition deletions** do not generate change data files.
- Databricks computes the feed **directly from the transaction log** in those cases.
- So the feed is still complete. Fewer files does not mean missing changes.
- This is also why enabling it usually costs less storage than people expect.

> ON THE EXAM
>
> The log already knows an insert-only commit added whole files. No separate change file is needed to describe it.

What to carry into the exam

- Statistics are **min, max, nulls and row count per file**, collected on write and used to skip files.
- **32 columns applies to external tables.** Managed tables have no such limit under predictive optimization.
- Changing statistics properties **does not recompute** existing data.
- Deletion vectors **mark rather than rewrite**. A compliance delete needs **REORG PURGE and VACUUM**.
- The change feed carries **updates and deletes** a streaming read cannot. It is not a permanent record.

> ON THE EXAM
>
> When a question describes a downstream consumer that needs to see updates and deletes incrementally, the change feed is almost always the answer.

> A wide Unity Catalog external table has 120 columns. Analysts filter constantly on a status column that sits at position 87. Queries scan the entire table every time and the team cannot work out why data skipping is not helping.
>
> The table is Delta format and statistics are otherwise working normally.

What is happening, and what is the fix?

A. Statistics are collected on the first 32 columns by default, so position 87 has none. Name it in the statistics columns property, then recompute.

B. Data skipping only works on partitioned tables. Partition by status.

C. The table needs OPTIMIZE before statistics take effect.

D. External tables do not support data skipping at all. Convert to managed.

◆ PAUSE AND DECIDE

What is happening, and what is the fix?

A Statistics are collected on the first 32 columns by default, so position 87 has none. Name it in the statistics columns property, then recompute.

The 32-column default applies to external tables, and naming columns supersedes the count. Recomputing is needed for existing files.

B Data skipping only works on partitioned tables. Partition by status.

Data skipping works on file statistics, independent of partitioning.

C The table needs OPTIMIZE before statistics take effect.

OPTIMIZE compacts files. It is not what collects statistics on an unindexed column.

D External tables do not support data skipping at all. Convert to managed.

They support it, with the 32-column default. Converting would remove the limit, which is a bigger change than needed.

ON THE EXAM

Note the second half of the answer. Naming the column fixes future writes only; existing files still need their statistics recomputed.

> A privacy team requires that records deleted under a right-to-erasure request are physically unrecoverable within 30 days. The engineering team runs DELETE statements on a Delta table with deletion vectors enabled and reports the records as removed.
>
> An auditor asks whether the data is actually gone from storage.

What is the correct answer?

A. Yes. DELETE removes the rows from the Parquet files immediately.

B. No. Deletion vectors mark rows as deleted; a REORG PURGE followed by VACUUM is required to rewrite and remove the files.

C. Yes, provided the table has been OPTIMIZEd since the delete.

D. No, and the only remedy is to disable deletion vectors and re-run the deletes.

◆ PAUSE AND DECIDE

What is the correct answer?

A. Yes. DELETE removes the rows from the Parquet files immediately.

That is the behaviour without deletion vectors. With them, the row is marked and filtered on read.

B. No. Deletion vectors mark rows as deleted; a REORG PURGE followed by VACUUM is required to rewrite and remove the files.

The row remains physically present until the file is rewritten and the old file vacuumed.

C. Yes, provided the table has been OPTIMIZEd since the delete.

Compaction may rewrite some files incidentally, which is not a guarantee of erasure.

D. No, and the only remedy is to disable deletion vectors and re-run the deletes.

Purge exists precisely so you can keep the write performance and still erase when required.

ON THE EXAM

The same feature that makes deletes fast is what makes them incomplete. Both facts are true and the requirement decides which matters.

> A silver table is fed by a streaming read from a bronze Delta table. Bronze now receives corrections that update existing rows. The silver stream fails, as it did in module 10's scenario.
>
> The team cannot move to a declarative pipeline before the next release, and needs an interim solution that propagates updates and deletes downstream.

What should they do?

A. Enable change data feed on bronze and read it with the read change feed option, handling each change type.

B. Enable skipChangeCommits so the stream ignores the updates.

C. Switch silver to a full batch reload each night.

D. Delete the checkpoint and restart the silver stream.

◆ PAUSE AND DECIDE

What should they do?

A Enable change data feed on bronze and read it with the read change feed option, handling each change type.

Exactly the objective: the change feed carries updates and deletes that a plain streaming read cannot express.

B Enable skipChangeCommits so the stream ignores the updates.

Stops the failure by discarding the corrections. Silver then holds values known to be wrong.

C Switch silver to a full batch reload each night.

Works, and abandons incremental processing and latency to avoid the problem.

D Delete the checkpoint and restart the silver stream.

Reprocesses everything and breaks again on the next correction.

ON THE EXAM

B is the dangerous option because it is a real setting that makes the error disappear. The corrections disappear with it.

> A compliance team asks for a complete row-level history of a customer table going back three years, for an investigation. Change data feed was enabled on the table eight months ago.
>
> The table has a standard vacuum retention and is vacuumed regularly.

What can they actually be given?

A Three years of row-level changes, since the transaction log retains everything.

B Only changes after CDF was enabled, and only within the retention window, since VACUUM deletes change data files.

C Eight months of changes, since CDF retains everything from the moment it is enabled.

D Nothing, since CDF cannot be queried retrospectively at all.

◆ PAUSE AND DECIDE

What can they actually be given?

A. Three years of row-level changes, since the transaction log retains everything.

Nothing before CDF was enabled was ever recorded as a change feed.

B. Only changes after CDF was enabled, and only within the retention window, since VACUUM deletes change data files.

Two limits stack: enablement date and retention. CDF is explicitly not a permanent record.

C. Eight months of changes, since CDF retains everything from the moment it is enabled.

Misses the second limit. Change data files are subject to VACUUM like anything else.

D. Nothing, since CDF cannot be queried retrospectively at all.

It can, within its window.

ON THE EXAM

C is the near-miss, and the more instructive wrong answer: it gets the enablement rule right and forgets that retention also applies.

> A downstream job reads a change data feed and computes a daily count of records processed. The numbers are consistently higher than the number of rows actually changing, and the discrepancy tracks the volume of updates rather than inserts.
>
> No duplicates exist in the source table.

What is the cause?

A Each update appears as two rows, a pre-image and a post-image. Filter on change type before counting.

B The change feed replays commits, so each change is delivered twice.

C Rate limits split commits across batches, causing double counting.

D Deletion vectors cause updated rows to appear in both the old and new file.

◆ PAUSE AND DECIDE

What is the cause?

A Each update appears as two rows, a pre-image and a post-image. Filter on change type before counting.

The feed represents an update as the value before and the value after. Counting rows counts both.

B The change feed replays commits, so each change is delivered twice.

It does not replay. A streaming read processes each commit once.

C Rate limits split commits across batches, causing double counting.

Rate limits apply atomically to whole commits, precisely so this cannot happen.

D Deletion vectors cause updated rows to appear in both the old and new file.

Deletion vectors affect storage layout, not what the change feed emits.

ON THE EXAM

The clue is that the discrepancy tracks updates and not inserts. An insert produces one row; an update produces two.

> A team is preparing to enable change data feed on a set of tables. One table fails with an error about column names. Its schema includes a column named change type, added years ago by a previous team to record an application-level event category.
>
> The column is used by several downstream reports.

What must they do?

A Rename the existing column before enabling change data feed, and update the downstream reports.

B Enable change data feed with a configuration option that renames the metadata columns.

C Drop the existing column, since change data feed will supply an equivalent.

D Enable change data feed on a clone of the table instead.

◆ PAUSE AND DECIDE

What must they do?

A Rename the existing column before enabling change data feed, and update the downstream reports.

Documented: CDF cannot be enabled where the schema already contains columns with the metadata names. Rename to resolve the conflict.

B Enable change data feed with a configuration option that renames the metadata columns.

The metadata column names are fixed. There is no such option.

C Drop the existing column, since change data feed will supply an equivalent.

It holds an application-level category, which is unrelated to the CDF change type. Dropping it loses real data.

D Enable change data feed on a clone of the table instead.

The clone has the same schema and the same conflict.

ON THE EXAM

C is the trap: two columns with the same name meaning entirely different things. One records a business event, the other records a row operation.

COST AND PERFORMANCE · DOMAIN 6 - COST & PERFORMANCE OPTIMIZATION

Liquid clustering, OPTIMIZE and VACUUM

Section 6. The layout decision you are allowed to change your mind about.

SECTION 6 of 10 WEIGHT 13% TOPIC Layout

admin@sthithapragna.com

Partitioning is a decision you make once and live with. Liquid clustering is a decision you can **revise**.

What liquid clustering actually is

- A data layout technique that **replaces table partitioning and ZORDER**. The documentation's own word is replaces.
- Files are organised around **clustering keys** rather than a directory structure.
- **Keys can be redefined without rewriting existing data**, so the layout evolves with analytic needs.
- It applies to **streaming tables and materialized views**, not only ordinary tables.

> ON THE EXAM
>
> Databricks recommends liquid clustering for all new tables. That is a blanket recommendation, not a conditional one.

Six scenarios the docs name

- Queries filtering on **high cardinality columns**. Partitioning cannot handle those without shattering the table.
- Tables with **heavy data skew**, where partitions would be wildly uneven.
- **Fast growing tables** that would otherwise need constant maintenance and tuning.
- Tables with **concurrent write** requirements, or **varied and changing access patterns**.
- Tables where a partition key would return **too many or too few partitions**.

> ON THE EXAM
>
> Notice how many of these are exactly the cases where choosing a partition column is hard. That is the point.

You cannot have both

- **Clustering is not compatible with partitioning or ZORDER.** The engine forbids combining them.
- You can enable it on an existing **unpartitioned** table, or at creation time.
- An **already-partitioned** table is not a small change. It needs a migration plan, not an ALTER statement.
- Recall from module 20 that predictive optimization also **ignores Z-ordered files**. The two facts compound.

> ON THE EXAM
>
> A partitioned table cannot simply adopt clustering. That single sentence decides several exam scenarios.

Manual, automatic, and off

FORM	WHAT IT DOES
`CLUSTER BY with columns`	You choose the keys. Up to four, and they must have statistics collected.
`CLUSTER BY AUTO`	Databricks chooses and adapts the keys. Managed tables, and requires predictive optimization.
`CLUSTER BY NONE`	Disables future clustering without rewriting existing data. New writes stay unclustered.

ON THE EXAM

The automatic form needs a Unity Catalog managed table and predictive optimization. On an external table it is simply not available.

How many, and which

- Up to **four** clustering keys. Choose the columns that actually appear in filters and joins.
- Clustering on **too many unrelated keys dilutes** the benefit for every one of them.
- A key with **no statistics collected** cannot drive file skipping, so it does nothing useful.
- If you cannot confidently name the columns, that is an argument for the automatic form rather than a guess.

> ON THE EXAM
>
> The connection to module 21 is direct. Clustering organises files, and statistics are what let the planner skip them.

OPTIMIZE and OPTIMIZE FULL

	WHAT IT DOES
Changing keys	Rewrites nothing. The new keys apply to future clustering only.
OPTIMIZE	Clusters incrementally. New and unclustered data only.
OPTIMIZE FULL	Forces reclustering of all data files. DBR 16.0 and above.

ON THE EXAM

After enabling clustering on a table that already holds data, plain OPTIMIZE will not move it. That is what OPTIMIZE FULL is for.

The one thing partitioning still does better

- **Metadata-only queries over partition columns** can be faster on a partitioned table.
- Selecting the distinct values of a partition column reads directory metadata, not data files.
- A clustered table has to **scan data** to answer the same question.
- So a workload that leans heavily on partition metadata is the honest exception to the recommendation.

> ON THE EXAM
>
> Worth knowing so the recommendation does not sound like marketing. Liquid clustering wins nearly everywhere, and not quite everywhere.

The sizing guidance that still applies

- Do not partition tables below roughly **one terabyte**. Smaller tables gain nothing and lose file size.
- If you do partition, aim for at least about **one gigabyte per partition**.
- Under-partitioning produces many tiny files, which is the small file problem OPTIMIZE exists to fix.
- A partition key that yields thousands of tiny partitions is worse than no partitioning at all.

> ON THE EXAM
>
> These thresholds are why so many partitioned tables perform badly. The scheme was chosen for readability rather than file size.

What compaction is actually for

- It rewrites many small files into fewer larger ones, which reduces per-file overhead at read time.
- On a clustered table it also **groups data by the clustering keys**, which is what makes skipping effective.
- Small files accumulate naturally from frequent small writes, particularly streaming ingestion.
- On managed tables, predictive optimization runs this for you when it judges it worthwhile.

> ON THE EXAM
>
> Compaction and clustering are the same command doing two jobs. On an unclustered table it only compacts.

Removing what nothing points at

- It deletes data files **no longer referenced** by any table version inside the retention window.
- Compaction and updates leave old files behind. Without vacuum, storage grows indefinitely.
- **Vacuum is what limits time travel.** Once the files are gone, older versions cannot be read.
- It also **deletes change data feed files**, which is why the feed is not a permanent record.

> ON THE EXAM
>
> Retention defaults to seven days, from module 20. Set it deliberately before enabling predictive optimization if you need longer history.

When you should not run either

- On **Unity Catalog managed tables with predictive optimization**, both operations are run for you.
- A scheduled weekly maintenance job on those tables is **duplicated work** you are paying for twice.
- Check what it decided with **DESCRIBE EXTENDED** before concluding it is not running.
- But remember Z-ordered files are ignored, so a Z-ordered table genuinely does get less.

> ON THE EXAM
>
> The modern answer to 'when should I schedule OPTIMIZE' is usually 'you should not', with two named exceptions.

Runtime versions, and a documentation disagreement

- Generally available for Delta Lake from around **DBR 15.4 LTS**. Public Preview for Apache Iceberg from **16.4 LTS**.
- The documentation is **inconsistent** here. Different pages state 15.4 and 15.2 for the same thing.
- **OPTIMIZE FULL requires DBR 16.0.** That figure is consistent everywhere.
- Managed Iceberg v3 adds deletion vectors, row tracking and automatic clustering, and needs **DBR 18.0**.

> ON THE EXAM
>
> Where documentation disagrees with itself, learn the shape rather than the number. Check your own runtime rather than trusting a course.

What to carry into the exam

- Liquid clustering **replaces partitioning and ZORDER**, and keys can change **without rewriting data**.
- **Not compatible with either.** A partitioned table needs a migration, not an ALTER.
- **OPTIMIZE clusters incrementally. OPTIMIZE FULL reclusters everything.**
- **CLUSTER BY AUTO** needs a managed table and predictive optimization. **CLUSTER BY NONE** stops without rewriting.
- On managed tables, scheduling OPTIMIZE and VACUUM yourself is usually duplicated work.

> ON THE EXAM
>
> Databricks recommends liquid clustering for all new tables. When a question offers a partitioning scheme and a clustering option, the burden of proof is on partitioning.

> A 4 TB fact table is partitioned by transaction date. Analysts increasingly filter on customer identifier, a high cardinality column, and those queries scan almost the whole table. A team member proposes adding liquid clustering on customer identifier with an ALTER TABLE statement.
>
> The table is a Unity Catalog managed table.

What is wrong with the proposal?

A. Nothing. ALTER TABLE with CLUSTER BY adds clustering to any table.

B. Clustering is not compatible with partitioning. The table must be migrated to an unpartitioned form first.

C. Customer identifier is too high cardinality to be a clustering key.

D. Clustering can only be set at table creation, never afterwards.

◆ PAUSE AND DECIDE

What is wrong with the proposal?

A. Nothing. ALTER TABLE with CLUSTER BY adds clustering to any table.

Only to an unpartitioned table. The engine forbids combining clustering with partitioning.

B. Clustering is not compatible with partitioning. The table must be migrated to an unpartitioned form first.

Enabling on an existing table is supported only when that table is unpartitioned. This is a migration.

C. Customer identifier is too high cardinality to be a clustering key.

Backwards. High cardinality filter columns are one of the named cases clustering handles well.

D. Clustering can only be set at table creation, never afterwards.

It can be enabled on an existing unpartitioned table.

ON THE EXAM

C is the instructive wrong answer: it applies partitioning intuition to clustering, where high cardinality is a strength rather than a problem.

> A team enables liquid clustering on an existing 900 GB unpartitioned table by setting clustering keys, then runs OPTIMIZE. Query performance improves only slightly, and the query profile shows poor file pruning on older data.
>
> New data written since the change is clustering well.

What should they do?

A Run OPTIMIZE FULL to force reclustering of all existing data files.

B Add more clustering keys so more queries benefit.

C Run VACUUM to remove the old unclustered files.

D Re-enable Z-ordering alongside clustering for the older data.

◆ PAUSE AND DECIDE

What should they do?

A Run OPTIMIZE FULL to force reclustering of all existing data files.

Plain OPTIMIZE clusters incrementally and leaves existing files alone. OPTIMIZE FULL reclusters everything.

B Add more clustering keys so more queries benefit.

Dilutes the benefit across keys, and does nothing about data that was never clustered.

C Run VACUUM to remove the old unclustered files.

Those files hold current data. Vacuum removes unreferenced files, not referenced ones.

D Re-enable Z-ordering alongside clustering for the older data.

The two cannot coexist, and Z-ordered files are also ignored by predictive optimization.

ON THE EXAM

The symptom is precise: new data clusters well, old data does not. That is incremental clustering behaving exactly as designed.

> A platform team wants clustering keys to adapt as query patterns change, without anybody revisiting them. Their tables are Unity Catalog managed with predictive optimization enabled.
>
> A second team wants the same for their tables, which are external tables in cloud storage they manage themselves.

What can each team do?

A Both can use CLUSTER BY AUTO, since it is a table-level setting.

B The first team can use CLUSTER BY AUTO. The second cannot, since it requires a managed table and predictive optimization.

C Neither can; CLUSTER BY AUTO applies only at table creation.

D Both can, but only after running OPTIMIZE FULL first.

◆ PAUSE AND DECIDE

What can each team do?

A Both can use CLUSTER BY AUTO, since it is a table-level setting.

It requires a Unity Catalog managed table. External tables are excluded.

B The first team can use CLUSTER BY AUTO. The second cannot, since it requires a managed table and predictive optimization.

Automatic key selection is driven by predictive optimization, which runs only on managed tables.

C Neither can; CLUSTER BY AUTO applies only at table creation.

It can be set on existing unpartitioned tables too.

D Both can, but only after running OPTIMIZE FULL first.

OPTIMIZE FULL reclusters data. It does not unlock the automatic form.

ON THE EXAM

The second team's realistic path is manual keys now, and converting to managed if they want the automatic form later.

> A reporting workload runs many queries of the form selecting distinct region values, and a dashboard that lists available periods. These run in milliseconds against the current partitioned table.
>
> The team is asked to migrate to liquid clustering, and is concerned about this specific workload.

Is the concern legitimate?

A. No. Clustering is faster than partitioning for every query shape.

B. Yes. Metadata-only queries over partition columns read directory metadata; a clustered table must scan data to answer them.

C. Yes, but only until OPTIMIZE FULL has been run once.

D. No, because clustering keys are stored as metadata and queried the same way.

◆ PAUSE AND DECIDE

Is the concern legitimate?

A. No. Clustering is faster than partitioning for every query shape.

Overstated. The recommendation is broad, not universal.

B. Yes. Metadata-only queries over partition columns read directory metadata; a clustered table must scan data to answer them.

The honest exception. Distinct values of a partition column come from directory names, and clustering has no directories.

C. Yes, but only until OPTIMIZE FULL has been run once.

Reclustering improves skipping. It does not create partition metadata.

D. No, because clustering keys are stored as metadata and queried the same way.

Statistics support file skipping. They are not a distinct-value index.

ON THE EXAM

A rare case where the concern is correct. Naming it makes the broader recommendation credible rather than promotional.

A team schedules a nightly job running OPTIMIZE and VACUUM across their Unity Catalog managed tables. Half the tables are Z-ordered from an earlier era; the rest are liquid clustered.

They notice the Z-ordered tables perform worse over time despite the nightly job, and the clustered ones perform well.

What best explains the difference?

A The nightly job is misconfigured and skips the Z-ordered tables.

B Predictive optimization ignores Z-ordered files, so those tables get less automatic maintenance. Migrating them to clustering resolves it.

C Z-ordering degrades naturally and must be re-run more frequently.

D VACUUM removes Z-order metadata, undoing the layout each night.

◆ PAUSE AND DECIDE

What best explains the difference?

A The nightly job is misconfigured and skips the Z-ordered tables.

Possible in general, and there is a documented explanation that fits without assuming a bug.

B Predictive optimization ignores Z-ordered files, so those tables get less automatic maintenance. Migrating them to clustering resolves it.

Module 20's finding: OPTIMIZE under predictive optimization does not run ZORDER and ignores Z-ordered files.

C Z-ordering degrades naturally and must be re-run more frequently.

Layouts do drift, and that does not explain a difference against clustered tables under the same job.

D VACUUM removes Z-order metadata, undoing the layout each night.

Vacuum removes unreferenced files. It does not undo a layout.

ON THE EXAM

Two modules converge here. Z-order excludes files from automatic maintenance, and clustering cannot be added alongside it.

> A team wants to stop clustering a table because the workload has changed and the keys no longer match any query. They want new writes to stop being clustered, but they cannot afford a full rewrite of the 12 TB table right now.
>
> They ask whether they must drop and recreate it.

What should they do?

A. Use CLUSTER BY NONE, which disables future clustering without rewriting existing data.

B. Drop and recreate the table without a CLUSTER BY clause.

C. Run OPTIMIZE FULL, which removes clustering when no keys are set.

D. Set the clustering keys to an empty list, which is equivalent to partitioning.

◆ PAUSE AND DECIDE

What should they do?

A Use CLUSTER BY NONE, which disables future clustering without rewriting existing data.

Exactly what it exists for: stop clustering future writes, leave the existing layout in place.

B Drop and recreate the table without a CLUSTER BY clause.

The full rewrite they said they cannot afford, plus the loss of history and permissions.

C Run OPTIMIZE FULL, which removes clustering when no keys are set.

OPTIMIZE FULL reclusters. It is the opposite of what they want.

D Set the clustering keys to an empty list, which is equivalent to partitioning.

Clustering and partitioning are not interchangeable, and the second clause is simply false.

ON THE EXAM

Note what the right answer does not do: it does not undo existing clustering. The old layout stays until something rewrites it.

COST AND PERFORMANCE · DOMAIN 6 - COST & PERFORMANCE OPTIMIZATION

Compute choices and what they cost

Section 6. The four principles, and the levers that actually move the bill.

SECTION 6 of 10 WEIGHT 13% TOPIC Compute cost

admin@sthithapragna.com

Compute type is the largest single lever on the bill, and it is chosen before anybody writes a line of code.

admin@sthithapragna.com

Use job compute for anything non-interactive

- The docs are plain: non-interactive workloads **cost significantly less** on job compute than on all-purpose.
- A production pipeline left running on an interactive cluster is the single most common avoidable overspend.
- Each job can run on new compute, which also **isolates workloads** from one another.
- Or a **multitask job can reuse compute** across its tasks, so start-up is paid once per job rather than per task.

> ON THE EXAM
>
> Notice the second option. Reusing compute across tasks is how you avoid paying start-up time repeatedly in a job with many small steps.

SQL warehouses, and Photon by default

- For interactive SQL, a SQL warehouse is described as **the most cost-efficient engine**.
- **All SQL warehouses come with Photon by default.** It is not something you enable.
- Serverless warehouses add **intelligent workload management** for handling many concurrent queries.
- Running interactive SQL on an all-purpose cluster pays a higher rate for a less suitable engine.

> ON THE EXAM
>
> Photon being on by default for warehouses is worth remembering, because on classic compute it is a choice you have to make.

When serverless wins on cost despite a higher rate

- Non-serverless warehouses start in **minutes**, so people leave them running rather than wait.
- Serverless starts and scales **in seconds**, so instant availability and idle termination are both achievable.
- Serverless also **scales down earlier** than non-serverless.
- So for **bursty** workloads the higher rate is offset by not paying for idle time at all.

> ON THE EXAM
>
> A higher per-unit rate can still be cheaper. What matters is rate multiplied by time, and serverless changes the time.

The rules of thumb the docs give

FAMILY	USE IT FOR
Memory optimised	Machine learning, and heavy shuffle and spill workloads.
Compute optimised	Structured Streaming, and maintenance jobs such as optimize and vacuum.
Storage optimised	Workloads that benefit from caching, such as ad-hoc and interactive analysis.
GPU optimised	Specific machine learning and deep learning workloads only.
General purpose	In the absence of specific requirements.

ON THE EXAM

Note the second row. Streaming and maintenance jobs want compute-optimised, which is not what most people reach for by default.

Three numbers that matter more than worker count

- **Total executor cores** determines the maximum parallelism available.
- **Total executor memory** determines how much can be held before spilling to disk.
- **Executor local storage** is used for spills during shuffles, and for caching.
- Databricks runs **one executor per worker node**, so executor and worker mean the same thing here.

> ON THE EXAM
>
> People size clusters by worker count. The docs point at cores, memory and local disk instead, which is what a workload actually consumes.

Autoscaling, and the streaming exception again

- Autoscaling adds workers for the demanding phases of a job and removes them afterwards, beating a static size.
- **It has limitations scaling down for Structured Streaming workloads.**
- Databricks recommends **Lakeflow pipelines with enhanced autoscaling** for streaming instead.
- That is the same recommendation module 7 gave from the pipelines side, arriving here as a cost argument.

> ON THE EXAM
>
> Module 6 said do not autoscale a hand-written stream. This is why, and this is the alternative.

Auto-termination and pools

- Configure **auto-termination on all interactive compute**. Idle clusters are pure waste.
- If start-up times are the reason people leave clusters running, use **pools** of idle ready-to-use instances.
- **Databricks does not charge DBUs while instances sit idle in a pool.**
- The instance provider does still bill for them, so a pool is not free. It moves the cost, and shortens start-up.

> ON THE EXAM
>
> Both halves of that pool fact matter. No DBUs, but the cloud provider still charges for the machines.

What a compute policy can enforce

- Require **autoscaling** with a set minimum number of workers.
- Require **auto-termination** with a sensible value, the docs suggest an hour as an example.
- Restrict which **VM instance types** may be chosen, and set maximum worker counts.
- Apply a **spot instance strategy**, and define standard sizes so choices are consistent.

> ON THE EXAM
>
> A policy fails cluster creation at submission. That is a much better control than a monthly cost review noticing it afterwards.

Judge the total, not the rate

- Photon accelerates SQL and DataFrame calls, and is compatible with Spark APIs. **No code changes.**
- It bills at a higher rate, and **a faster job can still be cheaper overall** because billing is by time.
- The docs say regular jobs **should be evaluated** to see whether they are not only faster but also cheaper.
- So it is a measurement, not an assumption. It does not help every workload equally.

> ON THE EXAM
>
> The same reasoning as serverless. Rate multiplied by time is what you pay, and a higher rate over less time can win.

Tagging, and the thing you cannot undo

- Tag workspaces, clusters, warehouses and pools. Tags propagate to **usage logs and to cloud provider resources**.
- Recommended minimum: **business unit** and **project**, plus **environment** where dev and production must be separated.
- **Tags only affect future usage. Missing tags cannot be added to past events.**
- So start detailed. You can ignore a tag later; you cannot backfill one.

> ON THE EXAM
>
> This is the only genuinely irreversible decision in the module. Everything else can be changed next week.

What a DBU does and does not include

- For classic compute, total cost is **DBUs plus the virtual machine, disk and network** charges.
- **For serverless, the DBU cost already includes the virtual machine cost.**
- So comparing a serverless DBU rate against a classic one is not comparing like with like.
- A migration to serverless can look like DBUs rising while infrastructure falls, with total spend barely moving.

> ON THE EXAM
>
> This single difference explains most confused conversations about whether serverless is more expensive.

Budgets, usage policies and system tables

- **Budgets** track account-wide or filtered spending, with email notification when a threshold is reached.
- **Usage policies** attribute serverless usage by tagging activity incurred by an assigned user.
- The **billable usage system table** is where custom tags land, so attribution becomes a query.
- Billable usage can also be downloaded as a file through the account API for offline analysis.

> ON THE EXAM
>
> Serverless needs usage policies specifically, because there is no cluster of yours to hang a tag on.

Sharing does not replicate, and may still cost

- Sharing avoids replication, which is the whole argument from module 15.
- But your cloud vendor **may charge data egress fees** when you share across clouds or regions.
- That cost lands on the **provider**, not the recipient, and there is guidance on monitoring it.
- So the zero-copy argument is about storage and staleness, not about network charges.

> ON THE EXAM
>
> A cost that appears on the cloud bill rather than the Databricks one, which is exactly why it goes unnoticed.

Always-on streaming is a choice, not a requirement

- Streaming suggests always on, and always on means paying for compute **every hour of the day**.
- If the business needs fresh data **every few hours**, a few runs a day meets that requirement.
- Databricks recommends the **availableNow trigger** for incremental workloads without low latency requirements.
- Same code, same checkpoint, dramatically less compute. Module 11's point, seen as a cost decision.

> ON THE EXAM
>
> Ask what the business actually requires, not what the source looks like. A continuous source does not compel continuous compute.

Cheaper capacity, with one rule

- Spot instances use excess cloud capacity at a lower price, and Databricks supports clusters built on them.
- **The first instance, the Spark driver, should always be on-demand.**
- They suit workloads where it is acceptable to take longer because instances were evicted.
- **Fleet instance types** let Databricks pick the physical instance with the best price and availability.

> ON THE EXAM
>
> Losing a worker to eviction costs you some rework. Losing the driver ends the job. That asymmetry is the whole rule.

What to carry into the exam

- **Job compute for non-interactive work.** The largest single lever, and the most common mistake.
- Serverless and Photon can be **cheaper at a higher rate**, because you pay rate multiplied by time.
- Autoscaling **does not scale down well for streaming**. Use pipelines with enhanced autoscaling.
- **Serverless DBU rates already include the VM cost.** Classic rates do not.
- **Tags cannot be backfilled.** Spot is fine for workers, never for the driver.

> ON THE EXAM
>
> That completes Section 6 at 13%. Section 7, security and compliance, is next at 10%.

> A finance review finds that 40% of Databricks spend comes from a cluster that runs continuously. Investigation shows it hosts eleven scheduled production pipelines, all configured to attach to it, plus occasional notebook work by two engineers.
>
> The cluster has autoscaling enabled and is correctly sized.

What is the largest single saving available?

A. Move the eleven pipelines to job compute, which costs significantly less for non-interactive work.

B. Reduce the autoscaling maximum to cap the cluster's size.

C. Enable Photon on the cluster to shorten pipeline runtimes.

D. Add spot instances to the worker pool to lower the instance cost.

◆ PAUSE AND DECIDE

What is the largest single saving available?

A Move the eleven pipelines to job compute, which costs significantly less for non-interactive work.

Scheduled pipelines are non-interactive. The docs are explicit that they cost significantly less on job compute.

B Reduce the autoscaling maximum to cap the cluster's size.

The scenario says it is correctly sized. Capping it constrains work without addressing the rate.

C Enable Photon on the cluster to shorten pipeline runtimes.

Worth evaluating, and it reduces time at a higher rate rather than changing the rate class.

D Add spot instances to the worker pool to lower the instance cost.

Touches the cloud infrastructure cost, not the DBU rate class, which is the larger factor here.

ON THE EXAM

The details about autoscaling and sizing are there to close off the tuning answers. The compute type is what is wrong.

> An analytics team of six queries intensively between 9 and 11 each morning, then barely touches the platform. They currently use a classic SQL warehouse, which they leave running all day because it takes several minutes to start.
>
> A colleague objects that serverless has a higher DBU rate and would cost more.

How should the objection be answered?

A The colleague is right; classic is always cheaper per DBU, so it wins.

B Serverless starts in seconds and scales down earlier, so a bursty pattern avoids six hours of idle time despite the higher rate.

C Set a shorter auto-stop on the classic warehouse; the start-up delay is acceptable.

D Move the workload to job compute, which is cheapest of all.

◆ PAUSE AND DECIDE

How should the objection be answered?

A The colleague is right; classic is always cheaper per DBU, so it wins.

The rate is only half the calculation. You pay rate multiplied by time.

B Serverless starts in seconds and scales down earlier, so a bursty pattern avoids six hours of idle time despite the higher rate.

The documented argument. Instant availability plus idle termination are only achievable together on serverless.

C Set a shorter auto-stop on the classic warehouse; the start-up delay is acceptable.

The team already rejected that trade, which is why the warehouse runs all day.

D Move the workload to job compute, which is cheapest of all.

Job compute is for non-interactive work. This is interactive SQL.

ON THE EXAM

The colleague's fact is correct and their conclusion is not. That is the shape of the reasoning being tested.

> A platform team is asked to attribute Databricks spend to five business units, going back twelve months. They discover that no custom tags were ever configured on any workspace, cluster or warehouse.
>
> They ask whether tags can be applied and backdated across the historical usage.

What should they be told?

A Yes. Applying tags updates the billable usage system table retrospectively.

B No. Tags affect future usage only and cannot be added to past events. Start tagging now, and attribute history another way.

C Yes, but only within the 365-day retention window of the usage table.

D No, but downloading billable usage through the account API produces tagged history.

◆ PAUSE AND DECIDE

What should they be told?

A Yes. Applying tags updates the billable usage system table retrospectively.

Tags are recorded with usage as it happens. Past records are not rewritten.

B No. Tags affect future usage only and cannot be added to past events. Start tagging now, and attribute history another way.

Stated directly in the guidance, and the reason it advises starting detailed from the beginning.

C Yes, but only within the 365-day retention window of the usage table.

Retention governs how long records are kept, not whether they can be edited.

D No, but downloading billable usage through the account API produces tagged history.

The API returns the same records. It cannot add tags that were never captured.

ON THE EXAM

History can still be partly attributed by joining usage to cluster and job identifiers. It is just far more work than a tag would have been.

> A team migrates several batch jobs from classic job compute to serverless. Afterwards the Databricks bill shows a marked rise in DBU consumption, and the platform team raises a concern that the migration was a mistake.
>
> Nobody has looked at the cloud provider invoice.

What should be checked before drawing a conclusion?

A Nothing. A rise in DBUs is a rise in cost.

B The cloud provider invoice, since serverless DBU rates already include the VM cost that was previously billed separately.

C Whether Photon was disabled during the migration.

D Whether the jobs are now running more frequently than before.

◆ PAUSE AND DECIDE

What should be checked before drawing a conclusion?

A. Nothing. A rise in DBUs is a rise in cost.
DBUs are one component. For serverless they carry a cost that used to appear elsewhere.

B. The cloud provider invoice, since serverless DBU rates already include the VM cost that was previously billed separately.
Classic bills DBUs plus VMs. Serverless folds the VM into the DBU, so DBUs up and infrastructure down can be a wash.

C. Whether Photon was disabled during the migration.
Worth knowing generally, and it would not explain a shift of cost between two invoices.

D. Whether the jobs are now running more frequently than before.
A reasonable check, and the sentence about the unread cloud invoice is the stronger clue.

ON THE EXAM

Total cost is DBUs plus infrastructure for classic, and DBUs alone for serverless. Comparing one invoice against itself is the error.

A Structured Streaming job ingests events continuously. The business confirms that dashboards built on the output are reviewed twice a day, and freshness of a few hours is entirely acceptable.

The team currently runs the stream continuously on an autoscaling cluster, and costs are high.

What is the strongest change?

A Switch to the availableNow trigger on a schedule, so compute runs only while there is work.

B Increase the autoscaling minimum so the cluster scales more smoothly.

C Move the stream to spot instances including the driver.

D Keep the continuous stream but enable Photon to shorten each micro-batch.

◆ PAUSE AND DECIDE

What is the strongest change?

A Switch to the availableNow trigger on a schedule, so compute runs only while there is work.

The documented recommendation for incremental workloads without low-latency requirements. Same code, same checkpoint.

B Increase the autoscaling minimum so the cluster scales more smoothly.

Raises the floor on a cluster that should not be running most of the time.

C Move the stream to spot instances including the driver.

The driver should always be on-demand. Losing it ends the job.

D Keep the continuous stream but enable Photon to shorten each micro-batch.

Faster batches on a cluster running 24 hours a day still means a cluster running 24 hours a day.

ON THE EXAM

The business requirement is the lever, not the technology. A continuous source does not compel continuous compute.

> A company shares several large tables with partners in another cloud region using Delta Sharing. Their Databricks bill is stable, but their cloud provider invoice has risen substantially in the same period.
>
> Nothing about their own workloads has changed.

What is the most likely cause?

A Sharing replicates the data to the recipient's region, duplicating storage.

B Data egress charges from the cloud vendor, incurred by the provider when recipients read across regions.

C The recipients' queries run on the provider's compute and appear as DBUs.

D Shared tables are excluded from predictive optimization, so storage grows unchecked.

◆ PAUSE AND DECIDE

What is the most likely cause?

A Sharing replicates the data to the recipient's region, duplicating storage.

Sharing does not replicate. That is its central property.

B Data egress charges from the cloud vendor, incurred by the provider when recipients read across regions.

Documented: sharing avoids replication, but the cloud vendor may charge egress across clouds or regions, and it lands on the provider.

C The recipients' queries run on the provider's compute and appear as DBUs.

Recipients use their own compute, and the Databricks bill is stated to be stable.

D Shared tables are excluded from predictive optimization, so storage grows unchecked.

Invented. Sharing does not change maintenance eligibility.

ON THE EXAM

The split between the two invoices is the clue. A cost on the cloud bill and not the Databricks bill points at network or storage, not compute.

SECURITY · DOMAIN 7 - ENSURING DATA SECURITY AND COMPLIANCE

Access control, secrets and service principals

Section 7. Least privilege, and the places a secret still leaks.

SECTION 7 of 10 WEIGHT 10% TOPIC Access control

admin@sthithapragna.com

Unity Catalog governs **data**. Workspace ACLs govern **objects**. Two systems, and a question is always about one or the other.

What everyone already has

- **Workspace admins have CAN MANAGE on every object** in the workspace. That cannot be lowered.
- **Users automatically have CAN MANAGE on objects they create.**
- Everything in the **Shared folder** is CAN MANAGE for all users. Moving something there grants it to everyone.
- So least privilege here is mostly about what you add, and about where you put things.

> ON THE EXAM
>
> Moving a notebook into Shared is a permission change, even though it looks like filing. That is worth knowing before somebody tidies up.

Inheritance, and a small information leak

- Objects in a folder **inherit all permission settings of that folder**. Managing by folder is the practical approach.
- Grant someone view on a notebook inside a folder they have no permissions on, and they can still **see the folder's name**.
- They cannot view or access anything else in it. Only the name leaks.
- So a folder named after a confidential project tells its own story to anyone with access to one file inside it.

> ON THE EXAM
>
> A minor leak with a real consequence. Name folders as though people who cannot open them will read the name.

Five levels, and one that is unusual

LEVEL	WHAT IT ALLOWS
CAN VIEW	See job details, settings and results.
CAN MANAGE RUN	Also view the Spark UI and logs, run now, run with parameters, and cancel.
IS OWNER	Also edit settings, delete the job, and modify permissions.
CAN MANAGE	The same abilities as owner, but it is a permission rather than ownership.

ON THE EXAM

A job cannot have more than one owner, and a group can never be the owner. Ownership is always exactly one identity.

Run Now uses the owner's permissions

- **A job triggered through Run Now assumes the permissions of the job owner**, not of the person who pressed it.
- So granting CAN MANAGE RUN lets somebody execute work **with the owner's access**, which may be far broader than their own.
- That is exactly why production jobs should be owned by a **service principal** with only the access the job needs.
- A job owned by a senior engineer runs with a senior engineer's access, every time anybody triggers it.

> ON THE EXAM
>
> This single fact is the strongest practical argument for service principal ownership, and it is easy to miss.

Two things they do not cover

- **Notebook workflows** that run other notebooks use the **notebook's own permissions**, not the job's.
- **Jobs submitted by API** use the notebook's default permissions unless you set an access control list in the request.
- So an externally orchestrated job can end up with permissions nobody reviewed.
- Setting the access control list explicitly at submission is the fix.

> ON THE EXAM
>
> Job permissions apply to jobs in the Jobs UI and their runs. Work reaching the platform another way follows different rules.

Four levels, and what attaching gets you

- **CAN ATTACH TO** lets somebody attach a notebook, view the Spark UI and see compute metrics.
- **CAN RESTART** adds terminating, starting and restarting.
- **CAN MANAGE** adds editing, resizing, attaching libraries and changing permissions.
- Note that attaching a notebook means running code **on that compute, as yourself**.

> ON THE EXAM
>
> Attach permission is not a small grant. It is the ability to execute arbitrary code on that machine with your own identity.

Secrets are not redacted from driver logs

- **Secrets are not redacted from the Spark driver's standard output and error streams.**
- So by default, driver logs are viewable **only by CAN MANAGE** on job and all-purpose compute in dedicated or standard access mode.
- A Spark property can relax that to let CAN ATTACH TO and CAN RESTART read them. Think before setting it.
- It is a **compute-level** property, so it must be set on each compute individually.

> ON THE EXAM
>
> The redaction people rely on happens in notebook output, not in driver logs. Printing a secret is safe; a library logging it is not.

Legacy no-isolation shared compute

- On legacy **no isolation shared** access mode, users with **CAN ATTACH TO can view service account keys** in the log file.
- The documentation says to use caution when granting that permission on such compute.
- Standard and dedicated access modes exist precisely to remove this class of problem.
- If a scenario mentions no isolation shared, treat every permission on it as broader than it looks.

> ON THE EXAM
>
> This is a legacy mode. Meeting it in a question is usually a signal that the answer involves moving off it.

Three levels, each containing the last

LEVEL	WHAT IT ALLOWS
READ	Read the secret scope, and list which secrets exist in it.
WRITE	Also write secrets to the scope.
MANAGE	Also modify the permissions on the scope.

ON THE EXAM

Access control is at the scope level, not per secret. If two secrets need different audiences, they need different scopes.

Four things about scopes

- The **creator of a scope automatically gets MANAGE**, and workspace admins have MANAGE on all scopes.
- Secret access control requires the **Premium plan or above**. Below that you grant MANAGE to all users.
- **Deleting a scope deletes every secret and every ACL in it.**
- Writing an ACL for a principal that already has one **overwrites** the existing level rather than adding to it.

> ON THE EXAM
>
> Principals are named differently by type: a user by email, a service principal by its application ID, a group by its name.

Whose permission applies

- When a secret is read from a notebook, **the permission of the user executing the command** is applied.
- They must have at least **READ** on the scope. The notebook's author is irrelevant.
- There is **no API to read the actual secret value outside a cluster**. It can only be resolved in a command.
- But remember that a job triggered by Run Now executes as the **job owner**, so it uses their scope permissions.

> ON THE EXAM
>
> Two identity rules interacting: interactive execution uses the runner, and a job run uses the owner. That combination decides several scenarios.

Why production should not run as a person

- A service principal is an identity for automation, with its own permissions and no interactive login.
- Production jobs owned by one survive the owner leaving, and run with **exactly the access the job needs**.
- Module 2 noted that production deployment mode **expects a service principal** for bundles.
- A job owned by a person is a job whose blast radius is that person's entire access.

> ON THE EXAM
>
> Least privilege on this platform is largely achieved by choosing the right identity, not by tuning permission levels.

Account group inheritance

- Workspace object permissions will soon be **inherited from all account groups a member belongs to**.
- That applies **regardless of whether those groups are assigned to the workspace**.
- So a permission granted to a group in one context may become effective somewhere it was not before.
- Databricks provides an **orphaned permissions analysis** notebook for reviewing grants ahead of the change.

> ON THE EXAM
>
> Flagged as upcoming rather than current. Worth knowing because it changes who can see what, without anybody granting anything.

What to carry into the exam

- **Unity Catalog governs data. Workspace ACLs govern objects.** Decide which system a question is about.
- **Run Now uses the job owner's permissions**, not the triggering user's.
- **Secrets are not redacted from driver logs.** That is why logs default to CAN MANAGE only.
- Secret ACLs are **per scope**, three levels, each containing the last. Reading applies the **executing user's** permission.
- Own production jobs with a **service principal**, granted only what the job needs.

> ON THE EXAM
>
> Least privilege here is achieved mostly by choosing identities and folders well, rather than by tuning individual permission levels.

A data engineering lead owns twelve production jobs. To let the support team restart failed runs overnight, they grant the support group CAN MANAGE RUN on all twelve.

A security review asks what access the support team now effectively has when they trigger a run.

What is the correct answer?

A The support team's own access, since permissions are evaluated per user at run time.

B The job owner's access, because a job triggered by Run Now assumes the owner's permissions.

C No data access at all; CAN MANAGE RUN only permits starting and cancelling.

D The access of whichever service principal the workspace runs jobs under by default.

◆ PAUSE AND DECIDE

What is the correct answer?

A The support team's own access, since permissions are evaluated per user at run time.

True for interactive execution. Not true for a job run, which assumes the owner's permissions.

B The job owner's access, because a job triggered by Run Now assumes the owner's permissions.

Stated directly in the docs. The support team can now execute work with the lead's full access.

C No data access at all; CAN MANAGE RUN only permits starting and cancelling.

Starting a run executes the job, and the job reads and writes data.

D The access of whichever service principal the workspace runs jobs under by default.

There is no such default. These jobs are owned by a person.

ON THE EXAM

The grant looks operational and is effectively a broad delegation of the owner's data access. Service principal ownership is the fix.

> An engineer reports that printing a secret in a notebook correctly shows REDACTED, so they conclude secrets cannot leak through logs. They ask for CAN ATTACH TO to be widened to the whole analytics team on a shared job cluster, so people can self-serve on debugging.
>
> A third-party library used by the job writes its connection string to standard error on failure.

What should the reviewer raise?

A Nothing; redaction covers all output paths.

B Secrets are not redacted from driver stdout and stderr, which is why driver logs default to CAN MANAGE only.

C CAN ATTACH TO does not permit log access under any configuration.

D The library should be removed, which resolves the issue entirely.

◆ PAUSE AND DECIDE

What should the reviewer raise?

A Nothing; redaction covers all output paths.

Redaction applies to notebook output. The docs state secrets are not redacted from driver streams.

B Secrets are not redacted from driver stdout and stderr, which is why driver logs default to CAN MANAGE only.

The engineer's observation is correct and their conclusion does not follow. The library's stderr write is exactly the exposed path.

C CAN ATTACH TO does not permit log access under any configuration.

A Spark property exists to allow precisely that, which is the risk being requested.

D The library should be removed, which resolves the issue entirely.

Addresses one library. The default log restriction protects against every future one.

ON THE EXAM

The redaction the engineer observed is real. It just protects a different path from the one their library uses.

A platform team creates one secret scope holding credentials for eight different source systems. They now need the finance team to access only the two finance credentials, and not the other six.

They ask how to grant READ on individual secrets within the scope.

What should they be told?

A Grant READ on the two secrets individually using the secrets CLI.

B Secret access control is at scope level. Split the credentials into separate scopes.

C Use a Unity Catalog row filter on the secret scope.

D Grant the finance team WRITE, which restricts them to their own secrets.

◆ PAUSE AND DECIDE

What should they be told?

A Grant READ on the two secrets individually using the secrets CLI.

Access control is applied to the scope, not to individual secrets.

B Secret access control is at scope level. Split the credentials into separate scopes.

The scope is the unit of access control, so different audiences require different scopes.

C Use a Unity Catalog row filter on the secret scope.

Row filters govern table data. A secret scope is a workspace object, not a table.

D Grant the finance team WRITE, which restricts them to their own secrets.

WRITE is broader than READ, not narrower. Each level contains the previous one.

ON THE EXAM

C is worth noticing as the two-systems confusion in action: a Unity Catalog feature applied to a workspace object.

> A tidy-up moves a set of analysis notebooks into the Shared folder so the team can find them. One notebook contains an embedded query against a sensitive HR table, though it holds no data itself.
>
> Nobody changed any permissions.

What has changed?

A Nothing. Moving a notebook does not alter permissions.

B All workspace users now have CAN MANAGE on those notebooks, though Unity Catalog still governs the HR data.

C All workspace users can now read the HR table through the notebook.

D The notebooks inherit the permissions of their previous folder, which travel with them.

◆ PAUSE AND DECIDE

What has changed?

A Nothing. Moving a notebook does not alter permissions.

Objects inherit the permissions of the folder they are in, and Shared is CAN MANAGE for all users.

B All workspace users now have CAN MANAGE on those notebooks, though Unity Catalog still governs the HR data.

Both halves matter: the object permission changed, and the data permission did not.

C All workspace users can now read the HR table through the notebook.

Running the notebook applies the runner's own Unity Catalog permissions, which they do not have.

D The notebooks inherit the permissions of their previous folder, which travel with them.

Inheritance follows the current location, not the previous one.

ON THE EXAM

C is the important wrong answer. The two systems are independent, so exposing the notebook does not expose the data.

> An external orchestrator submits jobs to Databricks through the API. A review finds that these runs have broader access than the equivalent jobs defined in the workspace, which are carefully permissioned.
>
> The submission request does not include an access control list.

What explains this?

A API-submitted jobs use the notebook's default permissions unless an access control list is set in the request.

B API-submitted jobs always run as a workspace admin.

C The orchestrator's service principal has been granted CAN MANAGE on the workspace.

D Job ACLs do not exist for jobs created outside the Jobs UI, so no permissions apply at all.

◆ PAUSE AND DECIDE

What explains this?

A API-submitted jobs use the notebook's default permissions unless an access control list is set in the request.

Stated in the docs as an exception to job access control. Setting the list explicitly at submission is the fix.

B API-submitted jobs always run as a workspace admin.

Invented, and it would be a far larger problem than described.

C The orchestrator's service principal has been granted CAN MANAGE on the workspace.

Possible in general, and the missing access control list is the documented explanation that fits.

D Job ACLs do not exist for jobs created outside the Jobs UI, so no permissions apply at all.

Permissions still apply; they are simply the notebook's defaults rather than a reviewed set.

ON THE EXAM

The last line of the scenario is the answer. When a question tells you a field was omitted, that omission is usually the mechanism.

> A production pipeline is owned by an engineer who is leaving the company. The team plans to transfer ownership to their group, so that the whole team collectively owns it.
>
> They also want the pipeline to keep only the access it currently needs.

What should they do?

A. Assign IS OWNER to the group, which distributes ownership across the team.

B. Transfer ownership to a service principal granted only the access the pipeline needs, and give the group CAN MANAGE.

C. Leave ownership with the departing engineer and disable their account.

D. Assign IS OWNER to a second engineer, and add the group as a co-owner.

◆ PAUSE AND DECIDE

What should they do?

A. Assign IS OWNER to the group, which distributes ownership across the team.

A group cannot be assigned IS OWNER. Ownership is always exactly one identity.

B. Transfer ownership to a service principal granted only the access the pipeline needs, and give the group CAN MANAGE.

Ownership survives people leaving, access is scoped to the job, and the team still administers it.

C. Leave ownership with the departing engineer and disable their account.

Leaves production owned by a disabled identity, and runs would carry that identity's permissions.

D. Assign IS OWNER to a second engineer, and add the group as a co-owner.

There cannot be more than one owner, and it recreates the same problem with a different person.

ON THE EXAM

Both constraints are in the answer: one owner only, never a group. A service principal is the identity that satisfies both.

SECURITY · DOMAIN 7 - ENSURING DATA SECURITY AND COMPLIANCE

Row filters and column masks

Section 7. Two mechanisms, one that the exam guide names and one that superseded it.

SECTION 7 of 10 WEIGHT 10% TOPIC Fine-grained access

admin@sthithapragna.com

Neither mechanism grants access. Both add restrictions on top of privileges you have already granted separately.

A boolean function, evaluated per row

- A **SQL user-defined function returning a boolean**, evaluated against each row at query time.
- Return true and the row is visible. Return false and it is not. The user never sees the unfiltered table.
- Bound to a table with **ALTER TABLE, set row filter**, naming the columns passed to the function.
- Typically the function checks group membership against a value in the row, such as a region or a cost centre.

> ON THE EXAM
>
> The filter runs at query time, so one physical table serves every audience. There is no filtered copy to keep in sync.

A function on one column, with three constraints

- A SQL function taking the column value and returning either the original or a masked version.
- **The return type must match, or be castable to, the column's data type.**
- **Each column can have exactly one mask.**
- A mask can take **other columns as inputs**, so behaviour can vary on several attributes at once.

> ON THE EXAM
>
> That last point is the useful one. A mask on an email column can consult a sensitivity column to decide how much to hide.

It must be a SQL UDF in Unity Catalog

- The function must be **registered in Unity Catalog**. That is what makes it governable and reusable.
- Python or Scala logic **must be wrapped in a SQL UDF**. The SQL function is what you attach.
- So the module 4 cost ladder applies here too. A pure SQL function is cheaper than one calling out to Python.
- This runs on every row of every query against that table, so the cost is not incidental.

> ON THE EXAM
>
> A masking function is the most frequently executed code you will write. Module 4's argument about UDF cost applies with full force.

Table-level against policy-based

	HOW IT WORKS
Table-level	Attached per table with ALTER TABLE. Managed by the table owner.
ABAC policy	Attached at catalog or schema level, matching tables and columns by governed tags.
Coverage	Table-level covers what you configured. ABAC covers new tables automatically as they are tagged.
Who controls it	Table owners can remove their own. ABAC is set by admins and owners cannot override it.

ON THE EXAM

That last row is the governance argument. A control the person being controlled can remove is not much of a control.

ABAC is GA, and the exam guide predates it

- ABAC policies, governed tags and data classification went **generally available in May 2026**.
- The **July 2026 exam guide names only row filters and column masks**, and never mentions policies.
- So answer exam questions with the table-level mechanism, and know that the docs now recommend policies at scale.
- Some regional documentation pages still say Preview or Beta. The blog and the main pages say GA.

> ON THE EXAM
>
> The fourth rename or reposition this course has met that the guide has not absorbed. Teach the examinable answer, name the current one.

Old runtimes fail securely

- **Databricks Runtime below 12.2 LTS does not support row filters or column masks.**
- Those runtimes **fail securely**. Attempting to read such a table returns **no data at all**.
- That is the correct design. The alternative would be silently returning unprotected data.
- ABAC needs more: serverless, or standard and dedicated compute on **DBR 16.4 or above**.

> ON THE EXAM
>
> An empty result set from an old runtime is the protection working, not a bug. Recognise that symptom.

Four documented limitations

- **You cannot apply row-level security or column masks to a view.** Tables only.
- **Iceberg REST catalog and Unity REST APIs cannot read tables with filters or masks.** Delta Lake APIs are unsupported.
- **Path-based access to the files is not supported** on tables with policies.
- **Providers cannot share tables carrying table-level filters or masks** through OpenSharing.

> ON THE EXAM
>
> That last one is a real design constraint. If a table must be shared and masked, the table-level mechanism will not do it.

How policies change the sharing answer

- Tables with **policy-based** filters or masks **can** be shared, unlike table-level ones.
- The condition: the **share owner must be exempt from the policy**, named in its exempting clause.
- Recipients can apply their own filters and masks to **shared tables and foreign tables**.
- Recipients **cannot** apply them to streaming tables or materialized views.

> ON THE EXAM
>
> So the two mechanisms differ on more than convenience. One of them makes a sharing requirement satisfiable and the other does not.

What a pipeline refresh bakes in

- When a pipeline refreshes a materialized view or streaming table, policies are evaluated using the **pipeline owner's or run-as identity**.
- If that identity is subject to a policy, the resulting table **permanently contains masked or filtered data**.
- The fix: add the pipeline owner or run-as identity to the **exempting clause** of every policy on tables the refresh reads.
- Note the word permanently. The masked values are written into the table, not applied at read time.

> ON THE EXAM
>
> Masking applied at write time is not masking. It is data loss, and no downstream grant will recover it.

Who receives masked data, and who does not

- A policy names **who it applies to**, which is the set of users and groups that receive masked or filtered data.
- And **who is exempt**, which is how service identities and stewards keep unrestricted access.
- So the exemption list is a security control in its own right, and belongs under review.
- An unreviewed exemption list is how a policy quietly stops applying to the people who matter.

> ON THE EXAM
>
> Policies are set by admins rather than table owners, which is the separation of duties argument for using them.

Compute requirements and quotas

- Policies need **serverless**, or **standard or dedicated compute on DBR 16.4 or above**.
- Older runtimes **cannot access** tables secured by a policy at all.
- Quotas: **10 policies per catalog**, **10 per schema**, **5 per table**.
- Creating one requires **EXECUTE** on the function and **MANAGE** on, or ownership of, the object.

> ON THE EXAM
>
> Those quotas are low enough to be a design constraint. Policies are meant to be broad and few, not narrow and many.

Dynamic views

- Before filters and masks, the pattern was a **view containing a case expression** checking group membership.
- It works, and it requires users to query the **view** rather than the table, and to be denied access to the table.
- Grant someone the underlying table by mistake and the protection is simply bypassed.
- Filters and masks attach to the **table itself**, so there is no unprotected object to reach around them.

> ON THE EXAM
>
> This is why filters and masks superseded dynamic views. Protection on the table cannot be routed around; protection in a view can.

What to carry into the exam

- **Neither mechanism grants access.** They restrict on top of privileges granted separately.
- Masks: return type must be castable, **one mask per column**, and they may read other columns.
- **Cannot apply to a view.** Providers cannot share tables with table-level filters or masks.
- Runtimes below 12.2 LTS **fail securely**, returning no data.
- **ABAC is GA and recommended at scale**, and the exam guide does not mention it.

> ON THE EXAM
>
> When a scenario needs one rule applied consistently across many tables, that is the policy argument, even if the exam expects the table-level answer.

> A team wants regional analysts to see only their own region's rows in a shared orders table. They create a row filter function checking group membership against the region column and attach it to the table.
>
> Analysts report they cannot see the table at all, and receive a permissions error rather than a filtered result.

What is missing?

A. The row filter function must be granted EXECUTE to each analyst group.

B. The analysts have no SELECT on the table. A row filter restricts access; it does not grant it.

C. Row filters require the table to be a view.

D. The filter must be applied to each region's rows separately.

◆ PAUSE AND DECIDE

What is missing?

A The row filter function must be granted EXECUTE to each analyst group.

Relevant when creating a policy. It is not what produces a permissions error on the table.

B The analysts have no SELECT on the table. A row filter restricts access; it does not grant it.

Stated directly: these controls add restrictions on top of object-level privileges, which must be granted separately.

C Row filters require the table to be a view.

The opposite. They cannot be applied to views at all.

D The filter must be applied to each region's rows separately.

One filter evaluates every row. There is nothing to apply per region.

ON THE EXAM

The error is the clue. A permissions error means no access at all, not filtered access, and filtering cannot create access.

> A governance team applies ABAC masking policies to columns tagged as personal data across a catalog. A declarative pipeline reads three of those tables and writes a gold materialized view.
>
> Weeks later, analysts with full access to the gold table find the personal data columns are masked there, and no grant restores them.

What happened?

A The gold table inherited the ABAC policy through the pipeline's lineage.

B The refresh evaluated policies as the pipeline's identity, which was subject to them, so masked values were written permanently.

C Materialized views cannot store personal data columns under Unity Catalog.

D The analysts need EXECUTE on the masking function to see unmasked values.

◆ PAUSE AND DECIDE

What happened?

A The gold table inherited the ABAC policy through the pipeline's lineage.

Policies do not propagate through lineage. The masking was written into the data.

B The refresh evaluated policies as the pipeline's identity, which was subject to them, so masked values were written permanently.

Documented behaviour. The fix is to exempt the pipeline owner or run-as identity from those policies.

C Materialized views cannot store personal data columns under Unity Catalog.

Invented restriction.

D The analysts need EXECUTE on the masking function to see unmasked values.

There is nothing to unmask. The original values were never written.

ON THE EXAM

The phrase no grant restores them is the tell. Read-time masking is reversible by permission; write-time masking is not.

> A provider must share a customer table with an external partner. Regulatory requirements mean two columns must be masked for that partner. The table currently has table-level column masks applied with ALTER TABLE.
>
> The share fails to include the table.

What is the explanation and the route forward?

A Providers cannot share tables with table-level column masks. Use ABAC policies, with the share owner exempt.

B The partner must be granted EXECUTE on the masking functions before the share works.

C Column masks only apply to Databricks recipients, so the share must use the open protocol.

D Materialise a masked copy of the table and share that instead, since masking cannot be shared at all.

◆ PAUSE AND DECIDE

What is the explanation and the route forward?

A Providers cannot share tables with table-level column masks. Use ABAC policies, with the share owner exempt.

A stated limitation, and the documented alternative. Policy-based masks can be shared when the share owner is exempt.

B The partner must be granted EXECUTE on the masking functions before the share works.

The recipient does not execute the provider's functions.

C Column masks only apply to Databricks recipients, so the share must use the open protocol.

The protocol is not the constraint. The table-level mechanism is.

D Materialise a masked copy of the table and share that instead, since masking cannot be shared at all.

Works, and creates the duplicate that sharing exists to avoid, plus the masking can be shared via policies.

ON THE EXAM

The two mechanisms differ on capability, not only on convenience. This requirement is satisfiable with one and not the other.

> A masked salary table is queried by two teams. One team, on a modern runtime, sees masked values as expected. Another team, still on Databricks Runtime 11.3, reports that the table appears to be empty.
>
> No errors appear in their query output.

What is happening?

A The older runtime does not support column masks and fails securely, returning no data.

B The masking function is returning null for every row on that runtime.

C The second team lacks SELECT, which presents as an empty table.

D Column masks are evaluated only on serverless compute.

◆ PAUSE AND DECIDE

What is happening?

A The older runtime does not support column masks and fails securely, returning no data.

Runtimes below 12.2 LTS do not support filters or masks and are documented as failing securely.

B The masking function is returning null for every row on that runtime.

That would return rows containing nulls, not an empty table.

C The second team lacks SELECT, which presents as an empty table.

Missing SELECT produces a permissions error, as in scenario 1.

D Column masks are evaluated only on serverless compute.

That is closer to the ABAC requirement, and table-level masks work on classic compute.

ON THE EXAM

Empty and unprotected are the two possible behaviours for an unsupported runtime. Databricks chose empty, which is the right choice.

> An organisation has 900 tables across four catalogs. New tables are created weekly. A compliance requirement says any column containing an email address must be masked for everyone outside the marketing team.
>
> A team proposes writing a masking function and applying it to each table with ALTER TABLE, then adding it to new tables as they appear.

What is the stronger approach?

A The proposal is fine; per-table application gives the most precise control.

B Use an ABAC policy at catalog level matched on a governed tag, so new tables are covered automatically as they are tagged.

C Create a dynamic view over each table and revoke access to the tables.

D Apply one mask at the schema level using ALTER SCHEMA.

◆ PAUSE AND DECIDE

What is the stronger approach?

A The proposal is fine; per-table application gives the most precise control.

Nine hundred manual applications, and it depends on somebody remembering every new table forever.

B Use an ABAC policy at catalog level matched on a governed tag, so new tables are covered automatically as they are tagged.

Exactly the case the docs recommend policies for: consistency across many tables and automatic coverage of new ones.

C Create a dynamic view over each table and revoke access to the tables.

900 views to maintain, and one mistaken grant on a base table bypasses the protection.

D Apply one mask at the schema level using ALTER SCHEMA.

Table-level masks attach to a column on a table. There is no schema-level form of them.

ON THE EXAM

Note that the exam may still expect the table-level answer. This scenario is about knowing why the recommendation moved.

> An engineer needs a mask on a customer identifier column. It must reveal the full value to fraud investigators, the last four characters to support staff, and nothing to everyone else. They also want a second mask on the same column for a separate regulatory rule.
>
> They ask how to layer the two masks.

What should they be told?

A Each column can have exactly one mask. Combine both rules into a single function.

B Apply the second mask to a duplicate of the column.

C Masks are applied in the order they were attached, so two can coexist.

D Use a row filter for the second rule, since filters and masks can be combined.

◆ PAUSE AND DECIDE

What should they be told?

A Each column can have exactly one mask. Combine both rules into a single function.

A documented constraint. One function can express several tiers, and may read other columns to decide.

B Apply the second mask to a duplicate of the column.

Duplicating a sensitive column to work around a constraint creates a second thing to protect.

C Masks are applied in the order they were attached, so two can coexist.

They cannot. One mask per column.

D Use a row filter for the second rule, since filters and masks can be combined.

A row filter removes rows. It cannot express a column-level regulatory rule.

ON THE EXAM

The three-tier requirement is not the problem. One function handles that easily; the constraint is on the number of masks.

SECURITY · DOMAIN 7 - ENSURING DATA SECURITY AND COMPLIANCE

PII, anonymisation and purging

Section 7. Four methods, one blueprint, and two retention numbers that are not the same.

SECTION 7 of 10 WEIGHT 10% TOPIC Privacy

admin@sthithapragna.com

Pseudonymised data is **still personal data**. Anonymised data is not. That distinction decides what the law asks of you.

Two pairs, not a list of four

	PSEUDONYMISATION / ANONYMISATION
Scope	Record level / The whole dataset
Reversible	Yes, by authorised users / No, irreversibly altered
Still personal data	Yes / No
Methods	Hashing, tokenisation / Suppression, generalisation

ON THE EXAM

The exam guide names all four methods. Knowing which pair each belongs to answers most questions before you read the options.

Hashing and tokenisation compared

- **Hashing** transforms a value irreversibly. The same input always gives the same output, so records stay **linkable**.
- That linkability is why it is still pseudonymisation. Without a salt, a dictionary attack can recover common values.
- **Tokenisation** replaces the value with a token, and a **separate vault** maps back. Only authorised users can re-identify.
- Spark SQL also offers native encryption functions where a reversible transform is wanted without an external vault.

> ON THE EXAM
>
> Hashing is irreversible and still not anonymisation, because the same person always hashes to the same value and can be tracked across a dataset.

Suppression and generalisation

- **Suppression** excludes the identifying columns entirely. The simplest method, and the one that loses most signal.
- **Generalisation** replaces a value with a broader category, keeping some analytic value while removing identifiability.
- It also covers **removing rows where a group is too small** to be non-identifying.
- Truncating an address to a wider range still supports geographic analysis without identifying anybody.

> ON THE EXAM
>
> Generalisation is the interesting one. It preserves the signal you actually wanted while removing the part that identifies a person.

The documentation is unambiguous. **Complete deletion is preferable to obfuscation**, because eliminating re-identification risk usually requires deleting the data.

Bronze first, driven by a control table

- Databricks recommends **deleting in the bronze layer first**, then propagating to silver and gold.
- Driven by a **scheduled job that queries a table of deletion requests**. You insert a row per request.
- Use **DELETE** for a single record. Use **MERGE against the control table** when deleting many at once.
- After the merge succeeds, **update the control table** to record that the request has been processed.

> ON THE EXAM
>
> The control table is what makes the process auditable. It is the evidence that a request arrived, and that it was actioned.

Sever the link and you delete less

- Using a **non-natural key** such as a user identifier, rather than a natural key like an email address, separates PII from the rest.
- You can then **delete the personal data and leave the non-personal data in place**.
- Behavioural analysis survives, because the events remain and only the identity is gone.
- Use the email as the join key everywhere and every table containing it becomes a table you must purge.

> ON THE EXAM
>
> This is a modelling decision made years before the first deletion request, and it determines how expensive every one of them will be.

Delete is not enough on its own

- With deletion vectors enabled, a delete **marks** rows. The data remains in the underlying files.
- You must also run **REORG TABLE with the purge option** to rewrite the files without those rows.
- That applies to Delta tables, and also to **materialized views and streaming tables**.
- Module 21 introduced this. Here it becomes a compliance requirement rather than a housekeeping one.

> ON THE EXAM
>
> A right-to-erasure request is not satisfied when the DELETE succeeds. It is satisfied after the purge, and after the old files are removed.

Two retention numbers, and they are different

- Delta retains **table history, including deleted records, for 30 days** by default, for time travel and rollback.
- **VACUUM by default reduces time travel to 7 days**, and removes those historical versions from cloud storage.
- Two different properties. A question can turn on which one it is describing.
- And note: even when previous versions are removed from the table, **the data is still in cloud storage** until vacuum runs.

> ON THE EXAM
>
> Thirty days of history and a seven-day vacuum window are not in conflict. They govern different things.

Three routes, in the order the docs recommend

- **Predictive optimization** for Unity Catalog managed tables. The recommended route, and it maintains streaming tables and materialized views.
- Failing that, **Lakeflow pipelines perform maintenance automatically within 24 hours** of a streaming table or materialized view being updated.
- Failing both, run **VACUUM** yourself on Delta tables.
- Doing none of the three means deleted records remain recoverable in cloud storage indefinitely.

> ON THE EXAM
>
> Module 20's automatic maintenance and this compliance requirement are the same mechanism. Predictive optimization is doing privacy work on your behalf.

Materialized views handle it themselves

- **Materialized views automatically handle deletions in their sources.** Nothing special is required.
- You must still **refresh** the view and run maintenance for the deletion to be fully processed.
- A materialized view always returns the correct result, using incremental computation **only when it is cheaper**.
- So a deletion in a source may force a **full recompute**. Correctness is never traded for speed.

> ON THE EXAM
>
> This is module 5's 'keeps a query's answer correct' arriving as a compliance property. Deletions propagate because correctness demands it.

Streaming tables need three deliberate steps

- Streaming tables process **append-only** data. Deleting from a streaming source is **not supported and breaks the stream**.
- The documented method: **delete in the source** with DML, **delete from the streaming table** with DML, then set **skipChangeCommits** on the read.
- The order matters. You apply the deletion to both tables yourself, and only then tell the stream to ignore the change commits.
- Setting the flag **without** the two deletions would simply discard the change and leave the data present.

> ON THE EXAM
>
> Module 21 called this flag a trap. Here it is the documented answer, and the difference is whether you deleted from both tables first.

Full refresh, and what it can cost

- You can instead delete from the source and then **fully refresh** the streaming table.
- That clears the streaming state and reprocesses everything from the source.
- **Any upstream source beyond its retention period will not be reprocessed**, which causes data loss.
- Recommended **only** where historical data is still available and reprocessing is not costly.

ON THE EXAM

Module 5's retention arithmetic, appearing for the fourth time. A Kafka topic ageing out after seven days cannot replay a year.

Stream from the change feed instead

- The documentation calls this the **more robust** implementation for streaming.
- Stream from the **change feeds** of the Delta tables and handle updates and deletes in your processing code.
- That removes the need for both the skip flag and the full refresh.
- Module 21 made this argument from the performance side. The privacy documentation makes it from the compliance side.

> ON THE EXAM
>
> Two modules, two motivations, the same recommendation. That is usually a sign it is the right design.

Upstream sources are in scope too

- **The regulations apply to all data**, including sources outside Delta such as message buses, files and databases.
- Deleting inside Databricks is not sufficient. You must delete in the upstream sources as well.
- Otherwise the next ingestion run simply brings the record back.
- You may also need to **export workspace data** for compliance or backup before running deletion workflows.

> ON THE EXAM
>
> A deletion that the next pipeline run undoes is not a deletion. Scope the purge to the whole path, not just the lakehouse.

What to carry into the exam

- **Pseudonymisation is still personal data**: hashing and tokenisation. Anonymisation is not: suppression and generalisation.
- **Complete deletion is preferable to obfuscation.** Bronze first, driven by a control table.
- With deletion vectors, **DELETE is not enough**. Reorg with purge, then maintenance.
- **Materialized views handle deletions automatically. Streaming tables need three deliberate steps.**
- History defaults to **30 days**; vacuum reduces time travel to **7**. Delete upstream too.

> ON THE EXAM
>
> That completes Section 7 at 10%. Section 8, Data Governance, is next at 7%.

> A team hashes customer email addresses with a standard hash function before loading them into an analytics catalog. They tell the privacy office the data is now anonymised and outside the scope of the regulation.
>
> Analysts routinely count distinct hashed values to measure unique customers.

What is wrong with that claim?

A Nothing. Hashing is irreversible, so the data is anonymised.

B Hashing is pseudonymisation. The same input always hashes identically, so records remain linkable and it is still personal data.

C The claim holds only if the hash is salted, which makes it anonymisation.

D Hashing is anonymisation, but counting distinct values re-identifies the data.

◆ PAUSE AND DECIDE

What is wrong with that claim?

A Nothing. Hashing is irreversible, so the data is anonymised.

Irreversible is not the test. Linkability is, and hashing preserves it.

B Hashing is pseudonymisation. The same input always hashes identically, so records remain linkable and it is still personal data.

The very behaviour analysts rely on for counting unique customers is what makes it pseudonymisation.

C The claim holds only if the hash is salted, which makes it anonymisation.

A salt frustrates dictionary attacks. It does not remove linkability within the dataset.

D Hashing is anonymisation, but counting distinct values re-identifies the data.

Gets the classification wrong, then invents a mechanism to explain it.

ON THE EXAM

The analysts' own usage proves the point. If you can count unique customers, the records are still linked to individuals.

> A right-to-erasure request is processed. The engineer runs DELETE on every table containing the user's records, confirms zero rows are returned by follow-up queries, and closes the ticket.
>
> The tables have deletion vectors enabled, and the estate does not use predictive optimization.

What remains to be done?

A Nothing. The queries confirm the records are gone.

B Run REORG TABLE with the purge option, then maintenance or VACUUM, since the rows are only marked and the files still exist.

C Re-run the DELETE with a longer retention setting.

D Convert the tables to materialized views, which delete permanently.

◆ PAUSE AND DECIDE

What remains to be done?

A Nothing. The queries confirm the records are gone.

Queries confirm the rows are filtered out. They say nothing about the files.

B Run REORG TABLE with the purge option, then maintenance or VACUUM, since the rows are only marked and the files still exist.

Deletion vectors mark rather than rewrite, and history plus cloud storage retain the data until purge and vacuum.

C Re-run the DELETE with a longer retention setting.

Retention governs how long files survive, and re-running the delete changes nothing.

D Convert the tables to materialized views, which delete permanently.

Materialized views also need the purge with deletion vectors enabled.

ON THE EXAM

Zero rows returned is exactly what a deletion vector produces. The evidence the engineer collected cannot distinguish marked from removed.

> A pipeline has a bronze streaming table, a silver streaming table, and two gold materialized views. A deletion request is processed by deleting from the source Delta tables only.
>
> The next pipeline run fails on the streaming tables. The materialized views are fine.

What explains both outcomes, and what is the fix?

A Materialized views handle source deletions automatically. Streaming tables need the rows deleted from them too, then skipChangeCommits on the read.

B Streaming tables need a longer watermark to tolerate deletions.

C Materialized views ignore deletions, which is why they did not fail. Both need a full refresh.

D The deletion should have been applied to the gold layer first, then propagated upstream.

◆ PAUSE AND DECIDE

What explains both outcomes, and what is the fix?

A Materialized views handle source deletions automatically. Streaming tables need the rows deleted from them too, then skipChangeCommits on the read.

Both halves are documented: MVs handle deletions, and streaming tables break because a delete is not an append.

B Streaming tables need a longer watermark to tolerate deletions.

Watermarks govern late-arriving event-time data, not deletes.

C Materialized views ignore deletions, which is why they did not fail. Both need a full refresh.

They do not ignore deletions; they process them correctly, which is why nothing broke.

D The deletion should have been applied to the gold layer first, then propagated upstream.

The recommendation is bronze first, propagating downstream.

ON THE EXAM

The two dataset types genuinely behave differently here, and the scenario shows both behaviours in the same pipeline.

> An auditor asks how quickly deleted personal data becomes unrecoverable. An engineer answers that Delta retains history for 30 days, so it takes a month.
>
> The tables are Unity Catalog managed with predictive optimization enabled, and the default vacuum retention has not been changed.

What is the more accurate answer?

A 30 days is correct, since that is the table history retention.

B History retention and vacuum retention are different. Vacuum removes files beyond the 7-day default, and predictive optimization runs it.

C The data is unrecoverable immediately after DELETE.

D It is indefinite, because cloud storage never removes the files.

◆ PAUSE AND DECIDE

What is the more accurate answer?

A 30 days is correct, since that is the table history retention.

That governs time travel history, not how long the underlying files survive.

B History retention and vacuum retention are different. Vacuum removes files beyond the 7-day default, and predictive optimization runs it.

Two distinct properties. Vacuum's default reduces time travel to 7 days and removes the files from cloud storage.

C The data is unrecoverable immediately after DELETE.

With deletion vectors it is not even rewritten, let alone removed.

D It is indefinite, because cloud storage never removes the files.

True only if nothing ever runs maintenance. Here predictive optimization does.

ON THE EXAM

Two numbers, two properties. Confusing them gives an auditor an answer that is wrong in the direction of over-promising.

> A company processes deletion requests carefully across every Delta table in its lakehouse. Three weeks later, a routine check finds records for several deleted users present again in the bronze layer.
>
> Ingestion is from a Kafka topic with 30-day retention, replayed after an unrelated incident.

What went wrong?

A The deletion was never committed, so the records were never removed.

B The regulations apply to upstream sources too. The records were never deleted from Kafka, so a replay reintroduced them.

C Time travel restored the deleted versions automatically.

D Predictive optimization restored the files during maintenance.

◆ PAUSE AND DECIDE

What went wrong?

A The deletion was never committed, so the records were never removed.

They were removed. Something put them back.

B The regulations apply to upstream sources too. The records were never deleted from Kafka, so a replay reintroduced them.

Stated directly: deleting inside Databricks is not sufficient. The source still held the records.

C Time travel restored the deleted versions automatically.

Time travel lets you read old versions. It does not write them back.

D Predictive optimization restored the files during maintenance.

Maintenance removes unreferenced files. It never resurrects data.

ON THE EXAM

Three weeks against thirty days of retention is the arithmetic. The records were still in the topic, waiting.

> A team must publish regional health statistics derived from patient records. Some regions have very few patients with a given condition. The published dataset must not be subject to privacy regulation at all.
>
> A colleague suggests hashing the patient identifiers before publishing.

What is the correct approach?

A Hash the identifiers, which removes the personal data from the dataset.

B Suppress the identifying columns and generalise, removing rows where a group is too small to be non-identifying.

C Tokenise the identifiers and keep the vault in a separate catalog.

D Apply a column mask on the identifier column before publishing.

◆ PAUSE AND DECIDE

What is the correct approach?

A Hash the identifiers, which removes the personal data from the dataset.

Hashing is pseudonymisation. The result is still personal data, so the requirement is unmet.

B Suppress the identifying columns and generalise, removing rows where a group is too small to be non-identifying.

Anonymisation, which is what takes the dataset out of scope. The small-group problem is exactly what generalisation addresses.

C Tokenise the identifiers and keep the vault in a separate catalog.

Also pseudonymisation, and a vault makes re-identification easier rather than impossible.

D Apply a column mask on the identifier column before publishing.

Masking is applied at query time on your platform. It does not travel with a published dataset.

ON THE EXAM

The requirement names the test: out of scope entirely. Only anonymisation achieves that, and only anonymisation handles the small-group risk.

GOVERNANCE · DOMAIN 8 - DATA GOVERNANCE

Unity Catalog permission inheritance

Section 8. Three things must be true, and ownership is not one of them.

SECTION 8 of 10 WEIGHT 7% TOPIC Permissions

admin@sthithapragna.com

To read a table, three separate things must be true. Two of them are on objects that are not the table.

Containers and everything else

- A metastore at the top, then a **three-level namespace**: catalog, schema, table.
- **Container objects** are **catalogs**, which hold schemas, and **schemas**, which hold tables, views, volumes and functions.
- Tables, views, volumes and functions are **non-container**. Nothing lives inside them.
- Only containers do inheritance, require usage privileges on themselves, and support creation privileges.

> ON THE EXAM
>
> The container distinction is not decoration. It determines which objects can pass privileges down and which cannot.

What each operation actually requires

OPERATION	PRIVILEGES NEEDED
Read a table or view	USE CATALOG, USE SCHEMA, and SELECT
Write to a table	USE CATALOG, USE SCHEMA, and MODIFY
Create a schema	USE CATALOG, and CREATE SCHEMA on the catalog
Create a table	USE CATALOG, USE SCHEMA, and CREATE TABLE on the schema
Execute a function	USE CATALOG, USE SCHEMA, and EXECUTE
Read files from a volume	USE CATALOG, USE SCHEMA, and READ VOLUME

ON THE EXAM

Notice the shape. Two usage privileges on the parents, then one specific privilege on the object. Learn the shape rather than the rows.

Usage privileges are an administrative boundary

- A table owner can grant SELECT on their table to anybody. That grant alone does nothing.
- The recipient still cannot read it without **USE CATALOG and USE SCHEMA** on the parents.
- Only **catalog and schema owners, or MANAGE holders**, can grant those usage privileges.
- So the design **prevents table owners from granting access outside approved boundaries**.

> ON THE EXAM
>
> This is stated as a purpose, not a side effect. Usage privileges exist so that higher-level administrators keep control of the perimeter.

Current and future, downward only

- A privilege on a **catalog** applies to every schema in it, and every table, view, volume and function in those schemas.
- A privilege on a **schema** applies to everything in that schema.
- It covers **current and future** objects. A table created tomorrow is already covered.
- That is powerful and blunt. Granting SELECT on a catalog grants it on tables nobody has designed yet.

> ON THE EXAM
>
> The future clause is the point of inheritance and its main risk. Grant broadly and you have granted on things that do not exist.

Metastore grants do not inherit

- **Privileges granted on a metastore do not inherit to child objects.** The chain starts at the catalog.
- Metastore grants cover metastore-scoped operations: create catalog, create external location, create share.
- There is **one documented exception**. **READ METADATA on a metastore does inherit** to every object in it.
- And that exception **needs no usage privileges** to exercise, unlike everything else.

> ON THE EXAM
>
> So the highest level privileges inherit from is the catalog, with a single named exception for metadata visibility.

Implied, not granted

- The principal that creates an object becomes its **initial owner**. An owner may be a user, service principal or group.
- Owners have all capabilities, and **ALL PRIVILEGES is not explicitly granted to them**.
- So **SHOW GRANTS will not return ALL PRIVILEGES for the owner**. The capability is implied.
- Owners also need **no usage privileges** at all.

> ON THE EXAM
>
> An empty grants list on an object you own is expected, not a misconfiguration. That distinction catches people auditing permissions.

Ownership does not inherit downward

- **Owning a catalog does not make you the owner of its schemas.** Ownership does not flow down.
- But an owner **automatically gets the ability to manage all child objects**, current and future.
- That too is **implied rather than explicitly granted**, so it will not show in a grants listing either.
- Manage is not the same as read. A catalog owner can administer a table without being able to query it.

> ON THE EXAM
>
> Two things are true at once: you do not own the children, and you can manage them. Questions are built on people assuming only one of those.

Ownership against the MANAGE privilege

OWNER

Capabilities implied

- All capabilities on the object, implied
- Manage child objects, implied
- No usage privileges required
- Exactly one principal

MANAGE

Administration without access

- Privileges must be granted separately
- But holders can grant them to themselves
- Usage needed only above where it is held
- Can be granted to many principals

THE TIEBREAKER

The practical difference: MANAGE lets several people administer an object, and ownership is a single principal that must be transferred.

MANAGE has reduced usage requirements

- You need usage only on levels **strictly above** where MANAGE is held, not at or below it.
- **MANAGE on a catalog requires no usage privileges at all.**
- MANAGE on a schema requires USE CATALOG on the parent, but not USE SCHEMA on the schema.
- MANAGE on a table requires both USE CATALOG and USE SCHEMA.

> ON THE EXAM
>
> Because MANAGE on a container is inherited by its children, holding it high up means you need no usage privileges below.

Data access still needs usage privileges

- The reduction applies **only to metadata capabilities** that MANAGE provides.
- **SELECT and MODIFY still require USE CATALOG and USE SCHEMA**, even for a MANAGE holder.
- So MANAGE on a catalog lets you administer everything and read nothing, until you grant yourself more.
- That separation is deliberate. Administration and access are different jobs.

> ON THE EXAM
>
> A MANAGE holder who grants themselves SELECT also needs the usage privileges. The reduction never applies to reading data.

What ALL PRIVILEGES does not include

- It **implies** all applicable privileges for the object type, evaluated at permission-check time rather than at grant time.
- It does **not** include **MANAGE**. Excluded explicitly to avoid accidental privilege escalation.
- It also excludes **READ METADATA**, **EXTERNAL USE SCHEMA** and **EXTERNAL USE LOCATION**.
- Evaluating at check time means a privilege added to the platform later is covered automatically.

> ON THE EXAM
>
> All does not mean all. Four named privileges sit outside it, three of them because they cross a security boundary.

The BROWSE privilege

- **BROWSE lets users discover objects and view metadata without any access to the underlying data.**
- They see the object exists, and its name, description and tags, and can **request access**.
- It works **without USE CATALOG or USE SCHEMA**, which is unusual and is the point.
- Granted at **catalog level**. Databricks recommends granting it on catalogs to **all account users**.

> ON THE EXAM
>
> This is the bridge to the next module. Discoverability is a governance objective, and BROWSE is how you get it without granting data access.

Composite and child privileges

- **MANAGE** is composite, and **READ METADATA** is its child, giving read-only metadata visibility without the management abilities.
- **MODIFY** is composite, and **INSERT, UPDATE and DELETE** are its children.
- They are granted and revoked **independently**. Granting the composite does not grant the children.
- And **revoking the composite does not revoke a child that was granted explicitly**.

> ON THE EXAM
>
> That second half is the trap. Revoke MODIFY from somebody who also holds DELETE explicitly, and they can still delete.

Two things owners deliberately do not get

- **Schema owners do not get EXTERNAL USE SCHEMA by default.**
- **External location owners do not get EXTERNAL USE LOCATION by default.**
- Both defaults exist to avoid **accidental data exfiltration**.
- So ownership is not quite unlimited. Two capabilities are withheld until somebody grants them deliberately.

> ON THE EXAM
>
> An exception to the rule that owners have every capability, and the reason given is exfiltration rather than tidiness.

What to carry into the exam

- **Three things must be true:** USE CATALOG, USE SCHEMA, and the specific privilege.
- Inheritance runs **catalog downward, current and future**. **Metastore grants do not inherit**, except READ METADATA.
- **Ownership does not inherit**, but owners can manage all children. Both are implied, not granted.
- **ALL PRIVILEGES excludes MANAGE**, READ METADATA, and the two external use privileges.
- **BROWSE** gives discovery without data access, and without usage privileges.

> ON THE EXAM
>
> When a permissions question looks like it needs one grant, count again. It usually needs three.

> A table owner grants SELECT on their table to the analytics group so they can build a report. The analysts report that the table does not appear and they cannot query it.
>
> The grant is confirmed present with SHOW GRANTS on the table.

What is missing?

A. USE CATALOG on the catalog and USE SCHEMA on the schema. All three are required.

B. The analysts need BROWSE on the table before SELECT takes effect.

C. The grant must be made at schema level to be inherited by the table.

D. The table owner must transfer ownership to the analytics group.

◆ PAUSE AND DECIDE

What is missing?

A USE CATALOG on the catalog and USE SCHEMA on the schema. All three are required.

Stated directly: SELECT alone is not sufficient without usage privileges on both parents.

B The analysts need BROWSE on the table before SELECT takes effect.

BROWSE aids discovery and is granted at catalog level. It is not a prerequisite for SELECT.

C The grant must be made at schema level to be inherited by the table.

A schema-level grant would work, and only because it comes with usage on the parents in practice. The table grant is valid.

D The table owner must transfer ownership to the analytics group.

Enormous overreach for a read requirement, and there can only be one owner.

ON THE EXAM

The table owner cannot fix this themselves. Only catalog and schema owners, or MANAGE holders, can grant the usage privileges.

> A governance lead owns the finance catalog. They are asked to audit permissions and find that SHOW GRANTS on a table within it returns nothing for them, though they can administer the table.
>
> They also cannot query the table's contents.

Which explanation fits all three observations?

A Permissions are broken and need to be re-granted from the catalog down.

B Catalog owners get implied management of child objects, not explicit grants, and management is not data access.

C They must be a metastore admin to see grants on child tables.

D SHOW GRANTS only lists grants made in the last 90 days.

◆ PAUSE AND DECIDE

Which explanation fits all three observations?

A Permissions are broken and need to be re-granted from the catalog down.

Nothing is broken. Every observation is documented behaviour.

B Catalog owners get implied management of child objects, not explicit grants, and management is not data access.

Explains all three: implied capabilities do not appear in SHOW GRANTS, and managing is not reading.

C They must be a metastore admin to see grants on child tables.

Object owners can view grants. The point is that their own implied capability is not listed.

D SHOW GRANTS only lists grants made in the last 90 days.

Invented.

ON THE EXAM

Three observations, one mechanism. Implied capabilities are real but invisible, and management is separate from access.

> A security review requires that a contractor be removed from write access to a table. The contractor holds MODIFY on the table, and also holds DELETE, granted separately six months earlier.
>
> An administrator revokes MODIFY and closes the ticket.

What is the problem?

A Nothing. Revoking MODIFY revokes its child privileges.

B Composite and child privileges are revoked independently. The explicit DELETE grant remains, so they can still delete rows.

C Revoking MODIFY also removes SELECT, so the contractor now has no access at all.

D DELETE cannot be granted separately from MODIFY.

◆ PAUSE AND DECIDE

What is the problem?

A Nothing. Revoking MODIFY revokes its child privileges.

The docs state the opposite: revoking a composite does not revoke an explicitly granted child.

B Composite and child privileges are revoked independently. The explicit DELETE grant remains, so they can still delete rows.

Granting and revoking are independent in both directions. The child grant survives.

C Revoking MODIFY also removes SELECT, so the contractor now has no access at all.

SELECT is a separate privilege, not a child of MODIFY.

D DELETE cannot be granted separately from MODIFY.

It can. That is what a child privilege is for.

ON THE EXAM

Revoking the broad privilege and assuming the narrow ones went with it is exactly the wrong direction to be wrong in.

> A data platform team wants analysts across the company to be able to find datasets and request access, without being able to read anything they have not been granted.
>
> A proposal suggests granting SELECT on all catalogs to all account users, then relying on row filters to restrict what they see.

What is the better approach?

A The proposal is sound; row filters will restrict the data appropriately.

B Grant BROWSE on the catalogs to all account users, which allows discovery and access requests without data access.

C Grant USE CATALOG to all account users, which allows discovery without SELECT.

D Grant READ METADATA on each table individually.

◆ PAUSE AND DECIDE

What is the better approach?

A. The proposal is sound; row filters will restrict the data appropriately.

Grants SELECT on all current and future tables, then relies on filters existing everywhere. One missing filter exposes a table.

B. Grant BROWSE on the catalogs to all account users, which allows discovery and access requests without data access.

Exactly what BROWSE is for, and Databricks recommends this pattern by name.

C. Grant USE CATALOG to all account users, which allows discovery without SELECT.

Closer, and BROWSE is the purpose-built privilege and needs no usage privileges.

D. Grant READ METADATA on each table individually.

Would work and is per-object toil. BROWSE is granted once at catalog level.

ON THE EXAM

A is the dangerous one: it grants SELECT on future tables too, so a table created next year is exposed until somebody remembers a filter.

A governance team needs to administer permissions across an entire catalog: granting, revoking, and transferring ownership. Policy says they must not be able to read the data itself.

Three people need this capability.

An administrator proposes transferring catalog ownership to the governance lead.

What is the better arrangement?

A Transfer ownership to the lead, then have them delegate to the other two.

B Grant MANAGE on the catalog to all three, which requires no usage privileges and does not confer data access.

C Grant ALL PRIVILEGES on the catalog to the governance group.

D Make the governance group the owner, since a group can own an object.

◆ PAUSE AND DECIDE

What is the better arrangement?

A Transfer ownership to the lead, then have them delegate to the other two.

Ownership implies all capabilities including data access, which policy forbids.

B Grant MANAGE on the catalog to all three, which requires no usage privileges and does not confer data access.

MANAGE can be held by many, needs no usage privileges at catalog level, and does not imply SELECT.

C Grant ALL PRIVILEGES on the catalog to the governance group.

Confers data access, and notably does not include MANAGE.

D Make the governance group the owner, since a group can own an object.

True that a group can own, and ownership still implies all capabilities including reading.

ON THE EXAM

Note that C fails in both directions at once: it grants the data access policy forbids, and withholds the MANAGE they actually need.

> An auditor must be able to see the name, description and lineage of every object in the metastore, across all catalogs, without any ability to read data.
>
> A proposal is to grant them BROWSE on each catalog as it is created.

Is there a simpler option?

A No. BROWSE per catalog is the only mechanism for metadata visibility.

B Grant READ METADATA on the metastore, which is the one privilege that inherits from the metastore and needs no usage privileges.

C Grant ALL PRIVILEGES on the metastore, which inherits to every object.

D Make the auditor a metastore admin, which is the standard approach.

◆ PAUSE AND DECIDE

Is there a simpler option?

A. No. BROWSE per catalog is the only mechanism for metadata visibility.

BROWSE works and is per catalog. READ METADATA on the metastore covers everything at once.

B. Grant READ METADATA on the metastore, which is the one privilege that inherits from the metastore and needs no usage privileges.

The documented exception to metastore grants not inheriting, and it needs no usage privileges to exercise.

C. Grant ALL PRIVILEGES on the metastore, which inherits to every object.

Metastore grants do not inherit, and ALL PRIVILEGES excludes READ METADATA anyway.

D. Make the auditor a metastore admin, which is the standard approach.

An enormous grant to solve a read-only metadata requirement.

ON THE EXAM

The requirement says every object across all catalogs. That phrasing points at the one privilege that inherits from the metastore.

GOVERNANCE · DOMAIN 8 - DATA GOVERNANCE

Metadata, tags, lineage and discoverability

Section 8. Making data findable, and knowing which of it to trust.

SECTION 8 of 10 WEIGHT 7% TOPIC Discoverability

admin@sthithapragna.com

Data nobody can find has no value. Data nobody trusts has **negative** value, because somebody will use it anyway.

How the documentation frames it

- **Catalog Explorer** to browse, search and inspect what exists.
- **AI-generated comments** to describe tables and columns without waiting for humans to write them.
- **Certified and deprecated tags** to signal which assets should be trusted.
- Plus **lineage**, which answers where something came from and what depends on it.

> ON THE EXAM
>
> Those first three are the documentation's own sentence about discoverability. If a question asks how to make data findable, they are the expected vocabulary.

Who can actually see them

- **Any user with BROWSE privileges can view comments.** That is what makes them a discoverability tool.
- So somebody with no access to the data at all can read what a table is for, and decide whether to request it.
- To **add or edit** a comment you must **own the object or hold MANAGE** on it.
- Comments cannot be edited on objects in a Delta Sharing catalog. In the legacy metastore, tables only.

> ON THE EXAM
>
> This is module 27's BROWSE arriving with a purpose. Granting BROWSE broadly is only useful if there is something written down to read.

Two commands, and one exception

- **COMMENT ON** works for any securable object **except table columns**.
- For a **column**, use the comment option on the alter statement instead.
- Comments can also be added and edited directly in Catalog Explorer.
- Modifying a table comment records a properties operation in the **table history**, capturing the SQL used.

> ON THE EXAM
>
> That last point is quietly useful. Documentation changes are versioned alongside the data, so you can see when a description changed and to what.

Markdown, and where it stops

- Comments support **basic Markdown** in Catalog Explorer. All basic syntax is supported **except images**.
- **Markdown does not render in DESCRIBE output.** There you get the raw characters.
- **Using Markdown comments has no impact on query performance.**
- So a table comment can be a genuine short document, with headings and links, rather than one sentence.

> ON THE EXAM
>
> Write for Catalog Explorer, and remember that anyone reading through DESCRIBE sees the syntax rather than the formatting.

AI-generated comments

- Databricks can generate comments for you, also called **AI-generated documentation**.
- Available for **tables and table columns only**. Not for catalogs, schemas or other objects.
- Offered as **suggestions you review and apply**, rather than written silently.
- That matters at scale. Nobody is going to hand-document nine hundred tables, so the realistic choice is generated or nothing.

> ON THE EXAM
>
> The scope limit is the examinable bit. If a question asks about documenting a catalog or a volume, generated comments are not the answer.

Certified and deprecated are a system tag

- There is a **system-governed tag** for certification status, with exactly **two values: certified and deprecated**.
- It is **not free-form**. You cannot invent a third value or define your own equivalent tag with the same meaning.
- It applies to catalogs, schemas, tables, dashboards, notebooks, apps and Genie Agents.
- The status is **displayed next to object names**, and influences how data appears in notebooks and the SQL editor.

> ON THE EXAM
>
> Being system-governed is the point. Everybody's certified means the same thing, which is exactly what a free-form tag cannot guarantee.

Setting and searching certification

- Assign it from the object's menu in the interface, choosing certified, deprecated, or none.
- Or in SQL with a **set tag** statement naming the system certification status key.
- **Search for it with the certification status keyword**, so you can list every certified asset.
- You can update or remove the status at any time. Deprecating is not permanent.

> ON THE EXAM
>
> Deprecated is the underused half. Marking a superseded table stops people finding it and trusting it, which is often more valuable than certifying a good one.

Keys, optional values, and governance

- A tag is an attribute with a **key and an optional value**, applicable to any securable object.
- Tags make objects **searchable and categorisable**, which is the discovery use.
- **Governed tags** control which tags may exist and **who is permitted to apply them**.
- Without that control, tagging degrades into inconsistent free text nobody can rely on.

> ON THE EXAM
>
> Governed tags went GA alongside ABAC and data classification. The three are designed as one system, not three features.

Tags do double duty

- The same tag that makes a column **findable** can be what a **masking policy matches on**.
- Applying coarse-grained tags at **catalog or schema level** means they cover nested objects.
- So one tagging scheme serves discovery and protection at once, which is why the two shipped together.
- It also raises the stakes. A tag that is wrong is now both a discovery problem and a security problem.

> ON THE EXAM
>
> This is the strongest argument for governed tags. When a tag drives access control, who can apply it matters as much as what it says.

Data classification

- Databricks can **automatically scan and tag sensitive data**, such as personal data, across tables.
- That closes the loop: classification finds it, tags mark it, and policies protect it.
- To manage the configuration you must be the **catalog owner**, or hold **USE CATALOG and MANAGE** on the catalog.
- The alternative is asking every team to correctly identify their own sensitive columns, forever.

ON THE EXAM

Classification, tags and policies are three parts of one pipeline: detect, mark, protect. Knowing the shape answers design questions.

What is captured, and how

- Lineage is captured **automatically at runtime**. You do not declare it or maintain it.
- Across **all languages**, and down to the **column level**.
- It includes the **notebooks, jobs and dashboards** related to the query, not only the tables.
- Visualised in **near real-time** in Catalog Explorer, and queryable from system tables.

> ON THE EXAM
>
> Automatic capture is the important word. Hand-maintained lineage diagrams are wrong within weeks; this is derived from what actually ran.

The questions it answers

- **Where did this come from?** Which sources feed a number somebody is querying.
- **What breaks if I change this?** Impact analysis before altering a schema or dropping a column.
- **Where did this personal data go?** Tracing sensitive columns downstream, which is module 26's problem.
- **Model lineage** also traces a model back to the datasets it was trained and evaluated on.

> ON THE EXAM
>
> Column-level lineage is what makes the third question answerable. Table-level would only tell you a table was involved somewhere.

The Insights tab

- Catalog Explorer browses and manages data, schemas, tables, permissions, owners, external locations and credentials.
- Its **Insights tab** shows the **most frequent recent queries and users** of any registered table.
- That answers a question nothing else does: **who actually uses this, and how?**
- Useful before deprecating something, and useful for finding the person who understands a table.

> ON THE EXAM
>
> Lineage tells you what is connected. Insights tells you what is actually being used, which is a different and often more useful question.

A four-level namespace was announced

- At the June 2026 summit, Databricks announced a **four-level namespace**: metastore, catalog, schema, table.
- The aim is a single address for every asset across accounts, regions and clouds, with one discovery experience.
- **Cross-region governance is described as coming soon in preview.** Cross-cloud and cross-account follow.
- The current documentation still describes the **three-level** namespace, so that is the examinable answer.

> ON THE EXAM
>
> Announced, not shipped. Module 27 teaches three-level because that is what the current docs and the exam describe.

What to carry into the exam

- Discoverability is **Catalog Explorer, AI-generated comments, and certified and deprecated tags**.
- **BROWSE lets people read comments** without data access. Writing them needs ownership or MANAGE.
- AI-generated comments cover **tables and columns only**. Markdown does not render in DESCRIBE.
- Certification is a **system-governed tag** with exactly two values. It is not free-form.
- **Lineage is automatic**, all languages, column level, including notebooks, jobs and dashboards.

> ON THE EXAM
>
> That completes Section 8 at 7%. Section 9, debugging and deploying, is next at 10%.

> A platform team grants BROWSE on all catalogs to every account user, expecting people to start finding datasets themselves. Six weeks later, discovery has not improved and the data team still receives the same requests.
>
> Investigation shows almost no tables have descriptions.

What is missing?

A. BROWSE does not permit viewing comments; USE CATALOG is also required.

B. BROWSE lets users read comments, but there are almost none to read. Add comments, using AI generation at scale.

C. Users also need SELECT before search returns results.

D. Discovery requires the certification tag on every table before assets appear in search.

◆ PAUSE AND DECIDE

What is missing?

A BROWSE does not permit viewing comments; USE CATALOG is also required.

BROWSE specifically allows viewing comments without usage privileges.

B BROWSE lets users read comments, but there are almost none to read. Add comments, using AI generation at scale.

The two mechanisms are designed to work together. Access to documentation is useless when none exists.

C Users also need SELECT before search returns results.

That would defeat the purpose of discovery without data access.

D Discovery requires the certification tag on every table before assets appear in search.

Certification signals trust. It is not a prerequisite for appearing in search.

ON THE EXAM

Half a mechanism does nothing. BROWSE is the permission to read documentation, and somebody still has to write it.

> A team wants to mark superseded tables so colleagues stop using them. They propose creating a tag with the key status and the value old, and asking everyone to check it before querying.
>
> Several other teams have already invented their own conventions for the same thing.

What is the better approach?

A Standardise on the proposed status tag and document the convention widely.

B Use the system-governed certification status tag with the deprecated value, which displays next to object names.

C Add a comment on each table saying it is deprecated.

D Revoke SELECT on the superseded tables so nobody can use them.

◆ PAUSE AND DECIDE

What is the better approach?

A Standardise on the proposed status tag and document the convention widely.

Reinvents a system tag, and relies on convention where a governed mechanism exists.

B Use the system-governed certification status tag with the deprecated value, which displays next to object names.

System-governed, two fixed values, displayed beside object names and searchable. Built for exactly this.

C Add a comment on each table saying it is deprecated.

Only helps someone who opens the table and reads the comment.

D Revoke SELECT on the superseded tables so nobody can use them.

Breaks anything still legitimately reading them, with no warning period.

ON THE EXAM

The several competing conventions in the scenario are the argument for a system-governed tag: everybody's deprecated means the same thing.

> A team is asked to remove a column from a widely used silver table. They need to know what will break before doing it, including reports nobody has mentioned.
>
> They have no diagram of their pipeline dependencies.

What should they use?

A Column-level lineage, which is captured automatically and includes related notebooks, jobs and dashboards.

B The Insights tab, which shows every downstream table.

C The audit system table, which records every read of the column.

D Ask each team to declare their dependencies, then build a diagram.

◆ PAUSE AND DECIDE

What should they use?

A Column-level lineage, which is captured automatically and includes related notebooks, jobs and dashboards.

Exactly the impact-analysis case, and it covers dashboards and jobs rather than only tables.

B The Insights tab, which shows every downstream table.

Insights shows frequent recent queries and users, not the dependency graph.

C The audit system table, which records every read of the column.

Audit records access events. Lineage is the purpose-built view of dependency.

D Ask each team to declare their dependencies, then build a diagram.

Slow, and it captures only what people remember. Lineage is derived from what actually ran.

ON THE EXAM

Note what A includes. Table-level lineage would find dependent tables; dashboards and jobs are where the unmentioned reports live.

> An engineer writes a detailed table comment using Markdown headings and a bulleted list. It renders correctly in Catalog Explorer. A colleague running DESCRIBE TABLE EXTENDED reports that the comment is full of hash symbols and asterisks.
>
> They ask whether the comment is corrupted.

What should they be told?

A The comment is corrupted and should be rewritten as plain text.

B Markdown renders in Catalog Explorer but not in DESCRIBE output, where the raw syntax is returned.

C Markdown in comments degrades query performance and should be removed.

D DESCRIBE only supports comments under a fixed character limit.

◆ PAUSE AND DECIDE

What should they be told?

A The comment is corrupted and should be rewritten as plain text.

Nothing is corrupted. Both views are showing the same stored string.

B Markdown renders in Catalog Explorer but not in DESCRIBE output, where the raw syntax is returned.

Documented behaviour. Worth knowing when deciding how heavily to format a comment.

C Markdown in comments degrades query performance and should be removed.

The docs state explicitly that Markdown comments have no impact on query performance.

D DESCRIBE only supports comments under a fixed character limit.

Invented.

ON THE EXAM

Neither view is wrong. One renders the formatting and one returns the stored string, which is worth knowing before writing heavily formatted documentation.

> A governance programme needs sensitive columns tagged consistently across 1,200 tables so that masking policies apply automatically. New tables appear weekly, created by eight different teams.
>
> A proposal is to circulate a tagging standard and ask each team to tag their own columns.

What is the stronger design?

A The proposal is sound; the teams know their own data best.

B Use data classification to scan and tag automatically, with governed tags controlling who may apply them, and policies matching on the tags.

C Add AI-generated comments to every column describing its sensitivity.

D Grant BROWSE widely so the governance team can find sensitive columns manually.

◆ PAUSE AND DECIDE

What is the stronger design?

A The proposal is sound; the teams know their own data best.

True and insufficient. It depends on eight teams applying one standard consistently, forever.

B Use data classification to scan and tag automatically, with governed tags controlling who may apply them, and policies matching on the tags.

The detect, mark, protect pipeline these three features were designed as.

C Add AI-generated comments to every column describing its sensitivity.

Comments aid discovery. Policies match on tags, not on comments.

D Grant BROWSE widely so the governance team can find sensitive columns manually.

Manual discovery at 1,200 tables and growing, which is the problem rather than the solution.

ON THE EXAM

Classification, governed tags and policies were released together because they are one system. Recognising the pipeline shape answers the question.

> Before deprecating a legacy table, a team wants to know whether anyone is actually still using it, and who. Lineage shows three downstream tables, all of which were themselves deprecated last year.
>
> They need evidence of real usage, not just connections.

Where should they look?

A Lineage, which already showed them everything connected to the table.

B The Insights tab in Catalog Explorer, which shows the most frequent recent queries and users of the table.

C The certification status tag, which records who certified it.

D Table comments, where users typically record their usage.

◆ PAUSE AND DECIDE

Where should they look?

A Lineage, which already showed them everything connected to the table.

Connected is not used. The scenario says the connected objects are themselves dead.

B The Insights tab in Catalog Explorer, which shows the most frequent recent queries and users of the table.

Answers the usage question directly, including who is querying it and how often.

C The certification status tag, which records who certified it.

It records a status, not usage.

D Table comments, where users typically record their usage.

Comments are documentation. Nobody logs their queries there.

ON THE EXAM

Lineage and Insights answer different questions. What is connected, and what is actually being used. A dead pipeline is connected and unused.

DEBUGGING · DOMAIN 9 - DEBUGGING AND DEPLOYING

Diagnosing failures

Section 9. Five places to look, and knowing which one before you start.

SECTION 9 of 10 WEIGHT 10% TOPIC Debugging

admin@sthithapragna.com

Each source answers a different question. **Choosing the wrong one wastes an afternoon before you learn anything.**

Five sources, five questions

SOURCE	WHAT IT ANSWERS
Spark UI	Task level. Skew, spill, slow stages, the physical plan.
Cluster logs	Driver level. Out of memory, init script failures, dependency errors.
Cluster event log	Lifecycle. Did it start, resize, terminate, or stop responding?
System tables	Historical and cross-workspace. Run history, cluster events, audit.
Query profile	SQL warehouse. Operator timing, pruning, join strategy.
Pipeline event log	Dataset level. Flow progress, expectations, data quality.

ON THE EXAM

Learn this table as a routing decision. When something fails, the first question is which row you are in.

admin@sthithapragna.com

Where your own code's output lands

- **Print and log statements from notebooks, jobs and libraries go to the Spark driver logs.**
- So your stack traces, your Python exceptions and your own logging are all here.
- Available from the **Driver logs tab** on the compute details page. Click a file name to download it.
- By default, **only users with CAN MANAGE can view them**, which is module 24's secret redaction warning.

> ON THE EXAM
>
> If your code printed it, it is in the driver logs. That single rule resolves most 'where do I look' questions about application errors.

Where the executors' problems land

- Worker-side errors do **not** appear in the driver logs. Task failures and executor out-of-memory errors are here.
- View them through the **Spark UI tab**, or in the delivered log location if you configured one.
- **Log delivery generates a log4j file only in the driver folder, not in executor folders.** That is by design.
- Executor folders carry standard output and standard error instead.

> ON THE EXAM
>
> The missing log4j file in executor folders is not a bug. Knowing that saves you investigating a delivery problem you do not have.

What happened to the machine

- The compute event log records **lifecycle events**: creation, termination, resizing, configuration edits.
- Click a row and then the JSON tab for the full detail of an event.
- **Events are stored for 60 days**, comparable to other retention windows on the platform.
- This is where a **driver not responding** event appears, often with a message about garbage collection.

> ON THE EXAM
>
> If the question is 'did it even start', or 'why did it resize', this is the source. Nothing else records those.

The silent failure

- An init script that fails can leave a cluster that **appears to start** while every job on it fails immediately.
- Init script logs are a **separate category**, delivered to their own folder when log delivery is configured.
- Library installation and environment setup failures land here, not in the driver logs.
- Without log delivery configured, diagnosing an init script failure is considerably harder.

> ON THE EXAM
>
> A cluster that starts and then fails everything is the signature. Look at the init script logs before reading a single stack trace.

The Spark UI dies with the cluster

- **The Spark UI is unavailable once a cluster is terminated.**
- Job compute typically terminates immediately after the run, which is exactly when you want to investigate.
- So **configure log delivery** to a storage location, and the logs survive the cluster.
- Delivered structure: a folder per cluster ID, containing driver, executor, event log and init script folders.

> ON THE EXAM
>
> This is the argument for configuring log delivery in advance. After the failure, the evidence has already gone.

The compute metrics UI

- Available for **classic all-purpose and jobs compute**, on the **Metrics tab** of the compute page.
- **Metrics are collected every minute**, and historical ranges are selectable with a date picker.
- Useful for memory pressure over time, which a single stack trace cannot show you.
- This replaced Ganglia, which is not available from DBR 13 onward. Older material still references it.

> ON THE EXAM
>
> An error tells you the moment something broke. Metrics tell you whether it had been degrading for an hour beforehand.

The four documented causes

- The **driver instance type is not optimal** for the load being placed on it.
- **Memory-intensive operations executed on the driver** rather than distributed.
- **Many notebooks or jobs running in parallel** on the same cluster, all sharing one driver.
- **Heavy Python logic** such as loops, UDFs and non-Spark operations, **which runs exclusively on the driver**.

> ON THE EXAM
>
> Three of the four are about work landing on the driver that should have been distributed. That is the pattern to recognise.

What driver memory pressure looks like

- The driver **crashes and restarts**, or becomes unresponsive through frequent full garbage collection.
- A **driver not responding** event appears, often saying the driver is up but unresponsive, likely due to GC.
- A notebook reports that the Spark driver stopped unexpectedly and will be reattached.
- The classic code cause is **collecting a large result to the driver** rather than keeping it distributed.

> ON THE EXAM
>
> Notice that unresponsive and crashed are the same underlying problem. Garbage collection thrashing is memory pressure that has not finished failing yet.

Out of memory and spill are not the same

- **Spill** means data did not fit in memory and was written to disk. The job continues, more slowly.
- **Out of memory** means it could not continue at all. The task fails and may take the executor with it.
- Spill on **most** tasks is memory pressure. Spill on a **few** is skew, from module 18.
- Both are worker-side, so both are in the **executor logs** rather than the driver logs.

> ON THE EXAM
>
> Spill is a performance symptom that a query profile and the Spark UI diagnose. Out of memory is a failure, and it is in the logs.

When the evidence is already gone

- System tables persist and are queryable, where the Spark UI is ephemeral and per run.
- Job run history, cluster events and audit records support **trend analysis** across the whole estate.
- Remember the retention differences from module 17. Node utilisation keeps only 90 days.
- And the cluster event log keeps **60 days**, which is shorter than most system tables.

> ON THE EXAM
>
> For 'has this been getting worse for months', system tables are the only source. Nothing else keeps that long a memory.

An order to work in

- **Did it start?** Cluster event log, and init script logs if it started and then failed everything.
- **Did the code fail?** Driver logs for the exception and the stack trace.
- **Did a task fail?** Executor logs, then the Spark UI for the pattern across tasks.
- **Was it just slow?** Query profile or Spark UI, and metrics for what was happening over time.

> ON THE EXAM
>
> Four questions in order. Most wasted debugging time is spent answering the third question with the tools for the first.

What to carry into the exam

- **Your code's output is in the driver logs. Worker errors are in the executor logs.** That split is examinable directly.
- **The Spark UI dies with the cluster.** Configure log delivery in advance.
- An init script failure looks like a cluster that starts and then fails everything.
- Driver OOM is usually work that should have been distributed. Watch for parallel notebooks sharing one driver.
- Event log keeps **60 days**. Metrics are collected **every minute**. System tables are for trends.

> ON THE EXAM
>
> When a question names a symptom, name the source before choosing an answer. Most distractors are the right technique applied to the wrong layer.

> A nightly job on job compute failed at 3am. At 9am an engineer opens the run and finds the Spark UI is unavailable. No log delivery is configured on the cluster.
>
> They need to know which stage failed and why.

What is the situation?

A. The Spark UI is unavailable because the cluster terminated. Without log delivery, that detail is gone; configure it before the next run.

B. The Spark UI takes several hours to become available after a run; check again later.

C. The Spark UI requires CAN MANAGE, which the engineer lacks.

D. Restart the terminated cluster, which restores its Spark UI with the previous run's data.

◆ PAUSE AND DECIDE

What is the situation?

A The Spark UI is unavailable because the cluster terminated. Without log delivery, that detail is gone; configure it before the next run.

The UI does not survive termination, and job compute terminates immediately after the run.

B The Spark UI takes several hours to become available after a run; check again later.

There is no such delay. It is gone with the cluster.

C The Spark UI requires CAN MANAGE, which the engineer lacks.

That gate applies to driver logs. This is an availability problem, not permissions.

D Restart the terminated cluster, which restores its Spark UI with the previous run's data.

Restarting re-creates the compute with the same ID. It does not restore the previous run's UI.

ON THE EXAM

An unsatisfying answer, and the honest one. The remedy is preventive, which is exactly why log delivery is worth configuring before you need it.

> A cluster starts successfully and reports healthy. Every job submitted to it fails within seconds with import errors for a library the team installed via an init script.
>
> The driver logs show only the import failures.

Where should they look next?

A The executor logs, since library installation happens on the workers.

B The init script logs, since a failed init script leaves a cluster that starts but cannot run anything.

C The compute metrics UI, to check whether memory prevented installation.

D The query profile, to see which operator is failing.

◆ PAUSE AND DECIDE

Where should they look next?

A The executor logs, since library installation happens on the workers.

Would show the same downstream symptom, not the cause.

B The init script logs, since a failed init script leaves a cluster that starts but cannot run anything.

The signature failure mode. Init script output is a separate log category with its own folder.

C The compute metrics UI, to check whether memory prevented installation.

Metrics show resource usage over time, not setup failures.

D The query profile, to see which operator is failing.

Wrong layer entirely. Nothing has reached a query.

ON THE EXAM

The import error is the symptom, and the driver logs faithfully report it. The cause is one layer earlier, at cluster startup.

A shared all-purpose cluster supports eight analysts. Through the morning it becomes unresponsive, and the event log shows a driver not responding event mentioning garbage collection. Individual notebooks report the driver stopped unexpectedly.

No single notebook is processing an unusually large dataset.

What is the most likely cause?

A Data skew in one analyst's query is overloading a single executor.

B Many notebooks running in parallel on the same cluster share one driver, which is a documented cause of driver memory pressure.

C Shuffle spill has filled the driver's local disk.

D The cluster needs more worker nodes.

◆ PAUSE AND DECIDE

What is the most likely cause?

A Data skew in one analyst's query is overloading a single executor.

Skew is a worker-side problem. The symptom here is on the driver.

B Many notebooks running in parallel on the same cluster share one driver, which is a documented cause of driver memory pressure.

One of the four documented causes, and it explains why no single notebook looks unusual.

C Shuffle spill has filled the driver's local disk.

Shuffle spill occurs on executors, and it is a disk and performance issue rather than driver GC.

D The cluster needs more worker nodes.

Adding workers does not relieve a driver that is out of memory.

ON THE EXAM

The clue is that no single notebook is unusual. Each analyst's work is reasonable, and the driver is carrying all eight at once.

> A job fails with an out-of-memory error. An engineer opens the driver logs, finds nothing but ordinary progress messages, and concludes the log delivery is broken.
>
> The error message referenced a task failure during a large shuffle.

What should they be told?

A. Log delivery is broken and needs reconfiguring.

B. Worker-side out-of-memory errors appear in the executor logs, not the driver logs.

C. Out-of-memory errors are only visible in the compute metrics UI.

D. The driver logs are truncated after a fixed size and the error was dropped.

◆ PAUSE AND DECIDE

What should they be told?

A Log delivery is broken and needs reconfiguring.
Nothing is broken. They are reading the wrong log.

B Worker-side out-of-memory errors appear in the executor logs, not the driver logs.
The driver and executor log split, and a task failure during a shuffle is unambiguously worker-side.

C Out-of-memory errors are only visible in the compute metrics UI.
Metrics show memory over time. The error itself is in the logs.

D The driver logs are truncated after a fixed size and the error was dropped.
Invented, and it would not explain a worker-side error being absent from a driver log.

ON THE EXAM

The phrase task failure is the giveaway. Tasks run on executors, so their failures are recorded there.

> A team is asked whether a nightly job's runtime has been degrading over the past six months, and whether cluster memory pressure has been increasing alongside it.
>
> The clusters are job compute and terminate after each run.

Which sources can answer each half?

A The Spark UI for both, comparing runs across the period.

B System tables for the run history over six months; node utilisation retains only 90 days, so the memory half is partly unanswerable.

C The cluster event log for both, since it retains a full year.

D The compute metrics UI for both, using the date picker across six months.

◆ PAUSE AND DECIDE

Which sources can answer each half?

A The Spark UI for both, comparing runs across the period.

Ephemeral and per run, and these clusters terminate.

B System tables for the run history over six months; node utilisation retains only 90 days, so the memory half is partly unanswerable.

Run history is retained a year; node utilisation is one of the 90-day exceptions from module 17.

C The cluster event log for both, since it retains a full year.

The event log retains 60 days, and it records lifecycle events rather than memory.

D The compute metrics UI for both, using the date picker across six months.

Metrics are per compute, and these clusters no longer exist.

ON THE EXAM

Two halves, two answers, and one of them is partly no. Retention decides which questions are answerable at all.

> A pipeline's silver streaming table stopped updating overnight. The pipeline itself reports a failure, and an engineer begins by opening the Spark UI to look for a slow stage.
>
> The failure message mentions an expectation.

What should they open instead?

A The pipeline event log, which records flow progress and data quality at dataset level.

B The driver logs, since pipeline failures always surface as Python exceptions.

C The cluster event log, since the pipeline's compute must have failed.

D The query profile, since the pipeline runs SQL.

◆ PAUSE AND DECIDE

What should they open instead?

A The pipeline event log, which records flow progress and data quality at dataset level.

An expectation is a pipeline-level construct, and the event log is where its results and failures are recorded.

B The driver logs, since pipeline failures always surface as Python exceptions.

Some do. An expectation violation is reported by the pipeline, at dataset level.

C The cluster event log, since the pipeline's compute must have failed.

Nothing suggests the compute failed. The pipeline reported the failure itself.

D The query profile, since the pipeline runs SQL.

Reachable from the pipelines UI, and it answers performance rather than an expectation failure.

ON THE EXAM

The word expectation places the failure at dataset level. Opening the Spark UI answers a performance question nobody asked.

DEBUGGING · DOMAIN 9 - DEBUGGING AND DEPLOYING

Job repair and parameter overrides

Section 9. Rerunning the four tasks that failed, not the eleven that worked.

SECTION 9 of 10 WEIGHT 10% TOPIC Repair

admin@sthithapragna.com

A repair is part of the original run, not a new one. It reruns only what failed, and it picks up your fix.

What repair actually does

- Re-runs **only the failed tasks**, and any dependent tasks that need to follow them.
- Everything that already succeeded is left alone, which is the whole point on a long job.
- **The run must not be in progress.** You cannot repair one failed task while others are still running.
- Contrast with a restart, which re-runs the entire job from the beginning.

> ON THE EXAM
>
> The 'not in progress' rule surprises people managing large fan-out jobs. You wait for the run to settle, then repair.

A repair is not a replay

- Repaired tasks use the **current job and task settings**, not the settings as they were at the original run.
- So the sequence is: diagnose, **fix the job definition**, then repair. The repair runs the fixed version.
- That also means a repair days later may behave differently from the original run, if somebody edited the job.
- The repair still appears in the **history of the original run**, keeping the record together.

> ON THE EXAM
>
> This is why repair is a remediation mechanism. A pure replay of the failed configuration would just fail again.

What you can ask a repair to do

OPTION	EFFECT
Rerun named tasks	Repair specific task keys that you choose.
Rerun all failed tasks	Repair everything that failed. Cannot be combined with naming tasks.
Rerun dependent tasks	Also repair everything downstream, even if it previously succeeded.
Job parameters	Override job-level parameters for this repair.
Performance target	Override the job's serverless performance target for this repair.
Full refresh	Trigger a full refresh on a declarative pipeline task.

ON THE EXAM

Naming tasks and rerunning all failed tasks are mutually exclusive. You pick one or the other.

Why rerunning dependents matters

- By default a repair reruns the failed tasks and the dependents that were **skipped** because of them.
- The dependent option goes further, rerunning downstream tasks **even if they previously succeeded**.
- You need that when a failed task **partially wrote data** that a successful downstream task then consumed.
- Otherwise the downstream result is built on data that has now changed underneath it.

> ON THE EXAM
>
> A task succeeding does not mean its output is still correct. If an upstream input changed on repair, the downstream needs redoing.

Disabled tasks and repair

- **Repairs use each task's run state to decide what to repair, not its disabled state.**
- So a task disabled after the original run may still be selected for repair based on how it ran.
- To **force** a disabled task to run during a repair, name it explicitly in the rerun list.
- Precise enough to be a question, and unlikely to be guessed correctly.

> ON THE EXAM
>
> Run state and disabled state are two different things, and repair looks at the first one.

Sequential repairs need the previous repair ID

- The first repair of a run needs no repair identifier.
- **Every subsequent repair of the same run must supply the identifier of the latest repair.**
- That is what keeps repairs sequential rather than conflicting.
- So a script that repairs automatically has to capture the returned identifier and pass it on next time.

> ON THE EXAM
>
> An easy thing to get wrong when automating. The first call works, and the second one fails without an obvious reason.

Overriding values for one repair

- The repair dialog lists the unsuccessful tasks **and the dependent tasks that will be re-run**.
- Parameters you enter there **override the existing values**, for that repair only.
- On a later repair, **clearing the key and value returns the parameter to its original value**.
- Overrides do not change the job definition. A scheduled run tomorrow uses the original values.

> ON THE EXAM
>
> That last point is the safety property. You can rerun with a corrected date without permanently editing the job.

Duration includes every repair

- The duration shown in the runs tab spans from **the first run's start to the latest repair's finish**.
- A run that failed twice and succeeded on the third attempt reports the time for **all three**.
- So a repaired run can trip a **run duration health threshold** even though the successful attempt was quick.
- That is module 19's duration warning notification, firing for a reason nobody expected.

> ON THE EXAM
>
> Worth knowing before somebody investigates a slow job that was never slow. It failed, waited, and was repaired.

Failed and skipped look similar

- The **matrix view** in the runs tab shows successful and unsuccessful runs for every task across many runs.
- A task run may be unsuccessful because it **failed**, or because it was **skipped when a dependency failed**.
- Those mean completely different things, and only one of them points at a problem in that task.
- Hovering over a failed task shows its metadata, which is how you tell them apart quickly.

> ON THE EXAM
>
> Chasing a task that was merely skipped is a common waste of time. The failure is upstream of it.

Full refresh is available, and dangerous

- A repair can trigger a **full refresh** on a declarative pipeline task.
- That clears the streaming state and reprocesses everything, exactly as in module 5.
- **Any source beyond its retention period will not be reprocessed**, and that data is lost.
- So reach for it when the pipeline logic changed, not merely because a run failed.

> ON THE EXAM
>
> Full refresh as a repair option is convenient and it is the same operation with the same consequences. Retention arithmetic first.

Is the task safe to run again?

- A repair reruns a task that may have **partially completed** before it failed.
- If the task is **idempotent**, rerunning is safe. A merge on a key, or an overwrite, is fine.
- If it **appends** without deduplication, or calls an external system, rerunning may double the effect.
- This is module 9's at-most-once problem, arriving from a different direction.

> ON THE EXAM
>
> Repair is safe when your tasks are idempotent. Designing for that is what makes recovery routine rather than an incident.

Exceeding maximum concurrent runs

- A run can fail simply because the job's **maximum concurrent runs** was exceeded.
- The documented remedies: **wait for other runs to complete**, or **increase the limit**.
- Worth recognising because the failure has nothing to do with your code or your data.
- Repairing without addressing it will simply fail the same way.

> ON THE EXAM
>
> A failure that no amount of log reading will explain, because nothing went wrong inside the job at all.

What to carry into the exam

- A repair reruns **only failed tasks and dependents**, as part of the **original run**, using **current settings**.
- The run **must not be in progress**. Naming tasks and rerunning all failed tasks are **mutually exclusive**.
- Subsequent repairs of the same run need the **latest repair identifier**.
- Parameter overrides apply **to that repair only**. Clearing them restores the original value.
- **Duration spans every attempt**, so a repaired run can trip a duration alert.

> ON THE EXAM
>
> When a question describes a long job failing near the end, repair is almost always the intended answer. The detail decides which option is right.

> A fifteen-task job fails at task twelve because a configuration value was wrong. The engineer corrects the value in the job definition, then wants to complete the run without repeating the eleven tasks that succeeded.
>
> Those eleven tasks took four hours.

What should they do?

A. Restart the job, since the definition changed and the new value must be picked up.

B. Repair the run, which reruns only the failed and dependent tasks using the current settings.

C. Clone the job, delete the first eleven tasks, and run the clone.

D. Repair the run, but the old configuration value will be used because repairs replay the original settings.

◆ PAUSE AND DECIDE

What should they do?

A Restart the job, since the definition changed and the new value must be picked up.

Repeats four hours of successful work. Repair also picks up the new value.

B Repair the run, which reruns only the failed and dependent tasks using the current settings.

Repaired tasks use current job and task settings, so the corrected value is applied without repeating the successful tasks.

C Clone the job, delete the first eleven tasks, and run the clone.

Creates a second job to maintain, and loses the run history connection.

D Repair the run, but the old configuration value will be used because repairs replay the original settings.

The opposite of documented behaviour. Repairs use current settings.

ON THE EXAM

D is the important distractor. If repairs replayed the original configuration, they would be useless for fixing configuration errors.

> A job's task C failed after writing roughly half its output. Task D depends on C, ran anyway on a schedule quirk, and completed successfully against that partial data. Task E depends on D and was skipped.
>
> The engineer plans to repair, selecting only task C.

What is the problem?

A. Nothing. Repairing C will automatically re-run D and E.

B. D succeeded against partial data, so it will not be re-run by default. Use the option to rerun dependent tasks.

C. Task C cannot be repaired because it partially wrote output.

D. The whole job must be restarted, since repair cannot handle partial writes.

◆ PAUSE AND DECIDE

What is the problem?

A Nothing. Repairing C will automatically re-run D and E.

E was skipped so it will re-run. D succeeded, so by default it will not.

B D succeeded against partial data, so it will not be re-run by default. Use the option to rerun dependent tasks.

That option re-runs downstream tasks even where they previously succeeded, which is exactly this case.

C Task C cannot be repaired because it partially wrote output.

It can. Whether that is safe depends on whether C is idempotent.

D The whole job must be restarted, since repair cannot handle partial writes.

Repair handles this with the dependent option, without repeating everything.

ON THE EXAM

Succeeded is not the same as still correct. D's output was computed from data that is about to change underneath it.

> A team automates recovery with a script that calls the repair API. The first repair of a failed run succeeds. When the repaired run fails again and the script repairs a second time, the API rejects the request.
>
> The script sends the same request body each time, with the run ID and the failed task keys.

What is missing?

A. The latest repair identifier, which is required on every repair after the first.

B. A run cannot be repaired more than once; the script must restart the job instead.

C. The script must wait 24 hours between repairs of the same run.

D. Both rerun_tasks and rerun_all_failed_tasks must be supplied together on subsequent repairs.

◆ PAUSE AND DECIDE

What is missing?

A The latest repair identifier, which is required on every repair after the first.

Not required for the first repair, mandatory on subsequent ones, to keep repairs sequential.

B A run cannot be repaired more than once; the script must restart the job instead.

Runs can be repaired repeatedly, which is why the sequencing identifier exists.

C The script must wait 24 hours between repairs of the same run.

Invented.

D Both rerun_tasks and rerun_all_failed_tasks must be supplied together on subsequent repairs.

They are mutually exclusive; only one may be used.

ON THE EXAM

The symptom is diagnostic: works the first time, fails the second. That pattern points at something required only on repeat calls.

> A daily job normally runs in eight minutes. One morning it fails, and an engineer repairs it four hours later. The repair completes in six minutes.
>
> A duration warning notification fires, reporting the run exceeded its threshold. The team investigates for performance problems and finds none.

What explains the alert?

A. Duration spans from the first run's start to the latest repair's finish, so it includes the four idle hours.

B. The repair ran on smaller compute, making it slower than reported.

C. Duration warnings fire on any repaired run regardless of timing.

D. The threshold was misconfigured and should be raised.

◆ PAUSE AND DECIDE

What explains the alert?

A Duration spans from the first run's start to the latest repair's finish, so it includes the four idle hours.

Documented: duration runs from the first start to the latest repair's finish, covering every attempt and the gaps.

B The repair ran on smaller compute, making it slower than reported.

The repair is stated to have completed in six minutes.

C Duration warnings fire on any repaired run regardless of timing.

They fire on the threshold, which this run genuinely exceeded as measured.

D The threshold was misconfigured and should be raised.

Raising it would hide genuinely slow runs to accommodate an artefact of repair timing.

ON THE EXAM

The alert is correct by its own definition. The team's mistake is assuming duration means the time spent computing.

> A job ingests a file whose path is built from a date parameter. One night the upstream system delivered the file a day late, so the run failed on a missing path. The file is now present under yesterday's date.
>
> The team wants to complete that run without changing tomorrow's scheduled behaviour.

What should they do?

A Edit the job's default date parameter, repair the run, then edit it back afterwards.

B Repair the run, entering the corrected date as a parameter override in the repair dialog.

C Create a one-off job with the corrected date and run that instead.

D Restart the job, which prompts for parameters at launch.

◆ PAUSE AND DECIDE

What should they do?

A. Edit the job's default date parameter, repair the run, then edit it back afterwards.
Works if nobody forgets the second edit, and mutates a production job to fix one run.

B. Repair the run, entering the corrected date as a parameter override in the repair dialog.
Overrides apply to that repair only, leaving the job definition and tomorrow's schedule untouched.

C. Create a one-off job with the corrected date and run that instead.
Leaves the original run failed in the history, and creates a second job.

D. Restart the job, which prompts for parameters at launch.
Repeats every task, and this run only needs the failed one.

ON THE EXAM

The safety property is that overrides are scoped to the repair. The job definition, and therefore tomorrow's run, is untouched.

> A job task calls an external payment provider's API to issue refunds, then records them in a table. It failed partway through, after issuing some refunds but before recording them all.
>
> An engineer is about to repair the run.

What should be checked first?

A Nothing. Repair reruns only what failed, so already-issued refunds will not be reissued.

B Whether the task is idempotent. A repair reruns a task that partially completed, and refunds are not idempotent.

C Whether the run is still in progress, which would prevent repair.

D Whether the latest repair identifier is available.

◆ PAUSE AND DECIDE

What should be checked first?

A Nothing. Repair reruns only what failed, so already-issued refunds will not be reissued.

The failed task is re-run in full. Work it completed before failing is repeated.

B Whether the task is idempotent. A repair reruns a task that partially completed, and refunds are not idempotent.

Repair re-runs the whole task, so a non-idempotent task repeats side effects it already performed.

C Whether the run is still in progress, which would prevent repair.

A real precondition, and a mechanical one rather than the risk here.

D Whether the latest repair identifier is available.

Relevant for a second repair, and not the question before the first.

ON THE EXAM

Repair reruns the task, not the part of it that failed. That distinction is what makes idempotency the first question.

DEBUGGING · DOMAIN 9 - DEBUGGING AND DEPLOYING

Bundles and Git-based delivery

Section 9. Deploying resources, deploying code, and knowing which you are doing.

SECTION 9 of 10 WEIGHT 10% TOPIC CI/CD

admin@sthithapragna.com

A bundle deploys **resources**: jobs, pipelines and their configuration. A Git folder deploys **code**. Those are different problems.

Asset Bundles are now Declarative Automation Bundles

- The documentation now says **Declarative Automation Bundles, formerly known as Databricks Asset Bundles**.
- **The acronym is unchanged**, and so is the command line. You still run **databricks bundle**.
- The exam guide uses the new name with the old one in brackets, so it has absorbed this one.
- Older material, including most practice banks, uses only the old name.

> ON THE EXAM
>
> Unusually, this is a rename the exam guide did catch. Expect either name in a question and treat them as identical.

The commands you will meet

COMMAND	WHAT IT DOES
`bundle init`	Create a new bundle project from a template.
`bundle validate`	Check the configuration and show what would be deployed.
`bundle deploy`	Create or update the resources in the workspace for a target.
`bundle run`	Run a job or pipeline defined in the bundle.
`bundle destroy`	Remove the resources the bundle previously deployed.

ON THE EXAM

Every one of them takes a target. The target is what makes the same source deploy differently to dev and production.

Three behaviours you get for free

- **Prefixes every resource** that is not a file or notebook with a dev marker and your short name, and **tags jobs and pipelines** as dev.
- **Marks all deployed pipelines as development**, which is module 5's development mode.
- **Enables the compute override flag**, letting you point the whole bundle at one existing cluster.
- That flag **overrides any and all existing cluster definitions** in the configuration. It is deliberately blunt.

> ON THE EXAM
>
> The prefix is what lets a whole team deploy the same bundle into one workspace without colliding. Each developer gets their own copy.

Two validations, and one you can override

- **Validates that every deployed pipeline is marked development false.** A dev pipeline cannot reach production by accident.
- **Validates that the current Git branch matches the branch named in the target**, if you named one.
- That branch check **can be overridden with a force flag**, which is worth knowing and worth using rarely.
- Unlike development mode, **production mode does not allow overriding** its behaviours.

> ON THE EXAM
>
> The branch validation is a guardrail against deploying to production from a feature branch, which is exactly the accident it sounds like.

What changes if you do not use a service principal

- Databricks recommends service principals for production, enforced by setting the run-as identity.
- **Without one, production mode applies two extra validations.**
- That the artifact, file, root and state paths are **not overridden to a specific user**.
- And that the **run-as and permissions mappings are specified**, so it is explicit who owns and can use the deployment.

> ON THE EXAM
>
> Module 24 explained why: a job runs with its owner's permissions, so a production job owned by a person carries that person's access.

Resource beats preset beats mode

- A **mode** supplies default behaviours. **Presets** customise them. **Individual resource settings** override the presets.
- Documented example: a job with a maximum of ten concurrent runs, under a preset of twenty, keeps **ten**.
- Another: a schedule set to unpaused, under a preset that pauses schedules, stays **unpaused**.
- So the most specific setting always wins, which is the behaviour you would hope for.

> ON THE EXAM
>
> Worth knowing precisely, because a preset that appears to have no effect is usually being overridden by an explicit resource setting.

Deploy to an immutable folder

- Databricks recommends deploying a bundle to an **immutable, read-only folder**.
- That guarantees deployed assets **cannot be modified by non-admin users**.
- And that **running jobs are not affected by an in-progress deployment**.
- Without it, somebody editing a deployed notebook creates drift that the next deploy silently reverses.

> ON THE EXAM
>
> The second reason is the one people forget. A deployment landing mid-run should not change what the running job is executing.

The stated division of labour

- **Use Git folders for interactive development.** That is the documentation's own sentence.
- **For CI/CD and production deployments, use bundles**, with versioned artifacts and workload identity federation.
- If you prefer to deploy **only code** into the workspace, a production Git folder is the supported alternative.
- So Git folders in production are a narrower answer, not a wrong one.

> ON THE EXAM
>
> Note what the alternative is scoped to: deploying only code. If you also need jobs and pipelines defined as code, that is a bundle.

User-level and production Git folders

- **User-level folders** live under the users path and act as **local checkouts** where developers work and push.
- **Production folders** are created by **admins, outside user folders**, and contain production deployment branches.
- A production folder **should only be updated programmatically when a pull request is merged**.
- Creating one requires a **service principal with a configured Git credential** for the repository.

> ON THE EXAM
>
> Two objects with the same name doing opposite jobs. One is a workspace for a person; the other is a deployment target nobody edits.

Who can do what to a production Git folder

- **Limit user access to run-only.** People need to run workflows, not to edit the checkout.
- **Allow only admins and service principals to edit.**
- Optionally grant view to all workspace users, to support discovery.
- Any human able to edit a production Git folder can put code into production without a pull request.

> ON THE EXAM
>
> Module 24's least privilege, applied to a deployment target. Edit access here bypasses your entire review process.

What Git folders will not do

- **No SSH support**, and **no commit signing**.
- **Sparse checkout cannot be disabled once enabled.**
- Databricks **recommends against Git folders backed by monorepos**. Cloning one can exceed memory and disk limits.
- For a repository holding many projects, **split it, or use sparse checkout** to limit which directories are cloned.

> ON THE EXAM
>
> The monorepo warning is specific and practical. Sparse checkout is the escape hatch, and it is a one-way decision.

They are not alternatives

- Databricks' own tip: **define jobs and pipelines in source using bundles, then manage those bundles in workspace Git folders**.
- Bundles in the workspace let a team develop and deploy a bundle from a Git folder in the interface, **without the command line**.
- That opens bundle-based delivery to analysts and scientists who were never going to install a CLI.
- So the exam's two objectives describe two halves of one workflow.

> ON THE EXAM
>
> The framing to carry: Git folders for how the code gets there and gets reviewed, bundles for what gets created in the workspace.

What to carry into the exam

- **Bundles deploy resources. Git folders deploy code.** The docs say use Git folders for interactive development.
- Development mode **prefixes and tags**, marks pipelines as development, and enables the compute override.
- Production mode **validates** pipelines are not development, and that the Git branch matches. Force overrides the branch check.
- Without a service principal, production mode adds **path and permissions validations**.
- Precedence is **resource, then preset, then mode**. Production Git folders are edit-restricted to admins and service principals.

> ON THE EXAM
>
> That completes Section 9 at 10%. Section 10, data modeling, is next and is the last teaching section at 6%.

> Six engineers work on the same bundle. When two deploy to the shared development workspace at once, they expect their jobs to overwrite each other. In practice each sees only their own copies.
>
> Their target sets mode to development.

Why do the deployments not collide?

A Development mode prefixes resources with a dev marker and the deploying user's short name, so each engineer gets their own set.

B Bundles lock the workspace during deployment, so deploys are serialised.

C Each deployment creates a separate workspace automatically.

D Development mode deploys to the user's home folder, which is why they are isolated.

◆ PAUSE AND DECIDE

Why do the deployments not collide?

A Development mode prefixes resources with a dev marker and the deploying user's short name, so each engineer gets their own set.

The documented behaviour, and precisely what makes a shared development workspace usable by a team.

B Bundles lock the workspace during deployment, so deploys are serialised.

There is no such lock, and serialising would not prevent overwriting.

C Each deployment creates a separate workspace automatically.

Workspaces are not created by a deploy.

D Development mode deploys to the user's home folder, which is why they are isolated.

Paths do differ per user, and the resource naming prefix is the documented mechanism for this.

ON THE EXAM

The prefix looks cosmetic and is doing real work. It is what makes one shared development workspace serve a whole team.

> A deployment to the production target fails validation. The engineer is on a feature branch, and the target specifies the main branch. They need to deploy urgently to fix a broken pipeline.
>
> A colleague suggests removing the branch specification from the target.

What is the better response?

A. Remove the branch specification, since it is blocking an urgent fix.

B. Deploy with the force flag for this one deployment, then merge the fix to main properly.

C. Switch the target to development mode, which skips the branch validation.

D. The deployment cannot proceed until the pull request is merged; there is no override.

◆ PAUSE AND DECIDE

What is the better response?

A. Remove the branch specification, since it is blocking an urgent fix.

Removes the guardrail permanently to solve a one-off problem. Nobody puts it back.

B. Deploy with the force flag for this one deployment, then merge the fix to main properly.

The documented override, scoped to one deployment, leaving the guardrail in place.

C. Switch the target to development mode, which skips the branch validation.

Would also prefix every resource and mark pipelines as development, changing production.

D. The deployment cannot proceed until the pull request is merged; there is no override.

The branch validation is explicitly overridable with force.

ON THE EXAM

A and B reach the same place. One removes the guardrail for everyone forever; the other bypasses it once and leaves it standing.

> A team sets a preset on their production target pausing all job schedules, so that deployments never start work unexpectedly. After deploying, they find one job's schedule is still running.
>
> That job's definition explicitly sets its schedule to unpaused.

What is happening?

A Presets override individual resource settings, so the preset should have won; this is a bug.

B Individual resource settings override presets, which override mode defaults. The explicit unpaused setting wins.

C Schedule pausing is only supported in development mode.

D The preset applies only to jobs created after it was added.

◆ PAUSE AND DECIDE

What is happening?

A Presets override individual resource settings, so the preset should have won; this is a bug.

The precedence runs the other way. Resource settings are the most specific and win.

B Individual resource settings override presets, which override mode defaults. The explicit unpaused setting wins.

Documented precedence, with this exact example given in the docs.

C Schedule pausing is only supported in development mode.

Presets apply to any target.

D The preset applies only to jobs created after it was added.

Presets apply at deployment, not at creation.

ON THE EXAM

This exact case is one of the documentation's own examples. The most specific setting always wins, which is usually what you want and occasionally not.

> A production bundle is deployed under a named engineer's identity rather than a service principal. Deployment succeeds, but production mode reports additional validation checks the team did not expect.
>
> They ask why their colleague's identical bundle in another workspace does not show them.

What is the explanation?

A. Production mode applies extra path and permissions validations when you do not deploy as a service principal.

B. The other workspace is on an older CLI version.

C. Validation behaviour depends on the workspace's pricing tier.

D. The extra checks appear only on first deployment of a bundle.

◆ PAUSE AND DECIDE

What is the explanation?

A Production mode applies extra path and permissions validations when you do not deploy as a service principal.

Documented: without a service principal, production mode additionally validates the paths and the run-as and permissions mappings.

B The other workspace is on an older CLI version.

Possible in general, and the identity difference is the documented explanation.

C Validation behaviour depends on the workspace's pricing tier.

Deployment mode behaviour is not tier-dependent.

D The extra checks appear only on first deployment of a bundle.

They depend on the deploying identity, not on how many times you have deployed.

ON THE EXAM

The extra validations are a nudge. Databricks would rather you used a service principal, and checks more carefully when you do not.

> A team clones their organisation's monorepo into a Git folder so that all projects are available in one place. Cloning is slow, some operations time out, and the folder occasionally fails to sync.
>
> The repository holds around forty projects.

What should they do?

A. Increase the cluster size, since Git operations run on the attached compute.

B. Databricks recommends against monorepo-backed Git folders. Split the repository, or use sparse checkout to clone only what is needed.

C. Switch to SSH, which handles large repositories better.

D. Disable sparse checkout, which is causing the sync failures.

◆ PAUSE AND DECIDE

What should they do?

A Increase the cluster size, since Git operations run on the attached compute.

The limits are on the Git folder, not on your cluster.

B Databricks recommends against monorepo-backed Git folders. Split the repository, or use sparse checkout to clone only what is needed.

Stated directly: cloning a monorepo can exceed Git folder memory and disk limits and slow operations.

C Switch to SSH, which handles large repositories better.

Git folders do not support SSH at all.

D Disable sparse checkout, which is causing the sync failures.

Sparse checkout is the remedy here, and it cannot be disabled once enabled anyway.

ON THE EXAM

D inverts the answer and also names something that cannot be done. Sparse checkout is a one-way decision.

> A team version-controls all their notebooks in a production Git folder, updated automatically when pull requests merge. Job definitions, schedules and permissions are still configured by hand in the workspace UI.
>
> After an incident, they cannot reconstruct what a job's configuration was last month.

What is missing?

A. Nothing structural; they should enable audit logging to reconstruct configuration history.

B. Git folders deploy code, not resources. Defining jobs and pipelines in a bundle puts their configuration under version control too.

C. They should move the production Git folder to a user folder so history is retained.

D. Job configuration cannot be version controlled on Databricks.

◆ PAUSE AND DECIDE

What is missing?

A Nothing structural; they should enable audit logging to reconstruct configuration history.

Audit records who changed something. It is not a substitute for the configuration being in source control.

B Git folders deploy code, not resources. Defining jobs and pipelines in a bundle puts their configuration under version control too.

The distinction this module opened with, arriving as a real consequence.

C They should move the production Git folder to a user folder so history is retained.

User-level folders are development checkouts, and the location is not the problem.

D Job configuration cannot be version controlled on Databricks.

That is exactly what bundles are for.

ON THE EXAM

Half the deployment is versioned and half is not. The half that is not is where the incident came from.

DATA MODELING · DOMAIN 10 - DATA MODELING

Designing Delta tables that scale

Section 10. Where the metadata lives, how you copy a table, and one question the guide publishes.

SECTION 10 of 10 WEIGHT 6% TOPIC Data modeling

admin@sthithapragna.com

A Delta table is **files plus a log**, and a **name in a metastore**. Knowing which layer an operation touches answers a surprising number of questions.

What renaming a table actually does

- The guide's first sample question renames a table with a typo in its name, and asks what happens.
- **The published answer: the table reference in the metastore is updated.**
- Not the transaction log. Not a new log. No files are moved, dropped or recreated.
- A rename is a **metastore operation**. The data and its log are untouched.

> ON THE EXAM
>
> The distractors are all about the transaction log, and every one is wrong. Renaming is the layer above.

The partition column question

- A posts table with user id, post text, post id, longitude, latitude, **post time as a timestamp**, and **date as a date**.
- The question: which column should be used for partitioning? **The published answer is date.**
- Not **post id**, which is unique, so every row becomes its own partition.
- Not **user id**, which is high cardinality. Not **post time**, a timestamp granular to the second.

> ON THE EXAM
>
> This is the objective's own published example. Partition-column reasoning is still tested, even though liquid clustering now supersedes partitioning.

What makes a column a good partition key

PROPERTY	WHY IT MATTERS
Low cardinality	Few distinct values means few, large partitions. Unique columns are the worst possible choice.
Queries filter on it	A partition only helps if queries actually restrict on that column.
Roughly even distribution	Wildly uneven partitions reproduce the skew problem from module 13.
Partitions of useful size	Databricks guidance: at least about 1 GB per partition.

ON THE EXAM

A date column usually satisfies all four. A timestamp satisfies the second and fails the first three.

But partitioning is not the modern answer

- Module 22 established that **liquid clustering replaces partitioning and ZORDER**, and is recommended for all new tables.
- Yet the guide's own sample question asks you to choose a partition column, and publishes an answer.
- So learn **both**. Answer the partitioning question if asked, and know clustering is the current recommendation.
- Note the sample questions are described as **retired from a previous version** of the exam.

> ON THE EXAM
>
> An honest tension rather than a contradiction. The reasoning behind a good partition key is still the reasoning behind a good clustering key.

Deep and shallow clones

- **Deep clone copies the data**, producing a complete independent table. **It is the default.**
- **Shallow clone copies the definition and references the source's files** without copying them.
- So a shallow clone is nearly instant and nearly free, whatever the size of the source.
- And it is **not independent** of the source's files, which matters for retention and lifecycle.

> ON THE EXAM
>
> Deep being the default is worth remembering. If a question shows CLONE with neither keyword, it is a deep clone.

What can be cloned, and how

SOURCE	SUPPORT
Delta	Deep and shallow.
Parquet	Deep and shallow.
Foreign Iceberg	Deep and shallow.
Managed Iceberg	Deep only, and the format cannot change during cloning.
Streaming tables and materialized views	Not supported as source or target.

ON THE EXAM

Shallow clone on Unity Catalog managed tables needs DBR 13.3 LTS or above. Below that there is no support for it in Unity Catalog at all.

When each clone is right

- **Shallow** for short-lived experiments, testing a migration, or trying a change against production-shaped data.
- **Deep** for anything that must outlive the source, or survive independently of it.
- A shallow clone whose source is vacuumed aggressively is a shallow clone with a lifespan.
- Deep clone is also how you take a genuine backup or move a table between environments.

> ON THE EXAM
>
> The question to ask is how long the copy must live, and whether it must survive the source being cleaned up.

Cloning a table as it used to be

- The clone source may include a **temporal specification**, so you can clone a table **as at a past version**.
- That gives you a named, queryable table holding a historical state, rather than a query with a version clause.
- Useful for reproducing an old report, or investigating what changed between two versions.
- Bounded by retention. You can only clone as far back as the files still exist.

> ON THE EXAM
>
> Clone plus time travel is more useful than either alone. A version becomes an object you can grant on and query normally.

How far back you can actually go

- Query a past version by **version number** or by **timestamp**.
- Bounded by two things: **history retention**, defaulting to 30 days, and **vacuum**, defaulting to 7.
- Once vacuum has removed the files, the version cannot be read however recent the log entry is.
- For longer reproducibility, set the retention property **before** enabling automatic maintenance.

> ON THE EXAM
>
> Module 26's two numbers again. History retention governs the log; vacuum retention governs the files, and files win.

MERGE, and what it does not know

- **MERGE** performs an upsert: update where a key matches, insert where it does not, optionally delete.
- It is the standard way to apply changes to a dimension, and it is **atomic**.
- But it applies changes **in the order they arrive**, which is module 8's whole argument.
- For an out-of-order change feed, **AUTO CDC** exists precisely because MERGE cannot sequence.

> ON THE EXAM
>
> MERGE is right when you control the ordering. It is wrong when a source can deliver a correction before the thing it corrects.

Constraints that the table enforces

- **NOT NULL** and **CHECK** constraints are enforced by the table itself. A violating write fails.
- That is stronger than an expectation, which lives in a pipeline and only governs data arriving through it.
- And blunter. A failed write is a failed job, with no warn or drop option.
- Expectations from module 14 offer three responses. A constraint offers one.

> ON THE EXAM
>
> Two mechanisms with different reach. A constraint protects the table from every writer; an expectation protects one pipeline's output.

Designing for a schema that will move

- Delta supports schema evolution, so a table's shape can change without rebuilding it.
- Module 21's caution applies: **a renamed column is treated as a new column added**, with the old one nulled for new rows.
- A dropped column is a **soft delete**, null for new rows rather than removed.
- So design for addition, which is cheap, rather than for renaming, which is not.

> ON THE EXAM
>
> Scalable table design is largely about which changes are cheap. Adding columns is; renaming and reshaping are not.

What to carry into the exam

- **Renaming a table updates the metastore**, not the transaction log. Published answer in the guide.
- **Partition on low cardinality columns that queries filter on.** The guide's answer is the date column, not the timestamp.
- **DEEP CLONE is the default.** Shallow references the source's files. Streaming tables and MVs cannot be cloned.
- Clone accepts a version, so you can clone a table as it used to be.
- **MERGE applies arrival order.** Constraints are enforced by the table; expectations by the pipeline.

> ON THE EXAM
>
> Two of these come from the guide's own published sample questions, which makes them the highest-confidence facts in the course.

> A table of user posts has columns for user id, post text, post id, longitude, latitude, a post time timestamp, and a date column.
>
> The team must choose a partitioning column. The table receives several million posts per day and analysts almost always filter by day.

Which column should be used?

A post_id

B post_time

C date

D user_id

◆ PAUSE AND DECIDE

Which column should be used?

A post_id

Unique per row, so partitioning on it creates one partition per post. The worst possible choice.

B post_time

A timestamp granular to the second is effectively unique at several million posts per day.

C date

Low cardinality, evenly distributed, and exactly what analysts filter on. The guide's published answer.

D user_id

High cardinality, and unevenly distributed since some users post far more than others.

ON THE EXAM

This is the guide's own sample question and published answer. Both wrong date-like options fail on cardinality, which is the first test.

> A table was created with a typo in its name. An engineer runs an ALTER TABLE statement to rename it, then worries that the rename may have rewritten the underlying data or invalidated existing time travel history.
>
> The table holds 40 TB.

What actually happened?

A All related files and metadata were dropped and recreated in a single transaction.

B The name change was recorded in the Delta transaction log.

C A new transaction log was created for the renamed table.

D The table reference in the metastore was updated.

◆ PAUSE AND DECIDE

What actually happened?

A All related files and metadata were dropped and recreated in a single transaction.

Nothing is rewritten. A 40 TB rename returns immediately, which is the practical tell.

B The name change was recorded in the Delta transaction log.

The log records changes to the table's data and metadata, not the name it is known by.

C A new transaction log was created for the renamed table.

The existing log is untouched, which is why time travel history survives.

D **The table reference in the metastore was updated.**

A rename is a metastore operation. The guide's published answer.

ON THE EXAM

The engineer's worry answers itself: a 40 TB rename that returns instantly cannot have moved any data.

> A team needs a copy of a 12 TB production table to test a destructive schema migration. The copy is needed for two days and will then be discarded. Production must not be affected.
>
> They plan to use a create-table-as-select.

What is the better approach?

A. A shallow clone, which copies the definition and references the source files, and is nearly instant.

B. A deep clone, since the migration is destructive and must not touch production files.

C. The create-table-as-select is correct; cloning would link the copy to production.

D. Time travel, querying production at a past version instead of copying.

◆ PAUSE AND DECIDE

What is the better approach?

A A shallow clone, which copies the definition and references the source files, and is nearly instant.

Two days, then discarded. Writes to the clone create new files and never modify the source's.

B A deep clone, since the migration is destructive and must not touch production files.

Also safe, and copies 12 TB for a copy that lives two days.

C The create-table-as-select is correct; cloning would link the copy to production.

Copies 12 TB, takes hours, and loses constraints and properties that a clone preserves.

D Time travel, querying production at a past version instead of copying.

Time travel is read-only. You cannot test a destructive migration against it.

ON THE EXAM

B is safe and expensive. The deciding facts are the two-day lifespan and that writing to a shallow clone never alters the source's files.

> An engineer wants to snapshot a materialized view before a pipeline change, so they can compare results afterwards. They attempt a deep clone of the materialized view and it fails.
>
> They ask whether the syntax is wrong.

What should they be told?

A Deep clone requires the LOCATION clause when the source is a materialized view.

B Streaming tables and materialized views are not supported as a source or target for CLONE.

C Materialized views can only be shallow cloned.

D The clone must specify a version, since materialized views have no current version.

◆ PAUSE AND DECIDE

What should they be told?

A Deep clone requires the LOCATION clause when the source is a materialized view.

LOCATION is optional and creates an external table. It is not the blocker.

B Streaming tables and materialized views are not supported as a source or target for CLONE.

A stated limitation. The syntax is fine; the source type is not supported.

C Materialized views can only be shallow cloned.

Neither kind is supported for them.

D The clone must specify a version, since materialized views have no current version.

Invented. Materialized views do have versions.

ON THE EXAM

The realistic alternative is a create-table-as-select from the view, which produces an ordinary table they can compare against later.

> A dimension table is maintained with a MERGE statement matching on customer id, run against a change feed from an operational system. The feed can deliver events out of order after network interruptions.
>
> Auditors find dimension rows holding older values than ones recorded earlier.

What is the cause and the remedy?

A. MERGE applies changes in arrival order. Use AUTO CDC, which sequences by a column you nominate.

B. The MERGE is matching on the wrong key; add the event timestamp to the match condition.

C. MERGE is not atomic, so concurrent writes are interleaving.

D. Enable change data feed on the dimension table so ordering is preserved.

◆ PAUSE AND DECIDE

What is the cause and the remedy?

A MERGE applies changes in arrival order. Use AUTO CDC, which sequences by a column you nominate.

Module 8's argument exactly: MERGE has no concept of when a change happened, only when it arrived.

B The MERGE is matching on the wrong key; add the event timestamp to the match condition.

Would make every event a distinct row rather than a version of one.

C MERGE is not atomic, so concurrent writes are interleaving.

MERGE is atomic. Ordering is the problem, not atomicity.

D Enable change data feed on the dimension table so ordering is preserved.

CDF records changes to a table. It does not order incoming events.

ON THE EXAM

An older value overwriting a newer one is the signature. The fix is sequencing, which MERGE has no way to express.

> A finance team must be able to reproduce a report exactly as it was produced on any month end for the past year. They plan to rely on time travel against the source tables.
>
> The tables are Unity Catalog managed with predictive optimization enabled and default retention.

What is the problem, and what would work better?

A. Nothing; time travel retains a year of history by default.

B. Vacuum removes the files after the default window, so year-old versions are unreadable. Deep clone each month end into a named table.

C. Shallow clone each month end, which is cheaper and equally durable.

D. Enable change data feed, which preserves every version indefinitely.

◆ PAUSE AND DECIDE

What is the problem, and what would work better?

A Nothing; time travel retains a year of history by default.

History defaults to 30 days and vacuum removes files after 7. Neither is a year.

B Vacuum removes the files after the default window, so year-old versions are unreadable. Deep clone each month end into a named table.

A deep clone is independent of the source's files, so it survives vacuum. Twelve durable snapshots.

C Shallow clone each month end, which is cheaper and equally durable.

A shallow clone references the source's files, which are exactly what vacuum removes.

D Enable change data feed, which preserves every version indefinitely.

CDF is explicitly not a permanent record, and vacuum deletes its files too.

ON THE EXAM

C is the trap. Cheaper, and it depends on precisely the files the source is entitled to delete.

DATA MODELING · DOMAIN 10 - DATA MODELING

Dimensional models on the lakehouse

Section 10. The last teaching module. Facts, dimensions, keys, and which constraints are real.

SECTION 10 of 10 WEIGHT 6% TOPIC Dimensional models

admin@sthithapragna.com

The lakehouse did not abandon structure.
It made structure cheaper to change.

Where a dimensional model lives

- **Bronze** holds raw ingested data, close to the source and append-only.
- **Silver** holds cleansed and conformed data, with quality rules applied and entities resolved.
- **Gold** holds business-level data shaped for consumption, and **this is where dimensional models live**.
- So the medallion architecture and dimensional modelling are not alternatives. One describes the journey, the other the destination.

> ON THE EXAM
>
> A common confusion. Medallion is a layering strategy; star schema is a modelling technique. Your gold layer is usually a star schema.

Facts and dimensions

	CHARACTER
Fact table	Measurements and events. Long and narrow, growing constantly. Mostly keys and numbers.
Dimension table	Descriptive context. Wide and comparatively short, changing slowly.
Fact grain	What one row means. The single most important design decision.
Dimension design	Denormalised deliberately, to avoid joins at query time.

ON THE EXAM

Grain first. Deciding what one row of the fact table represents settles almost every other question about the model.

Why dimensions repeat themselves

- Databricks states it plainly: **normalisation maximises write integrity and storage efficiency**.
- **Denormalisation maximises read speed.** Star schemas duplicate descriptive attributes to optimise scan performance.
- On a lakehouse, storage is cheap and analyst time is not, so the trade usually favours reading.
- The cost is that an attribute changing means updating it in more than one place.

> ON THE EXAM
>
> A deliberate trade rather than sloppiness. A dimension repeating a region name a million times is doing that on purpose.

Surrogate and business keys do different jobs

- A **business key** comes from the source system. A **surrogate key** is system-generated and meaningless.
- Surrogates protect you when **business keys collide across source systems**, which they eventually do.
- In a Type 2 dimension: **the business key tells you who, and the surrogate key tells you which version of them**.
- Facts join on the surrogate, so a fact row is tied to the version of the dimension that was current at the time.

> ON THE EXAM
>
> That third line is the clearest statement of why Type 2 dimensions need surrogate keys at all.

Identity columns, and the two forms

- An identity column generates a sequential value automatically. It is the standard surrogate key mechanism.
- **Generated always** prohibits inserting your own values. **Generated by default** permits them.
- Always is safer for a true surrogate. By default is useful when migrating existing keys in.
- **Gaps in the sequence are normal and do not matter**, because the value is meaningless by design.

> ON THE EXAM
>
> People worry about gaps. A surrogate key has no meaning, so a gap has no meaning either.

Identity columns and AUTO CDC do not combine

- **Identity columns are not supported on AUTO CDC targets**, from module 8.
- They may also be **recomputed during materialized view updates**, so Databricks recommends them only with streaming tables.
- So the standard surrogate key mechanism collides with the standard SCD mechanism.
- The documented alternative: a **deterministic hash of the business key** and the attributes that define a version.

> ON THE EXAM
>
> Worth knowing because both halves are recommended practice, and they do not work together. A hash sidesteps it entirely.

Primary and foreign keys are informational

- Supported since **DBR 11.3 LTS**, generally available as of **15.2**.
- **They are informational and not enforced at write time.** A violating row is accepted.
- They still earn their place: they **document relationships**, the **optimizer can use them**, and BI tools discover joins from them.
- Databricks says **enforceable referential integrity is on the roadmap**.

> ON THE EXAM
>
> Not enforced does not mean not useful. It means the guarantee has to come from your pipeline rather than from the table.

Two kinds of constraint on the same table

- **Primary key and foreign key: informational.** Not enforced. Documentation and optimisation.
- **NOT NULL and CHECK: enforced.** A violating write fails, from module 32.
- So a Delta table carries both kinds at once, and knowing which is which is directly examinable.
- If referential integrity actually matters, it must be guaranteed by your pipeline, not by the constraint.

> ON THE EXAM
>
> The single most likely confusion in this module. Enforcement is per constraint type, not a table-wide property.

The dimension types, in one place

- **Type 1** overwrites. The dimension holds current truth and no history.
- **Type 2** adds a row per version, with start and end columns, and a null end meaning current.
- Module 8's AUTO CDC implements both, sequenced by a column you nominate rather than by arrival order.
- Choosing is a business question: does anybody need to know what was true on a past date?

> ON THE EXAM
>
> The technique is settled. The decision is not, and it belongs to whoever owns the reporting requirement.

Missing members and degenerate dimensions

- A **missing member** is a dimension row representing unknown or not applicable, so facts always have something to join to.
- That keeps joins inner rather than outer, and stops unmatched facts vanishing from a report.
- A **degenerate dimension** is an attribute kept in the fact table rather than given its own dimension.
- Typically an identifier like an order number, where a separate table would hold nothing but the key.

> ON THE EXAM
>
> Both are named patterns in Databricks' own material, which makes their vocabulary fair game in a question.

How the model meets the storage

- Fact tables are large, frequently filtered and joined, which is exactly what **liquid clustering** suits.
- Cluster on the columns queries filter and join on, most often a date and one or two dimension keys.
- Remember the limit: **up to four clustering keys**, and clustering on unrelated columns dilutes the benefit.
- Dimension tables are usually small enough that layout barely matters.

> ON THE EXAM
>
> Module 22's material applied to a model. The fact table is where layout decisions pay, and the dimensions rarely need them.

Why this is not a legacy topic

- **Metric views depend entirely on well-structured relational models** to work, and they are a current feature.
- Semantic layers need proper facts and dimensions to deliver consistent metrics across an organisation.
- Databricks also notes that AI and machine learning perform best on clean, structured, tabular data.
- Use the **three-level namespace** to organise the model, which is module 27's catalog, schema and table.

> ON THE EXAM
>
> The argument for dimensional modelling on Databricks is a forward-looking one, not nostalgia for the warehouse.

What to carry into the exam

- **Gold is where dimensional models live.** Medallion is layering; star schema is modelling. Not alternatives.
- **PK and FK are informational and not enforced. NOT NULL and CHECK are enforced.**
- Identity columns generate surrogates. **Generated always prohibits your own values; by default permits them.**
- **Identity columns are not supported on AUTO CDC targets.** Hash the business key instead.
- Business key says who; surrogate key says **which version**. Facts join on the surrogate.

> ON THE EXAM
>
> That completes the teaching content: ten sections, thirty-two modules. Two cross-cutting modules and three practice exams remain.

> A team defines a foreign key constraint from a fact table to a dimension table. They then remove the referential integrity checks from their pipeline, reasoning that the constraint now enforces it.
>
> Three months later, reports show a growing number of transactions attributed to an unknown customer.

What went wrong?

A The constraint was defined with the wrong syntax and was never applied.

B Primary and foreign key constraints are informational and not enforced at write time. The pipeline still has to guarantee integrity.

C Foreign keys are only enforced on managed tables.

D The constraint requires the dimension to have a NOT NULL primary key to take effect.

◆ PAUSE AND DECIDE

What went wrong?

A The constraint was defined with the wrong syntax and was never applied.

An invalid constraint would have errored at definition. This one is working as designed.

B Primary and foreign key constraints are informational and not enforced at write time. The pipeline still has to guarantee integrity.

Not enforced means violating rows are accepted silently, which is exactly the symptom.

C Foreign keys are only enforced on managed tables.

They are not enforced on any table type.

D The constraint requires the dimension to have a NOT NULL primary key to take effect.

No configuration makes a foreign key enforced.

ON THE EXAM

The constraint did its job, which was documentation and optimisation. It was never going to reject a row.

> A team builds a Type 2 customer dimension using AUTO CDC, and defines the surrogate key as an identity column. The pipeline fails to deploy.
>
> They need a stable surrogate key that identifies each version of a customer.

What is the problem and the alternative?

A. Identity columns are not supported on AUTO CDC targets. Generate the surrogate as a deterministic hash of the business key and versioning attributes.

B. Type 2 dimensions cannot have surrogate keys; use the business key instead.

C. The identity column must be declared as generated by default rather than generated always.

D. AUTO CDC does not support Type 2; use MERGE.

◆ PAUSE AND DECIDE

What is the problem and the alternative?

A Identity columns are not supported on AUTO CDC targets. Generate the surrogate as a deterministic hash of the business key and versioning attributes.

Two recommended practices that do not combine, with a documented alternative.

B Type 2 dimensions cannot have surrogate keys; use the business key instead.

The business key cannot identify a version. That is what the surrogate is for.

C The identity column must be declared as generated by default rather than generated always.

Neither form is supported on an AUTO CDC target.

D AUTO CDC does not support Type 2; use MERGE.

AUTO CDC supports Type 2 directly, and MERGE cannot sequence out-of-order changes.

ON THE EXAM

Two things Databricks recommends separately that do not work together. Recognising the collision is the skill.

> A report of sales by region is missing about 3% of total revenue compared to the raw fact table. Investigation shows those fact rows have a region key that does not appear in the region dimension, because the source system occasionally sends a blank region.
>
> The report uses an inner join to the dimension.

What is the standard modelling fix?

A. Change the report to a left outer join and handle the nulls in the reporting tool.

B. Add a missing member row to the dimension, and map unmatched facts to it during load.

C. Add a foreign key constraint so the bad rows are rejected at write time.

D. Filter the blank regions out of the fact table during ingestion.

◆ PAUSE AND DECIDE

What is the standard modelling fix?

A. Change the report to a left outer join and handle the nulls in the reporting tool.
Pushes the problem into every report that ever joins this dimension.

B. Add a missing member row to the dimension, and map unmatched facts to it during load.
The named pattern. Every fact has something to join to, revenue reconciles, and the gap is visible as its own category.

C. Add a foreign key constraint so the bad rows are rejected at write time.
Foreign keys are informational and reject nothing.

D. Filter the blank regions out of the fact table during ingestion.
Makes the numbers reconcile by deleting revenue, which is worse than the symptom.

ON THE EXAM

The missing member turns silently vanishing rows into a visible category called Unknown, which is what you want a report to show.

> A team is migrating a customer dimension from an existing warehouse. They need to preserve the surrogate keys already used by historic fact rows, then generate new keys for customers added afterwards.
>
> They plan to use an identity column.

Which form should they use?

A Generated always, which guarantees the keys are unique.

B Generated by default, which permits inserting the existing keys during migration and generates new ones afterwards.

C Neither; identity columns cannot coexist with pre-existing key values.

D Generated always, then update the keys afterwards to match the originals.

◆ PAUSE AND DECIDE

Which form should they use?

A Generated always, which guarantees the keys are unique.

It prohibits inserting your own values, which is exactly what the migration requires.

B Generated by default, which permits inserting the existing keys during migration and generates new ones afterwards.

The difference between the two forms is precisely whether your own values may be supplied.

C Neither; identity columns cannot coexist with pre-existing key values.

That is what the by-default form exists for.

D Generated always, then update the keys afterwards to match the originals.

Rewrites every key after loading, and breaks the fact joins in between.

ON THE EXAM

The two forms exist for exactly this distinction. Always for a clean new dimension; by default when you are bringing keys with you.

> A gold fact table holding four billion rows is queried mostly by date range and by store. Query times are poor. The table is unpartitioned and unclustered, and is a Unity Catalog managed table.
>
> A team member proposes clustering on eight columns to cover every query pattern the analysts use.

What is the better approach?

A Cluster on all eight columns, since more coverage means more queries benefit.

B Cluster on the columns actually filtered and joined on, within the four-key limit, or use automatic clustering.

C Partition by date and Z-order by store, which covers both access patterns.

D Leave it unclustered; predictive optimization will handle layout automatically.

◆ PAUSE AND DECIDE

What is the better approach?

A Cluster on all eight columns, since more coverage means more queries benefit.

Exceeds the four-key limit, and clustering on unrelated columns dilutes the benefit for all of them.

B Cluster on the columns actually filtered and joined on, within the four-key limit, or use automatic clustering.

Date and store are named in the scenario, and automatic clustering is available on a managed table with predictive optimization.

C Partition by date and Z-order by store, which covers both access patterns.

Clustering replaces both, and Z-ordered files are ignored by predictive optimization.

D Leave it unclustered; predictive optimization will handle layout automatically.

It maintains tables and compacts files. Choosing clustering keys needs CLUSTER BY AUTO.

ON THE EXAM

The scenario names the access pattern, which is the answer. Everything else is the team member guessing at coverage.

> A team argues their gold layer should be a single wide denormalised table rather than a star schema. Their reasoning is that joins are expensive and storage is cheap.
>
> The business needs consistent metric definitions across several reporting tools, and wants to use metric views.

What is the strongest counter-argument?

A. A single wide table is always slower to query than a star schema.

B. Metric views and semantic layers depend on well-structured relational models with proper facts and dimensions.

C. Denormalisation is not supported on Delta tables.

D. Wide tables cannot exceed the column limit for Unity Catalog tables.

◆ PAUSE AND DECIDE

What is the strongest counter-argument?

A. A single wide table is always slower to query than a star schema.
Overstated. A wide table can be very fast for the queries it was shaped for.

B. Metric views and semantic layers depend on well-structured relational models with proper facts and dimensions.
Databricks states this directly, and the scenario names metric views as a requirement.

C. Denormalisation is not supported on Delta tables.
Star schemas denormalise dimensions deliberately. It is a technique, not a restriction.

D. Wide tables cannot exceed the column limit for Unity Catalog tables.
A limit exists in principle and is not the argument here.

ON THE EXAM

The team's reasoning about joins and storage is sound. It just does not address the requirement the business actually stated.

CONFUSION PAIRS · EXAM STRATEGY - ALL DOMAINS

Confusion pairs

Cross-cutting. Eight scenarios built from the places two things look alike and are not.

TYPE Cross-cutting SCENARIOS 8 TOPIC Discrimination

admin@sthithapragna.com

A good distractor is not wrong nonsense. It is a true statement about the wrong thing.

admin@sthithapragna.com

Where the default is not what people assume

THING	THE ACTUAL DEFAULT
Expectations	warn. The invalid record is written to the target and counted.
Auto Loader schema evolution	addNewColumns when no schema is supplied, none when one is.
CLONE	Deep. Shallow must be asked for explicitly.
Window frame	RANGE, not ROWS. Peers with the same ordering value group together.
Job retries	None, except on serverless and in continuous mode.
Delta streaming source	Append-only. An update in the source fails the stream.

ON THE EXAM

Every row here is a default somebody assumes is the opposite. Defaults are where the exam finds free marks.

What actually stops something happening

MECHANISM	ENFORCED?
NOT NULL and CHECK constraints	Yes. A violating write fails.
Primary key and foreign key	No. Informational, for documentation and optimisation.
Expectations	Depends on the action you chose. Warn does not stop anything.
Row filters and column masks	They restrict. They never grant.
A BROADCAST hint	Honoured even above the size threshold. That is how it kills a driver.
Runtimes below 12.2 with masks	Fail securely. No data at all is returned.

ON THE EXAM

The first two rows sit on the same table and behave oppositely. That is the single most likely confusion in Section 10.

Right technique, wrong layer

IF THE QUESTION IS ABOUT	LOOK AT
Your code's exception or print output	Driver logs, not executor logs.
A task failing during a shuffle	Executor logs, not driver logs.
Whether the cluster started at all	The cluster event log, not either of those.
An expectation failing	The pipeline event log, not the Spark UI.
Who can query a table	Unity Catalog grants, not workspace ACLs.
Who can start a cluster	Workspace ACLs, not Unity Catalog grants.

ON THE EXAM

Most wasted debugging time, and most wrong answers, come from applying a good technique at the wrong layer.

One flag, two opposite verdicts

- In module 21, **skip change commits was the trap**. Corrections needed to propagate, and the flag discarded them.
- In module 26, **the same flag is the documented answer**, for propagating a deletion.
- The difference: in module 26 the deletion is applied **explicitly to both the source and the streaming table first**.
- So the flag is right when you have already handled the change yourself, and wrong when you have not.

> ON THE EXAM
>
> The same setting, the same stream, opposite verdicts. No amount of memorising 'skipChangeCommits is bad' survives this.

Micro-batches do not overlap

- A published sample question describes a stream whose micro-batches sometimes exceed the trigger interval.
- The **correct** answer: shorten the interval so records do not back up and cause large batches that spill.
- The **distractor**: shorten it so idle executors start the next batch while earlier tasks finish.
- That is false. **A new micro-batch does not begin while the previous one is running.**

> ON THE EXAM
>
> Two options recommending the same action for different reasons. Only the reasoning separates them, which is a very professional-exam thing to do.

There is no cross-task transaction

- Another published question: task A succeeds, B succeeds, C fails. What is the state?
- The published answer: A and B completed, and **some operations in C may have completed successfully**.
- **Nothing is rolled back across tasks.** A job is not a transaction.
- Which is exactly why module 30 insisted on asking whether a task is idempotent before repairing it.

> ON THE EXAM
>
> Individual Delta operations are atomic. A multi-task job is not, and two of that question's distractors offer group rollback.

Controlling file size without moving data

- A published question: write 1 TB of JSON to Parquet with 512 MB target files, **without rearranging the data**.
- The answer: set the **maximum partition bytes** setting to 512 MB, which controls partition size **on read**.
- Every distractor rearranges. **Repartition** shuffles. **Coalesce** moves data. **Sorting** repartitions.
- Output file size follows partition count, and partition count is decided when you read.

> ON THE EXAM
>
> The phrase 'without rearranging the data' is the whole question. It rules out every option that causes a shuffle.

Sometimes the objective names the wrong tool

- A published question's stated objective is about **Delta Lake clone**.
- The scenario: share a table with another team, renaming some fields and hiding others.
- **The published answer is a view**, with aliased column names and only approved fields selected.
- The question asks for the solution emphasising **simplicity**, and renaming and subsetting columns is a view's job.

> ON THE EXAM
>
> Read the requirement, not the objective heading. The objective tells you the topic area; the scenario tells you the answer.

How to use this under time pressure

- **Find the requirement sentence.** It is usually one clause, and it eliminates two options immediately.
- Ask **which layer** the question is about before choosing: data or object, driver or executor, table or pipeline.
- If two options recommend the same action, the difference is the **reasoning**. Read both fully.
- Be suspicious of an option that is true and answers a question nobody asked.

> ON THE EXAM
>
> Roughly two minutes per question. Most of that should go on understanding the requirement, not on weighing four options equally.

> A silver streaming table stopped updating. The bronze Delta table it reads from now receives corrective UPDATE statements from an operations team. An engineer sets skipChangeCommits on the streaming read and the stream runs cleanly again.
>
> A reviewer asks whether this is correct.

Is it?

A Yes. skipChangeCommits is the documented way to handle changes in a streaming source.

B No. The corrections are being discarded. The documented use requires the change to be applied to both tables first.

C No. skipChangeCommits only applies to deletes, not updates.

D Yes, provided the pipeline is fully refreshed afterwards.

◆ PAUSE AND DECIDE

Is it?

A Yes. skipChangeCommits is the documented way to handle changes in a streaming source.

It is documented for propagating deletions already applied to both tables. Not for discarding corrections.

B No. The corrections are being discarded. The documented use requires the change to be applied to both tables first.

Silver now runs cleanly while holding values the business knows are wrong.

C No. skipChangeCommits only applies to deletes, not updates.

It skips change commits generally. The scope is not the problem.

D Yes, provided the pipeline is fully refreshed afterwards.

A full refresh would reprocess, and the flag would discard the corrections again.

ON THE EXAM

The stream running cleanly is the symptom, not the cure. It is quiet because it is ignoring the thing it should be processing.

> A job has three tasks. Task A loads a staging table and succeeds. Task B enriches it and succeeds. Task C writes to an external payments API and fails halfway through.
>
> An engineer states that because task C failed, the whole run is rolled back, so it is safe to simply rerun the job from the start.

What is wrong with that reasoning?

A Nothing. Multi-task jobs are transactional across tasks.

B There is no cross-task transaction. Some operations in C may have completed, so rerunning may repeat them.

C Only Delta writes are rolled back; external API calls are not.

D Tasks A and B would also be rolled back, so rerunning is wasteful but safe.

◆ PAUSE AND DECIDE

What is wrong with that reasoning?

A Nothing. Multi-task jobs are transactional across tasks.

They are not. The guide's own published answer says some operations in the failed task may have completed.

B There is no cross-task transaction. Some operations in C may have completed, so rerunning may repeat them.

A job is not a transaction, and an external payments call is not idempotent.

C Only Delta writes are rolled back; external API calls are not.

Even the Delta writes in A and B are not rolled back. They committed.

D Tasks A and B would also be rolled back, so rerunning is wasteful but safe.

They completed and committed. Nothing undoes them.

ON THE EXAM

C is the sophisticated wrong answer: it half-knows the rule and invents a boundary where Delta writes roll back and external calls do not.

> A wide managed table has 90 columns. Analysts filter on a column at position 78 and complain of full scans. An engineer recalls that Delta collects statistics on the first 32 columns and proposes reordering the schema so the column moves into the first 32.
>
> The table is Unity Catalog managed with predictive optimization enabled.

What should they be told?

A The reordering is correct; 32 columns is the universal limit.

B The 32-column limit applies to external tables. Managed tables under predictive optimization have no such limit, so something else explains the scans.

C Reorder the schema and also run OPTIMIZE to rebuild statistics.

D Statistics cannot be collected on managed tables at all.

◆ PAUSE AND DECIDE

What should they be told?

A The reordering is correct; 32 columns is the universal limit.

It is not universal. That is the update most material has missed.

B The 32-column limit applies to external tables. Managed tables under predictive optimization have no such limit, so something else explains the scans.

Statistics columns are chosen intelligently on managed tables. The diagnosis needs to start again.

C Reorder the schema and also run OPTIMIZE to rebuild statistics.

Two actions built on a premise that does not apply here.

D Statistics cannot be collected on managed tables at all.

They are collected, and chosen more intelligently than on external tables.

ON THE EXAM

The engineer's fact is correct and its scope is not. The honest answer is that the diagnosis has to restart, which is often the right answer.

> A compliance officer asks how long deleted records remain recoverable. An engineer answers 30 days, citing Delta's default history retention. A second engineer answers 7 days, citing vacuum.
>
> The tables are managed with predictive optimization enabled and default settings throughout.

Who is right?

A The first. History retention governs recoverability, and it defaults to 30 days.

B The second, in practice. Vacuum removes the underlying files after 7 days, and once they are gone no version can be read.

C Neither. Deleted records are unrecoverable immediately.

D Both are wrong; deletion vectors keep records indefinitely.

◆ PAUSE AND DECIDE

Who is right?

A The first. History retention governs recoverability, and it defaults to 30 days.

History retention governs the log. It cannot make files readable after vacuum has removed them.

B The second, in practice. Vacuum removes the underlying files after 7 days, and once they are gone no version can be read.

Two properties, and the files are the binding constraint. Predictive optimization runs vacuum automatically.

C Neither. Deleted records are unrecoverable immediately.

With deletion vectors the row is not even rewritten, let alone removed.

D Both are wrong; deletion vectors keep records indefinitely.

Only until purge and vacuum, which predictive optimization performs.

ON THE EXAM

Both engineers cite a real number. Only one of them is the binding constraint, and it is the one governing the files.

A job must write a large dataset out with a specific target file size. A colleague proposes a repartition before writing. Another proposes coalesce. A third proposes sorting the data, which repartitions as a side effect.

The requirement states explicitly that the data must not be rearranged.

Which approach meets the requirement?

A Repartition to the calculated number of partitions before writing.

B Set the maximum partition bytes configuration to the target size, then read, transform and write.

C Coalesce to the calculated number of partitions before writing.

D Set the shuffle partitions configuration and sort the data before writing.

◆ PAUSE AND DECIDE

Which approach meets the requirement?

A Repartition to the calculated number of partitions before writing.

Repartition shuffles, which is rearranging.

B Set the maximum partition bytes configuration to the target size, then read, transform and write.

Controls partition size on read, so output files match without any shuffle. The published answer.

C Coalesce to the calculated number of partitions before writing.

Moves data between partitions. Cheaper than a repartition, still rearranging.

D Set the shuffle partitions configuration and sort the data before writing.

Sorting repartitions as a side effect, which the option itself admits.

ON THE EXAM

One clause decides everything. Three options are eliminated by 'must not be rearranged' before you evaluate their arithmetic.

> An analyst has SELECT on a table but cannot query it. A colleague suggests the table's row filter is blocking them and should be removed. Another suggests adding a column mask so they see something rather than nothing.
>
> The analyst receives a permissions error, not an empty result.

What is actually needed?

A Remove the row filter, which is denying access.

B USE CATALOG and USE SCHEMA on the parents. Filters and masks restrict; they never grant.

C Add a column mask so the analyst sees masked values instead of an error.

D Grant the analyst ownership of the table.

◆ PAUSE AND DECIDE

What is actually needed?

A Remove the row filter, which is denying access.

A row filter returns fewer rows. It does not produce a permissions error.

B USE CATALOG and USE SCHEMA on the parents. Filters and masks restrict; they never grant.

A permissions error means no access at all, and SELECT alone is never sufficient.

C Add a column mask so the analyst sees masked values instead of an error.

Masks restrict what a permitted user sees. They cannot create permission.

D Grant the analyst ownership of the table.

Enormous overreach, and ownership is a single principal.

ON THE EXAM

The error type is the discriminator. A permissions error means no access; an empty result would mean access plus a filter.

> A long-running job succeeded after a repair. A duration warning fired. Separately, a Structured Streaming job on the same cluster is showing one task far slower than the median with lopsided shuffle read, and spill on two tasks.
>
> A single engineer must explain both.

What are the two explanations?

A Both are memory pressure; the cluster needs resizing.

B Duration spans every repair attempt including idle time; and a few slow tasks with skewed shuffle read is data skew, not memory pressure.

C Both are skew; the repair took long because of the same skewed data.

D The duration warning is a false positive and the spill indicates memory pressure across the cluster.

◆ PAUSE AND DECIDE

What are the two explanations?

A Both are memory pressure; the cluster needs resizing.

Neither is. One is duration accounting, one is skew.

B Duration spans every repair attempt including idle time; and a few slow tasks with skewed shuffle read is data skew, not memory pressure.

Two unrelated symptoms with two documented explanations, and spill on two tasks is a consequence of skew.

C Both are skew; the repair took long because of the same skewed data.

Invents a connection between two unrelated jobs.

D The duration warning is a false positive and the spill indicates memory pressure across the cluster.

The warning is correct by its own definition, and spill on two tasks is localised.

ON THE EXAM

Two symptoms, two causes, no connection. The temptation is to find one explanation for everything on one cluster.

> A team must share a curated table with another department. The field names differ from the other department's conventions, and three columns are not approved for them to see.
>
> The recipient works in the same Databricks workspace. The team asks whether to use a deep clone, a shallow clone, or Delta Sharing.

What is the simplest solution?

A. A deep clone into a table with the required schema, synchronised on a schedule.

B. A view selecting only approved fields, with the column names aliased to their conventions.

C. A shallow clone, which is instant and references the same files.

D. Delta Sharing, which is designed for sharing between teams.

◆ PAUSE AND DECIDE

What is the simplest solution?

A. A deep clone into a table with the required schema, synchronised on a schedule.
A clone cannot rename or drop columns, and this adds a copy to keep in sync.

B. A view selecting only approved fields, with the column names aliased to their conventions.
Renaming and subsetting columns is exactly a view's job, with no copy and nothing to synchronise. The guide's published answer.

C. A shallow clone, which is instant and references the same files.
Also cannot rename or drop columns, and exposes everything.

D. Delta Sharing, which is designed for sharing between teams.
Built for sharing outside your metastore. The recipient is in the same workspace.

ON THE EXAM

Three of the four options are copying mechanisms answering a question about presentation. The requirement is rename and subset.

EXAM MECHANICS · EXAM STRATEGY - ALL DOMAINS

Exam mechanics and clock management

Cross-cutting. The rules, the clock, and what this course found that most material has not.

TYPE Cross-cutting TIME 120 min TOPIC Exam day

admin@sthithapragna.com

The exam at a glance

	DETAIL
Questions	59 scored, multiple choice.
Time	120 minutes.
Fee	USD 200, plus local taxes.
Delivery	Online proctored, or test centre proctored.
Test aids	None. Explicitly including API documentation.
Prerequisite	None required. One year of hands-on experience is recommended.
Validity	2 years, then recertify on the current version.

ON THE EXAM

No pass mark is published anywhere in the guide. Aim to be comfortable rather than to hit a number.

Unscored items, and what they do to your budget

- The exam **may include unscored items**, used to gather statistics for future versions.
- They are **not identified on the form**. You cannot tell which they are, and you should not try.
- They do **not affect your score**, and **additional time is factored in** for them.
- So do not budget 120 divided by 59. Budget roughly **two minutes per question you actually see**.

> ON THE EXAM
>
> The arithmetic people do in their heads is slightly wrong, and in your favour. There are more questions and more time than 59 and 120 suggest.

Where the marks actually are

SECTION	WEIGHT
1. Developing code for data processing	22%
6. Cost and performance optimization	13%
3, 5, 7, 9. Transformation, monitoring, security, deploying	10% each
2, 8. Ingestion, governance	7% each
10. Data modeling	6%
4. Data sharing and federation	5%

ON THE EXAM

Section 1 alone is more than a fifth of the paper. If revision time is short, that is where it goes.

The guide changes, and it tells you so

- The guide says directly: **check back two weeks before your exam** to make sure you have the current version.
- This course is built on the version live as of **3 July 2026**.
- Objectives get reworded, renamed and reweighted. It has happened repeatedly in the last year.
- Two minutes of checking beats an afternoon of studying an objective that has moved.

> ON THE EXAM
>
> Databricks publishes guide updates ahead of the change taking effect, so a check tells you what is coming as well as what is current.

What Databricks recommends studying

- Instructor-led: **Advanced Data Engineering With Databricks**.
- Self-paced: **Advanced Techniques with Spark Declarative Pipeline**, **Databricks Data Privacy**, **Databricks Performance Optimization**, and **Automated Deployment with Bundles**.
- Those four map onto **Sections 1, 7, 6 and 9**, which are three of the four heaviest sections.
- Treat the list as a statement about emphasis rather than a required reading list.

> ON THE EXAM
>
> Databricks chose four self-paced courses out of everything it publishes. Which four is itself information.

Two minutes, and what that buys

- Two minutes is **plenty** for a question you know, and **not enough** for one you are reasoning out from scratch.
- So the budget is really: bank time on the ones you know, and spend it on the ones you do not.
- The failure mode is spending six minutes on question four and then rushing the last fifteen.
- Aim to reach the halfway point with **more than half the time left**.

> ON THE EXAM
>
> Everyone knows this and it is still the most common way people lose marks they had already earned.

Three passes, not one

- **First pass:** answer everything you are confident about, and **flag anything that takes more than about ninety seconds**.
- **Second pass:** return to the flagged ones with the clock in hand and a calmer head.
- **Third pass:** if time remains, revisit only those you flagged as genuinely uncertain.
- **Always answer everything.** There is no penalty for a wrong answer, so an unanswered question is a wasted one.

> ON THE EXAM
>
> The flag exists so you can leave a question without deciding you have failed it. Use it early and often.

Find the requirement sentence first

- Most scenarios contain **one clause that decides the answer**. Find it before reading the options.
- Examples from this course: **without rearranging the data**, **without failing the job**, **must not be subject to the regulation**.
- Also: **latency under 400 milliseconds**, **the team is comfortable in SQL**, **no constraint requires hand-written streaming**.
- Reading the options first invites you to justify one, rather than to test all four against the requirement.

> ON THE EXAM
>
> Reading the options before the requirement is the single most expensive habit in a multiple-choice exam.

How the wrong answers are usually wrong

- **Right technique, wrong layer.** A real tool applied one level away from the problem.
- **True but irrelevant.** A correct statement that does not address the requirement.
- **Right conclusion, wrong reason.** Reaches the correct answer through a false claim.
- **Outdated but formerly correct.** True two years ago, and this course has met several.

> ON THE EXAM
>
> Notice that none of these is nonsense. Every category is something a competent engineer might say.

Things that changed, part one

- **Delta Sharing is now OpenSharing.** Renamed 10 June 2026, three weeks before the guide, which still says delta sharing.
- **The pipelines Python API was renamed** a week after the guide. The import, the decorators and the change capture function all changed.
- **ABAC row filters and column masks went GA** in May 2026, and the docs now recommend them. The guide names only the table-level form.
- **The change data feed table property is now described as legacy.** An automatic form exists.

> ON THE EXAM
>
> Four renames or repositions the guide has not absorbed. In each case, answer with what the guide names and know what is current.

Things that changed, part two

- **The 32-column statistics limit now applies only to external tables.** Managed tables under predictive optimization have no limit.
- **Predictive optimization does not run ZORDER, and ignores Z-ordered files.** A tuned table gets less maintenance, not more.
- **Liquid clustering replaces partitioning and ZORDER**, and is recommended for all new tables.
- **Asset Bundles are now Declarative Automation Bundles.** This one the guide did catch.

> ON THE EXAM
>
> Where the guide and the docs disagree, answer the guide's language and know the docs' position. Both appear in questions.

The week before

- **Check the guide** for a newer version. Two minutes, and it is the highest-value thing on this list.
- Work the three practice exams in this course under timed conditions, not as a reading exercise.
- Revisit **module 34**. Confusion pairs are where marks are won and lost.
- Do not start a new topic in the last three days. Consolidate what you have.

> ON THE EXAM
>
> If you have one hour left, spend it on Section 1 and on the confusion pairs, in that order.

Practical things that go wrong

- For an online proctored exam, test your setup **the day before**, not fifteen minutes beforehand.
- Clear your desk completely. Proctors are strict, and a delay here eats your exam time.
- Read every option before answering, even when the first one looks obviously right.
- If your mind goes blank, **flag it and move on**. The next question often unlocks it.

> ON THE EXAM
>
> None of this is clever. It is the list of things that cost people marks for reasons unrelated to what they know.

What to carry into the exam

- **59 scored questions, 120 minutes, no test aids.** Unscored items exist with extra time allowed.
- **Section 1 is 22%.** If revision time is short, that is where it goes.
- **Three passes.** Flag anything over ninety seconds. Answer everything; there is no penalty.
- **Find the requirement sentence before reading the options.**
- **Check the guide two weeks before.** It changes, and it tells you to.

> ON THE EXAM
>
> Three practice exams follow. Work them timed, and treat a wrong answer as the most useful thing on the page.

PRACTICE EXAM 1 · EXAM STRATEGY - ALL DOMAINS

Practice exam 1

Twenty questions, weighted to the real section proportions. Forty minutes if you are pacing properly.

TYPE Practice exam QUESTIONS 20 TOPIC All sections

admin@sthithapragna.com

How to get value from this

- **Pause on each question slide and commit to an answer** before continuing. Reading ahead teaches you nothing.
- Write your answers down. You want to know what you chose, not what you would have chosen.
- **A wrong answer is the most useful thing on the page.** Read the reasoning for every option, not only the right one.
- If you get one right for the wrong reason, count it as wrong. That is the one that will catch you on the day.

> ON THE EXAM
>
> Score interpretation comes at the end. Do not look at it first, and do not stop partway to check how you are doing.

> A team packages shared transformation logic as a Python wheel and deploys it with a bundle. Several notebook tasks import it. After a change to the wheel, some tasks pick up the new version and others continue using the old one within the same job run.
>
> The tasks run on a shared all-purpose cluster.

What is the most likely cause?

A. The wheel was not rebuilt, so both versions are identical.

B. The library is installed at cluster scope and already loaded into running Python processes.

C. Bundles cannot deploy wheels to all-purpose compute.

D. Notebook tasks always pin the wheel version at first import.

◆ PAUSE AND DECIDE

What is the most likely cause?

A The wheel was not rebuilt, so both versions are identical.

Then every task would behave the same. The scenario says they differ.

B The library is installed at cluster scope and already loaded into running Python processes.

A cluster-scoped library loaded into a live process is not swapped underneath it. Job compute avoids this entirely.

C Bundles cannot deploy wheels to all-purpose compute.

They can. The compute type is the risk, not a restriction.

D Notebook tasks always pin the wheel version at first import.

There is no such pinning behaviour.

ON THE EXAM

The shared all-purpose cluster is the clue. Fresh job compute per run gives each task a clean environment.

> A transformation applies a business rule to 3 billion rows. It is currently implemented as a Python user-defined function and takes four hours. The rule is a series of conditional string comparisons producing a category label.
>
> The team asks how to make it faster.

What is the first thing to try?

A Rewrite it as a pandas UDF, which vectorises the operation.

B Express the rule in native SQL, since conditional logic on strings needs no UDF at all.

C Increase the number of shuffle partitions so the UDF runs on more tasks.

D Register the UDF in Unity Catalog so it can be optimised.

◆ PAUSE AND DECIDE

What is the first thing to try?

A Rewrite it as a pandas UDF, which vectorises the operation.

A real improvement over a row-at-a-time Python UDF, and still slower than not leaving the engine at all.

B Express the rule in native SQL, since conditional logic on strings needs no UDF at all.

Conditional string logic is expressible in SQL, which keeps execution in the engine and avoids serialisation entirely.

C Increase the number of shuffle partitions so the UDF runs on more tasks.

Parallelises the serialisation overhead rather than removing it.

D Register the UDF in Unity Catalog so it can be optimised.

Registration governs and shares a function. It does not change how it executes.

ON THE EXAM

A is the trap because it is genuinely better. The question asks for the first thing to try, and the first thing is to stop leaving the engine.

> A pipeline computes a daily aggregate of orders by region. The source is a Delta table that receives late-arriving corrections to orders from up to two weeks earlier. The business needs the aggregate to always be correct.
>
> An engineer proposes a streaming table for cost reasons.

What should they use, and why?

A. A streaming table, which reprocesses corrections automatically.

B. A materialized view, which always returns the correct result and recomputes when incremental processing cannot.

C. A streaming table with a two-week watermark.

D. A materialized view is unnecessary; add a scheduled full refresh to the streaming table.

◆ PAUSE AND DECIDE

What should they use, and why?

A. A streaming table, which reprocesses corrections automatically.

A streaming table processes each row once and does not revisit it. Corrections to old rows are not reprocessed.

B. A materialized view, which always returns the correct result and recomputes when incremental processing cannot.

Correctness is the stated requirement, and a materialized view never trades it for speed.

C. A streaming table with a two-week watermark.

A watermark bounds state in a streaming aggregation. It does not make a streaming table revisit processed rows.

D. A materialized view is unnecessary; add a scheduled full refresh to the streaming table.

A full refresh recomputes everything nightly, which costs more than the materialized view being avoided.

ON THE EXAM

Always correct is the requirement sentence. A streaming table's process-once guarantee is exactly what the corrections violate.

A supplier dimension is maintained with AUTO CDC as Type 2. A supplier is deleted in the source system, and the delete condition is configured.

An analyst reports that the supplier has vanished from the dimension entirely, and asks whether the history was lost.

What actually happened?

A The history was deleted along with the record, as Type 2 deletes remove all versions.

B The current version was closed with an end value; the history remains. The analyst is filtering on a null end date.

C AUTO CDC does not support deletes on Type 2 targets.

D The supplier was moved to the rescued data column.

◆ PAUSE AND DECIDE

What actually happened?

A The history was deleted along with the record, as Type 2 deletes remove all versions.

Type 2 preserves history through deletion. That is much of the point of it.

B The current version was closed with an end value; the history remains. The analyst is filtering on a null end date.

A deleted record has its final version closed rather than removed, so it disappears from a current-state filter.

C AUTO CDC does not support deletes on Type 2 targets.

It supports them through the delete condition, which the scenario says is configured.

D The supplier was moved to the rescued data column.

Rescued data captures records that did not parse. This one parsed and was deleted.

ON THE EXAM

Vanished from the dimension and vanished from the analyst's query are different claims. The second is what happened.

> A job must process a different set of country files each night, determined by which countries delivered data. The list changes daily. A first task queries the landing zone and produces the list.
>
> An engineer configures a For each task fed by that list, and the job fails immediately with nothing having run.

What is missing?

A. A nested task inside the For each task. It has nothing to run.

B. A task value must be set explicitly; For each cannot read a task's output.

C. The For each task requires a static array; dynamic lists are unsupported.

D. An If/else task must precede the For each to check the list is non-empty.

◆ PAUSE AND DECIDE

What is missing?

A. A nested task inside the For each task. It has nothing to run.

A For each task requires both itself and a nested task. Neither does anything alone.

B. A task value must be set explicitly; For each cannot read a task's output.

Dynamically generated input arrays are supported and are the powerful case.

C. The For each task requires a static array; dynamic lists are unsupported.

The opposite. Generating the array dynamically is the documented pattern.

D. An If/else task must precede the For each to check the list is non-empty.

Sensible defensive design, and not what causes an immediate failure.

ON THE EXAM

The structural trap from module 9. A For each task is half a construction, and the other half is a nested task.

> An Auto Loader stream ingests CSV files with an explicit schema supplied at creation. A source adds two new columns. The stream continues without error, and the new columns never appear downstream.
>
> The team expected the stream to fail so they would notice.

Why did nothing happen?

A Supplying a schema sets the evolution mode default to none, which ignores new columns without failing.

B CSV does not support schema evolution.

C The new columns were written to the rescued data column, which nobody queried.

D Schema evolution only triggers when a column's type changes, not when one is added.

◆ PAUSE AND DECIDE

Why did nothing happen?

A Supplying a schema sets the evolution mode default to none, which ignores new columns without failing.

The two-defaults rule. With a schema supplied the default is none, which ignores new columns and rescues nothing.

B CSV does not support schema evolution.

It does. CSV is one of the formats where evolution matters most.

C The new columns were written to the rescued data column, which nobody queried.

None mode does not rescue unless a rescued data column is explicitly requested.

D Schema evolution only triggers when a column's type changes, not when one is added.

Adding a column is the canonical evolution case.

ON THE EXAM

The team's expectation was reasonable and describes addNewColumns, which is the default only when no schema is supplied.

> A query must return, for each product, the most recent price change and the price immediately before it. An engineer writes a window function with a partition by product and an order by change date, then filters on the row number in a WHERE clause.
>
> The query fails to resolve the column.

What is the cleanest fix on Databricks?

A Move the filter to HAVING.

B Use QUALIFY, which filters window results in the same query block.

C Replace row_number with dense_rank, which is resolvable in WHERE.

D Add the row number to the GROUP BY.

◆ PAUSE AND DECIDE

What is the cleanest fix on Databricks?

A Move the filter to HAVING.

Window functions are evaluated after HAVING too.

B Use QUALIFY, which filters window results in the same query block.

Exactly what QUALIFY exists for, and it is Databricks-specific, which is why the question says on Databricks.

C Replace row_number with dense_rank, which is resolvable in WHERE.

No ranking function is resolvable in WHERE. The function is not the problem.

D Add the row number to the GROUP BY.

Collapses the rows being selected.

ON THE EXAM

A CTE would also work and is not offered. The phrase 'on Databricks' is pointing at the platform-specific answer.

> A full outer join between two large tables is slow. The Spark UI shows median task duration of six seconds and maximum of forty minutes, with shuffle read similarly lopsided.
>
> An engineer says adaptive query execution will split the skewed partition automatically once statistics refresh.

What is the correct assessment?

A Correct. AQE splits skewed partitions in shuffle-based joins once statistics are current.

B AQE does not skew-optimise a FULL OUTER JOIN on either side. A manual approach such as salting is required.

C AQE handles this, but only if the join is converted to a broadcast join first.

D The maximum-to-median ratio indicates memory pressure, not skew.

◆ PAUSE AND DECIDE

What is the correct assessment?

A — Correct. AQE splits skewed partitions in shuffle-based joins once statistics are current.

Nearly right, and the join type is the exception. AQE also uses runtime statistics, not collected ones.

B — AQE does not skew-optimise a FULL OUTER JOIN on either side. A manual approach such as salting is required.

A documented limit. Skew handling applies to shuffle joins, and full outer is excluded on both sides.

C — AQE handles this, but only if the join is converted to a broadcast join first.

Broadcast joins are never skew-optimised, and both tables are large.

D — The maximum-to-median ratio indicates memory pressure, not skew.

Uniformly slow tasks indicate resources. A few far slower than the median is skew.

ON THE EXAM

A is wrong twice: the join type is excluded, and AQE works from runtime statistics precisely so it does not wait for collected ones.

> A healthcare provider shares a patient table with a research partner using WITH HISTORY for performance. Records are routinely deleted under right-to-erasure requests. Vacuum runs on the default retention.
>
> A privacy review asks what the recipient can currently read.

What is the accurate answer?

A Only the current table contents, since deletes are applied immediately.

B The current contents plus, via the Delta log, commit history and deleted data not yet vacuumed.

C Nothing beyond current contents; recipients cannot access the Delta log.

D Everything ever written, since WITH HISTORY disables vacuum.

◆ PAUSE AND DECIDE

What is the accurate answer?

A Only the current table contents, since deletes are applied immediately.

True of a query against the table, and not of direct file access to its directory.

B The current contents plus, via the Delta log, commit history and deleted data not yet vacuumed.

WITH HISTORY grants cloud tokens scoped to the table's root directory, which includes the log.

C Nothing beyond current contents; recipients cannot access the Delta log.

Cloud token access is precisely access to the directory, log included.

D Everything ever written, since WITH HISTORY disables vacuum.

It does not disable vacuum. The window is bounded by retention.

ON THE EXAM

The vacuum window bounds the exposure, which is why D overstates it and A understates it. The truthful answer sits between them.

A cost report built from the billable usage system table shows one job costing far more than the team expects. The report joins usage to the jobs table on job identifier and sums the usage quantity.

That job's definition has been edited eleven times this year. Other jobs, rarely edited, look correct.

What is wrong?

A. The job genuinely costs more because each edit redeploys it.

B. The jobs table is SCD Type 2, so the join multiplies usage by the number of job versions.

C. Usage records for edited jobs are duplicated by the billing system.

D. The report must filter to record type ORIGINAL to avoid double counting.

◆ PAUSE AND DECIDE

What is wrong?

A The job genuinely costs more because each edit redeploys it.

Editing a definition does not run it.

B The jobs table is SCD Type 2, so the join multiplies usage by the number of job versions.

Every edit emits a new row, and joining without restricting to one version multiplies the usage.

C Usage records for edited jobs are duplicated by the billing system.

Usage is recorded per run. The duplication is in the join.

D The report must filter to record type ORIGINAL to avoid double counting.

That filter causes overcounting by discarding retractions, and is a different trap entirely.

ON THE EXAM

The correlation with edit count is the tell, and D is a real trap from the same table applied to the wrong symptom.

A job runs a SQL alert task checking that a loaded table's null rate is below a threshold, then a publication task depending on it. On a night when the null rate breached the threshold, the alert notified the team and publication proceeded.

The dependency uses the default run-if condition.

How should the job be changed?

A Change the run-if condition to None failed.

B Branch on the alert task's alert state with an If/else task, since task status reports whether the alert evaluated, not what it found.

C Configure the alert to fail the task when triggered.

D Move the alert to a schedule so it runs before the job.

◆ PAUSE AND DECIDE

How should the job be changed?

A Change the run-if condition to None failed.

Loosens the condition. The alert task succeeded either way.

B Branch on the alert task's alert state with an If/else task, since task status reports whether the alert evaluated, not what it found.

Task status is Succeeded whenever evaluation completed. The verdict is a value, so it needs If/else.

C Configure the alert to fail the task when triggered.

No such setting. Failed means an evaluation error.

D Move the alert to a schedule so it runs before the job.

Decouples the check from the load, so the two can disagree about which data was checked.

ON THE EXAM

Module 9's distinction: run-if branches on outcome, If/else branches on a value. The alert's verdict is a value.

> An organisation has 400 external Delta tables and no automatic maintenance. They run a nightly OPTIMIZE and VACUUM job costing a significant amount. A proposal suggests converting the tables to Unity Catalog managed and retiring the job.
>
> Historic time travel of 60 days must be preserved on all tables.

What must they do, and in what order?

A Convert with SET MANAGED, then set the deleted file retention property to cover 60 days.

B Set the deleted file retention property to cover 60 days first, then convert and enable predictive optimization.

C Convert using CREATE TABLE AS SELECT, which preserves history.

D Keep the tables external; predictive optimization works on external tables too.

◆ PAUSE AND DECIDE

What must they do, and in what order?

A Convert with SET MANAGED, then set the deleted file retention property to cover 60 days.

Right operation, wrong order. Automatic vacuum may run on the 7-day default before the property is set.

B Set the deleted file retention property to cover 60 days first, then convert and enable predictive optimization.

The documented sequence: set retention before enabling, so automatic vacuum respects the longer window.

C Convert using CREATE TABLE AS SELECT, which preserves history.

CTAS creates a new table with no history at all, which is the opposite of the requirement.

D Keep the tables external; predictive optimization works on external tables too.

It runs only on Unity Catalog managed tables.

ON THE EXAM

A and B differ only in order, and the order is the answer. The docs say set retention before enabling.

> A privacy team requires that records deleted under erasure requests be unrecoverable within 30 days. Deletion vectors are enabled on all tables. The engineering team runs DELETE statements and confirms the rows no longer appear in queries.
>
> Predictive optimization is enabled on all tables.

What still needs to happen?

A Nothing further; predictive optimization runs VACUUM, which removes the files.

B REORG TABLE with the purge option, so the files are rewritten without the soft-deleted rows, then vacuum removes the old files.

C Disable deletion vectors and re-run the deletes.

D Set the deleted file retention property to 30 days.

◆ PAUSE AND DECIDE

What still needs to happen?

A Nothing further; predictive optimization runs VACUUM, which removes the files.

Vacuum removes unreferenced files. Files containing soft-deleted rows are still referenced.

B REORG TABLE with the purge option, so the files are rewritten without the soft-deleted rows, then vacuum removes the old files.

With deletion vectors the row stays in the file until a purge rewrites it. The GDPR page requires this step.

C Disable deletion vectors and re-run the deletes.

Purge exists so you keep the write performance and still erase when required.

D Set the deleted file retention property to 30 days.

Lengthening retention keeps files longer, which is the opposite of what erasure needs.

ON THE EXAM

A is the sophisticated wrong answer: vacuum is real and automatic, and it only removes files nothing references.

A 3 TB unpartitioned table is converted to liquid clustering with keys on region and event date. OPTIMIZE is run and completes. Queries filtering on region show little improvement, though data written after the change clusters well.

The runtime is DBR 16.2.

What should they run?

A OPTIMIZE again, since the first run may have timed out.

B OPTIMIZE FULL, which forces reclustering of all existing data files.

C ZORDER BY region alongside the clustering keys.

D VACUUM, to remove the unclustered files.

◆ PAUSE AND DECIDE

What should they run?

A OPTIMIZE again, since the first run may have timed out.

It completed, and plain OPTIMIZE clusters incrementally however many times you run it.

B OPTIMIZE FULL, which forces reclustering of all existing data files.

Incremental clustering leaves existing files alone. OPTIMIZE FULL is required, and DBR 16.2 supports it.

C ZORDER BY region alongside the clustering keys.

Clustering and Z-order are incompatible, and Z-ordered files are ignored by predictive optimization.

D VACUUM, to remove the unclustered files.

Those files hold current data. Vacuum removes only unreferenced files.

ON THE EXAM

The runtime is in the question for a reason. OPTIMIZE FULL needs DBR 16.0, and 16.2 clears it.

> A production job is owned by a platform engineer with broad access across the workspace. To let an on-call rota restart it overnight, the team grants the rota group CAN MANAGE RUN.
>
> A security review asks what the on-call engineers can effectively do.

What is the correct assessment?

A They can start and cancel the job, executing it with their own permissions.

B They can execute the job with the owner's permissions, which are broader than their own.

C They can start the job but it will fail, since they lack access to the underlying data.

D CAN MANAGE RUN grants no data access, so the review has no finding.

◆ PAUSE AND DECIDE

What is the correct assessment?

A They can start and cancel the job, executing it with their own permissions.

True for interactive execution. A job run assumes the owner's permissions.

B They can execute the job with the owner's permissions, which are broader than their own.

A job triggered through Run Now assumes the permissions of the job owner, not the triggering user.

C They can start the job but it will fail, since they lack access to the underlying data.

It will not fail, because it runs as the owner.

D CAN MANAGE RUN grants no data access, so the review has no finding.

Starting a run executes work that reads and writes data.

ON THE EXAM

The fix is service principal ownership scoped to what the job needs, which is why bundles expect one in production mode.

A masking policy is applied to columns tagged as personal data across a catalog. A declarative pipeline whose run-as identity is a service principal reads two of those tables and writes a gold streaming table.

Analysts with full access find the personal data masked in gold, and no grant restores it.

What is the cause and the fix?

A The gold table inherited the policy through lineage. Remove the policy from gold.

B The refresh evaluated policies as the pipeline's run-as identity, writing masked values permanently. Exempt that identity from the policies.

C Streaming tables cannot carry unmasked personal data. Use a materialized view.

D Analysts need EXECUTE on the masking function.

◆ PAUSE AND DECIDE

What is the cause and the fix?

A The gold table inherited the policy through lineage. Remove the policy from gold.

Policies do not propagate through lineage. The masked values are in the data.

B The refresh evaluated policies as the pipeline's run-as identity, writing masked values permanently. Exempt that identity from the policies.

Policies are evaluated using the pipeline owner or run-as identity, and masked values are written in.

C Streaming tables cannot carry unmasked personal data. Use a materialized view.

Invented, and a materialized view would hit the same problem.

D Analysts need EXECUTE on the masking function.

There is nothing to unmask. The original values were never written.

ON THE EXAM

No grant restores it is the tell. Read-time masking is reversible by permission; write-time masking is data loss.

A data steward is granted MANAGE on a catalog so they can administer permissions across it. They can grant and revoke on every table inside, and transfer ownership.

They report they cannot query any of the tables, and ask whether the grant is incomplete.

What should they be told?

A The grant is incomplete; MANAGE on a catalog should confer SELECT on its tables.

B MANAGE confers administration, not data access. SELECT plus USE CATALOG and USE SCHEMA are still needed.

C They must be the catalog owner rather than a MANAGE holder to query tables.

D MANAGE on a catalog requires USE CATALOG, which they are missing.

◆ PAUSE AND DECIDE

What should they be told?

A The grant is incomplete; MANAGE on a catalog should confer SELECT on its tables.

The separation is deliberate. Administering is not accessing.

B MANAGE confers administration, not data access. SELECT plus USE CATALOG and USE SCHEMA are still needed.

MANAGE holders do not automatically have data privileges, and data access still needs the usage privileges.

C They must be the catalog owner rather than a MANAGE holder to query tables.

An owner has capabilities implied, and MANAGE holders can simply grant themselves what they need.

D MANAGE on a catalog requires USE CATALOG, which they are missing.

MANAGE on a catalog requires no usage privileges. That is the reduced requirement.

ON THE EXAM

D is the near miss: it names the right privilege for the wrong reason. Usage is needed for the SELECT, not for the MANAGE.

> An overnight job on job compute failed. The next morning an engineer finds the job's error message mentions a task failure during a shuffle, opens the driver logs, and sees only routine progress messages.
>
> No log delivery is configured.

What should they do?

A The driver logs are truncated. Increase the log retention setting.

B Worker-side failures are in the executor logs, which are unavailable because the cluster terminated. Configure log delivery for future runs.

C Check the cluster event log, which records task failures.

D Re-run the job with a larger driver, since driver logs are incomplete.

◆ PAUSE AND DECIDE

What should they do?

A The driver logs are truncated. Increase the log retention setting.

They are not truncated. The error is worker-side and was never in them.

B Worker-side failures are in the executor logs, which are unavailable because the cluster terminated. Configure log delivery for future runs.

Both halves: a task failure is worker-side, and job compute terminates taking the Spark UI with it.

C Check the cluster event log, which records task failures.

It records lifecycle events, not task failures.

D Re-run the job with a larger driver, since driver logs are incomplete.

Changes the compute to address a logging misunderstanding.

ON THE EXAM

Two facts in one answer: the right log, and the fact that it is already gone. The remedy is preventive.

> A deployment to a bundle's production target fails validation, reporting that a pipeline is marked as a development pipeline. The engineer confirms the pipeline definition sets development to true, which they added while testing.
>
> They are on the correct Git branch.

What is happening?

A. Production mode validates that all deployed pipelines are marked development false. Remove the setting.

B. Deploy with the force flag, which overrides production mode validations.

C. Switch the target to development mode to deploy successfully.

D. Production mode requires a service principal, which is the actual failure.

◆ PAUSE AND DECIDE

What is happening?

A Production mode validates that all deployed pipelines are marked development false. Remove the setting.

Exactly the stated validation, and it is doing its job: a development pipeline cannot reach production by accident.

B Deploy with the force flag, which overrides production mode validations.

Force overrides the Git branch validation specifically, not this one.

C Switch the target to development mode to deploy successfully.

Would prefix every resource and mark pipelines as development, changing production rather than deploying to it.

D Production mode requires a service principal, which is the actual failure.

It recommends one. Without it you get extra validations, not this error.

ON THE EXAM

B is the near miss. The force flag exists, and it overrides the branch check rather than every production validation.

A gold star schema is built with primary keys on dimensions and foreign keys from the fact table. A data quality report finds fact rows whose customer key does not exist in the customer dimension.

The team is surprised, since the foreign key constraint is in place and the load succeeded.

What explains it?

A The constraint was created but not validated. Run a validation command.

B Primary and foreign key constraints are informational and not enforced at write time.

C Foreign keys are enforced only when both tables are in the same schema.

D The dimension is Type 2, so the surrogate keys change and break the reference.

◆ PAUSE AND DECIDE

What explains it?

A The constraint was created but not validated. Run a validation command.

There is no validation step that makes a foreign key enforced.

B Primary and foreign key constraints are informational and not enforced at write time.

They document relationships and inform the optimizer. They never reject a row.

C Foreign keys are enforced only when both tables are in the same schema.

They are not enforced in any arrangement.

D The dimension is Type 2, so the surrogate keys change and break the reference.

Type 2 adds versions with new surrogate keys. Existing keys remain valid.

ON THE EXAM

Contrast with NOT NULL and CHECK on the same table, which are enforced. Enforcement is per constraint type.

How to read your result

- **17 or more.** Comfortable. Work exams 2 and 3, and revisit anything you got right by guessing.
- **13 to 16.** On track. Identify which sections your misses cluster in and revisit those modules.
- **Below 13.** More study needed before booking. Start with Section 1, which is 22% of the paper.
- No pass mark is published, so treat these bands as guidance rather than prediction.

> ON THE EXAM
>
> The more useful number is how many you got right for the right reason. Go back and check the reasoning on every one you guessed.

PRACTICE EXAM 2 · EXAM STRATEGY - ALL DOMAINS

Practice exam 2

Twenty more questions, none repeating exam 1. Same rules, same clock.

TYPE Practice exam QUESTIONS 20 TOPIC All sections

admin@sthithapragna.com

> A fraud detection workload must flag transactions and publish to a Kafka topic with end-to-end latency under 500 milliseconds. The platform team's standard is that all new streaming work is built as declarative pipelines.
>
> Events arrive from Kafka.

What should the team do?

A Build it as a pipeline in continuous mode with the shortest available trigger.

B Build it as Structured Streaming in real-time mode on dedicated compute, since pipelines cannot run real-time mode.

C Build it as a pipeline on serverless, which supports real-time mode.

D Build it as a pipeline with a foreach batch sink writing to Kafka.

◆ PAUSE AND DECIDE

What should the team do?

A Build it as a pipeline in continuous mode with the shortest available trigger.

Continuous mode still runs micro-batches. It does not reach sub-second latency.

B Build it as Structured Streaming in real-time mode on dedicated compute, since pipelines cannot run real-time mode.

Real-time mode is unsupported on pipelines, classic or serverless, and Kafka to Kafka is what it supports.

C Build it as a pipeline on serverless, which supports real-time mode.

Factually false, and it is the option that sounds like a compromise between the requirement and the standard.

D Build it as a pipeline with a foreach batch sink writing to Kafka.

Solves the sink and ignores the latency.

ON THE EXAM

A team standard is not a requirement. When a stated latency target is unreachable by the standard approach, the target wins.

> A serverless job has three tasks. Two are Python wheel tasks and one is a notebook task. An engineer defines an environment with the required libraries and attaches it to all three by environment key.
>
> The notebook task fails, reporting the libraries are unavailable.

Why?

A. Notebook tasks cannot take a task environment specification. Their dependencies come from the notebook's environment panel.

B. An environment can be attached to only one task.

C. The environment version must match the notebook's runtime version.

D. Wheel tasks and notebook tasks cannot share a serverless job.

◆ PAUSE AND DECIDE

Why?

A Notebook tasks cannot take a task environment specification. Their dependencies come from the notebook's environment panel.

The documented exception. It is also why the high memory setting lives in that same panel.

B An environment can be attached to only one task.

Sharing one environment across tasks is the point of the environment key.

C The environment version must match the notebook's runtime version.

Invented requirement.

D Wheel tasks and notebook tasks cannot share a serverless job.

They can. Mixed task types in one job are normal.

ON THE EXAM

The two wheel tasks working and the notebook task failing is the tell. One task type behaves differently.

> A Structured Streaming job ingests from cloud storage. The business confirms the downstream dashboard is checked twice daily and a few hours of latency is acceptable. The stream currently runs continuously on an autoscaling cluster.
>
> The finance team has queried the cost.

What is the strongest change?

A Reduce the autoscaling minimum so the cluster is smaller between batches.

B Switch to the availableNow trigger on a schedule, so compute runs only while there is work.

C Switch to the once trigger, which processes a single micro-batch per run.

D Move the stream to a declarative pipeline in continuous mode.

◆ PAUSE AND DECIDE

What is the strongest change?

A Reduce the autoscaling minimum so the cluster is smaller between batches.

Autoscaling has known scale-down limitations for streaming, and the cluster still runs all day.

B Switch to the availableNow trigger on a schedule, so compute runs only while there is work.

Consumes everything available and stops. Same code, same checkpoint, a fraction of the compute.

C Switch to the once trigger, which processes a single micro-batch per run.

Deprecated from DBR 11.3 LTS, and a single micro-batch may not clear the backlog.

D Move the stream to a declarative pipeline in continuous mode.

Continuous mode keeps compute running, which is the cost being questioned.

ON THE EXAM

The business requirement is the lever. A continuous source does not compel continuous compute.

A serverless task sends onboarding emails to new customers through a third-party provider. During a transient network failure the task failed, and afterwards several customers reported receiving the email twice.

The task has no retry policy configured, and the code sends each email exactly once per invocation.

What should be changed?

A Set maximum retries to zero on the task.

B Disable serverless auto-optimization for this task, since it retries failed tasks by default.

C Move the task to a continuous job, which handles retries more carefully.

D Nothing; the provider duplicated the sends.

◆ PAUSE AND DECIDE

What should be changed?

A Set maximum retries to zero on the task.

Does not disable auto-optimization, which is what performed the retry.

B Disable serverless auto-optimization for this task, since it retries failed tasks by default.

Auto-optimization retries failed tasks by default. Sending email is not idempotent, so this needs at-most-once.

C Move the task to a continuous job, which handles retries more carefully.

Continuous mode retries automatically with backoff and cannot be configured.

D Nothing; the provider duplicated the sends.

Blames a third party for documented default behaviour on this platform.

ON THE EXAM

No retry policy is configured, and a retry still happened. That is the whole question.

> A team reads a Kafka topic carrying Avro payloads into a bronze streaming table. They select the value column and apply an Avro deserialisation function to it, and the job fails with a type error.
>
> A schema registry is configured and reachable.

What is missing?

A. The value must be cast to string before deserialisation.

B. Nothing is missing in the reader; the value column is binary, which is what from_avro expects.

C. The topic must be subscribed with the assign option rather than subscribe.

D. Avro payloads require the schema to be supplied as a table property.

◆ PAUSE AND DECIDE

What is missing?

A The value must be cast to string before deserialisation.

Casting binary Avro to string corrupts it. That is the fix for JSON, not for Avro.

B Nothing is missing in the reader; the value column is binary, which is what from_avro expects.

Kafka returns binary, and the Avro function consumes binary. The error is elsewhere, most likely the schema argument.

C The topic must be subscribed with the assign option rather than subscribe.

Any of subscribe, subscribePattern or assign works. It is not a type problem.

D Avro payloads require the schema to be supplied as a table property.

Invented mechanism.

ON THE EXAM

This inverts exam 1's habit. Kafka returning bytes is right for Avro, and the JSON fix would break it.

> A monthly reconciliation job runs an Auto Loader stream against a bucket holding several million files, using file notification with file events enabled. Each monthly run now takes hours before processing begins, though very little new data arrives.
>
> Correctness has never been in question.

What explains it?

A The file events cache expires; streams should run at least every seven days, so a monthly run falls back to a full directory listing.

B The checkpoint has grown too large to load.

C Backfill interval must be set manually for streams running less than daily.

D File events are unsupported for buckets above a certain file count.

◆ PAUSE AND DECIDE

What explains it?

A The file events cache expires; streams should run at least every seven days, so a monthly run falls back to a full directory listing.

A month is well outside the seven-day window, so the stored read position is invalid and a full listing is required.

B The checkpoint has grown too large to load.

Checkpoint size tracks files processed, and very little new data arrives.

C Backfill interval must be set manually for streams running less than daily.

Backfill is handled automatically once file events are enabled.

D File events are unsupported for buckets above a certain file count.

No such limit. The limit is on time between runs.

ON THE EXAM

Correctness has never been in question is the clue. The fallback is complete and correct, and slow.

> A daily job filters a large orders table to those whose product appears in a small recalled-products table, using an inner join on product id. Finance reports the recalled order totals are overstated.
>
> The recalled-products table lists some products twice, once per affected batch.

What is the correct fix?

A. Deduplicate the recalled-products table before the join.

B. Replace the inner join with a LEFT SEMI join, which returns each qualifying order exactly once.

C. Add SELECT DISTINCT to the join output.

D. Replace the inner join with a LEFT ANTI join.

◆ PAUSE AND DECIDE

What is the correct fix?

A Deduplicate the recalled-products table before the join.

Fixes today's data. The duplicates are legitimate, being one row per affected batch, so they will return.

B Replace the inner join with a LEFT SEMI join, which returns each qualifying order exactly once.

A semi join is a filter that cannot multiply rows, whatever the right side contains.

C Add SELECT DISTINCT to the join output.

Masks the duplication at the cost of a shuffle, and would collapse genuinely identical orders.

D Replace the inner join with a LEFT ANTI join.

Returns orders with no recalled product, which is the opposite set.

ON THE EXAM

The duplicates are legitimate here, which rules out deduplication as a fix. The join type is the thing that is wrong.

A pipeline defines an expectation on a silver table checking that a delivery date is not in the future. Metrics show violations being counted every day, and a downstream report still contains future-dated deliveries.

The expectation was written with no violation clause.

What is happening?

A The expectation is malformed and is being ignored.

B Warn is the default action, so violating records are written to the target and counted.

C The metrics are counting rows that were dropped, and the report reads a stale version.

D Expectations do not apply to date comparisons.

◆ PAUSE AND DECIDE

What is happening?

A The expectation is malformed and is being ignored.

The metrics prove it is being evaluated.

B Warn is the default action, so violating records are written to the target and counted.

An expectation with no violation clause counts rather than prevents. It is working exactly as written.

C The metrics are counting rows that were dropped, and the report reads a stale version.

Dropped rows would not appear downstream at all.

D Expectations do not apply to date comparisons.

Any SQL boolean expression works.

ON THE EXAM

Both observations are true at once. It is evaluated, and it lets the record through, because that is what warn does.

> A team federates a Snowflake source for exploratory analysis. It works well. The business now wants that data feeding an hourly production pipeline over several hundred million rows, with tight freshness requirements.
>
> The source is supported by both Lakehouse Federation and Lakeflow Connect.

What does Databricks recommend?

A Keep federation and scale up the SQL warehouse.

B Use Lakeflow Connect, which Databricks recommends when performance at volume or lower latency is a priority.

C Keep federation and enable result caching to reduce repeated load.

D Switch to catalog federation, which is more performance-optimized.

◆ PAUSE AND DECIDE

What does Databricks recommend?

A Keep federation and scale up the SQL warehouse.

The constraint is the source and the network, not Databricks compute.

B Use Lakeflow Connect, which Databricks recommends when performance at volume or lower latency is a priority.

Stated directly: where a source supports both, prefer Lakeflow Connect at volume or for lower latency.

C Keep federation and enable result caching to reduce repeated load.

Databricks query caching is not supported for federated queries at all.

D Switch to catalog federation, which is more performance-optimized.

True in general, and it is a different query path for catalog platforms rather than a faster setting.

ON THE EXAM

Federation is scoped to ad hoc reporting, BI and proof of concept. Growing past that scope is the signal to ingest.

> A finance analyst reconciling Databricks costs finds their monthly total is consistently higher than the invoice. They built the report from the billable usage system table, summing usage quantity grouped by month.
>
> To be careful, they excluded records with a negative usage quantity as obvious data errors.

What is wrong?

A. Negative quantities are retractions that cancel incorrect original records. Excluding them leaves the uncorrected values in.

B. The usage table reports list prices rather than negotiated ones.

C. They must also filter to record type RESTATEMENT to get corrected values.

D. Monthly grouping double counts usage spanning a month boundary.

◆ PAUSE AND DECIDE

What is wrong?

A Negative quantities are retractions that cancel incorrect original records. Excluding them leaves the uncorrected values in.

A correction is a negative retraction plus a restatement. Removing the negatives keeps the wrong figures.

B The usage table reports list prices rather than negotiated ones.

The table records quantities, and a pricing difference would be a consistent ratio.

C They must also filter to record type RESTATEMENT to get corrected values.

Filtering to restatements alone discards all correct original records.

D Monthly grouping double counts usage spanning a month boundary.

Invented.

ON THE EXAM

Exam 1 had this trap as a filter on record type. Here it is a filter on the sign of the quantity, which is more natural and equally wrong.

> A declarative pipeline ingesting from Kinesis must alert when it falls more than fifteen minutes behind. An engineer adds a streaming backlog notification with a backlog seconds threshold to the job that runs the pipeline task.
>
> During a confirmed three-hour backlog, no notification fires and no error appears anywhere.

What is the explanation and the alternative?

A Backlog alerting uses a ten-minute average, which smoothed the backlog away.

B Streaming backlog notifications are not supported for pipeline tasks. Use the event log's flow progress backlog metrics.

C Backlog seconds is not a supported metric; only bytes and records are.

D The threshold belongs on the pipeline rather than in the job's health rules.

◆ PAUSE AND DECIDE

What is the explanation and the alternative?

A Backlog alerting uses a ten-minute average, which smoothed the backlog away.

The averaging smooths brief spikes. A sustained three-hour backlog clears any average.

B Streaming backlog notifications are not supported for pipeline tasks. Use the event log's flow progress backlog metrics.

A documented limitation, and it fails silently. The event log carries backlog metrics in flow progress events.

C Backlog seconds is not a supported metric; only bytes and records are.

All four exist: bytes, records, seconds and files.

D The threshold belongs on the pipeline rather than in the job's health rules.

Health rules are the right place. The task type is the problem.

ON THE EXAM

Everything configured was correct. The unsupported combination fails silently, which is what makes it hard to diagnose.

> Two managed tables in the same schema receive identical write patterns. One is Z-ordered on two columns from an earlier project; the other has no clustering. Over six months the Z-ordered table's query performance degrades while the other's holds steady.
>
> Predictive optimization is enabled and no manual maintenance runs.

What explains it?

A Z-ordering degrades faster than no clustering and needs periodic re-running.

B Predictive optimization does not run ZORDER and ignores Z-ordered files, so that table receives less automatic maintenance.

C The unclustered table benefits from data skipping that Z-order disables.

D Predictive optimization only maintains tables with clustering keys defined.

◆ PAUSE AND DECIDE

What explains it?

A Z-ordering degrades faster than no clustering and needs periodic re-running.

Layouts do drift, and that does not explain one table getting less maintenance than its neighbour.

B Predictive optimization does not run ZORDER and ignores Z-ordered files, so that table receives less automatic maintenance.

A tuned table receives less automatic benefit than an untuned one, which is counterintuitive and documented.

C The unclustered table benefits from data skipping that Z-order disables.

Z-order does not disable data skipping. It is a layout technique that supports it.

D Predictive optimization only maintains tables with clustering keys defined.

It maintains managed tables generally, which is why the unclustered one is fine.

ON THE EXAM

The engineering effort is causing the problem. Migrating to liquid clustering restores full automatic maintenance.

> A compliance investigation asks for a complete row-level history of a transactions table covering the past two years. Change data feed was enabled on the table fourteen months ago and has never been disabled.
>
> The table is managed with predictive optimization enabled and default retention.

What can actually be provided?

A Fourteen months of row-level changes, from the point CDF was enabled.

B Only changes within the retention window, since CDF is not a permanent record and vacuum deletes change data files.

C Two years, since the transaction log retains all history regardless of CDF.

D Nothing, since CDF cannot be queried retrospectively.

◆ PAUSE AND DECIDE

What can actually be provided?

A Fourteen months of row-level changes, from the point CDF was enabled.

Gets the enablement rule right and forgets that retention also applies.

B Only changes within the retention window, since CDF is not a permanent record and vacuum deletes change data files.

Two limits stack: nothing before enablement, and nothing beyond retention.

C Two years, since the transaction log retains all history regardless of CDF.

Nothing before enablement was ever recorded as a change feed.

D Nothing, since CDF cannot be queried retrospectively.

It can, within its window.

ON THE EXAM

A is the near miss and the more instructive wrong answer. Half the rule is easier to remember than both halves.

A platform team migrated a set of batch jobs from classic job compute to serverless. The Databricks bill shows DBU consumption up 60%. A manager concludes the migration should be reversed.

Runtimes and schedules are unchanged.

What should be checked first?

A Nothing; a 60% rise in DBUs is a 60% rise in cost.

B The cloud provider invoice, since serverless DBU rates already include the VM cost billed separately for classic.

C Whether Photon was disabled during the migration.

D Whether autoscaling limits were carried over correctly.

◆ PAUSE AND DECIDE

What should be checked first?

A Nothing; a 60% rise in DBUs is a 60% rise in cost.
DBUs are one component, and for serverless they carry a cost that used to appear on another invoice.

B The cloud provider invoice, since serverless DBU rates already include the VM cost billed separately for classic.
Classic bills DBUs plus VMs. Serverless folds the VM into the DBU, so DBUs up and infrastructure down can be a wash.

C Whether Photon was disabled during the migration.
Would affect runtime, and the scenario says runtimes are unchanged.

D Whether autoscaling limits were carried over correctly.
Serverless does not use your autoscaling configuration.

ON THE EXAM

Total cost is DBUs plus infrastructure for classic, and DBUs alone for serverless. Judging from one invoice is the error.

> A platform team stores twelve database credentials in a single secret scope. A new requirement says the reporting team may access only the two reporting credentials.
>
> An engineer proposes granting the reporting team READ on those two secrets.

What should they be told?

A READ must be granted at scope level, so split the credentials into separate scopes.

B Grant WRITE instead, which is narrower than READ.

C Use a row filter on the secret scope to restrict which secrets are visible.

D Grant READ on the scope and rely on the reporting team not reading the others.

◆ PAUSE AND DECIDE

What should they be told?

A READ must be granted at scope level, so split the credentials into separate scopes.

Secret access control applies to the scope, not to individual secrets. Two audiences means two scopes.

B Grant WRITE instead, which is narrower than READ.

WRITE contains READ. The levels are nested, not alternatives.

C Use a row filter on the secret scope to restrict which secrets are visible.

Row filters govern table data. A secret scope is a workspace object.

D Grant READ on the scope and rely on the reporting team not reading the others.

Not a control. READ on the scope permits listing and reading every secret in it.

ON THE EXAM

C is the two-systems confusion: reaching for a Unity Catalog feature to govern a workspace object.

> A team must publish a dataset of customer purchase patterns to an external research partner. The dataset must fall outside the scope of privacy regulation entirely. Customer identifiers will be replaced with values generated by a secure tokenisation service, with the vault retained internally.
>
> Legal asks whether this meets the requirement.

Does it?

A. Yes. Tokenisation removes the identifiers, so the data is anonymised.

B. No. Tokenisation is pseudonymisation and is reversible, so the result is still personal data. Anonymisation requires suppression or generalisation.

C. Yes, provided the vault is stored in a separate catalog.

D. No, because tokenisation is not supported on Databricks.

◆ PAUSE AND DECIDE

Does it?

A Yes. Tokenisation removes the identifiers, so the data is anonymised.

Tokenisation replaces identifiers reversibly. Reversible means still personal data.

B No. Tokenisation is pseudonymisation and is reversible, so the result is still personal data. Anonymisation requires suppression or generalisation.

The requirement is to be out of scope entirely, and only anonymisation achieves that.

C Yes, provided the vault is stored in a separate catalog.

Where the vault lives does not change that re-identification remains possible.

D No, because tokenisation is not supported on Databricks.

Right conclusion, wrong reason. Tokenisation is a perfectly available technique.

ON THE EXAM

D reaches the correct verdict through a false claim, which is a distractor pattern worth recognising.

> An organisation wants every employee to be able to discover which datasets exist and request access, while ensuring nobody can read data they have not been granted.
>
> A proposal grants USE CATALOG on all catalogs to all account users, reasoning that usage privileges alone confer no data access.

What is the better approach?

A The proposal is sound; USE CATALOG alone grants no data access.

B Grant BROWSE on the catalogs, which is purpose-built for discovery and needs no usage privileges.

C Grant SELECT on all catalogs and rely on row filters.

D Grant READ METADATA on each table individually.

◆ PAUSE AND DECIDE

What is the better approach?

A The proposal is sound; USE CATALOG alone grants no data access.

The reasoning is correct and BROWSE is the purpose-built privilege, granting discovery and the ability to request access.

B Grant BROWSE on the catalogs, which is purpose-built for discovery and needs no usage privileges.

Exactly what BROWSE is for, and Databricks recommends granting it on catalogs to all account users.

C Grant SELECT on all catalogs and rely on row filters.

Grants SELECT on future tables too, so one missing filter exposes a table.

D Grant READ METADATA on each table individually.

Would work per object. BROWSE is granted once at catalog level.

ON THE EXAM

A is not wrong about the mechanics. It is just not the purpose-built answer, and it does not enable access requests.

> A governance programme wants every catalog, schema and table in the estate documented. A team proposes using AI-generated comments across the whole estate to avoid months of manual writing.
>
> There are 40 catalogs, 300 schemas and 12,000 tables.

What limitation should they plan around?

A AI-generated comments are available for tables and table columns only, not catalogs or schemas.

B AI-generated comments cannot be edited once applied.

C Comments are limited to plain text and cannot use Markdown.

D Only the metastore admin can add comments to any object.

◆ PAUSE AND DECIDE

What limitation should they plan around?

A AI-generated comments are available for tables and table columns only, not catalogs or schemas.

The documented scope. Tables and columns are the bulk of the work, and catalogs and schemas need writing by hand.

B AI-generated comments cannot be edited once applied.

They are suggestions you review and apply, and remain editable.

C Comments are limited to plain text and cannot use Markdown.

Markdown is supported in Catalog Explorer, though it does not render in DESCRIBE output.

D Only the metastore admin can add comments to any object.

The object owner or a MANAGE holder can.

ON THE EXAM

The good news is the limitation falls where the volume does not. 340 objects by hand is achievable; 12,000 is not.

> A job's task B failed. Task C depends on B and was skipped. Task D depends on A only, and completed successfully using an intermediate table that task B also writes to.
>
> An engineer repairs the run, selecting only task B.

What should they also consider?

A Nothing; repairing B automatically re-runs everything downstream of it.

B Task D succeeded against data B had partially written, so the option to re-run dependent tasks may be needed.

C Task D must be repaired first, since it completed before B.

D The whole job must be restarted, since repair cannot handle partial writes.

◆ PAUSE AND DECIDE

What should they also consider?

A Nothing; repairing B automatically re-runs everything downstream of it.

C was skipped and will re-run. D succeeded and is not a declared dependent of B.

B Task D succeeded against data B had partially written, so the option to re-run dependent tasks may be needed.

Succeeding is not the same as still being correct, and D's declared dependency hides its real one.

C Task D must be repaired first, since it completed before B.

Repair order is not something you control this way.

D The whole job must be restarted, since repair cannot handle partial writes.

Repair handles this without repeating the tasks that are still valid.

ON THE EXAM

D depends on A in the job graph and on B in reality. The graph does not know about the shared table.

> A team wants a point-in-time snapshot of a materialized view before a major pipeline change, so they can compare results afterwards. They attempt a shallow clone and it fails.
>
> They ask whether deep clone would work instead.

What should they be told?

A Deep clone works; only shallow clone is unsupported for materialized views.

B Neither works. Materialized views are unsupported as a CLONE source or target. Use CREATE TABLE AS SELECT.

C Shallow clone requires DBR 13.3 LTS, which is the actual blocker.

D Clone the underlying Delta table the materialized view is built on.

◆ PAUSE AND DECIDE

What should they be told?

A Deep clone works; only shallow clone is unsupported for materialized views.

Neither kind is supported. The restriction is on the object type, not the clone type.

B Neither works. Materialized views are unsupported as a CLONE source or target. Use CREATE TABLE AS SELECT.

A stated limitation, and CTAS produces exactly the comparison snapshot they wanted.

C Shallow clone requires DBR 13.3 LTS, which is the actual blocker.

A real requirement for shallow clones of managed tables, and not what blocks this.

D Clone the underlying Delta table the materialized view is built on.

Snapshots the input rather than the computed result, which is not the comparison they want.

ON THE EXAM

C names a genuine version requirement that is not the blocker, which is a distractor pattern worth recognising.

How to read your result

- **17 or more.** Comfortable. Compare against exam 1 and look at whether the same sections are weak.
- **13 to 16.** On track. A section you missed in both exams is worth a re-watch, not a re-read.
- **Below 13.** Work through exam 1's misses again before attempting exam 3.
- Compare the two exams by **section**, not by total. A steady total can hide one section going badly twice.

> ON THE EXAM
>
> Exam 3 is the last one, and it closes with a read of what the current guide emphasises. Save it until the other two are solid.

PRACTICE EXAM 3 · EXAM STRATEGY - ALL DOMAINS

Practice exam 3

The last twenty questions, and a read of where this exam is heading.

TYPE Practice exam QUESTIONS 20 TOPIC All sections

admin@sthithapragna.com

> A streaming table has ingested from a Kafka topic for eighteen months and holds 4 billion rows. The topic retains seven days. A schema change requires the pipeline logic to be rewritten, and an engineer plans a full refresh afterwards.
>
> The team asks what they should prepare for.

What is the risk?

A The full refresh will take several hours but complete correctly.

B A full refresh clears the streaming state and reprocesses from source. Everything older than seven days cannot be reprocessed and is lost.

C The full refresh will fail because the checkpoint is incompatible with the new logic.

D Nothing; streaming tables retain their own copy of source data.

◆ PAUSE AND DECIDE

What is the risk?

A The full refresh will take several hours but complete correctly.

It will complete, and with only seven days of data. The duration is not the problem.

B A full refresh clears the streaming state and reprocesses from source. Everything older than seven days cannot be reprocessed and is lost.

Retention arithmetic: eighteen months of history behind a seven-day source. The difference is unrecoverable.

C The full refresh will fail because the checkpoint is incompatible with the new logic.

A full refresh clears the checkpoint deliberately. That is what it does.

D Nothing; streaming tables retain their own copy of source data.

They hold processed output, which a full refresh discards before reprocessing.

ON THE EXAM

A full refresh that succeeds is the dangerous outcome here. It completes cleanly and the table is 4 billion rows lighter.

A serverless wheel task fails at import time on a library that works fine in a developer's notebook. The developer installed it in the notebook's environment panel; the task uses an environment defined in the bundle.

Both specify the same library.

What is the most likely cause?

A The environment defined in the bundle does not include a transitive dependency the notebook panel resolved automatically.

B Wheel tasks cannot use environments; only notebook tasks can.

C Serverless does not support third-party libraries.

D The bundle must install libraries at cluster scope rather than in an environment.

◆ PAUSE AND DECIDE

What is the most likely cause?

A The environment defined in the bundle does not include a transitive dependency the notebook panel resolved automatically.

Both specify the same library and only one works, so the difference is in what else got resolved alongside it.

B Wheel tasks cannot use environments; only notebook tasks can.

The reverse. Notebook tasks are the ones that cannot take a task environment specification.

C Serverless does not support third-party libraries.

It supports them through environments, which is what the bundle is using.

D The bundle must install libraries at cluster scope rather than in an environment.

There is no cluster to scope to on serverless.

ON THE EXAM

B is the inversion trap. The exception genuinely exists, and it applies to notebook tasks, not wheel tasks.

> A team on the Core pipeline edition must build an SCD Type 2 dimension. Their source is a legacy system with no change data capture, delivering a nightly full extract to cloud storage.
>
> They plan to write the pipeline in SQL, as nobody on the team writes Python.

What must change?

A. Nothing; AUTO CDC FROM SNAPSHOT works in SQL on any edition.

B. Move to serverless or Pro or Advanced, and accept that snapshot processing is Python only.

C. Enable change data capture on the legacy source, which AUTO CDC requires.

D. Upgrade the Databricks Runtime, which unlocks the CDC APIs.

◆ PAUSE AND DECIDE

What must change?

A Nothing; AUTO CDC FROM SNAPSHOT works in SQL on any edition.

Wrong twice. The edition gate is real, and snapshot processing has no SQL interface.

B Move to serverless or Pro or Advanced, and accept that snapshot processing is Python only.

Two independent blockers, and both must be cleared. Their SQL preference cannot be met.

C Enable change data capture on the legacy source, which AUTO CDC requires.

Would let them use the SQL interface, and the snapshot API exists precisely so this is not required.

D Upgrade the Databricks Runtime, which unlocks the CDC APIs.

The gate is the pipeline edition, not the runtime version.

ON THE EXAM

Two blockers, not one. Scan for every stated constraint before choosing, because these questions often hide a second.

> A job's ingestion task loads files and its three downstream transformation tasks are expensive. On days when no files arrive, ingestion succeeds having loaded nothing, and the transformations run anyway.
>
> The team wants the transformations skipped on those days.

What should they build?

A Set the transformations' run-if condition to At least one succeeded.

B Have ingestion set a task value with the row count, then branch on it with an If/else task.

C Make the ingestion task fail when no files arrive, so downstream tasks are skipped.

D Set the transformations' run-if condition to None failed.

◆ PAUSE AND DECIDE

What should they build?

A Set the transformations' run-if condition to At least one succeeded.

Run-if branches on outcome. Ingestion succeeds whether or not it found anything.

B Have ingestion set a task value with the row count, then branch on it with an If/else task.

Branching on a value a previous task computed is exactly what task values and If/else exist for.

C Make the ingestion task fail when no files arrive, so downstream tasks are skipped.

Makes a normal day look like an incident, and poisons alerting and run history.

D Set the transformations' run-if condition to None failed.

Also branches on outcome, and nothing failed.

ON THE EXAM

Outcome or value is the first question. Nothing failed here, so no run-if condition can express what is wanted.

A team must load a one-off historical archive of 1.2 million Parquet files from a decommissioned system into a Delta table, then never read that location again. They work in SQL and want the simplest approach.

A colleague insists Auto Loader is always the recommended tool for file ingestion.

What is the better assessment?

A The colleague is right; Auto Loader is always preferred.

B COPY INTO suits a bounded one-off load, is idempotent on rerun, and leaves no checkpoint to maintain.

C Neither works at this scale; use a Spark batch read and insert.

D Auto Loader is required because COPY INTO does not support Parquet.

◆ PAUSE AND DECIDE

What is the better assessment?

A The colleague is right; Auto Loader is always preferred.

The recommendation is for incremental ingestion of arriving files. This load is bounded and then finished.

B COPY INTO suits a bounded one-off load, is idempotent on rerun, and leaves no checkpoint to maintain.

Idempotence covers the retry risk, and there is no ongoing stream to operate afterwards.

C Neither works at this scale; use a Spark batch read and insert.

Both handle far more. Auto Loader alone scales to billions of files.

D Auto Loader is required because COPY INTO does not support Parquet.

Both support the same eight formats, Parquet included.

ON THE EXAM

Recommendations have scope. Auto Loader is recommended for incremental ingestion, and this load has a defined end.

> A report computes a running total of monthly revenue ordered by month. Several months happen to have identical revenue figures. The engineer wrote an ORDER BY with no explicit frame.
>
> The running total jumps by several months' revenue at once rather than increasing month by month.

What explains it?

A The default frame is RANGE, so rows sharing an ordering value are treated as one peer group.

B The ORDER BY needs a PARTITION BY to reset correctly.

C The default frame includes UNBOUNDED FOLLOWING, so future rows are counted.

D Running totals require a cumulative sum function rather than a window aggregate.

◆ PAUSE AND DECIDE

What explains it?

A The default frame is RANGE, so rows sharing an ordering value are treated as one peer group.
With ORDER BY and no explicit frame the default is RANGE, and peers are included together.

B The ORDER BY needs a PARTITION BY to reset correctly.
A running total across all months is what was wanted.

C The default frame includes UNBOUNDED FOLLOWING, so future rows are counted.
The default stops at the current row, not the end of the partition.

D Running totals require a cumulative sum function rather than a window aggregate.
A window aggregate is exactly how a running total is written.

ON THE EXAM

Note the detail: the duplicates are in the revenue figures, and the ordering is by month. Read carefully which column ties.

A nightly job joining a 6 TB fact table to a currency reference table has run reliably for two years in twelve minutes. This month it began failing with driver out-of-memory errors.

The query carries a BROADCAST hint on the reference table. New currencies and historical rates have been added over time.

What is the cause?

A The fact table has grown beyond what a broadcast join supports.

B The hint is honoured above the broadcast threshold, so the grown reference table is collected to the driver.

C AQE stopped converting the join because statistics are stale.

D Shuffle partitions must be increased for the larger reference table.

◆ PAUSE AND DECIDE

What is the cause?

A The fact table has grown beyond what a broadcast join supports.

The broadcast side is the small one. The large side is never collected.

B The hint is honoured above the broadcast threshold, so the grown reference table is collected to the driver.

A hint overrides the automatic size check, which is exactly what makes it dangerous as data grows.

C AQE stopped converting the join because statistics are stale.

The hint forces the broadcast. AQE is not making this decision.

D Shuffle partitions must be increased for the larger reference table.

A broadcast join does not shuffle the large side.

ON THE EXAM

Two years of reliability then a sudden failure is the signature. Nothing changed except the size of a table somebody once measured.

A team maintains a curated view that joins four tables and masks two columns. They must make it available to a partner organisation that does not use Databricks.

They add the view to a share and create an open sharing recipient. The view never appears for the recipient.

What should they do?

A Views are Databricks-to-Databricks only. Materialise the view as a table and share that.

B Share the view WITH HISTORY, which makes it visible to open sharing recipients.

C Grant the recipient SELECT on the four underlying tables as well.

D The recipient must install a connector before views appear.

◆ PAUSE AND DECIDE

What should they do?

A Views are Databricks-to-Databricks only. Materialise the view as a table and share that.

A view is a query needing a catalog to resolve against, and a non-Databricks recipient has none. Materialising is the intended pattern.

B Share the view WITH HISTORY, which makes it visible to open sharing recipients.

WITH HISTORY concerns cloud token eligibility for tables. Views are not eligible at all.

C Grant the recipient SELECT on the four underlying tables as well.

Defeats the masking the view exists to provide.

D The recipient must install a connector before views appear.

A connector is needed to read anything, and it cannot make an unsupported asset type appear.

ON THE EXAM

Materialising is the intended pattern, not a workaround. The masking is applied when the table is written.

> A dashboard query takes 55 seconds. The query profile shows wall-clock duration of 53 seconds and aggregated task time of 4 seconds. The query is a simple filter and aggregate over a well-clustered managed table.
>
> No operator shows anything unusual.

Where should the engineer look?

A The query logic, since a simple aggregate should not take 53 seconds.

B Compute availability, since aggregated task time far below wall-clock means tasks waited for nodes.

C Data skew, since one task must dominate.

D File layout, since poor pruning would explain the elapsed time.

◆ PAUSE AND DECIDE

Where should the engineer look?

A The query logic, since a simple aggregate should not take 53 seconds.

The work took four seconds. The query is not the problem.

B Compute availability, since aggregated task time far below wall-clock means tasks waited for nodes.

Documented: task time shorter than wall-clock indicates tasks waiting for available nodes.

C Data skew, since one task must dominate.

Skew shows as a long aggregated task time unevenly distributed, not a short one.

D File layout, since poor pruning would explain the elapsed time.

Bad pruning means more data scanned, which raises task time rather than lowering it.

ON THE EXAM

Four seconds of work inside fifty-three seconds of elapsed time. Forty-nine seconds went somewhere that was not computing.

A support team needs to monitor data quality trends across four pipelines by querying their event logs. The pipelines use default settings and the support team members are not the owners.

An engineer suggests each support member call the event log function with the pipeline identifiers.

Why will that not work?

A The function is owner-only and its views cannot be shared. Publish the event logs, create views, and grant on those.

B The function requires DBR 13.3 LTS on the support team's compute.

C Event logs must be enabled in Advanced settings before any query works.

D Support members must be made pipeline owners in turn.

◆ PAUSE AND DECIDE

Why will that not work?

A The function is owner-only and its views cannot be shared. Publish the event logs, create views, and grant on those.

Two documented facts, plus the recommended route. The function also applies only to legacy publishing mode.

B The function requires DBR 13.3 LTS on the support team's compute.

A real requirement, and not the blocker here.

C Event logs must be enabled in Advanced settings before any query works.

They exist by default as a hidden table. Publishing changes where they live.

D Support members must be made pipeline owners in turn.

Changes who the pipeline runs as, to solve a read-access problem.

ON THE EXAM

Note also that Databricks recommends a view over the event log for security reasons, not merely convenience.

> A team wants clustering keys chosen and adapted automatically as query patterns change. Half their tables are Unity Catalog managed with predictive optimization enabled; the other half are external tables in storage they manage.
>
> They want one consistent approach.

What is possible?

A CLUSTER BY AUTO on all tables, since it is a table-level setting.

B CLUSTER BY AUTO on the managed tables only. External tables need keys declared manually.

C Neither set can use it; automatic clustering applies only at table creation.

D Both, after running OPTIMIZE FULL on the external tables first.

◆ PAUSE AND DECIDE

What is possible?

A CLUSTER BY AUTO on all tables, since it is a table-level setting.

It requires a Unity Catalog managed table, so external tables are excluded.

B CLUSTER BY AUTO on the managed tables only. External tables need keys declared manually.

Automatic key selection is driven by predictive optimization, which runs only on managed tables.

C Neither set can use it; automatic clustering applies only at table creation.

It can be set on existing unpartitioned tables too.

D Both, after running OPTIMIZE FULL on the external tables first.

OPTIMIZE FULL reclusters data. It does not unlock the automatic form.

ON THE EXAM

The honest answer refuses the premise. They cannot have one consistent approach without converting the external tables.

> A team is sizing compute for two workloads: a Structured Streaming job with modest state, and a nightly OPTIMIZE and VACUUM maintenance job on external tables.
>
> They ask which instance family to choose for each.

What does Databricks recommend?

A Memory optimised for both, since streaming holds state and maintenance rewrites files.

B Compute optimised for both, which the docs name for Structured Streaming and for maintenance jobs.

C Storage optimised for both, since both are IO-heavy.

D General purpose for both, in the absence of specific requirements.

◆ PAUSE AND DECIDE

What does Databricks recommend?

A Memory optimised for both, since streaming holds state and maintenance rewrites files.

Memory optimised is named for machine learning and for heavy shuffle and spill.

B Compute optimised for both, which the docs name for Structured Streaming and for maintenance jobs.

The documented rule of thumb names both workloads for compute optimised explicitly.

C Storage optimised for both, since both are IO-heavy.

Named for workloads benefiting from caching, such as ad-hoc interactive analysis.

D General purpose for both, in the absence of specific requirements.

There are specific requirements here, and the docs name them.

ON THE EXAM

This is the row people get wrong. Streaming and maintenance both want compute optimised, which is not most people's instinct.

> A masked salary table is queried by two teams. One team sees masked values as expected. The other, on an older long-term-support runtime, reports the table appears completely empty with no error message.
>
> Both teams have identical Unity Catalog grants.

What is happening?

A Runtimes below 12.2 LTS do not support column masks and fail securely, returning no data.

B The second team lacks SELECT, which presents as an empty table.

C The masking function returns null on that runtime.

D Column masks require serverless compute.

◆ PAUSE AND DECIDE

What is happening?

A Runtimes below 12.2 LTS do not support column masks and fail securely, returning no data.

Documented: unsupported runtimes fail securely rather than returning unprotected data.

B The second team lacks SELECT, which presents as an empty table.

Missing SELECT produces a permissions error, and the grants are stated to be identical.

C The masking function returns null on that runtime.

That would return rows containing nulls, not an empty table.

D Column masks require serverless compute.

Closer to the ABAC requirement. Table-level masks work on classic compute.

ON THE EXAM

The scenario rules out permissions explicitly. An empty result with no error is the protection working correctly.

> A right-to-erasure request must be propagated to a silver streaming table fed from a bronze Delta table. An engineer deletes the records from bronze only, and the pipeline fails on the next run.
>
> They want the deletion to propagate correctly.

What is the documented method?

A Delete from bronze, then fully refresh the streaming table.

B Delete in the source, delete from the streaming table with DML, then set skipChangeCommits on the read.

C Set skipChangeCommits on the read, which handles the deletion automatically.

D Enable change data feed on bronze, which propagates deletes without further work.

◆ PAUSE AND DECIDE

What is the documented method?

A Delete from bronze, then fully refresh the streaming table.

A documented alternative, and it risks data loss where a source is beyond its retention period.

B Delete in the source, delete from the streaming table with DML, then set skipChangeCommits on the read.

The three documented steps, in order. The flag comes last, after the deletion is applied to both tables.

C Set skipChangeCommits on the read, which handles the deletion automatically.

Skips the change without applying it downstream, so silver keeps the record that was supposed to be erased.

D Enable change data feed on bronze, which propagates deletes without further work.

Streaming from the change feed is the more robust design, and it requires handling deletes in your code.

ON THE EXAM

C is the same flag as B, without the two deletions. Order is what separates the documented method from the failure mode.

> An engineer owns a schema and can create, alter and drop every table in it. Running SHOW GRANTS on one of those tables returns no privileges for them at all.
>
> They raise a ticket asking why their permissions have been removed.

What should they be told?

A Permissions were removed and must be re-granted from the schema down.

B Schema owners get implied management of child objects. Implied capabilities do not appear in SHOW GRANTS.

C SHOW GRANTS only lists grants made by the current user.

D They must be metastore admin to see grants on tables they do not own.

◆ PAUSE AND DECIDE

What should they be told?

A Permissions were removed and must be re-granted from the schema down.

Nothing was removed. They can still administer the tables, which the scenario states.

B Schema owners get implied management of child objects. Implied capabilities do not appear in SHOW GRANTS.

Ownership and its implied capabilities are not explicitly granted, so a grants listing shows nothing.

C SHOW GRANTS only lists grants made by the current user.

Invented.

D They must be metastore admin to see grants on tables they do not own.

They can view grants. The point is that their own capability is implied rather than listed.

ON THE EXAM

An empty grants list on an object you can administer is expected. It catches people auditing permissions.

> A cluster starts and reports healthy. Every notebook attached to it fails within seconds with import errors for a library the team installs through an init script. The driver logs show only those import failures.
>
> Log delivery is configured to a storage location.

Where should they look?

A The executor logs, since library installation happens on the workers.

B The init script logs in the delivered log location, since a failed init script leaves a cluster that starts but cannot run anything.

C The compute metrics UI, to check whether memory prevented installation.

D The cluster event log, which records library installation.

◆ PAUSE AND DECIDE

Where should they look?

A The executor logs, since library installation happens on the workers.

Would show the same downstream symptom, not the cause.

B The init script logs in the delivered log location, since a failed init script leaves a cluster that starts but cannot run anything.

The signature failure mode, and log delivery is configured so the init script folder is available.

C The compute metrics UI, to check whether memory prevented installation.

Metrics show resource usage over time, not setup failures.

D The cluster event log, which records library installation.

It records lifecycle events such as creation, termination and resizing.

ON THE EXAM

The import errors are the symptom, faithfully reported. The cause is one layer earlier, at cluster startup.

> A nightly job failed because a source file arrived under the wrong date path. The file is now available under yesterday's date. The team wants to complete that run without altering tomorrow's scheduled behaviour.
>
> A colleague proposes editing the job's default date parameter, repairing, then editing it back.

What is the better approach?

A The colleague's approach is fine if the second edit is not forgotten.

B Repair the run, entering the corrected date as a parameter override, which applies to that repair only.

C Restart the job, which prompts for parameters at launch.

D Create a one-off job with the corrected date.

◆ PAUSE AND DECIDE

What is the better approach?

A The colleague's approach is fine if the second edit is not forgotten.

Works, and mutates a production job to fix one run, with a manual step nobody will remember at 3am.

B Repair the run, entering the corrected date as a parameter override, which applies to that repair only.

Overrides are scoped to the repair, leaving the job definition and tomorrow's schedule untouched.

C Restart the job, which prompts for parameters at launch.

Repeats every task when only the failed one needs running.

D Create a one-off job with the corrected date.

Leaves the original run failed in the history, and adds a second job.

ON THE EXAM

On a later repair, clearing the key and value returns a parameter to its original value, which is a nice touch worth knowing.

> A team version-controls every notebook in a production Git folder updated automatically on merge. Job schedules, cluster configuration and permissions are set by hand in the workspace.
>
> After an outage, they cannot determine what a job's schedule was three weeks ago.

What is missing?

A Enable audit logging to reconstruct configuration history.

B Git folders deploy code, not resources. Defining jobs in a bundle puts schedules, compute and permissions under version control.

C Move the production Git folder under a user folder so history is retained.

D Job configuration cannot be version controlled on Databricks.

◆ PAUSE AND DECIDE

What is missing?

A Enable audit logging to reconstruct configuration history.

Audit records who changed something. It is not the configuration in source control, nor is it redeployable.

B Git folders deploy code, not resources. Defining jobs in a bundle puts schedules, compute and permissions under version control.

Half the deployment is versioned and half is not, and the half that is not is where the question came from.

C Move the production Git folder under a user folder so history is retained.

User-level folders are development checkouts, and location is not the problem.

D Job configuration cannot be version controlled on Databricks.

That is exactly what bundles are for.

ON THE EXAM

The documentation's own division: Git folders for interactive development, bundles for CI/CD and production deployment.

> A sensor readings table has columns for device id, reading value, latitude, longitude, a captured-at timestamp, and a reading date. It receives around 80 million readings per day from 12,000 devices.
>
> Analysts almost always filter by day, occasionally by device.

Which column should be used for partitioning?

A device_id

B captured_at

C reading_date

D latitude

◆ PAUSE AND DECIDE

Which column should be used for partitioning?

A device_id

Twelve thousand devices means twelve thousand partitions, and unevenly sized ones.

B captured_at

A timestamp at 80 million readings a day is effectively unique. The classic trap.

C reading_date

Low cardinality, evenly distributed, useful partition size, and exactly what analysts filter on.

D latitude

A float with no relationship to how the data is queried.

ON THE EXAM

The same shape as the exam guide's own published question. Low cardinality, queries filter on it, evenly distributed, useful size.

A team is building a customer dimension in a new gold schema. Surrogate keys will be generated by an identity column. No keys are being migrated from an existing system.

They ask which identity column form to use.

Which is appropriate, and why?

A Generated by default, which allows values to be supplied if ever needed.

B Generated always, which prohibits supplying your own values and is safer for a new surrogate.

C Either; the two forms are equivalent in behaviour.

D Neither; surrogate keys should be hashes of the business key.

◆ PAUSE AND DECIDE

Which is appropriate, and why?

A. Generated by default, which allows values to be supplied if ever needed.

Permitting supplied values on a brand new surrogate invites a conflict with the generated sequence.

B. Generated always, which prohibits supplying your own values and is safer for a new surrogate.

Nothing is being migrated, so nothing needs to supply its own values. Prohibiting them removes a class of error.

C. Either; the two forms are equivalent in behaviour.

They differ precisely on whether your own values may be supplied.

D. Neither; surrogate keys should be hashes of the business key.

A hash is the documented alternative when identity columns are unavailable, such as on an AUTO CDC target.

ON THE EXAM

D is worth remembering for the case it does fit: identity columns are unsupported on AUTO CDC targets.

How to read your result

- **17 or more across all three exams.** You are ready. Book it.
- **13 to 16 consistently.** Close. Identify the two weakest sections and re-watch those modules before booking.
- **Below 13.** Work the misses from all three exams, then re-attempt them in a fortnight.
- Look at the **trend across the three**, not any single score. Improving is a better signal than a high one-off.

> ON THE EXAM
>
> Sixty questions is a reasonable sample. If one section has gone badly in all three, that is a real signal rather than bad luck.

Where this exam appears to be heading

- **Declarative over hand-written.** Pipelines are the recommended default, and hand-written streaming is the exception you justify.
- **Managed over external.** Managed tables are the recommended type, and most automatic optimisation only reaches them.
- **Automatic over configured.** Predictive optimization, automatic clustering, file events, enhanced autoscaling.
- **Policy over per-object.** ABAC over per-table masks, governed tags over conventions, bundles over hand configuration.

> ON THE EXAM
>
> Four movements in the same direction: declare the intent and let the platform decide the mechanics. Questions increasingly reward that instinct.

What that means for answering questions

- When two options are both workable, the one that **declares intent** rather than configuring mechanics is usually intended.
- But **not always.** Module 10 chose COPY INTO over Auto Loader, and module 22 defended partitioning for metadata queries.
- The exam still rewards **reading the requirement**, and a requirement can rule out the modern answer.
- And the guide lags the platform. **Answer the guide's language**, and know what the docs now say.

> ON THE EXAM
>
> A heuristic, not a rule. The trend tells you where to look first; the requirement sentence still decides.

That is the course

- Thirty-eight modules. Ten sections. Sixty practice questions and over two hundred worked scenarios.
- Every fact checked against current documentation, with the places the guide has fallen behind flagged as they arose.
- **Check the exam guide two weeks before you sit.** It changes, and it says so itself.
- Good luck.

> ON THE EXAM
>
> If something in this course turns out to be wrong or out of date, that is worth knowing. The platform moves faster than any course can.